
ANALYST TO ARCHITECT · PART 5

Data Engineering, Integration & Scale

Ingestion, pipelines and orchestration, data quality, big data, warehouses and lakehouses, streaming, data activation, and the cloud.

Contents

Part 5 — Data Engineering, Integration & Scale	1
Chapter 45. Data Ingestion & Integration	2
45.1 Where company data comes from	4
45.2 Setting up the practice environment	5
45.3 Full loads	9
45.4 Incremental loads, and the update they miss	12
45.5 Detecting change with hashes	14
45.6 Change data capture	22
45.7 Loading files reliably	25
45.8 When the source changes shape	31
45.9 Pulling data from APIs	32
45.10 Build or buy: connectors versus custom code	36
45.11 Web data: legal and ethical limits	37
Chapter 46. Pipelines & Orchestration	48
46.1 From scheduled scripts to pipelines	50
46.2 The core ideas	51
46.3 Building Riverstone's daily pipeline	53
46.4 Idempotency, proven with a failure test	68
46.5 Partitions and backfills	72
46.6 Retries, timeouts, and alerting	74
46.7 Delivery only after the data passes its checks	78
46.8 The same pipeline in Airflow	82
46.9 Choosing an orchestrator	85
46.10 Operating pipelines	86
Chapter 47. Data Quality, Observability & Contracts	95
47.1 The dimensions of data quality	96
47.2 Testing data inside the pipeline	97
47.3 A check library for Riverstone	98
47.4 Write-audit-publish	107
47.5 Observability: freshness, volume, and unusual values	112
47.6 Lineage: what breaks when this breaks	116
47.7 Data contracts in practice	117
47.8 Handling a data incident	122
47.9 Alerts people still read	123
47.10 The tools landscape	126
Chapter 48. Big Data & Distributed Compute	139
48.0 Setting up Spark, Polars, and the data	141
48.1 Most "big data" isn't	145
48.2 What "distributed" actually means	146
48.3 PySpark on the sensor data	147
48.4 Reading the plan, and finding the shuffle	150
48.5 Making a job faster	153
48.6 The same question in three tools	158
48.7 Writing Spark that doesn't surprise you	164
48.8 The wider landscape in one page	166
Chapter 49. Storage, Warehouses & Lakehouses	175
49.0 Setting up	177

49.1 Where data sits	178
49.2 Row versus columnar, measured	179
49.3 Inside a Parquet file	183
49.4 ACID, properly	186
49.5 Table formats: a folder of files that behaves like a table	187
49.6 Partitioning and file sizing	197
49.7 Warehouses: storage and compute, separated	198
49.8 What it costs	199
49.9 Choosing a layout	201
Chapter 50. Streaming & Real-Time	210
50.0 Setting up	212
50.1 What "real-time" actually means	212
50.2 The log: topics, partitions, offsets, groups	213
50.3 Delivery guarantees, and the duplicates you will get	218
50.4 Streaming with Spark: the same code, running forever	224
50.5 Event time, windows, and watermarks	226
50.6 Writing a stream into a table	233
50.7 Operating a stream	235
50.8 Riverstone's plant monitoring case	235
Chapter 51. Data Activation: Reverse ETL, APIs & System Integration	245
51.0 Setting up the practice environment	247
51.1 The loop nobody finishes	249
51.2 Computing what to sync	250
51.3 Writing to a system through an API	255
51.4 Making writes idempotent, and proving it	260
51.5 System of record and conflict rules	263
51.6 Webhooks: pushing instead of polling	264
51.7 Reconciliation for writes	267
51.8 Integration patterns	270
51.9 Choosing how to build it	271
51.10 When there's no API: RPA and file drops	272
Chapter 52. The Cloud, Containers & Infrastructure as Code	283
52.0 Setting up	285
52.1 Cloud fundamentals	286
52.2 Containers	292
52.3 Orchestrating containers	303
52.4 Infrastructure as Code	309
52.5 CI/CD	316
52.6 Secrets, done properly	320
52.7 Cost	321
52.8 Choosing a deployment target	323

Part 5 — Data Engineering, Integration & Scale

Part 5 builds the plumbing that moves data reliably: ingestion, pipelines and orchestration, data quality, big data, warehouses and lakehouses, streaming, data activation, and the cloud.

Read the chapters in this order:

Chapter	Time needed
45. Data Ingestion & Integration	15-20 hours
46. Pipelines & Orchestration	16-20 hours
47. Data Quality, Observability & Contracts	14-18 hours
48. Big Data & Distributed Compute	14-18 hours
49. Storage, Warehouses & Lakehouses	14-17 hours
50. Streaming & Real-Time	15-19 hours
51. Data Activation: Reverse ETL, APIs & System Integration	14-18 hours
52. The Cloud, Containers & Infrastructure as Code	20-26 hours

In total, allow 122-156 hours, including the exercises and projects.

Chapter 45. Data Ingestion & Integration

Chapter at a glance

You will learn to: name the kinds of source systems a data platform ingests from, and what makes each one hard · choose between full loads, incremental loads, and change data capture, and explain what each one can miss · show, with a real test, why a "new rows only" load silently loses updates · compute and use SHA-256 and MD5 fingerprints · detect inserted, changed, and deleted rows with hashes, and skip files that haven't changed · read real change events from PostgreSQL's log · load messy CSV files without silent type errors, and reconcile what you loaded against the file · recognize schema changes before they break a load · pull data from a paginated, authenticated, rate-limited API with retries · decide when to build a connector and when to buy one · judge the legal and ethical limits of collecting web data.

Before you start: Chapter 2 (formats; the idea of a file fingerprint; what an API is). Chapter 12 (transactions, upsert, loading a CSV). Chapters 13 and 28 (SQL). Chapter 17 (the virtual environment and Jupyter). Chapter 18 (calling an API, `.env` files). Chapter 20 (idempotency with a run key). Chapter 26 section 26.0 (the terminal). Chapter 29 (Python as software; the CRM client). Chapter 32 (dbt) helps but isn't required.

Time needed: 15–20 hours of reading and practice, spread over two to three weeks. Plan four sittings: sections 45.1–45.4; sections 45.5–45.6 (hashes, upserts, change data capture); sections 45.7–45.8 (files and schema changes); sections 45.9–45.11 and the project.

Tools: PostgreSQL 16, and Python in the book's virtual environment (Chapter 17) with `requests` and `python-dotenv` (Chapter 18), plus two libraries you install in section 45.2: `psycopg2` and `duckdb`. The Chapter 45 companion folder. Everything runs on your own computer.

Practice data: `riverstone_source`, a private copy of the Riverstone one-year database that plays the part of the ERP and the CRM; two dispatch files from the Bhiwandi Main warehouse; and a small local API that stands in for the CRM. Every output shown is from a real run.

In this chapter

START	Why this matters · In plain English	3
LEARN	45.1 Where company data comes from	4
	45.2 Setting up the practice environment	5
	45.3 Full loads	9
	45.4 Incremental loads, and the update they miss	12
	45.5 Detecting change with hashes	14
	45.6 Change data capture	22
	45.7 Loading files reliably	25
	45.8 When the source changes shape	31
	45.9 Pulling data from APIs	32
	45.10 Build or buy: connectors versus custom code	36
	45.11 Web data: legal and ethical limits	37
APPLY	Common mistakes · In the real world · Project	37
REVIEW	Recap · Key terms · Check yourself	41
PRACTISE	Exercises	42
	Answers	44
NEXT	Where this leads	47

START

Why this matters

Every report, dashboard, model, and automation you've built so far started with data that was already sitting in a table. Somebody put it there. In most companies, that somebody is a data engineer, and the first job is **ingestion**: getting data out of the systems where the business runs and into a place built for analysis, correctly, every day, without anyone copying files by hand.

Ingestion looks simple and isn't. The ERP doesn't tell you which orders changed yesterday. The warehouse's dispatch file arrives with dates in a different format and quantities written as text. The CRM's API only returns 20 records at a time, and refuses your requests if you ask too quickly. A source system adds a column on a Tuesday and your load breaks on Wednesday, or worse, keeps running with the wrong numbers.

The failures that hurt most in ingestion aren't crashes. They're loads that succeed and quietly deliver the wrong data. This chapter shows three of them happening on Riverstone data, and how to catch each one. Get this layer right and everything built on top of it can be trusted. Get it wrong and every chart in the company is wrong in ways nobody notices.

In plain English

Think about how a large kitchen gets its ingredients.

- Ingredients arrive from many **suppliers**: a vegetable vendor, a dairy, a spice wholesaler. Each delivers differently: some bring crates, some send a list and a truck, some you must collect. Those are **source systems**, each with its own way of handing over data.
- On the first day, the kitchen takes **everything** each supplier has. That's a **full load**.

- After that, it only wants what's **new or different** since yesterday. That's an **incremental load**. But if a supplier quietly swaps yesterday's milk for a different brand and keeps the same crate number, a kitchen that only counts new crates will never notice. That's the update a naive incremental load misses.
- A careful kitchen compares a **fingerprint** of each item against yesterday's: same weight, same label, same batch code. Any difference is a change. That's **change detection with hashes**.
- The best suppliers send a **delivery log** listing every change as it happens: "added two crates, replaced one". That's **change data capture**.
- At the receiving door, someone checks each delivery against the delivery note, sets aside anything damaged, and writes down what arrived. That's **reconciliation** and a **load log**.
- One supplier changes its packaging without warning. If the receiving clerk only checks crate counts, the new packaging sails through. That's a **schema change**.

Ingestion is the receiving door of the data platform. This chapter is about running it well.

LEARN

45.1 Where company data comes from

Before you design any load, know what kind of source you're dealing with. Each kind fails in its own way.

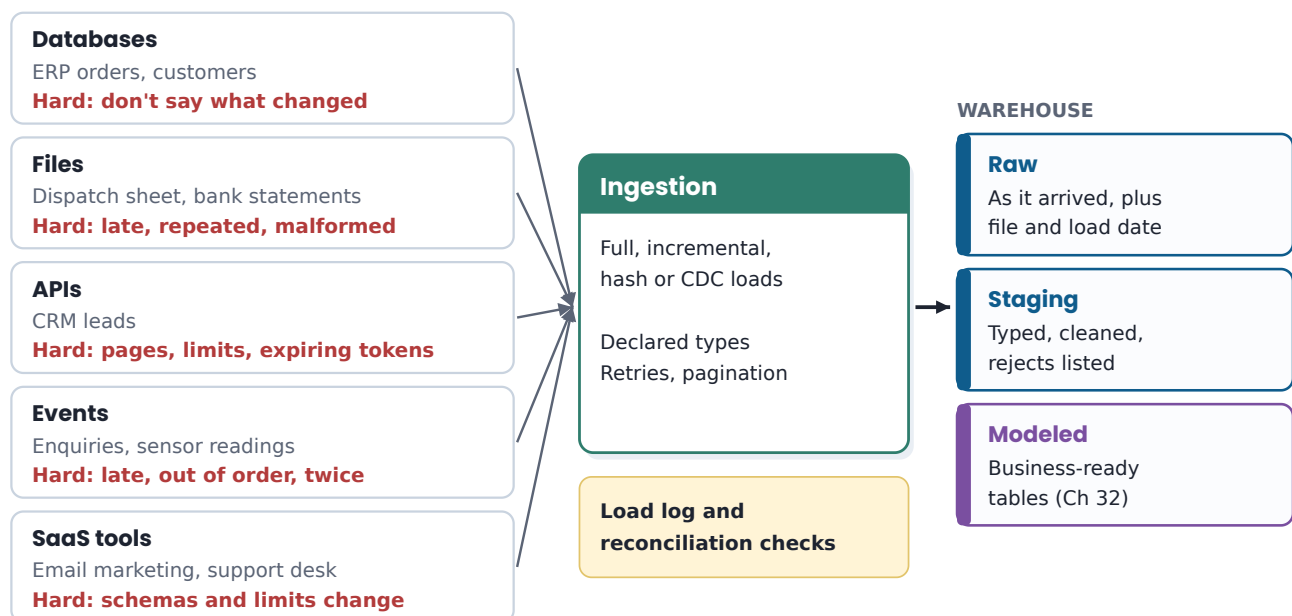


Figure 45.1 – The main kinds of source, and the difficulty each brings. Ingestion writes to a raw layer first, so the original data is kept even when later steps change.

Source type	Riverstone example	How you usually read it	What makes it hard
Operational databases	The ERP's orders, order lines, customers	SQL queries, or the database's change log	They're built to run the business, not to tell you what changed; heavy queries can slow the business down

Source type	Riverstone example	How you usually read it	What makes it hard
Files	Bhiwandi Main's daily dispatch sheet; bank statements	Reading CSV, Excel, JSON, or Parquet files from a folder, SFTP server, or cloud storage	Late, duplicated, renamed, or malformed files; formats chosen by a person, not a system
APIs	The CRM's leads	HTTP requests with authentication (Chapter 2)	Pagination, rate limits, expiring tokens, changing versions
Events	Website enquiries; plant sensor readings (Chapter 50)	Streams and message queues	Huge volumes; events arrive late, out of order, or twice
SaaS tools	Email marketing, the support desk	Their APIs, usually through a connector	Limits and schemas you don't control, and change without notice

Two terms appear throughout this part of the book.

- A **source system** is any system where data is first recorded: the ERP, the CRM, a spreadsheet the warehouse maintains.
- A **system of record** is the source that's officially correct for a piece of data. Riverstone's ERP is the system of record for orders; the CRM is the system of record for leads. When two systems disagree, the system of record wins. Chapter 51 relies on this when data flows back the other way.

Layers in the warehouse

Ingested data lands in layers, each with a purpose.

- The **raw** layer holds data as it arrived, with as little change as possible. If a later step is wrong, you can rebuild from raw without asking the source again.
- The **staging** layer holds the same data cleaned and typed: dates as dates, quantities as numbers, consistent names.
- The **modeled** layer holds the business-ready tables that reports use, often built with dbt (Chapter 32).

This chapter covers raw and staging. Chapter 49 covers how the warehouse stores these layers, and Chapter 32 covers modeling on top of them.

Simplification note. The chapter uses a local DuckDB file as the warehouse, so everything runs on your computer for free. The patterns are the same on cloud warehouses such as Snowflake, BigQuery, Redshift, or Databricks; Chapter 49 compares them.

45.2 Setting up the practice environment

The companion folder `companion/ch45/` contains four small helper files. You'll use them, but you won't need to change them.

File	What it does
<code>reset_ch45.py</code>	Creates <code>riverstone_source</code> , a private copy of <code>riverstone_2025</code> , deletes the local warehouse, and writes the practice files. Run it any time to start again.
<code>apply_day.py</code>	Simulates later business days in the ERP: new orders, and status changes to existing ones.
<code>mock_crm_api.py</code>	A tiny local web server that behaves like the CRM's API, including authentication, pagination, and rate limiting.
<code>make_files.py</code>	Writes Bhiwandi Main's dispatch files, messy in realistic ways.

Your copy of `riverstone_source` is disposable. The shared practice databases from Chapter 12 are never changed.

Change data capture in section 45.6 needs one PostgreSQL setting, `wal_level = logical`. It's covered there, with the exact steps.

Step 1. Your practice folder

Work the way Chapter 17 did. In your file manager, copy the folder `companion/ch45` and paste it inside `work`, so you have `work/ch45` to practise in and the original stays untouched.

Step 2. Install two new libraries

Open a terminal (Chapter 26, section 26.0), go to the book folder, then into your practice folder, and activate the book's virtual environment:

```
# terminal
$ cd work/ch45
$ source ../../.venv/bin/activate
$ python -m pip install psycopg2-binary==2.9.13 duckdb==1.5.6
```

How it works:

- `cd work/ch45` moves into your practice folder. `source ../../.venv/bin/activate` activates the environment from Chapter 17, two levels up; on Windows PowerShell it's `..\..\venv\Scripts\Activate.ps1`. The prompt starts with `(.venv)` when it's active.
- `psycopg2-binary` is a PostgreSQL **driver**: the library Python uses to talk to PostgreSQL directly. "Binary" means it comes pre-built, so nothing needs compiling. Chapter 18 installed a newer driver, `psycopg` (version 3), for SQLAlchemy to use. This chapter's companion files, and the next chapters that build on them, use the older and still very common `psycopg2`. The two work almost the same way.
- `duckdb` is the analytical database this chapter uses as its warehouse (section 45.1's simplification note).
- `==2.9.13` and `==1.5.6` ask for the exact versions used for this chapter's outputs. `requests` and `python-dotenv` are already installed, from Chapter 18.

Add `psycopg2-binary` and `duckdb` to your `requirements.txt`, one per line, so the environment can be rebuilt (Chapter 26). Then check that both libraries import:

```
# terminal
$ python -c "import psycopg2, duckdb; print(duckdb.__version__)"
1.5.6
```

`python -c` runs the Python code in quotes and exits. If both imports work, it prints DuckDB's version. An error such as `ModuleNotFoundError` means the environment wasn't active when you installed: activate it and install again.

Step 3. Tell Python where the source database is

Python finds `riverstone_source` through a **connection string**: space-separated `key=value` pairs, such as `dbname=riverstone_source user=postgres password=your-password host=localhost port=5432`. Any key you leave out takes a default: your computer's login name as the user, this computer as the host, and port 5432. On Windows, the PostgreSQL installer from Chapter 12 set a password for the `postgres` user, so you need `user` and `password`.

Keep the string out of your code. Put it in a `.env` file in `work/ch45`, as Chapter 18 did, with your own password in place of `your-password`:

```
RIVERSTONE_SOURCE=dbname=riverstone_source user=postgres password=your-password host=localhost
```

The notebook reads it with `load_dotenv()`. If your PostgreSQL needs no password (common with `Postgres.app` and on Linux), you can skip the file: the code falls back to plain `dbname=riverstone_source`.

Step 4. Start the notebook

In VS Code, open the `analyst-to-architect` folder, create a new Jupyter notebook, choose the `.venv` kernel, and save it as `ch45.ipynb` in `work/ch45` (Chapter 17, section 17.0). Its working folder is then `work/ch45`, where the companion files are.

Run each code block in this chapter as its own cell, top to bottom, **without restarting the kernel**. Later cells use names that earlier cells create, such as `wh`, `fetch_rows`, and `new_rows`. If you restart, run the cells again from the top.

The first cell:

```
import os, time, hashlib
import psycopg2, duckdb, requests
from dotenv import load_dotenv

load_dotenv()
from reset_ch45 import reset
from apply_day import apply_day

reset()
SRC = os.environ.get("RIVERSTONE_SOURCE", "dbname=riverstone_source")
wh = duckdb.connect("warehouse/riverstone_wh.duckdb")
print("source and warehouse ready")
```


How it works, line by line.

- `os` reads environment variables and file sizes; `time` lets the code wait (section 45.9); `hashlib` computes fingerprints (section 45.5). All three come with Python.
- `psycopg2` talks to PostgreSQL, `duckdb` is the warehouse, and `requests` calls APIs (Chapter 18).
- `load_dotenv()` copies the `.env` file's variables into the environment, exactly as in Chapter 18. It comes before the companion imports, because they read `RIVERSTONE_SOURCE` when they load.
- `reset()` builds a fresh environment. It copies `riverstone_2025` into a new `riverstone_source` with `CREATE DATABASE ... TEMPLATE`, deletes the old warehouse, and creates the folders `warehouse/` and `exports/`, with the two dispatch files in `exports/`. PostgreSQL copies a database only while nobody else is connected to it, so `reset()` first closes any other connection to `riverstone_2025` or `riverstone_source`, such as an open DBeaver tab (DBeaver reconnects the next time you use it). It needs a user that may create databases; the local `postgres` user can. It reads the same `RIVERSTONE_SOURCE` string as the notebook.
- `SRC` is the connection string: the one from `.env` if there is one, otherwise the default. `os.environ.get(name, default)` returns the default instead of stopping when the variable isn't set.
- `duckdb.connect()` opens (or creates) the warehouse as a single file. **DuckDB** is an analytical database that runs inside your Python process, with no server to install.

Talking to DuckDB from Python

Chapter 18 mentioned DuckDB; this is its first real use. Three calls show everything the chapter needs:

```
print(wh.execute("SELECT 1 + 1").fetchone())
print(wh.execute("SELECT 42 AS x, 'a' AS y").fetchall())
print(wh.execute("SELECT ?::INTEGER * 2", [21]).fetchone())
```

```
(2,)
[(42, 'a')]
(42,)
```

- `wh.execute(sql)` runs one SQL statement and returns its result. `.fetchone()` takes the first row, as a tuple: `(2,)` is a tuple with one item, which is why it has a comma.
- `.fetchall()` takes every row, as a list of tuples: here one row with two columns.
- `?` marks a value you pass separately, in a list after the SQL: `[21]` fills the one `?`. `::INTEGER` is the cast shorthand from Chapter 28; DuckDB accepts it too.

Tool note. The outputs in this chapter were produced with Python 3.11.15, DuckDB 1.5.6, `psycopg2` 2.9.13, `requests` 2.33.1, `python-dotenv` 1.2.3, and PostgreSQL 16.13. Newer versions may format a few outputs slightly differently.

45.3 Full loads

A **full load** copies everything from a source table into the warehouse, replacing what was there. It's the simplest pattern, and often the right one.

Talking to PostgreSQL from Python

First, one query to the source, with nothing hidden:

```
conn = psycopg2.connect(SRC)
cur = conn.cursor()
cur.execute("SELECT COUNT(*) FROM orders")
print(cur.fetchone())
conn.close()
```

```
(175,)
```

- **psycopg2.connect(SRC)** opens a **connection**: the line to the database. It uses the connection string from section 45.2.
- **conn.cursor()** creates a **cursor**: the pen you write queries with. A connection can have several.
- **cur.execute(...)** sends one query. **cur.fetchone()** takes the first row of the answer, as a tuple, exactly as in DuckDB: 175 orders.
- **conn.close()** hangs up. Every connection you open, close.

The two libraries mark values you pass separately with different symbols. It's an easy slip, so keep this in mind:

Library	Placeholder	Example
psycopg2 (PostgreSQL)	%s	cur.execute("... WHERE order_id > %s", (10175,))
DuckDB	?	wh.execute("... WHERE order_id > ?", [10175])

A small helper, then the load

Every query to the source opens a connection, runs, and closes. A function saves repeating that:

```
def fetch_rows(sql, params=None):
    conn = psycopg2.connect(SRC)
    with conn, conn.cursor() as cur:
        cur.execute(sql, params)
        rows = cur.fetchall()
        cols = [d[0] for d in cur.description]
    conn.close()
    return cols, rows

cols, rows = fetch_rows(
    "SELECT order_id, customer_id, order_date, status, sales_rep_id FROM orders")
print(cols)
print(rows[0])
```



```
['order_id', 'customer_id', 'order_date', 'status', 'sales_rep_id']
(10001, 2, datetime.date(2025, 1, 2), 'Delivered', 3)
```

How it works, line by line.

- `params=None` lets you pass values for `%s` placeholders; with `None`, the query runs as written.
- `with conn, conn.cursor() as cur:` opens two things at once. `with conn` commits the work if the block succeeds and rolls it back if anything fails (Chapter 12's transactions), but it does **not** close the connection; that's why `conn.close()` follows. `conn.cursor() as cur` creates the cursor and closes it at the end of the block.
- `cur.fetchall()` returns every row as a list of tuples.
- `cur.description` describes the answer's columns, one entry per column; `d[0]` is each column's name.
- The function returns two things, and `cols, rows = ...` unpacks them into two names. The first row is order 10001: `psycopg2` turns a PostgreSQL `date` into a Python `datetime.date`.

Now load those rows into the warehouse:

```
wh.execute("CREATE SCHEMA IF NOT EXISTS raw")
wh.execute("""
    CREATE OR REPLACE TABLE raw.orders (
        order_id      INTEGER PRIMARY KEY,
        customer_id   INTEGER NOT NULL,
        order_date     DATE NOT NULL,
        status         VARCHAR NOT NULL,
        sales_rep_id   INTEGER
    )""")
wh.executemany("INSERT INTO raw.orders VALUES (?, ?, ?, ?, ?)", rows)

print("rows read from source :", len(rows))
print("rows in raw.orders      :",
      wh.execute("SELECT COUNT(*) FROM raw.orders").fetchone()[0])
```

```
rows read from source : 175
rows in raw.orders     : 175
```

How it works, line by line.

- `CREATE SCHEMA IF NOT EXISTS raw` makes the `raw` layer, a schema as in Chapter 12.
- `CREATE OR REPLACE TABLE` throws away yesterday's copy and creates an empty table with explicit types. Declaring types, instead of letting the tool guess, is the first habit of reliable ingestion.
- `executemany()` runs the same `INSERT` once for every tuple in `rows`. The `?` placeholders keep values separate from the SQL text, which avoids quoting mistakes and SQL injection.
- The last two lines compare the count read with the count stored. `.fetchone()[0]` takes the first item of the first row. Chapter 12 asked you to run `COUNT(*)` against the source after loading; this is the same check, automated.

Reconcile more than the row count

Row counts can match while the contents don't. A slightly stronger check compares counts by a meaningful category. You'll run it again after every load, so it's a function:

```
def compare_status():
    cols, status_rows = fetch_rows("SELECT status, COUNT(*) FROM orders GROUP BY status")
    src_status = dict(status_rows)
    wh_status = dict(wh.execute(
        "SELECT status, COUNT(*) FROM raw.orders GROUP BY status").fetchall())
    for s in sorted(set(src_status) | set(wh_status)):
        src_n, wh_n = src_status.get(s, 0), wh_status.get(s, 0)
        flag = "" if src_n == wh_n else "    <-- differs"
        print(f"{s:<10} source={src_n:>3} warehouse={wh_n:>3}{flag}")

compare_status()
```

```
Cancelled  source=  2 warehouse=  2
Delivered  source=171 warehouse=171
Pending    source=  1 warehouse=  1
Shipped    source=  1 warehouse=  1
```

How it works, line by line.

- Each query returns rows like ('Delivered', 171). `dict(...)` turns a list of two-item tuples into a dictionary: 'Delivered' becomes a key and 171 its value.
- `set(src_status) | set(wh_status)` is every status found on *either* side (`|` is set union). Loop over both sides, or a status that exists only in the warehouse is invisible to the check.
- `.get(s, 0)` reads a count, or 0 when that side has no such status.
- `flag` is empty when the counts agree and points at them when they don't. `{s:<10}` pads the status to 10 characters, left-aligned, and `{src_n:>3}` right-aligns a number in 3 (Chapter 17's f-strings).

Everything matches: 175 orders, 171 of them delivered. A **reconciliation check** like this compares a summary of the source with the same summary of the warehouse. It's cheap, and it catches a surprising number of problems.

When a full load is the right choice

Full loads are **simple, self-correcting** (every run fixes any earlier mistake), and they **capture updates and deletes** automatically, because the whole table is replaced. Choose them when:

- the table is small or medium: Riverstone's customers, products, and employees are ideal;
- the source can handle reading the whole table without slowing the business down;
- the load finishes comfortably inside the time you have.

They stop being practical when a table is large, changes slowly, or sits in a busy production system. Reading a hundred million order lines every night to find the few thousand that changed is slow, expensive, and hard on the ERP. That's where incremental loads come in.

45.4 Incremental loads, and the update they miss

An **incremental load** reads only the rows that are new or changed since the last load. To know what "since the last load" means, it keeps a **watermark**: a stored value marking how far the previous load got, such as the highest ID or the latest timestamp it saw.

By hand first

Take a tiny orders table. On day 1 the warehouse loaded all four orders, so the watermark is 4, the highest ID it saw. By day 2, the source has changed:

order_id	In the warehouse after day 1	In the source on day 2
1	Delivered	Delivered
2	Delivered	Delivered
3	Pending	Cancelled
4	Shipped	Shipped
5	(not there)	Pending (new)

Before reading on, answer two questions. The day-2 load reads `WHERE order_id > 4`: which rows does it read? And after it, what does the warehouse say about order 3?

It reads only order 5. The warehouse still says order 3 is Pending, which is wrong: the change happened below the watermark, where the load never looks. Figure 45.2 draws the same thing for the real orders you're about to load.

1 JANUARY: warehouse loaded up to order 10175

2 JANUARY: what happens in the ERP, and what an ID-watermark load reads

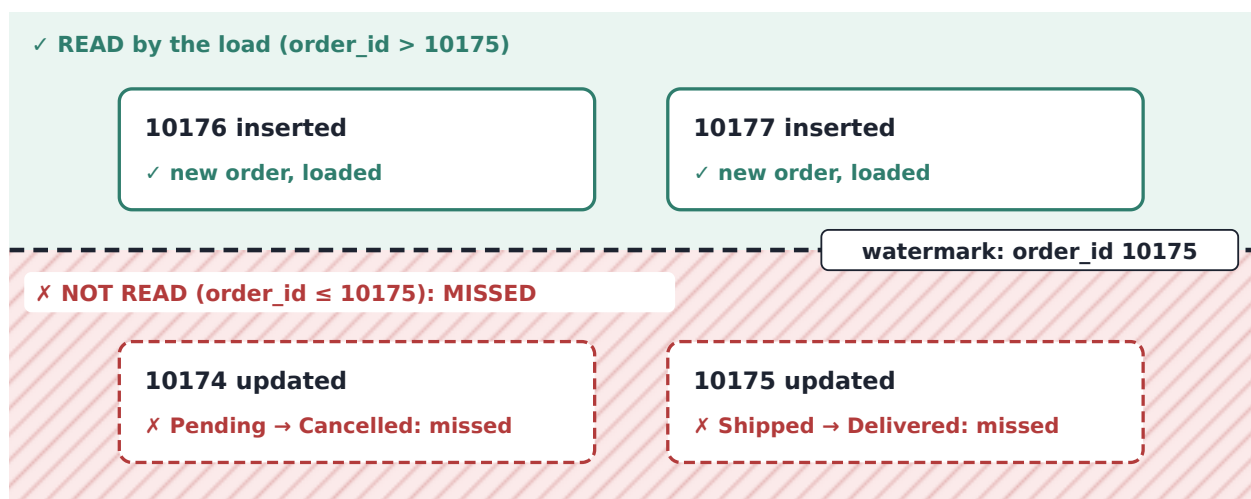


Figure 45.2 — Why an ID watermark misses updates. The load only looks above the line; changes to older rows happen below it.

The same thing, on Riverstone's orders

It's now 2 January 2026. The ERP has had a business day. Apply it, then run the most common incremental pattern: load rows with an ID above the watermark.

```

apply_day(1)

watermark = wh.execute("SELECT MAX(order_id) FROM raw.orders").fetchone()[0]
cols, new_rows = fetch_rows("""
    SELECT order_id, customer_id, order_date, status, sales_rep_id
    FROM orders
    WHERE order_id > %s
    ORDER BY order_id""", (watermark,))
wh.executemany("INSERT INTO raw.orders VALUES (?, ?, ?, ?, ?)", new_rows)

print("watermark before load:", watermark)
print("new rows loaded      :", [r[0] for r in new_rows])

```

```

watermark before load: 10175
new rows loaded      : [10176, 10177]

```

How it works.

- `apply_day(1)` runs 2 January's changes in the ERP.
- The watermark is the highest `order_id` already in the warehouse.
- `%s` in the query is filled from `(watermark,)`, a tuple with one item: the trailing comma is what makes it a tuple. `psycopg2` sends the value separately from the SQL, as section 45.3's placeholder table showed.
- `[r[0] for r in new_rows]` lists the first item, the order ID, of each new row.

The load ran, found two new orders, and finished without errors. Now run the reconciliation check from section 45.3 again, and look at two older orders:

```

compare_status()
print(wh.execute("SELECT order_id, status FROM raw.orders "
                "WHERE order_id IN (10174, 10175) ORDER BY order_id").fetchall())

```

```

Cancelled source= 3 warehouse= 2 <-- differs
Delivered source=172 warehouse=171 <-- differs
Pending   source= 2 warehouse= 3 <-- differs
Shipped   source= 0 warehouse= 1 <-- differs
[(10174, 'Pending'), (10175, 'Shipped')]

```

Reading it. The row counts match (177 in both), but four status counts don't. On 2 January, the ERP also *updated* two existing orders: order 10174 was cancelled, and order 10175 was delivered. Their IDs are below the watermark, so the load never looked at them. The warehouse still says 10174 is Pending and 10175 is Shipped, so it has a Shipped order that the source no longer has. A dashboard built on it would count a cancelled order as open business.

Watch out: a successful load is not a correct load. The watermark load above raised no error and loaded exactly the rows it was designed to load. Only a reconciliation check against the source revealed the problem. Never judge a pipeline by whether it finished.

Better watermarks, and their limits

The usual fix is a **last-updated timestamp**: a column such as `updated_at` that the source sets every time a row changes. Then the load reads `WHERE updated_at > watermark`, and updates are caught.

Riverstone's `orders` table, like many real ERP tables, has no such column. And where one exists, check three things before trusting it:

- **Does every change update it?** Bulk fixes run directly in the database often skip it.
- **Is it set when the change is committed, or when it started?** A long transaction that began before your last load can commit a timestamp you've already passed.
- **What about deletes?** A deleted row has no timestamp to find. Incremental loads by timestamp never see deletes.

A common safety margin is to **look back**: load rows where `updated_at > watermark - interval '1 hour'` and let the upsert (section 45.5) absorb the duplicates. When no reliable column exists, you need a different way to detect change, which is the next section.

45.5 Detecting change with hashes

What a hash is

Chapter 2 introduced the idea: a **hash** is a fixed-length fingerprint of any data. The same data always gives the same fingerprint, and the smallest change gives a completely different one. Chapter 2 showed two payment lines with different fingerprints. Here is how they were made:

```
text = "Pay Rs 14,700 to Riverstone Supplies"
print(hashlib.sha256(text.encode("utf-8")).hexdigest())
```

```
f4251ff3fb7191f7e79677f3b1871db06c0935c3cc08164496995f1f909f6f71
```

- `text.encode("utf-8")` turns the text into bytes (Chapter 2's bytes), because hashes work on bytes, not on letters.
- `hashlib.sha256(...)` runs the **SHA-256** algorithm on those bytes. `hashlib` comes with Python; the setup cell imported it.
- `.hexdigest()` gives the fingerprint as 64 hexadecimal characters (0–9 and a–f). Each character carries 4 bits, so 64 of them are 256 bits: the "256" in the name.

What happens if you change the text? Change it to the second payment line, `Pay Rs 17,400 ...`, and then run the first line again:

```
print(hashlib.sha256("Pay Rs 17,400 to Riverstone Supplies".encode("utf-8")).hexdigest())
print(hashlib.sha256("Pay Rs 14,700 to Riverstone Supplies".encode("utf-8")).hexdigest())
```

```
9d2f842c50110e140abd578e7e8460d1f6f2bbca7eba341ce1e5bee43566d0fa
f4251ff3fb7191f7e79677f3b1871db06c0935c3cc08164496995f1f909f6f71
```

Swapping two digits changed every part of the fingerprint. Running the first line again gave exactly the same fingerprint as before. Does a longer input give a longer fingerprint? Predict, then run:

```
for sample in ["Riverstone", "x" * 1000000]:
    digest = hashlib.sha256(sample.encode("utf-8")).hexdigest()
    print(len(sample), "characters in,", len(digest), "characters out")
```

```
10 characters in, 64 characters out
1000000 characters in, 64 characters out
```

"x" * 1000000 repeats the letter a million times: about 1 MB of text. Both fingerprints are 64 characters long.

Four properties of a hash.

- **Deterministic:** the same input always gives the same fingerprint, on any computer.
- **Fixed length:** one word or a million characters, SHA-256 always gives 64 characters.
- **Avalanche:** a tiny change to the input changes the whole fingerprint, not just one part of it.
- **One-way:** you can't work back from a fingerprint to the text.

Two different inputs *could* in theory share a fingerprint (a **collision**), but with SHA-256 it's not something you will ever meet.

Where hashes are used.

- **Checking downloads.** A **checksum** is a hash published next to a file, such as the SHA-256 listed beside the Python installer. Hash what you downloaded; if it matches, the file arrived undamaged.
- **Checking backups** against the original (Chapter 2's exercise 9).
- **Passwords.** Systems store a fingerprint of your password, not the password. Real systems use a deliberately *slow* hash with a random extra value mixed in (a **salt**), such as bcrypt or Argon2, never plain SHA-256 or MD5, so that stolen fingerprints can't be tested against billions of guesses a second.
- **Pipelines.** Fingerprints of rows and of files tell you what changed since yesterday. That's the rest of this section.

You'll also meet an older algorithm, **MD5**. It's the one databases offer as a function called `md5()`:

```
print(hashlib.md5(text.encode("utf-8")).hexdigest())
```

```
f3e2b15a5bacba44d562207000e67838
```

Algorithm	Length	Speed	Use it for
MD5	32 hex characters (128 bits)	Fast	Change detection. It's broken for security: people can deliberately make two different inputs with the same MD5
SHA-256	64 hex characters (256 bits)	Fast enough	Anything security-related, and files you must be sure of

Row hashes, by hand

Go back to the tiny table in section 45.4. Build one string from order 3's values, with a `|` between them, and fingerprint it before and after the change:

```
print(hashlib.md5("3|Pending".encode("utf-8")).hexdigest())
print(hashlib.md5("3|Cancelled".encode("utf-8")).hexdigest())
```

```
b8379e4c6efbe5b331ea3a2fdb9858e5
c6777f82bb14454418119822ba5680a0
```

Different strings, different fingerprints. Keep yesterday's fingerprint of every row, compute today's, and every row whose fingerprint differs has changed, whatever its ID. No watermark is needed.

Row hashes in SQL

Both databases can build the string and hash it themselves, so only the fingerprints travel. One detail first. SQL's `concat_ws` (concatenate with separator) joins values with a separator, but it **skips NULLs**:

```
print(wh.execute("SELECT concat_ws('|', 1, NULL, 3), "
                 "concat_ws('|', 1, coalesce(NULL::text, 'ø'), 3)").fetchone())
```

```
('1|3', '1|ø|3')
```

`1|3` loses the fact that the middle value was missing, so a value changing to or from NULL could go unnoticed. `coalesce(value, 'ø')` (Chapter 12) puts a marker in place of a NULL, so the string keeps its shape. `sales_rep_id` is the one column in `orders` that can be NULL, so it gets the marker. Now fingerprint every order, on both sides, and look at an unchanged order and a changed one:

```
HASH_SQL = """
    SELECT order_id,
           md5(concat_ws('|', order_id, customer_id, order_date, status,
                        coalesce(sales_rep_id::text, 'ø'))) AS row_hash
    FROM orders"""
WH_HASH_SQL = """
    SELECT order_id,
           md5(concat_ws('|', order_id, customer_id,
                        strftime(order_date, '%Y-%m-%d'), status,
                        coalesce(sales_rep_id::text, 'ø'))) AS row_hash
    FROM raw.orders"""

cols, hash_rows = fetch_rows(HASH_SQL)
source_hashes = dict(hash_rows)
wh_hashes = dict(wh.execute(WH_HASH_SQL).fetchall())
for order_id in [10173, 10174]:
    print(order_id, "source  :", source_hashes[order_id])
    print(order_id, "warehouse:", wh_hashes[order_id])
```

```
10173 source   : 31e3dbe0caabc12b48bed4c669ee5886
10173 warehouse: 31e3dbe0caabc12b48bed4c669ee5886
10174 source   : ce97b59465f0cd081ee9a4803c26bd63
10174 warehouse: 600caf6f8c577cd8f16ebfc0cfe0c939
```


How it works.

- `concat_ws('|', ...)` joins a row's values into one string with `|` between them, for example `10174|9|2025-12-21|Pending|5`. The separator matters: without it, the values `1` and `23` would join to the same `123` as `12` and `3`.
- `sales_rep_id::text` casts the number to text, so `coalesce` can offer the text marker as the alternative.
- `md5()` turns the string into a 32-character fingerprint. MD5 is fine for detecting change; it isn't safe for security, as the table above explains.
- PostgreSQL writes a date in its `DateStyle` setting, `2025-12-21` by default. On the warehouse side, `strftime(order_date, '%Y-%m-%d')` makes DuckDB write exactly the same. **Both sides must build the string identically**, or every row will look changed.
- `dict(...)` gives two dictionaries, order ID $\rightarrow$ fingerprint. Order 10173 didn't change, so both sides agree. Order 10174 was cancelled in the source, so its fingerprints differ.

Comparing the two dictionaries finds every difference at once. You'll need this again in section 45.6, so it's a function:

```
def compare_hashes():
    cols, hash_rows = fetch_rows(HASH_SQL)
    source_hashes = dict(hash_rows)
    wh_hashes = dict(wh.execute(WH_HASH_SQL).fetchall())
    inserted = sorted(set(source_hashes) - set(wh_hashes))
    deleted = sorted(set(wh_hashes) - set(source_hashes))
    changed = sorted(k for k in set(source_hashes) & set(wh_hashes)
                     if source_hashes[k] != wh_hashes[k])
    return inserted, deleted, changed

inserted, deleted, changed = compare_hashes()
print("inserted:", inserted)
print("deleted :", deleted)
print("changed :", changed)
```

```
inserted: []
deleted : []
changed : [10174, 10175]
```

How it works.

- `set(source_hashes)` is the set of the dictionary's keys, the order IDs.
- `-` between sets keeps what's in the first and not the second: a key only in the source was **inserted**; a key only in the warehouse was **deleted**.
- `&` keeps what's in both. Of those, a key whose two fingerprints differ was **changed**. The `for ... if ...` inside `sorted()` works like a list comprehension without the brackets.
- The function returns three lists, and `inserted, deleted, changed = ...` unpacks them.

Reading it. The hash comparison finds exactly the two orders the watermark load missed, 10174 and 10175, and confirms there were no deletes. (The two new orders were already loaded, so they're not "inserted" here.)

Applying the changes with an upsert

You wrote an **upsert** in Chapter 12: insert a row if its key is new, update it if the key already exists, with PostgreSQL's `INSERT ... ON CONFLICT ... DO UPDATE`. DuckDB has a shorthand, `INSERT OR REPLACE`, which replaces every column of the existing row. That's what a raw copy wants. Fetch the current version of the changed orders, and upsert them:

```
cols, fresh = fetch_rows("""
    SELECT order_id, customer_id, order_date, status, sales_rep_id
    FROM orders WHERE order_id = ANY(%s)""", (changed,))
wh.executemany("INSERT OR REPLACE INTO raw.orders VALUES (?, ?, ?, ?, ?)", fresh)
print(wh.execute("SELECT order_id, status FROM raw.orders "
    "WHERE order_id IN (10174, 10175) ORDER BY order_id").fetchall())
```

```
[(10174, 'Cancelled'), (10175, 'Delivered')]
```

- `= ANY(%s)` with `(changed,)`: `psycopg2` turns the Python list `[10174, 10175]` into a PostgreSQL array, and `order_id = ANY(array)` matches any ID in it. It's the placeholder version of `IN (10174, 10175)`.
- `INSERT OR REPLACE` updates both rows in place.

Both orders are now correct. DuckDB also accepts Chapter 12's long form. It does the same job here, and lets you choose which columns to update:

```
wh.executemany("""
    INSERT INTO raw.orders VALUES (?, ?, ?, ?, ?)
    ON CONFLICT (order_id) DO UPDATE SET
        customer_id = EXCLUDED.customer_id, order_date = EXCLUDED.order_date,
        status = EXCLUDED.status, sales_rep_id = EXCLUDED.sales_rep_id""", fresh)
print(wh.execute("SELECT order_id, status FROM raw.orders "
    "WHERE order_id IN (10174, 10175) ORDER BY order_id").fetchall())
```

```
[(10174, 'Cancelled'), (10175, 'Delivered')]
```

As in Chapter 12, `EXCLUDED` means "the row you tried to insert". Many cloud warehouses spell the same idea `MERGE`.

Why upserts make loads safe to repeat

A load that can be run twice with the same result is **idempotent**. Chapter 20 made the Flash email idempotent with a run key; here is the same property for a table load. It matters because loads fail halfway and get re-run, schedulers retry, and people press "run" again. A plain `INSERT` isn't idempotent. What do you expect if yesterday's two new orders are inserted a second time? Run it:

```
try:
    wh.executemany("INSERT INTO raw.orders VALUES (?, ?, ?, ?, ?)", new_rows)
except duckdb.ConstraintException as e:
    print("plain INSERT, second run:", e)

wh.executemany("INSERT OR REPLACE INTO raw.orders VALUES (?, ?, ?, ?, ?)", new_rows)
wh.executemany("INSERT OR REPLACE INTO raw.orders VALUES (?, ?, ?, ?, ?)", new_rows)
print("upsert, run twice  :",
      wh.execute("SELECT COUNT(*) FROM raw.orders").fetchone()[0], "rows")
```

```
plain INSERT, second run: Constraint Error: Duplicate key "order_id: 10176" violates primary key
constraint.
upsert, run twice      : 177 rows
```

- `except duckdb.ConstraintException as e:` catches only this kind of error, a broken constraint such as a duplicate primary key, and names it `e`. Any other error still stops the cell (Chapter 17's `try` and `except`).
- The upsert then runs twice, and the count is unchanged.

Here the primary key stopped the duplicate, which is exactly why raw tables should have one.

Without a primary key, the plain insert would have silently doubled two orders. The upsert can run any number of times and leave 177 rows. Chapter 46 builds whole pipelines on this property, and tests it by making a run fail on purpose.

The cost of row hashing

Hash comparison catches inserts, updates, and deletes without any help from the source. The price is that it reads **every row's key and hash** on every run. For Riverstone's 177 orders that's nothing; for a table of hundreds of millions of rows, it's a heavy scan of a production database each night. Common compromises:

- hash only a recent window, such as the last 90 days, where changes usually happen, plus a full comparison once a week;
- hash in chunks (by month or ID range), compare chunk hashes first, and drill into rows only where a chunk differs;
- use change data capture instead (section 45.6), which doesn't need to scan at all.

File hashes: skipping files that haven't changed

The same idea applies to whole files. Bhiwandi Main saves its dispatch sheet into a shared folder every evening. Sometimes someone saves it twice; sometimes the file is re-sent unchanged. A file hash tells you whether you've already loaded exactly this content.

For a small file, read it all and hash it:

```
path = "exports/dispatch_2026-01-02.csv"
h = hashlib.sha256()
with open(path, "rb") as f:
    h.update(f.read())
print(h.hexdigest())
```

```
56ac5c9f3112a1bdeed5e1f95e8cc7e5530c5a91bbe24c606c5accc5e1001d0b
```

- `"rb"` opens the file to read **bytes**, not text, so nothing is changed by decoding: the hash covers exactly what's on disk.
- `h.update(...)` feeds bytes into the hash; `hexdigest()` reads the fingerprint out.

A large file shouldn't be read into memory in one go. `update` can be called many times, and gives the same result as hashing everything at once, so read the file in pieces:

```
def file_sha256(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        while True:
            chunk = f.read(65536)
            if not chunk:
                break
            h.update(chunk)
    return h.hexdigest()

print(file_sha256(path))
```

```
56ac5c9f3112a1bdeed5e1f95e8cc7e5530c5a91bbe24c606c5accc5e1001d0b
```

- `f.read(65536)` reads at most 65,536 bytes (64 KB). At the end of the file it returns nothing, an empty bytes value, which counts as false, so `if not chunk: break` leaves the loop.
- The fingerprint is identical to the one above. This version works on files far larger than memory.

Now a **load log**: a table recording each file that was loaded, its fingerprint, and the day it was loaded for.

```

wh.execute("""
    CREATE TABLE IF NOT EXISTS raw.file_log (
        file_name    VARCHAR,
        sha256       VARCHAR,
        loaded_for   DATE,
        rows_loaded  INTEGER,
        loaded_at    TIMESTAMP,
        PRIMARY KEY (sha256, loaded_for))""")

def already_loaded(digest, loaded_for):
    row = wh.execute(
        "SELECT file_name FROM raw.file_log WHERE sha256 = ? AND loaded_for = ?",
        [digest, loaded_for]).fetchone()
    return row[0] if row else None

def record_loaded(path, digest, loaded_for, rows_loaded):
    wh.execute("INSERT INTO raw.file_log VALUES (?, ?, ?, ?, current_timestamp)",
        [os.path.basename(path), digest, loaded_for, rows_loaded])

print("already loaded?", already_loaded(file_sha256(path), "2026-01-02"))

```

```
already loaded? None
```

How it works.

- The key is the **pair** of fingerprint and day, `PRIMARY KEY (sha256, loaded_for)`: the same content may legitimately arrive for two different days (see the Watch out below).
- `already_loaded()` only *reads* the log. It returns the name of a file with the same content already loaded for that day, or `None`. `fetchone()` returns `None` when there's no row, so `row[0] if row else None` handles both cases.
- `record_loaded()` writes one line to the log, with the number of rows and the time (`current_timestamp`). `os.path.basename(path)` keeps just the file name.
- The two jobs are separate on purpose. Record a file only **after** it has loaded successfully. A log that records the file first, and then fails to load it, would make every later run skip the file as "already loaded", and its data would be lost without a sound.

Nothing has been loaded yet, so the answer is `None`. Section 45.7 loads this file, and only then records it.

Watch out: the same content isn't always the same file. A hash tells you the *content* is identical. If a warehouse legitimately sends two identical dispatch files for two different days (no dispatches on both), skipping the second would be wrong. That's why the log's key includes the day the file was loaded for. Decide explicitly what "already loaded" means for each source.

45.6 Change data capture

Every change to a PostgreSQL database is first written to a **write-ahead log** (WAL): a running record of every insert, update, and delete, used for crash recovery and replication. **Change data capture** (CDC) reads changes from that log instead of querying tables. It sees every insert, update, and delete, in commit order, without scanning anything.

Here are the four ways to keep a warehouse table in step with its source, side by side. Notice which ones can see deletes.

	Full load	Incremental (watermark)	Hash comparison	Change data capture
Reads	The whole table	Rows past the last ID or time	Every key and hash	The database's change log
Sees inserts	✓ Yes	✓ Yes	✓ Yes	✓ Yes
Sees updates	✓ Yes	✗ Only with a reliable <code>updated_at</code>	✓ Yes	✓ Yes, in order
Sees deletes	✓ Yes	✗ No	✓ Yes	✓ Yes
Load on the source	✗ Heavy for big tables	✓ Light	Medium to heavy	✓ Very light
Complexity	✓ Simplest	✗ Simple, but misses changes silently	Moderate	✗ Settings, permissions, slots

Turning CDC on

CDC needs PostgreSQL to write enough detail into the log. That's controlled by one setting, `wal_level`, which needs a server restart. The default is `replica`.

1. In DBeaver's SQL editor (Chapter 12), connected as `postgres`, run `ALTER SYSTEM SET wal_level = logical;`. **ALTER SYSTEM** writes the setting into a file PostgreSQL reads when it starts (`postgresql.auto.conf`), so you don't have to find and edit configuration files by hand.
2. Restart PostgreSQL. **Windows:** open the Services app, find `postgresql-x64-16`, and choose Restart. **macOS with Homebrew:** `brew services restart postgresql@16`; **with Postgres.app:** Stop, then Start. **Linux:** `sudo systemctl restart postgresql`.
3. Check from the notebook, in the next cell.

Your local `postgres` user is a **superuser**, allowed to do anything, including creating the replication slot below. On a company database this is a change for the database administrator, not for you: it increases log volume, and CDC needs a user with the `REPLICATION` permission. Managed cloud databases expose the same setting under a different name; check the provider's documentation.

```
cdc_conn = psycopg2.connect(SRC)
cdc_conn.autocommit = True
cur = cdc_conn.cursor()
cur.execute("SHOW wal_level")
print(cur.fetchone())
```

```
('logical',)
```

- `cdc_conn` is a connection kept open for this section, named so it doesn't clash with other names.
- `cdc_conn.autocommit = True` makes every statement take effect the moment it runs. By default `psycopg2` opens a transaction and waits for `commit()`; that suits `fetch_rows()`, which commits and closes at once, but not a connection you keep open and use cell by cell. It also matters for the slot below: PostgreSQL refuses to create a slot inside a transaction that has already written anything, and with `autocommit` every statement is a transaction of its own.
- `SHOW wal_level` reads the setting. If it says `('replica',)`, the restart hasn't happened yet.

Simplification note. Production CDC usually runs through a dedicated tool, such as the open-source Debezium, or through a managed connector, and it streams changes continuously (Chapter 50). This section uses PostgreSQL's built-in `test_decoding` plugin, which prints changes as readable text. It's meant for learning and testing, not for production pipelines.

Reading real changes

A **replication slot** is a bookmark in the log. PostgreSQL keeps every change after the bookmark until a reader consumes it, so nothing is lost between reads. Create a slot, then let the ERP have another business day (5 January 2026):

```
cur.execute("SELECT slot_name "
            "FROM pg_create_logical_replication_slot('ch45_slot', 'test_decoding')")
print("slot created:", cur.fetchone()[0])

apply_day(2)

cur.execute("SELECT data FROM pg_logical_slot_get_changes('ch45_slot', NULL, NULL)")
for (line,) in cur.fetchall():
    if line.startswith("table "):
        print(line)
```

```
slot created: ch45_slot
table public.orders: UPDATE: order_id[integer]:10176 customer_id[integer]:9 order_date[date]:'2026-01-02'
    status[character varying]:'Shipped' sales_rep_id[integer]:5
table public.orders: INSERT: order_id[integer]:10178 customer_id[integer]:17 order_date[date]:'2026-01-05'
    status[character varying]:'Pending' sales_rep_id[integer]:3
table public.order_items: INSERT: order_item_id[integer]:334 order_id[integer]:10178 product_id[integer]:102
    quantity[integer]:25 unit_price[numeric]:750.00 discount_pct[numeric]:0.00
```

How it works.

- `pg_create_logical_replication_slot()` creates the bookmark and names the plugin that turns log records into text.
- `apply_day(2)` runs the day's changes in the ERP, in one transaction.
- `pg_logical_slot_get_changes()` returns every change since the bookmark, then moves the bookmark forward. The two `NULL`s mean "no limit": read everything waiting.
- Each row comes back as a one-item tuple; `for (line,) in ...` unpacks that one item into `line`.
- The plugin also returns `BEGIN` and `COMMIT` lines with transaction numbers, which change on every run, so `line.startswith("table ")` keeps only the table changes.

Reading a change line. Each one reads `table <schema>.<table>: <INSERT|UPDATE|DELETE>:`

`column[type]:value ...`. An INSERT or UPDATE carries the full new row. A DELETE carries only the primary key, for example `table public.orders: DELETE: order_id[integer]:10150`, which is all you need to delete the row from the warehouse.

Reading it. Three changes, in the order they were committed: order 10176 moved to Shipped, a new order 10178 was created, and its order line was inserted. The warehouse can apply each one with an upsert. No table was scanned.

Read the slot again, and there's nothing waiting:

```
cur.execute("SELECT data FROM pg_logical_slot_get_changes('ch45_slot', NULL, NULL)")
print("changes waiting:", len(cur.fetchall()))
```

```
changes waiting: 0
```

Both methods agree

As a check, run the hash comparison from section 45.5. It should find what CDC found.

```
inserted, deleted, changed = compare_hashes()
print("inserted:", inserted, " deleted:", deleted, " changed:", changed)
```

```
inserted: [10178]  deleted: []  changed: [10176]
```

The hash comparison, which scanned every order, found the same two changes to `orders` that CDC reported from the log.

Watching a slot

PostgreSQL keeps every log record a slot hasn't consumed. So the one thing to monitor is how far each slot is **behind**: how much log is waiting for it.


```
cur.execute("""
    SELECT slot_name, active,
           pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),
                                         confirmed_flush_lsn)) AS behind
    FROM pg_replication_slots""")
print(cur.fetchall())
cdc_conn.close()
```

```
[('ch45_slot', False, '0 bytes')]
```

- **pg_replication_slots** lists every slot. **active** says whether a reader is connected right now.
- **pg_current_wal_lsn()** is the log's current position, and **confirmed_flush_lsn** is how far the slot has read. **pg_wal_lsn_diff()** is the distance between them in bytes, and **pg_size_pretty()** writes it for people. Nothing is waiting, because the cell above read everything; if the database has written a little since, you may see a few bytes.
- **cdc_conn.close()** closes this section's connection.

Run this check on a schedule, and **alert when behind grows day after day**: it means the reader has stopped.

Watch out: an abandoned slot fills the disk. If a CDC reader stops for days and nobody notices, the log grows until the database server runs out of disk space, and the ERP stops. Watch slot lag with the query above, and drop slots you no longer use: **SELECT pg_drop_replication_slot('ch45_slot');**. The companion's **reset()** does this for you.

Choosing a strategy

If...	Consider
The table is small, or you're not sure	A full load
Rows are only ever added, never changed (events, log entries)	Incremental by ID or timestamp
Rows change, and a trustworthy updated_at exists	Incremental by timestamp , with a look-back window and an upsert
Rows change, no trustworthy column, table not huge	Hash comparison
Rows change, the table is large or busy, or you need deletes and low delay	Change data capture

Many platforms mix them: CDC for the big, busy order tables; full loads for products and employees; files and APIs for everything else.

45.7 Loading files reliably

Chapter 12 promised this chapter would cover loading large and messy files reliably. Files are still one of the most common ways data arrives, especially from spreadsheets, partners, banks, and older systems.

Bhiwandi Main's dispatch sheet for 2 January 2026 was saved from a spreadsheet. Here's the file as it appears in a text editor:

```
dispatch_id,order_id,dispatch_date,warehouse,qty_units,remarks
D-2601,10175,02-01-2026,Bhiwandi Main,100,"Delivered to site, signed"
D-2602,10173,02-01-2026,Bhiwandi Main,"1,200",Part load
D-2603,10172,02-01-2026,Bhiwandi Main,N/A,Awaiting count
D-2604,10171,02-01-2026,Bhiwandi Main,85,
D-2605,10170,02-01-2026,Bhiwandi Main,"2,040","Two trucks; second at 4 pm"
```

It looks tidy. It has five traps: an invisible **byte-order mark** (BOM) at the very start, which some tools read as part of the first column name; dates written day-first (02-01-2026 is 2 January, not 1 February); quantities with thousands separators, which have to be quoted; a quantity of N/A; and Windows line endings with a blank line at the end.

What an automatic reader does

Modern readers try to detect everything for you. DuckDB's `read_csv` is good at it. This cell prints the first three rows, one per line, and then each column's name and detected type:

```
for row in wh.execute(
    "SELECT * FROM read_csv('exports/dispatch_2026-01-02.csv') LIMIT 3").fetchall():
    print(row)
for row in wh.execute(
    "DESCRIBE SELECT * FROM read_csv('exports/dispatch_2026-01-02.csv')").fetchall():
    print(row[:2])
```

```
('D-2601', 10175, datetime.date(2026, 1, 2), 'Bhiwandi Main', '100', 'Delivered to site, signed')
('D-2602', 10173, datetime.date(2026, 1, 2), 'Bhiwandi Main', '1,200', 'Part load')
('D-2603', 10172, datetime.date(2026, 1, 2), 'Bhiwandi Main', 'N/A', 'Awaiting count')
('dispatch_id', 'VARCHAR')
('order_id', 'BIGINT')
('dispatch_date', 'DATE')
('warehouse', 'VARCHAR')
('qty_units', 'VARCHAR')
('remarks', 'VARCHAR')
```

- `read_csv('...')` reads a CSV file as if it were a table, so you can `SELECT` from it.
- **DESCRIBE** lists the columns a query would return, with their types. Each of its rows has six items; `row[:2]` keeps the first two, the name and the type.

Reading it. DuckDB handled the byte-order mark, the quoted commas, and the blank line. For the dates, it *guessed* day-first. It had no way to know, because 02-01-2026 fits both orders. It happened to be right for an Indian file; a file from a US system would have been silently wrong. And look at the types: `qty_units` came in as **VARCHAR**, text, because of "1,200" and N/A. Nothing failed. A report that sums this column will either error much later or, in a spreadsheet, quietly skip the text values. Automatic detection is a helpful first look, and a risky way to run a daily load, because tomorrow's file may be guessed differently.

Watch out: guessed types change from file to file. If tomorrow's file happens to contain only plain numbers, the same column will be detected as an integer; the day after, as text again. Downstream code that worked yesterday breaks, or worse, silently changes behavior. In a scheduled load, declare the columns.

Declare the columns, keep the raw text, then type in staging

A reliable file load has two steps. First, load into raw with **declared columns**, keeping awkward fields as text so that nothing is lost or guessed. Second, convert to proper types in staging, and **set aside** the values that can't be converted.

The raw table is created once. Besides the file's six columns, every row records the file it came from and when it was loaded, so any number can be traced back:

```
wh.execute("""
    CREATE TABLE IF NOT EXISTS raw.dispatch (
        dispatch_id  VARCHAR,
        order_id     INTEGER,
        dispatch_date VARCHAR,
        warehouse    VARCHAR,
        qty_units     VARCHAR,
        remarks       VARCHAR,
        source_file   VARCHAR,
        loaded_at     TIMESTAMP)""")
```

`dispatch_date` and `qty_units` are text on purpose: raw keeps exactly what the file says. Now the load, using the load log from section 45.5:

```
def load_dispatch(path, day):
    digest = file_sha256(path)
    seen = already_loaded(digest, day)
    if seen:
        return f"skip: same content as {seen}, already loaded for {day}"
    wh.execute("DELETE FROM raw.dispatch WHERE source_file = ?", [path])
    wh.execute("""
        INSERT INTO raw.dispatch
        SELECT *, current_timestamp
        FROM read_csv(?, header = true, filename = true,
            columns = {'dispatch_id': 'VARCHAR', 'order_id': 'INTEGER',
                'dispatch_date': 'VARCHAR', 'warehouse': 'VARCHAR',
                'qty_units': 'VARCHAR', 'remarks': 'VARCHAR'})""", [path])
    n = wh.execute("SELECT COUNT(*) FROM raw.dispatch WHERE source_file = ?",
        [path]).fetchone()[0]
    record_loaded(path, digest, day, n)
    return f"loaded {n} rows from {path}"

print(load_dispatch("exports/dispatch_2026-01-02.csv", "2026-01-02"))
print(load_dispatch("exports/dispatch_2026-01-02.csv", "2026-01-02"))
```

```
loaded 5 rows from exports/dispatch_2026-01-02.csv
skip: same content as dispatch_2026-01-02.csv, already loaded for 2026-01-02
```

How it works, line by line.

- The function first asks the load log whether this content was already loaded for this day. The second call finds it, and skips.
- `DELETE FROM raw.dispatch WHERE source_file = ?` removes any rows an earlier load of the same file left. If a *corrected* file arrives under the same name, its content and fingerprint are new, so it loads, and the delete stops its rows from doubling up. Delete-then-insert by file makes the reload idempotent, like the upsert in section 45.5.
- `read_csv(?, ...)` takes the file name as a parameter. `header = true` says the first line holds column names. `columns = {...}` declares every column's name and type, as `'name': 'TYPE'` pairs inside braces; DuckDB then guesses nothing.
- `filename = true` adds one more column, the path of the file each row came from. With `current_timestamp` after `SELECT *`, the rows match the raw table's eight columns: the file's six, then `source_file`, then `loaded_at`.
- Only after the insert succeeds does `record_loaded()` write to the load log, with the real row count.

Look at what landed:

```
for row in wh.execute("SELECT dispatch_id, dispatch_date, qty_units, remarks, source_file "
                      "FROM raw.dispatch").fetchall():
    print(row)
```

```
('D-2601', '02-01-2026', '100', 'Delivered to site, signed', 'exports/dispatch_2026-01-02.csv')
('D-2602', '02-01-2026', '1,200', 'Part load', 'exports/dispatch_2026-01-02.csv')
('D-2603', '02-01-2026', 'N/A', 'Awaiting count', 'exports/dispatch_2026-01-02.csv')
('D-2604', '02-01-2026', '85', None, 'exports/dispatch_2026-01-02.csv')
('D-2605', '02-01-2026', '2,040', 'Two trucks; second at 4 pm', 'exports/dispatch_2026-01-02.csv')
```

The raw table holds exactly what the warehouse team wrote, including N/A, and each row names its file. Now convert in staging:

```
wh.execute("CREATE SCHEMA IF NOT EXISTS staging")
wh.execute("""
    CREATE OR REPLACE TABLE staging.dispatch AS
    SELECT dispatch_id,
           order_id,
           strptime(dispatch_date, '%d-%m-%Y')::DATE           AS dispatch_date,
           warehouse,
           TRY_CAST(replace(qty_units, ',', '')) AS INTEGER) AS qty_units,
           remarks
    FROM raw.dispatch""")
bad = wh.execute("""
    SELECT dispatch_id, qty_units
    FROM raw.dispatch
    WHERE TRY_CAST(replace(qty_units, ',', '')) AS INTEGER) IS NULL""").fetchall()
for row in wh.execute("SELECT dispatch_id, dispatch_date, qty_units "
                      "FROM staging.dispatch ORDER BY dispatch_id").fetchall():
    print(row)
print("rows needing attention:", bad)
```

```
( 'D-2601', datetime.date(2026, 1, 2), 100)
( 'D-2602', datetime.date(2026, 1, 2), 1200)
( 'D-2603', datetime.date(2026, 1, 2), None)
( 'D-2604', datetime.date(2026, 1, 2), 85)
( 'D-2605', datetime.date(2026, 1, 2), 2040)
rows needing attention: [( 'D-2603', 'N/A')]
```

How it works, line by line.

- `strptime(dispatch_date, '%d-%m-%Y')` reads the date with an **explicit format**: day, month, year. If a date doesn't match, the load fails loudly instead of swapping day and month. `strptime` returns a date *and time* (midnight); `::DATE` keeps only the date. `::` is the cast shorthand from Chapter 28, which DuckDB accepts too.
- `replace(qty_units, ',', '')` removes the thousands separators, so "1,200" becomes 1200.
- `TRY_CAST(... AS INTEGER)` converts to a whole number, and returns NULL instead of an error when it can't. A plain `CAST` would stop the whole load on N/A.
- The second query lists the rows where conversion failed, so a person can follow up.

Reconcile the file

Now prove the load is complete: compare what's in the file with what's in the tables.

```
with open("exports/dispatch_2026-01-02.csv", encoding="utf-8-sig") as f:
    data_lines = [line for line in f.read().splitlines()[1:] if line.strip()]
loaded = wh.execute("SELECT COUNT(*) FROM staging.dispatch").fetchone()[0]
usable = wh.execute(
    "SELECT COUNT(*) FROM staging.dispatch WHERE qty_units IS NOT NULL").fetchone()[0]
print("data lines in file :", len(data_lines))
print("rows loaded          :", loaded)
print("rows with a usable quantity:", usable)
print("total units (usable rows):",
      wh.execute("SELECT SUM(qty_units) FROM staging.dispatch").fetchone()[0])
```

```
data lines in file : 5
rows loaded          : 5
rows with a usable quantity: 4
total units (usable rows): 3425
```

- **encoding="utf-8-sig"** reads UTF-8 and drops the invisible byte-order mark if there is one: the fix for the first trap.
- `f.read().splitlines()` splits the file into lines, and `[1:]` keeps everything after the header line.
- **if `line.strip()`** skips lines that are empty or only spaces, such as the blank line at the end. `strip()` removes spaces from both ends, and an empty result counts as false.
- The three queries count the loaded rows, the rows with a quantity, and the total units.

What to tell the dispatch supervisor. All five dispatches were loaded. Four have quantities, totaling 3,425 units. Dispatch D-2603 for order 10172 has no quantity ("N/A, awaiting count"), so it's excluded from unit totals until the count arrives. Please send the corrected quantity, or update the sheet before tonight's load.

Simplification note. Counting lines works here because no field contains a line break. Real files sometimes do (an address or a remark typed over two lines, inside quotes), so production reconciliation counts *records* with a proper CSV parser, not raw lines, and compares against a count or total supplied by the sender when one exists.

A robust file-loading checklist

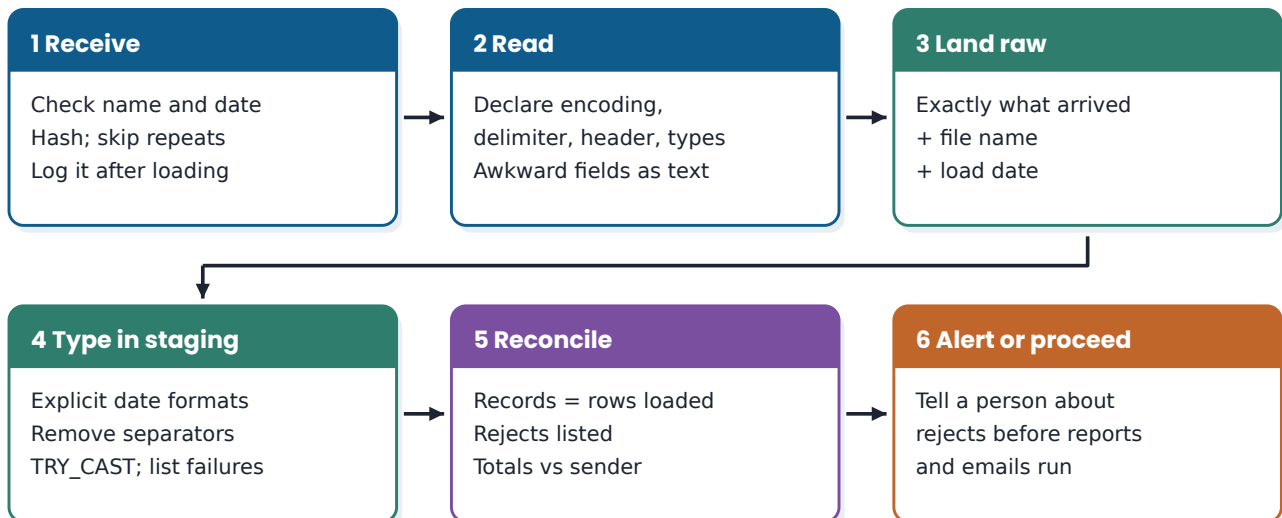


Figure 45.3 – A reliable file load, step by step. Every step either proves something or records something.

1. **Receive:** check the file name matches the expected pattern and date; hash it; skip exact repeats; record it in the load log once it has loaded.
2. **Read with declared settings:** encoding, delimiter, quote character, header, and column names and types. Don't rely on guessing in a scheduled load.
3. **Land raw:** keep the original values, plus the file name and load date on every row, so any number can be traced back to its file.
4. **Type in staging:** explicit date formats, clean separators, `TRY_CAST`, and a list of values that failed.
5. **Reconcile:** records in the file against rows loaded; rejected rows listed; totals checked against any control total the sender provides.
6. **Alert or proceed:** if anything was rejected or doesn't reconcile, tell a person before reports and emails run. Chapter 46 wires this into a pipeline.

Parquet: when files are designed for data

Chapter 2 introduced **Parquet**, a columnar file format that stores types, compresses well, and reads fast. Once a file has been cleaned, writing it as Parquet keeps its types for every later reader:

```
wh.execute("COPY staging.dispatch TO 'warehouse/dispatch_2026-01-02.parquet' "  
          "(FORMAT parquet)")  
for row in wh.execute(  
    "DESCRIBE SELECT * FROM 'warehouse/dispatch_2026-01-02.parquet'").fetchall():  
    print(row[:2])  
print("bytes: csv =", os.path.getsize("exports/dispatch_2026-01-02.csv"),  
      " parquet =", os.path.getsize("warehouse/dispatch_2026-01-02.parquet"))
```

```
( 'dispatch_id', 'VARCHAR')
( 'order_id', 'INTEGER')
( 'dispatch_date', 'DATE')
( 'warehouse', 'VARCHAR')
( 'qty_units', 'INTEGER')
( 'remarks', 'VARCHAR')
bytes: csv = 374  parquet = 1208
```

- **COPY table TO 'file' (FORMAT parquet)** writes a table to a Parquet file.
- **SELECT * FROM 'file.parquet'** reads it straight back; **DESCRIBE** shows the types it stored.
- **os.path.getsize()** gives a file's size in bytes.

Reading it. The Parquet file carries its types with it: `dispatch_date` is a `DATE` and `qty_units` an `INTEGER`, so no later reader has to guess. For this five-row file, Parquet is *larger* than the CSV, because it stores metadata and structure that only pays off with more data, just as Chapter 2 found for four orders. Chapter 49 measures large files, where Parquet is far smaller and faster than CSV. Choose Parquet for data at scale and for types; don't expect it to shrink a tiny file.

Dialect note: other tools. The same ideas apply in other tools, with different syntax. In pandas, pass `dtype=`, `parse_dates=` with `dayfirst=True`, and `thousands=`, `","` to `read_csv`. In PostgreSQL, `COPY ... FROM ... WITH (FORMAT csv, HEADER true)` loads into a table whose types you've declared, and rejects the whole file on the first bad value, so load into a text-typed raw table first.

45.8 When the source changes shape

A **schema** is the structure of a dataset: its column names, types, and order. A **schema change** (or **schema drift**) is when a source changes that structure: a column is added, removed, renamed, or changes type.

On 5 January, Bhiwandi Main started recording which truck carried each dispatch, and someone tidied up a column name. The new file begins:

```
dispatch_id,order_id,dispatch_date,warehouse,units,remarks,vehicle_no
```

Compare its columns with what the load expects, before loading anything:

```
EXPECTED = ["dispatch_id", "order_id", "dispatch_date", "warehouse", "qty_units", "remarks"]
new_cols = [r[0] for r in wh.execute(
    "DESCRIBE SELECT * FROM "
    "read_csv('exports/dispatch_2026-01-05.csv', all_varchar = true)").fetchall()]
print("missing columns:", [c for c in EXPECTED if c not in new_cols])
print("new columns      :", [c for c in new_cols if c not in EXPECTED])
```

```
missing columns: ['qty_units']
new columns      : ['units', 'vehicle_no']
```


- `all_varchar = true` reads every column as text, so the check looks only at names and never fails on a value.
- `[r[0] for r in ...]` keeps each column's name from `DESCRIBE`.
- The two list comprehensions find expected columns that are missing, and columns that are new.

Reading it. One expected column is missing (`qty_units`), and two new ones appeared (`units` and `vehicle_no`). Matching columns by *position* happens to work today, because `units` sits where `qty_units` was. But the extra seventh column makes DuckDB's declared-columns read stop with a confusing "sniffing" error, not a clear message. And the next time someone inserts a column before `units`, a positional load would put `remarks` where quantities belong. Matching by name finds no `qty_units` at all. The column check turns all of these into one clear message, before loading.

How to handle schema changes

Different changes deserve different responses.

Change	Usually safe?	Sensible response
New column added	Yes, if you match columns by name	Load known columns; log the new one; add it to raw when someone confirms what it means
Column removed	No	Stop the load and alert; downstream reports depend on it
Column renamed	No, because it looks like one removed and one added	Stop and alert; map the new name only after confirming it means the same thing
Type changed (number becomes text)	No	Raw keeps text, so it loads; staging's rejected-values check will catch it; alert
Column order changed	Yes, if you match by name	Always match by name, never by position

Here, the right response is to **stop**, tell the warehouse team and the pipeline's owner, confirm that `units` means the same as `qty_units`, add the mapping, and add `vehicle_no` to the raw table. That conversation is part of a **data contract**: an agreement with a source's owner about what they'll send and how they'll warn you before changing it. Chapter 47 covers data contracts in practice.

Real-life example: the tidy-up that cost a week. A common story in data teams: someone renames a spreadsheet column to make it clearer, a nightly load keeps "succeeding", and a dashboard shows zero sales for a region for several days before anyone asks why. The fix takes minutes. Finding it takes days, because nothing failed. A column check like the one above turns that into an alert on the first night.

45.9 Pulling data from APIs

Chapter 2 introduced APIs and Chapter 18 called one from Python. Ingesting from an API adds four practical problems: **authentication**, **pagination**, **rate limits**, and **transient failures**, the temporary kind that may succeed if you try again.

Riverstone's CRM exposes its leads through a REST API. The companion's `mock_crm_api.py` behaves like a typical one: it needs a token, returns at most 20 leads per request, and, like real APIs under load, sometimes answers "too many requests".

Chapter 29 built `CrmClient`, a class with the same retry, backoff, and pagination logic, for a practice API with a different key header and page format. We write the loop out again here so every line is visible. In your project, you can adapt `CrmClient` to this API instead.

Start the practice API, and set up the token:

```
from mock_crm_api import start_server
server = start_server()
BASE = "http://127.0.0.1:8045/api/leads"
TOKEN = os.environ.get("CRM_API_TOKEN", "practice-token-45") # practice default only
HEADERS = {"Authorization": f"Bearer {TOKEN}"}
print(server.server_address)
```

```
('127.0.0.1', 8045)
```

- `start_server()` starts the practice API in the background of the notebook, at port 8045 on this computer; `server_address` confirms where. Against a real CRM, you'd skip this and use its address.
- `HEADERS` carries the **token** that proves who you are. **Bearer** is the standard word for "whoever holds this token". Real tokens are secrets: never write them into code or commit them to Git. The token is read from the environment variable `CRM_API_TOKEN`, which you'd put in `.env` as in Chapter 18. The default after it is the practice API's public token, and exists only so this notebook runs without one; Chapters 52 and 64 cover secrets in production.

The function that makes one request, and retries only what might succeed next time:

```
RETRY_STATUSES = {429, 503}

def get_with_retry(url, params, max_attempts=4):
    for attempt in range(1, max_attempts + 1):
        try:
            resp = requests.get(url, params=params, headers=HEADERS, timeout=10)
        except (requests.Timeout, requests.ConnectionError) as e:
            problem, wait = type(e).__name__, 2 ** (attempt - 1)
        else:
            if resp.status_code not in RETRY_STATUSES:
                resp.raise_for_status()
                return resp.json()
            problem = f"{resp.status_code} {resp.reason}"
            wait = int(resp.headers.get("Retry-After", 2 ** (attempt - 1)))
        if attempt == max_attempts:
            break
        print(f" page {params['page']}: {problem}, waiting {wait}s (attempt {attempt})")
        time.sleep(wait)
    raise RuntimeError(f"gave up after {max_attempts} attempts (last: {problem})")
```

How it works, line by line.

- `timeout=10` stops a request waiting forever for an answer: after 10 seconds it raises `requests.Timeout`.

- `except (requests.Timeout, requests.ConnectionError)` catches a timeout or a dropped connection. Both are worth another try; `type(e).__name__` is the error's name, for the message.
- `else:` runs only when the request didn't raise. A status outside `RETRY_STATUSES` is final: `raise_for_status()` turns an error such as 401, 404, or 500 into an exception, so a failure is never mistaken for empty data, and a success returns the JSON.
- **429 Too Many Requests** and **503 Service Unavailable** are temporary. `resp.reason` is the status's name. If the server sends a **Retry-After** header, the wait is that many seconds; otherwise it's `2 ** (attempt - 1): 1, 2, then 4 seconds`. Waits that double each time are called **exponential backoff**.
- `max_attempts=4` is the number of tries in all, including the first. On the last attempt there's no point waiting, so `break` leaves the loop, and the function raises a clear error.

Some APIs send **Retry-After** as a date instead of a number of seconds, and `int()` would fail on it. Production code handles both; the `tenacity` package does this for you.

Now follow the pages:

```
leads, page = [], 1
while page is not None:
    body = get_with_retry(BASE, {"page": page, "page_size": 20})
    print(f"  page {page}: {len(body['data'])} leads")
    leads.extend(body["data"])
    page = body["next_page"]
print("leads fetched:", len(leads))
```

```
page 1: 20 leads
page 2: 429 Too Many Requests, waiting 1s (attempt 1)
page 2: 20 leads
page 3: 3 leads
leads fetched: 43
```

- The while loop follows **pagination**: it asks for page 1, then whatever `next_page` the API returns, until `next_page` is empty (`None`).
- `leads.extend(...)` adds the page's leads to the list.

Reading it. The API returned 20, 20, and 3 leads. Page 2 was rate limited once; the code waited and retried, and nothing was lost. That's 43 leads in total.

What happens without a token

```
resp = requests.get(BASE, params={"page": 1}, timeout=10)
print(resp.status_code, resp.json())
```

```
401 {'error': 'missing or invalid token'}
```

A **401 Unauthorized** response means the token is missing, wrong, or expired. Retrying won't help, which is why `get_with_retry()` doesn't retry it. The rule: **retry what might succeed next time** (429, 503, timeouts, and dropped connections; a 500 at most once or twice), and **fail fast on other 4xx errors** (400, 401, 403, 404).

Load and reconcile

```
wh.execute("""
    CREATE OR REPLACE TABLE raw.crm_leads (
        lead_id      INTEGER PRIMARY KEY,
        created_at   TIMESTAMP,
        company_name VARCHAR,
        email        VARCHAR,
        source       VARCHAR,
        owner_id     INTEGER,
        loaded_for   DATE)""")
wh.executemany("INSERT INTO raw.crm_leads VALUES (?, ?, ?, ?, ?, ?, DATE '2026-01-05')",
               [(lead["lead_id"], lead["created_at"], lead["company_name"], lead["email"],
                    lead["source"], lead["owner_id"])
                 for lead in leads])
cols, count_rows = fetch_rows("SELECT COUNT(*) FROM leads")
src_count = count_rows[0][0] # first row, first column
wh_count = wh.execute("SELECT COUNT(*) FROM raw.crm_leads").fetchone()[0]
print("source:", src_count, " warehouse:", wh_count, " match:", src_count == wh_count)
server.shutdown()
```

```
source: 43  warehouse: 43  match: True
```

- `raw.crm_leads` declares the API's six fields with their types, plus `loaded_for`, and `lead_id` as its primary key.
- Each lead is a dictionary from the API's JSON; the list comprehension turns each one into a tuple in the table's column order. `DATE '2026-01-05'` fills `loaded_for`, the day this load is for.
- `count_rows[0][0]` is the first column of the first row: the count.
- `server.shutdown()` stops the practice API. (The CDC connection was closed at the end of section 45.6.)

All 43 leads arrived. Against a real CRM you usually can't count the source directly; instead, compare with the total the API reports (many APIs return one), or with a count exported from the CRM's own screen.

Watch out: offset pagination and moving data. This API pages by position ("records 21–40"). If a lead is added or removed while you're reading, every later record shifts by one, and you can skip or repeat a record. For data that changes during the read, prefer APIs that page with a **cursor** (a token pointing at the next record) or filter by a stable key or timestamp, and always reconcile the final count.

API ingestion checklist

- Keep **credentials** out of code; rotate them; use the least access the load needs.
- Always set **timeouts**.
- **Retry** 429, 503, timeouts, and dropped connections with increasing waits, respecting `Retry-After`; a 500 at most once or twice; fail fast on other 4xx errors (400, 401, 403, 404).
- Follow **pagination** to the end, and prefer cursor pagination for changing data.

- Ask for **only what changed** when the API supports it (for example `updated_since=`), with a look-back window.
- Land the **raw response** (or its fields) with a load date, so you can re-process without calling the API again.
- **Reconcile** against a total from the source.
- Respect the provider's **limits and terms**. Some APIs charge per call or suspend keys that exceed limits.

45.10 Build or buy: connectors versus custom code

Everything in this chapter so far was custom code. In practice, many companies don't write ingestion for common tools at all. They use **managed connectors**: services and open-source tools that already know how to extract data from hundreds of popular databases and SaaS products, handle pagination, schema changes, and CDC, and load into a warehouse. Well-known examples include Fivetran and Airbyte, and cloud providers offer their own.

Question	Leans toward a managed connector	Leans toward custom code
Is the source a popular product with a maintained connector?	Yes	No, or the connector lacks what you need
How much engineering time do you have?	Little	Enough to build <i>and</i> maintain
How much data, and how does the tool charge?	Moderate volumes where pricing is affordable	Very high volumes where per-row pricing becomes expensive
How unusual is the source?	Standard APIs and databases	Custom files, older systems, internal tools, odd formats
Where can the data go?	Allowed to pass through a vendor's service	Must stay inside your network or country
Who fixes it when it breaks?	The vendor, for connector bugs	You

The honest summary: **buying saves building, not owning**. A connector still needs someone to choose what to sync, watch costs, reconcile results, and respond when a source changes. And custom code is rarely "free": the first version takes days; keeping it working as sources change takes years.

For Riverstone, a reasonable split would be a managed connector for the CRM if a maintained one exists for its product, CDC or scheduled queries for the ERP, and custom loading for the warehouse's dispatch sheet, which no vendor will ever have a connector for.

Interview extra point. When asked "how would you ingest data from X?", don't start with a tool. Ask three questions first: how big is it, how does it change (append-only, updates, deletes), and how fresh does it need to be? Then choose full, incremental, hash, or CDC, and say how you'd reconcile it. Chapter 77 has data engineering system design questions that use exactly this structure.

45.11 Web data: legal and ethical limits

Sometimes the data a business wants isn't in any of its systems: competitors' published prices, public tender notices, product reviews. Collecting data from websites automatically is **web scraping**. It can be legitimate, and it has real limits.

Before collecting any web data, check these, in order.

1. **Is there an official way?** An API, a data download, or a licensed data feed is almost always better: more reliable, and clearly permitted.
2. **What do the site's terms of use say?** Many sites prohibit automated collection. Breaking terms of use can have legal consequences, and it can get your company's network blocked.
3. **What does robots.txt say?** Most sites publish a file at `/robots.txt` saying which parts automated tools may visit. Respecting it is the minimum standard of good behavior.
4. **Does the data include personal information?** Names, phone numbers, email addresses, and profile details are personal data. Data protection laws, including India's Digital Personal Data Protection Act, 2023, and the EU's GDPR, apply to collecting and using personal data even when it's publicly visible. Treat personal data as off-limits unless your legal or compliance team has confirmed a lawful basis.
5. **Is the content protected by copyright?** Reviews, articles, images, and databases can be. Collecting facts is different from copying creative content.
6. **Will your collection harm the site?** Send requests slowly, identify your tool truthfully, and never try to get around logins, paywalls, or blocking measures.

When in doubt, ask your company's legal or compliance team before collecting anything. This section is general guidance, not legal advice; laws differ between countries and change over time. Chapter 64 covers privacy and data governance in depth.

Watch out: "it's public" is not permission. Being visible in a browser doesn't make data free to collect, store, and reuse. The questions above apply to public pages too.

APPLY

Common mistakes

Mistake	Symptom	Fix
Judging a load by whether it finished	Dashboards drift from the source; nobody knows since when	Reconcile counts and totals against the source after every load (45.3)
Incremental load by ID on rows that get updated	Status counts differ from the source; cancelled orders still open (45.4)	Use <code>updated_at</code> with a look-back, hash comparison, or CDC
Trusting <code>updated_at</code> without checking it	Occasional missing changes, especially after bulk fixes	Test it against a hash comparison before relying on it
Forgetting deletes	Rows deleted in the source live forever in the warehouse	Full load, hash comparison, or CDC; or a source "deleted" flag

Mistake	Symptom	Fix
Plain <code>INSERT</code> in a re-runnable load	Duplicate rows after a retry	Primary keys in raw, and upserts (45.5)
Building hash strings differently on each side	Every row looks changed	Same columns, order, separator, and date format on both sides
NULLs dropped from the hash string	A change to or from NULL goes unnoticed	Wrap nullable columns in <code>coalesce(..., '')</code> on both sides (45.5)
Recording a file as loaded before loading it	A failed load is skipped forever as "already loaded"	Record in the load log only after the load succeeds (45.5)
Leaving a CDC slot unconsumed	Source database disk fills; the ERP stops	Monitor slot lag; drop unused slots (45.6)
Letting a reader guess column types in a daily load	Columns switch between number and text from day to day	Declare columns; type in staging (45.7)
Parsing dates without an explicit format	2 January loaded as 1 February	<code>strptime</code> with the source's exact format
<code>CAST</code> on messy values	One bad value stops the whole load	<code>TRY_CAST</code> , and list the failures
Matching file columns by position	Values land in the wrong columns after a change	Match by name; check expected columns first (45.8)
Retrying every error	Endless retries on a bad token; accounts locked	Retry 429, 503, and timeouts; fail fast on other 4xx errors (400, 401, 403, 404)
No timeout on API calls	A load hangs all night	Always set a timeout
Tokens in code	A secret leaks through Git	Environment variables or a secrets manager
Scraping without checking terms or personal data	Legal risk, blocked networks	Official sources first; check terms, robots.txt, privacy law (45.11)

In the real world: the orders that never cancelled

Riverstone's first proper data pipeline went live in February, built by a data engineer the company had hired on contract, with Meera as the business owner. Every night it loaded new orders from the ERP into the warehouse, and every morning the Daily Sales Flash came from the warehouse instead of from Meera's spreadsheet. It was fast, it never failed, and for three weeks everyone was pleased.

Then Vikram brought a printout to the Monday sales review. "The Flash says we have eleven open orders for Green Leaf Hotels. Their buyer told me on Friday they cancelled six of those in January."

Meera opened the ERP. The six orders were marked Cancelled. She opened the warehouse. The same six orders said Pending.

She'd read enough about ingestion to guess the cause before she found it. The pipeline loaded orders with an ID higher than the last one it had seen. Cancelling an order changes its status, not its ID. Every cancellation, every delivery, every correction made to an existing order after its first night had never reached the warehouse.

"How many orders are wrong?" Anita asked.

"I don't know yet," Meera said. "That's the worse problem. Nothing told us."

That afternoon, the engineer and Meera did three things. First, they measured the damage: a hash comparison across every order since the pipeline started. It found 214 orders whose status in the warehouse didn't match the ERP. (This was the real ERP, which takes about 4,000 orders a month, not the 175-order practice copy you've been using.) Second, they fixed the data, with an upsert of every changed row, and re-ran the Flash for the affected days, so the sales team had correct history. Third, they fixed the design. The ERP had no reliable last-updated column, so the engineer set up a nightly hash comparison over the last 90 days of orders, plus a full comparison every Sunday, and added a reconciliation check: order counts by status in the warehouse must match the ERP before the Flash is sent.

The check failed once more, two weeks later, when someone corrected a batch of orders from the previous quarter, outside the 90-day window. This time nothing reached the sales team. The Flash was held, an alert went to Meera at 6:10 a.m., the Sunday full comparison was run early, and the Flash went out at 7:30 with the right numbers.

At the monthly review, Anita asked the question Meera had been expecting. "Why didn't the pipeline do this from the start?"

"Because it looked like it worked," Meera said. "It loaded exactly what it was built to load. We built it to load the wrong thing, and never checked it against the ERP."

What made this work.

- **Meera recognized the pattern:** a load that never fails, with numbers that drift, points to what the load isn't reading.
- **They measured before fixing:** the hash comparison showed the full size of the problem, not only the six orders someone happened to notice.
- **They fixed data, design, and detection:** corrected history, a change-detection method that catches updates, and a reconciliation check that stops bad numbers before they're sent.
- **The check proved its value** the next time something changed outside the plan.

Project: incremental ingestion for Riverstone

Goal: a small, re-runnable ingestion job that loads Riverstone orders, order lines, CRM leads, and dispatch files into the warehouse, detects changes correctly, and proves every load with a reconciliation report. Chapter 46 turns it into an orchestrated pipeline, and Chapter 47 adds quality checks and freshness alerts.

Tools you'll need

- **PostgreSQL 16**, with `wal_level = logical` for section 45.6. On a company database, ask the administrator; don't change it yourself.
- **Python** in the book's virtual environment (Chapter 17), with `psycopg2` (PostgreSQL driver) and `duckdb` (local analytical database), installed in section 45.2 with `python -m pip install psycopg2-binary==2.9.13 duckdb==1.5.6`, and `requests` and `python-dotenv` from Chapter 18.

- **The Chapter 45 companion folder** (`companion/ch45/`): `reset_ch45.py`, `apply_day.py`, `mock_crm_api.py`, `make_files.py`, copied to `work/ch45`. Run the notebook from that folder. If your PostgreSQL needs a user or password, put the connection string in `work/ch45/.env` as `RIVERSTONE_SOURCE=dbname=riverstone_source user=postgres password=your-password host=localhost` (section 45.2). The notebook, `reset()`, `apply_day()`, and the practice API all read it.
- **Versions used for the outputs shown:** Python 3.11.15, DuckDB 1.5.6, psycopg2 2.9.13, requests 2.33.1, python-dotenv 1.2.3, PostgreSQL 16.13.
- **Worth knowing about, not needed here:** Debezium for production CDC; managed connectors such as Fivetran and Airbyte; `dlt` and Singer-style tools for writing connectors in Python.

Option A: Riverstone. Use the companion environment.

Option B: your own data. Use a database, API, or regular file you have permission to read. Don't use production systems for practice without your administrator's agreement, and keep credentials out of anything you share.

Steps

1. **Plan the sources.** For each of `orders`, `order_items`, `customers`, `products`, CRM leads, and dispatch files, write down: source type, system of record, size, whether rows are updated or deleted, and your chosen strategy (full, incremental, hash, CDC) with one sentence of reasoning.
2. **Create the raw tables** in the warehouse with declared types and primary keys, plus `loaded_at_run` (a run ID) on every table.
3. **Build a load log table:** run ID, source, strategy, rows read, rows inserted, rows updated, rows rejected, reconciliation result.
4. **Full loads** for `customers` and `products`.
5. **Change detection for orders and order_items**, using hash comparison or CDC, applied with upserts.
6. **The CRM leads API**, with authentication from `.env`, pagination, retries on 429, 503, and timeouts, and a final count check.
7. **Dispatch files**, with file hashing, declared columns, typed staging, a rejected-rows list, and a column check that stops the load on missing or renamed columns.
8. **A reconciliation report**, printed at the end of every run: for each source, rows in the source (or file), rows in the warehouse, and order counts by status. Mark any mismatch clearly.
9. **Test it:** run `reset()`, run your job, `apply_day(1)`, run it again, `apply_day(2)`, run it again, then run it a fourth time with no changes. The reconciliation report must match the source every time, and the fourth run must change nothing.
10. **Break it on purpose:** load `dispatch_2026-01-05.csv` and confirm your column check stops it with a clear message.

Stretch goals

- Replace the hash comparison for orders with the CDC slot, applying each change with an upsert.
- Add a deleted order to `apply_day` (in your own copy) and show which strategies notice it.
- Add a look-back window to an `updated_at`-style incremental load of lead stage history, and show it handles a late change.

REVIEW

Recap

- **Ingestion** moves data from source systems into the warehouse. Its dangerous failures are loads that succeed with wrong data.
- Sources include **databases**, **files**, **APIs**, **events**, and **SaaS tools**; the **system of record** is the source that's officially correct.
- Data lands in a **raw** layer as it arrived, then a typed **staging** layer, then **modeled** tables.
- A **full load** replaces everything: simple, self-correcting, and sees updates and deletes. It stops being practical for large or busy tables.
- An **incremental load** uses a **watermark**. By ID it misses **updates** and **deletes**; `updated_at` helps, with a **look-back** window, if it's trustworthy.
- A **hash** is a fixed-length fingerprint of any data: the same data always gives the same fingerprint, and the smallest change gives a completely different one. **SHA-256** gives 64 hex characters; **MD5** gives 32 and is fine for change detection, not for security.
- **Row hashes** detect inserts, updates, and deletes without help from the source, at the cost of scanning keys; wrap nullable columns so NULLs aren't dropped. **File hashes** skip content already loaded, recorded in a load log only after a successful load.
- **Upserts** make loads **idempotent**: safe to run again.
- **Change data capture** reads the database's **write-ahead log** through a **replication slot**: every change, in order, without scanning. Unconsumed slots fill disks.
- **Reliable file loads** declare columns, keep raw text, type in staging with explicit formats and `TRY_CAST`, list rejects, and **reconcile** against the file.
- **Schema changes** should be detected before loading; match columns **by name**, and stop on removed or renamed columns.
- **API ingestion** needs secure **authentication**, **timeouts**, **pagination**, and **retries** only for temporary failures: 429, 503, and timeouts.
- **Managed connectors** save building, not owning.
- **Web data** needs an official source where possible, and checks on terms, robots.txt, personal data, and copyright.

Key terms

ingestion · source system · system of record · raw layer · staging layer · modeled layer · DuckDB · full load · reconciliation check · incremental load · watermark · last-updated timestamp · look-back window · hash (row hash, file hash) · SHA-256 · MD5 · checksum · hexdigest · connection string · cursor · upsert · idempotent · load log · write-ahead log (WAL) · change data capture (CDC) · replication slot · slot lag · byte-order mark (BOM) · declared columns · TRY_CAST · rejected rows · Parquet · schema · schema change (schema drift) · data contract · authentication token · pagination (offset, cursor) · rate limit (429) · Retry-After · exponential backoff · timeout · managed connector · web scraping · robots.txt · personal data

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ You can name the main kinds of source system and one typical difficulty of each.
- ☐ You can explain raw, staging, and modeled layers, and why raw keeps data as it arrived.
- ☐ You can write a full load with declared types and reconcile it by count and by category.
- ☐ You can explain, and demonstrate, why an incremental load by ID misses updates and deletes.
- ☐ You can list three ways an `updated_at` watermark can still miss changes.
- ☐ You can compute a SHA-256 fingerprint in Python and explain why a one-character change alters all of it.
- ☐ You can detect inserted, changed, and deleted rows with row hashes, and explain why both sides must build the string identically.
- ☐ You can explain idempotency and show that an upsert can be safely re-run.
- ☐ You can skip already-loaded files with a file hash, and say when that would be wrong.
- ☐ You can explain change data capture, replication slots, and the risk of an abandoned slot.
- ☐ You can choose between full, incremental, hash, and CDC for a given table, with reasons.
- ☐ You can load a messy CSV with declared columns, explicit date formats, `TRY_CAST`, and a rejected-rows list, and reconcile it against the file.
- ☐ You can detect a schema change before loading and choose the right response.
- ☐ You can ingest from a paginated, authenticated, rate-limited API with retries and timeouts.
- ☐ You can argue build versus buy for a given source.
- ☐ You can list the checks to make before collecting web data.

PRACTISE

Exercises

Run `reset()` before each exercise that uses the companion environment, unless it says otherwise.

Warm-up

1. For each Riverstone table or source, choose a loading strategy (full, incremental by ID, incremental by timestamp, hash comparison, or CDC) and give one reason: (a) `products`; (b) website enquiry events that are only ever added; (c) `orders`, which are updated but have no `updated_at`; (d) the CRM's lead stage history, where every stage change adds a row with a timestamp.
2. Which HTTP status codes should an ingestion job retry, and which should make it fail immediately? Classify: 200, 401, 404, 429, 500, 503, and a timeout.
3. Explain in two sentences why `concat_ws(' ', ...)` uses a separator when building a row hash.
4. Hash Pay Rs 14,700 to Riverstone Supplies with SHA-256, with and without a space at the end. How many of the 64 characters of the two fingerprints match, position by position? Is that what you expected?

Core

5. After `reset()` and a full load of orders, run `apply_day(1)` and then `apply_day(2)` *without loading in between*. Using the hash comparison from section 45.5, which order IDs will be inserted and which changed? Explain why order 10176 appears in only one of the lists.
6. Write a function `load_orders_full()` that performs a full load of `orders` into `raw.orders` and returns the source and warehouse counts by status. Why does a full load not need a separate method for detecting updates?
7. A dispatch file has the line `D-2610,10180,13-01-2026,Bhiwandi Main,"3,150",.` What will `staging.dispatch` contain for `dispatch_date` and `qty_units`? What would happen if the date were parsed with the format `'%m-%d-%Y'`?
8. Using the reconciliation output in section 45.7, a manager asks: "What were total units dispatched on 2 January?" Write the answer you'd send, including what's missing.
9. `get_with_retry()` in section 45.9 waits 1, 2, then 4 seconds between attempts when the server sends no `Retry-After` header. Chapter 29's `CrmClient` also added a small random amount to each wait, called **jitter**. Change `get_with_retry()` so each of those waits is `2 ** (attempt - 1)` plus a random amount of up to half of that. Why is waiting longer each time better than retrying immediately, and what does the jitter add?
10. Explain why the plain `INSERT` in section 45.5 raised an error on the second run. What would have happened if `raw.orders` had no primary key?

Stretch

11. Extend the column check from section 45.8 into a function `check_columns(path, expected)` that returns "ok", "new columns only" (safe to load known columns), or "stop" (a column is missing), and a list of the differences. Run it on both dispatch files.
12. Design a strategy for an `invoices` table with 80 million rows, where invoices are sometimes corrected up to 60 days after they're raised, rows are never deleted, and there's no trustworthy `updated_at`. The ERP team won't allow CDC. Describe what you'd load nightly and weekly, and how you'd reconcile.
13. The CRM API pages by offset, and leads are added and removed while you read. Describe a sequence in which a lead is skipped, using page size 20, and propose two fixes.

Think about it (no code needed)

14. Riverstone is choosing between a managed connector for its CRM and custom code. List three questions you'd ask before deciding, and what answer would push you each way.
15. A marketing colleague wants to collect the names and phone numbers of shop owners listed on a public business directory, to call them about Riverstone's products. What would you tell them before anything is collected?

Answers

1. (a) **Full load**: `products` is small and changes occasionally, so reloading it is cheap and always correct. (b) **Incremental by ID or timestamp**: rows are only ever added, so a watermark can't miss updates. (c) **Hash comparison** (or CDC if the ERP team allows it): rows are updated, and there's no trustworthy column to find the changes. (d) **Incremental by timestamp** on the stage-change timestamp column (in Riverstone's `lead_stage_history` table, `entered_at`), with a look-back window: each change adds a new row with a timestamp. A full load is also reasonable for (d) at Riverstone's size.
2. Retry: **429** (rate limited), **503** (temporarily unavailable), and **timeouts**. **500** is borderline: retry at most once or twice, because it's sometimes temporary, then fail. Fail immediately on the other 4xx errors: **401** (authentication won't fix itself) and **404** (the address is wrong). **200** is success: no retry.
3. Without a separator, different rows can produce the same string: values 1 and 23 join to 123, exactly like 12 and 3, so a change could go unnoticed. A separator that doesn't appear in the data keeps each value's boundaries, so different rows give different strings and different hashes.
4. Only 2 of the 64 positions match. The fingerprints are:

```
f4251ff3fb7191f7e79677f3b1871db06c0935c3cc08164496995f1f909f6f71    no space
71e96ba2039f8e4139716f2287b78cfe1e12842a28e6f3dfab748321bbcaa77a    one space at the end
```

That's the avalanche property: one invisible character changes the whole fingerprint. Any position has a 1 in 16 chance of matching by luck (16 possible hex characters), so about 4 matches is what chance alone gives; 2 is well within that. It's also a warning: a trailing space in a file makes it a different file to a hash.

5. Inserted: **10176, 10177, 10178**. Changed: **10174, 10175**. Order 10176 was created on day 1 and changed on day 2, but the warehouse never saw its first version, so it counts as new: its latest version is inserted. Hash comparison sees the *current* difference between source and warehouse, not the history of changes in between; CDC would report both the insert and the update.

6. A sample answer:

```
def load_orders_full():
    cols, rows = fetch_rows(
        "SELECT order_id, customer_id, order_date, status, sales_rep_id FROM orders")
    wh.execute("DELETE FROM raw.orders")
    wh.executemany("INSERT INTO raw.orders VALUES (?, ?, ?, ?, ?)", rows)
    src = dict(fetch_rows("SELECT status, COUNT(*) FROM orders GROUP BY status")[1])
    tgt = dict(wh.execute(
        "SELECT status, COUNT(*) FROM raw.orders GROUP BY status").fetchall())
    return src, tgt
```

A full load replaces every row with the source's current version, so updated rows arrive in their new state and deleted rows disappear. There's nothing to detect. For a production version, wrap the delete and insert in one transaction, so readers never see an empty table.

7. `dispatch_date = 2026-01-13` (13 January 2026), and `qty_units = 3150`. With `'%m-%d-%Y'`, the parser would read month 13, which doesn't exist, so the load would fail with an error. That failure is useful: with a day of 12 or less, the wrong format would not fail, and would silently swap day and month.

8. *"Bhiwandi Main recorded five dispatches on 2 January. Four have quantities, totaling 3,425 units. The fifth, D-2603 for order 10172, has no quantity yet (marked 'awaiting count'), so the true total is 3,425 plus that dispatch. I've asked the warehouse for the count and will update the figure once it arrives."* The key points are the total, what's excluded, and that it's being followed up.

9. A sample answer:

```
import random

def get_with_retry(url, params, max_attempts=4):
    for attempt in range(1, max_attempts + 1):
        try:
            resp = requests.get(url, params=params, headers=HEADERS, timeout=10)
        except (requests.Timeout, requests.ConnectionError) as e:
            problem, retry_after = type(e).__name__, None
        else:
            if resp.status_code not in RETRY_STATUSES:
                resp.raise_for_status()
                return resp.json()
            problem = f"{resp.status_code} {resp.reason}"
            retry_after = resp.headers.get("Retry-After")
        if attempt == max_attempts:
            break
        if retry_after is not None:
            wait = int(retry_after)
        else:
            base = 2 ** (attempt - 1)
            wait = base + random.uniform(0, base / 2)
        time.sleep(wait)
    raise RuntimeError(f"gave up after {max_attempts} attempts (last: {problem})")
```


Without a `Retry-After` header, the waits between the four attempts are 1, 2, and 4 seconds, each plus up to half as much again. `random.uniform(a, b)` returns a random number between `a` and `b`. There's no wait after the fourth attempt, because the function gives up then. This is **exponential backoff** with **jitter**. Retrying immediately hits a server that's already overloaded or limiting you, which usually fails again and can make things worse for everyone; waiting longer each time gives it room to recover. The jitter spreads out many clients that failed at the same moment, so they don't all retry at the same moment too.

10. `raw.orders` has `order_id` as its **primary key**, and orders 10176 and 10177 already existed, so inserting them again violated the key and DuckDB refused. Without a primary key, the insert would have **succeeded silently**, creating duplicate rows: order counts and revenue for those orders would be doubled in every report, with no error anywhere.

11. A sample answer:

```
def check_columns(path, expected):
    cols = [r[0] for r in wh.execute(
        f"DESCRIBE SELECT * FROM read_csv('{path}', all_varchar = true)".fetchall())
    missing = [c for c in expected if c not in cols]
    extra = [c for c in cols if c not in expected]
    if missing:
        return "stop", {"missing": missing, "new": extra}
    if extra:
        return "new columns only", {"new": extra}
    return "ok", {}
```

On `dispatch_2026-01-02.csv`, it returns **ok**. On `dispatch_2026-01-05.csv`, it returns **stop**, with `qty_units` missing and `units` and `vehicle_no` new. The rename is reported as one missing and one new column, which is why it must stop rather than guess.

12. One reasonable design: **Nightly**, compute row hashes for invoices raised in the **last 60 days** (the correction window) plus a small margin, say 70 days, compare with the warehouse, and upsert changes; load new invoices in the same step. **Weekly**, compare hashes for the whole table **in monthly chunks**: compute one hash per month (for example, a hash of the sorted row hashes) on both sides, and only compare row by row for months whose chunk hash differs. **Reconcile** every night: invoice count and total amount by month for the last 70 days, and for all months weekly. Run the heavy queries against a read replica or outside business hours, with the ERP team's agreement. Because rows are never deleted, delete detection can be weekly only. Other designs are acceptable if they cover the 60-day correction window, limit the load on the ERP, and reconcile.

13. The practice API has 43 leads. Suppose it lists them newest first, 20 per page. You read page 1 (positions 1–20). Before you ask for page 2, a lead in positions 1–20 is **deleted**, or merged into another. Everything after it moves up one place: the lead that was at position 21 is now at 20, and page 2 (positions 21–40) never returns it. It is **skipped**. (A new lead arriving at position 1 does the opposite: the old 20 becomes 21 and is read **twice**.) Fixes: (1) **cursor pagination**, or a stable filter such as `lead_id > last_seen_id` read in `lead_id` order, which isn't affected by changes elsewhere in the list; (2) de-duplicate by `lead_id` on load and **reconcile** the final count against the API's total, re-reading if it doesn't match.

14. Useful questions: (a) *Is there a maintained connector for our exact CRM product, covering the objects we need?* Yes pushes toward buying; no, or only partly, toward custom code. (b) *What will it cost at our data volume and sync frequency, and what does our engineering time cost?* Affordable pricing and little engineering time push toward buying; high per-row costs at large volumes, or spare engineering capacity, push toward building. (c) *Is our customer and lead data allowed to pass through a vendor's service, and where is it processed?* If policy or law requires it to stay inside our systems or country, that pushes toward custom code or a self-hosted tool. Other good questions: who fixes it when it breaks, how quickly it handles schema changes, and whether it supports incremental syncs.

15. Tell them to stop and check before collecting anything. **Names and phone numbers are personal data**, and data protection laws such as India's Digital Personal Data Protection Act, 2023, apply even when the directory is public; using them for sales calls needs a lawful basis, which the company's legal or compliance team must confirm. Check the directory's **terms of use**, which often prohibit automated collection and commercial reuse, and its `robots.txt`. Ask whether there's an **official route**: a paid listing service, a licensed data provider, or the directory's own advertising options. And note that India's rules on unsolicited commercial calls also apply. The safest recommendation is to involve legal or compliance first, and to prefer sources where shop owners have agreed to be contacted.

NEXT

Where this leads

- **Chapter 46, Pipelines & Orchestration**, turns this chapter's loads into a scheduled, orchestrated pipeline, with idempotency tested by a deliberate failure, backfills, alerts, and the Daily Sales Flash sent only after its data passes checks.
- **Chapter 47, Data Quality, Observability & Contracts**, turns reconciliation checks into automated tests, adds freshness monitoring, and puts data contracts like the one in section 45.8 into practice.
- **Chapter 49, Storage, Warehouses & Lakehouses**, explains how warehouses store raw and staging layers, and covers ACID transactions, partitioning, and cloud warehouse costs.
- **Chapter 50, Streaming & Real-Time**, takes change data capture from a daily read to a continuous stream.
- **Chapter 51, Data Activation**, reverses the direction: writing warehouse data back into the CRM and ERP through APIs, with the same care for idempotency, retries, and systems of record.
- **Chapter 64** covers privacy, security, and governance, including the rules behind section 45.11.
- **Part 8**: ingestion and system design questions appear in Chapter 77, and API and webhook integration questions in Chapter 78.

Chapter 46. Pipelines & Orchestration

Chapter at a glance

You will learn to: explain what an orchestrator adds to scheduled scripts · describe a pipeline as a graph of assets with dependencies, schedules, and partitions · build Riverstone's daily pipeline in Dagster, from ingestion to a delivered Daily Sales Flash · prove a pipeline step is idempotent with a deliberate failure test, and show what happens when it isn't · rebuild history with a backfill without re-sending old reports · use retries for transient failures without hiding real bugs · stop delivery when the data fails its checks, and send an alert that tells someone what to do · handle late-arriving corrections · read and write the same pipeline in Airflow · choose an orchestrator, and run pipelines responsibly.

Before you start: Chapter 45 (ingestion, hashes, upserts, idempotency; your ingestion project), Chapter 20 (the Daily Sales Flash, cron, and scheduling), Chapter 29 (modules, decorators, generators, tests), Chapter 32 (dbt's DAG), Chapter 12 (transactions: `BEGIN`, `COMMIT`, `ROLLBACK`), Chapter 26, section 26.0 (the terminal), and Chapter 17 (the virtual environment and Jupyter).

Time needed: 16–20 hours of reading and practice, spread over two to three weeks, in four sittings: sections 46.1–46.3 (the ideas and the first run); 46.4–46.5 (idempotency, backfills, late data); 46.6–46.7 (retries, sensors, checks, alerts); 46.8–46.10 and the project.

Tools: Python in the book's virtual environment (Chapter 17) with `duckdb`, `psycopg2`, and `requests` from Chapter 45, plus `dagster` and `dagster-webserver`, which section 46.3 installs; PostgreSQL 16; and the Chapter 46 companion folder. Everything runs on your own computer.

Practice data: the `riverstone_source` database from Chapter 45 (a private copy of the one-year database, playing the ERP and CRM), Bhiwandi Main's dispatch files, and the practice CRM API. Every output shown is from a real run.

In this chapter

START	Why this matters · In plain English	49
LEARN	46.1 From scheduled scripts to pipelines	50
	46.2 The core ideas	51
	46.3 Building Riverstone's daily pipeline	53
	46.4 Idempotency, proven with a failure test	68
	46.5 Partitions and backfills	72
	46.6 Retries, timeouts, and alerting	74
	46.7 Delivery only after the data passes its checks	78
	46.8 The same pipeline in Airflow	82
	46.9 Choosing an orchestrator	85
	46.10 Operating pipelines	86
APPLY	Common mistakes · In the real world · Project	86
REVIEW	Recap · Key terms · Check yourself	89
PRACTISE	Exercises	91
	Answers	92
NEXT	Where this leads	94

Why this matters

In Chapter 20, the Daily Sales Flash became a script that runs every morning and emails the managers. In Chapter 45, you built the loads that bring orders, dispatches, and leads into the warehouse. Each piece works on its own. Now they have to work *together*, every day, in the right order, without anyone watching.

That's harder than it sounds. The dispatch file arrives late. The CRM drops a connection. The order-line load hits a bug on the same morning a manager is waiting for the numbers. A script scheduled for 6:30 a.m. doesn't know that the orders it's reporting on haven't finished loading. And when something fails at 6:31, someone has to know what failed, what's safe to re-run, and whether the managers already received a wrong email.

An **orchestrator** is the software that runs these steps in the right order, on schedule, and keeps a record of every run. Good orchestration turns a collection of scripts into a **pipeline**: something you can trust, re-run, repair, and explain. This chapter builds Riverstone's first real pipeline, breaks it on purpose, one fault at a time, and shows how a well-designed pipeline survives each one.

In plain English

Think about how a restaurant kitchen gets dinner out.

- The **recipe** says what depends on what: the sauce needs the stock, the plate needs the sauce and the rice. That's the **dependency graph** of a pipeline.
- The **head chef** doesn't cook everything. The head chef makes sure each station starts at the right time, in the right order, and knows when something's gone wrong. That's the **orchestrator**.
- Each **service** (Monday lunch, Monday dinner) is prepared separately. That's a **partition**: one day's run of the pipeline.
- If the rice burns, you remake the rice. You don't serve two plates of rice, and you don't throw away the sauce. Remaking a step without doubling anything is **idempotency**.
- Before a plate leaves the kitchen, someone checks it. A bad plate goes back; it never reaches the table. That's a **check that blocks delivery**.
- If the fish supplier is late, a runner calls again in five minutes. That's a **retry**. But if the fish is off, calling again won't fix it.
- Reprinting last month's menus for the archive isn't the same as handing them to today's customers. Rebuilding history without re-sending it is a **backfill**.

The Daily Sales Flash is a plate that leaves the kitchen every morning at 6:30. This chapter is about making sure it's the right plate.

46.1 From scheduled scripts to pipelines

The simplest scheduler is **cron**, which you used in Chapter 20 (section 20.7): the tool on Linux and macOS that runs a command at set times. A cron line such as `30 6 * * * cd /opt/riverstone && .venv/bin/python daily_flash.py` runs the Flash script at 6:30 every morning, by the server's clock. cron starts in your home folder with almost no settings, so the line first moves into the project folder and then names the project's own Python, as Chapter 20 explained. Windows Task Scheduler (Chapter 20) and Apps Script's time-driven triggers (Chapter 19, section 19.13) do the same job.

For a single script, that's often enough. Problems start when there are several steps that depend on each other:

- **Order.** If orders load at 6:00 and the Flash runs at 6:30, what happens on the day orders take 40 minutes?
- **Failure handling.** If the order-line load fails, does the Flash still run on stale data?
- **Retries.** If the CRM drops the connection, who tries again?
- **Re-running.** If you fix a bug and re-run, do you get duplicate rows or duplicate emails?
- **History.** Which runs succeeded last week? How long did each step take? What did it load?
- **Backfills.** How do you rebuild March after fixing a calculation, without re-sending March's emails?
- **Visibility.** Which reports depend on the dispatch file? If it breaks, what else is affected?

An orchestrator answers all of these in one place.

Need	Cron plus scripts	An orchestrator
Run steps in dependency order	You wire it by hand, often with time gaps	Declared once; the orchestrator waits for upstream steps
Skip downstream steps when something fails	Each script has to check	Built in
Retry transient failures	Custom code in every script	A retry policy per step
Run history, logs, durations	Scattered log files, if any	Every run recorded and searchable
Re-run one day, or rebuild a date range	Manual, error-prone	Partitions and backfills
See what depends on what	In people's heads	A graph you can view
Alert the right person	Custom code	Failure hooks and sensors (section 46.2)

Simplification note. Cron isn't wrong. A single report with no dependencies, run by one person who checks it, doesn't need an orchestrator. Section 46.9 covers when plain scheduling is the better choice.

46.2 The core ideas

Pipelines as graphs

A pipeline is a **directed acyclic graph** (DAG): a set of steps connected by arrows that show which step needs which, with no loops. "Acyclic" means you can never follow the arrows back to where you started, so there's always a valid order to run things in.

You met a DAG in Chapter 32: dbt built its models in the order their `ref()`s implied. Try the idea by hand with four boxes: orders, order lines, Flash, and deliver. The Flash needs orders and order lines; delivery needs the Flash. Two run orders are valid: orders, order lines, Flash, deliver; or order lines, orders, Flash, deliver. Both work, because orders and order lines don't need each other. Now add one more arrow, from deliver back to orders ("load orders only after the Flash has gone out"). Orders now needs deliver, deliver needs the Flash, and the Flash needs orders: there's no step you can start with, so no valid order exists. That loop is a **cycle**, and it's why a pipeline must be acyclic.

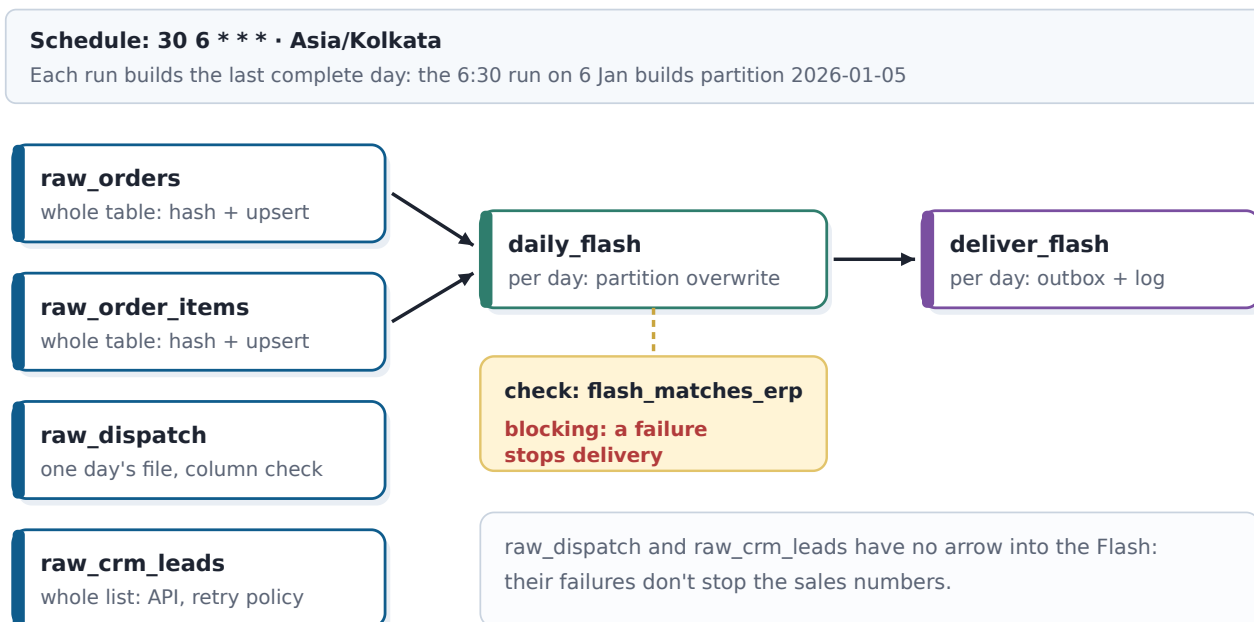


Figure 46.1 – Riverstone's daily pipeline as a graph. Notice what the Flash does and doesn't depend on: a problem with the dispatch file shouldn't stop the sales numbers.

Tasks and assets

There are two ways to think about the steps.

- **Task-based** orchestrators, such as Airflow, describe *what to run*: "run the load script, then run the report script". The data each task produces is implied.
- **Asset-based** orchestrators, such as Dagster, describe *what should exist*: "the table `raw.orders` should exist, built from the ERP; the `daily_flash` table depends on `raw.orders` and `raw.order_items`". An **asset** is a piece of data that the pipeline produces and keeps: a table, a file, a model, a report.

Both work, and both end up as a DAG. The asset view makes it easier to answer questions such as "which reports use this table?" and "is this table up to date?", which is why this chapter builds in Dagster. Section 46.8 shows the same pipeline written as Airflow tasks.

Schedules, time zones, and partitions

A **schedule** says when a pipeline runs. It's usually written as a **cron expression**: the five fields you met in Chapter 20, for minute, hour, day of month, month, and day of week.

Expression	Meaning
30 6 * * *	6:30 every day
0 7 * * 1	7:00 every Monday
0 */2 * * *	Every two hours, on the hour
15 22 1 * *	10:15 p.m. on the 1st of every month

A cron expression means nothing without a **time zone**, as Chapter 20 found with cron on a server. Servers usually run in **UTC** (Coordinated Universal Time); Riverstone runs on **IST** (India Standard Time, Asia/Kolkata), which is UTC plus 5 hours 30 minutes. A schedule of `30 6 * * *` in UTC would run at noon in Mumbai.

A **partition** is one slice of the pipeline's work that runs as a unit, most often one day. Partitioning means each run knows exactly which day it's building, so you can re-run 5 January without touching 2 January, and rebuild a whole month one day at a time. This isn't the `PARTITION BY` of Chapter 13's window functions (section 13.3): here a partition is one day's slice of the pipeline's work. Chapter 48 uses the word a third way, for the pieces of a large dataset spread across machines.

Watch out: "yesterday" depends on where you are. At 3:00 a.m. IST on 6 January, it's still 5 January in UTC. A pipeline that computes "yesterday" from the server clock in UTC will build the wrong day for five and a half hours every night. Set the time zone explicitly on schedules and partitions, and pass the partition date into every step that works on one day, instead of reading the clock.

Sensors: runs triggered by events

A schedule starts a run because the clock says so. A **sensor** starts one because something happened. It's a small function that the orchestrator calls every few minutes; if the function finds something (a new file, a finished upstream job, a failed run), it asks for a run or sends a message. The dispatch file that arrives late is the classic case: instead of guessing a time, a sensor watches the folder and starts the dispatch load when the file appears.

Someone has to watch the clock and call the sensors, even when nobody is looking at the pipeline. In Dagster that's the **daemon**: a background program that checks the schedules and sensors and starts the runs they ask for. Section 46.6 builds a sensor.

46.3 Building Riverstone's daily pipeline

Setting up

Chapter 45 set up the practice source, `riverstone_source`, and the practice CRM API. This chapter reuses both, and adds Dagster.

Step 1. Your practice folder. As in Chapter 45, copy the folder `companion/ch46` and paste it inside `work`, so you have `work/ch46`. If you made a `.env` file in `work/ch45` (Chapter 45, section 45.2), copy it into `work/ch46` too: this chapter's files read the same `RIVERSTONE_SOURCE` connection string.

Step 2. Install Dagster. Open a terminal, go into your practice folder, and activate the book's virtual environment:

```
# terminal
$ cd work/ch46
$ source ../../.venv/bin/activate
$ python -m pip install dagster==1.13.23 dagster-webserver==1.13.23
```

- **dagster** is the library you write pipelines with: assets, checks, schedules, and sensors, and the code that runs them.
- **dagster-webserver** is the local web page that shows your pipeline, its runs, and its schedules. You need it for the `dagster dev` command at the end of this section; without it, `dagster dev` stops with an error.
- `==1.13.23` asks for the version used for this chapter's outputs. Dagster works on Windows, macOS, and Linux; on Windows PowerShell, activate with `../../.venv\Scripts\Activate.ps1`, as in Chapter 45.

Add both packages to your `requirements.txt`, one per line. Then check the install:

```
# terminal
$ dagster --version
dagster, version 1.13.23
```

If the terminal says it can't find `dagster`, the environment wasn't active when you installed: activate it and install again.

Step 3. A new notebook. If Chapter 45's notebook is still open, shut down its kernel first. It holds its own warehouse file open and its practice API on port 8045, and a second notebook can't use either while the first one has them. Then create a new notebook, choose the `.venv` kernel, and save it as `ch46.ipynb` in `work/ch46`. Run every code block in this chapter as its own cell, top to bottom, without restarting the kernel.

The practice folder holds these files:

File	What it does
reset_ch46.py	Recreates <code>riverstone_source</code> , and empties the warehouse, the <code>outbox</code> folder (where delivered reports go), and the <code>alerts</code> folder
ingest.py	Your Chapter 45 loads, tidied into functions (next subsection)
riverstone_pipeline.py	The finished pipeline as one file, for Dagster's web page (end of this section)
apply_day.py, make_files.py, mock_crm_api.py	Copied from Chapter 45

What's in `ingest.py`

Every pipeline step in this chapter calls a function from `ingest.py`, so read it before you use it. It's your Chapter 45 project, tidied into functions. It starts with three settings:

```
SRC = os.environ.get("RIVERSTONE_SOURCE", "dbname=riverstone_source")
BROKEN_ITEMS_SYNC = False    # section 46.7 turns this on: a faulty order-line load
CONFIRMED_RENAMES = {}      # section 46.7 fills this in: a confirmed column rename
```

- `SRC` is the connection string, read from the environment (your `.env` file) as in Chapter 45.
- `BROKEN_ITEMS_SYNC` and `CONFIRMED_RENAMES` are two **switches** this chapter flips to simulate a fault and a fix. Nothing uses them until section 46.7.

Then two small functions for the source:

```
def fetch_rows(sql, params=None):
    """Run one query on the source and return just its rows (a list of tuples)."""
    conn = psycopg2.connect(SRC)
    with conn, conn.cursor() as cur:
        cur.execute(sql, params)
        rows = cur.fetchall()
    conn.close()
    return rows

def run_sql(sql, params=None):
    """Run one statement that changes the source, such as an UPDATE, and commit it."""
    conn = psycopg2.connect(SRC)
    with conn, conn.cursor() as cur:
        cur.execute(sql, params)
    conn.close()
```

- `fetch_rows()` is Chapter 45's function (section 45.3) with one change: it returns **just the rows**, not `(columns, rows)`. `params` holds the values for any `%s` placeholders in the SQL, as in Chapter 45. So `fetch_rows(sql)[0]` is the first row.
- `run_sql()` runs a statement that changes the source and commits it. Section 46.5 uses it to correct an order in the ERP.

The tables are loaded by the hash comparison and upsert from Chapter 45, section 45.5:

```
def row_hash(row):
    """One fingerprint per row: every column as text, joined, then hashed."""
```

```

        return hashlib.md5("".join(str(v) for v in row).encode()).hexdigest()

def sync_table(wh, table):
    """Bring raw.<table> up to date with the source. Returns (new, changed, deleted)."""
    key, cols, ddl = TABLES[table]
    wh.execute("CREATE SCHEMA IF NOT EXISTS raw")
    wh.execute(f"CREATE TABLE IF NOT EXISTS raw.{table} ({ddl})")
    if table == "order_items" and BROKEN_ITEMS_SYNC:
        return (0, 0, 0) # the simulated bug: load nothing, say nothing
    col_list = ", ".join(cols)
    src = {r[0]: r for r in fetch_rows(f"SELECT {col_list} FROM {table}")}
    tgt = {r[0]: r for r in wh.execute(f"SELECT {col_list} FROM raw.{table}").fetchall()}
    new = [k for k in src if k not in tgt]
    changed = [k for k in src if k in tgt and row_hash(src[k]) != row_hash(tgt[k])]
    deleted = [k for k in tgt if k not in src]
    wh.execute("BEGIN")
    try:
        if new or changed:
            wh.executemany(f"INSERT OR REPLACE INTO raw.{table} VALUES ({', '.join('?' * len(cols))})",
                           [src[k] for k in new + changed])
        for k in deleted:
            wh.execute(f"DELETE FROM raw.{table} WHERE {key} = ?", [k])
        wh.execute("COMMIT")
    except Exception:
        wh.execute("ROLLBACK")
        raise
    return (len(new), len(changed), len(deleted))

```

- `TABLES` (a dictionary near the top of the file) holds, for `orders` and `order_items`, the key column, the columns to copy, and the warehouse table's column definitions.
- `row_hash()` is section 45.5's row fingerprint, computed in Python: the row's values joined with `|`, then `md5`. Both sides are built by the same Python code, so they match whenever the values match.
- `sync_table()` reads the **whole** table from the source and from the warehouse, and sorts every key into new, changed, or deleted, exactly as in section 45.5. Then it applies all of them with an upsert and deletes, inside one transaction. It returns the three counts, so a step can report them.
- The `BROKEN_ITEMS_SYNC` line is the simulated bug: when the switch is on, the order-line load quietly does nothing.

The dispatch file is loaded with the column check from section 45.8, and the day's rows are replaced:

```

def load_dispatch(wh, day):
    """Load exports/dispatch_<day>.csv into raw.dispatch, replacing that day's rows."""
    path = f"exports/dispatch_{day}.csv"
    wh.execute("CREATE SCHEMA IF NOT EXISTS raw")
    wh.execute("CREATE TABLE IF NOT EXISTS raw.dispatch (dispatch_id VARCHAR PRIMARY KEY, order_id INTEGER, "
               "dispatch_date VARCHAR, qty_units VARCHAR, file_day DATE)")
    if not os.path.exists(path):
        return "no file for this day, nothing to load"
    columns = [r[0] for r in wh.execute(f"DESCRIBE SELECT * FROM read_csv('{path}', all_varchar = "
                                     "true)").fetchall()]
    renamed = {c: CONFIRMED_RENAMES.get(c, c) for c in columns} # file name -> expected name
    missing = [c for c in EXPECTED_DISPATCH if c not in renamed.values()]
    if missing:
        raise ValueError(f"schema change in {path}: missing {missing}")
    qty_column = next(c for c, name in renamed.items() if name == "qty_units")
    wh.execute("BEGIN")
    try:
        wh.execute("DELETE FROM raw.dispatch WHERE file_day = ?", [day])
        wh.execute(f"""INSERT INTO raw.dispatch
                       SELECT dispatch_id, order_id, dispatch_date, {qty_column}, CAST(? AS DATE)
                       FROM read_csv('{path}', header = true, all_varchar = true)""", [day])
        wh.execute("COMMIT")
    except Exception:
        wh.execute("ROLLBACK")
        raise

```

```
except Exception:
    wh.execute("ROLLBACK")
    raise
n = wh.execute("SELECT COUNT(*) FROM raw.dispatch WHERE file_day = ?", [day]).fetchone()[0]
return f"{n} rows (the day's rows replaced)"
```

- The function works on **one day's file**, `exports/dispatch_<day>.csv`. A day with no file (a holiday, a weekend) isn't an error: it returns a message saying so.
- `DESCRIBE SELECT * FROM read_csv(...)` lists the file's columns, as in section 45.7. `renamed` maps each file column to the name the load expects, using `CONFIRMED_RENAMES` for any rename someone has confirmed. `EXPECTED_DISPATCH` (near the top of the file) lists the six columns the file must have. If any is missing, the load **stops with a clear error** instead of guessing (section 45.8).
- `qty_column` is the file's name for the quantity column: `qty_units`, or whatever a confirmed rename says.
- The load **replaces** the day's rows: delete that day, then insert the file, in one transaction. Running it again gives the same rows, so it's idempotent. It doesn't skip unchanged files by their hash, as section 45.5's file log did; the file is small, and replacing it is simpler.

The CRM leads come from the practice API, as in section 45.9:

```
def fetch_leads(base="http://127.0.0.1:8045/api/leads",
               token=os.environ.get("CRM_API_TOKEN", "practice-token-45")):
    """Read every lead, page by page. Waits and retries on 429; raises on any other error."""
    headers = {"Authorization": f"Bearer {token}"}
    leads, page = [], 1
    while page is not None:
        resp = requests.get(base, params={"page": page, "page_size": 20}, headers=headers, timeout=10)
        if resp.status_code == 429:
            time.sleep(int(resp.headers.get("Retry-After", "1")))
            continue
        resp.raise_for_status()
        body = resp.json()
        leads.extend(body["data"])
        page = body["next_page"]
    return leads

def load_leads(wh, leads):
    """Replace raw.crm_leads with this list of leads: the same table as Chapter 45."""
    wh.execute("CREATE SCHEMA IF NOT EXISTS raw")
    wh.execute("""CREATE OR REPLACE TABLE raw.crm_leads (
        lead_id INTEGER PRIMARY KEY, created_at TIMESTAMP, company_name VARCHAR,
        email VARCHAR, source VARCHAR, owner_id INTEGER, loaded_for DATE)""")
    wh.executemany("INSERT INTO raw.crm_leads VALUES (?, ?, ?, ?, ?, ?, CURRENT_DATE)",
                  [(l["lead_id"], l["created_at"], l["company_name"], l["email"], l["source"],
                     l["owner_id"])
                   for l in leads])
    return wh.execute("SELECT COUNT(*) FROM raw.crm_leads").fetchone()[0]
```

- `fetch_leads()` is section 45.9's pagination loop. `base` is the API's address, and `headers` carries the token in the `Authorization` header, as in section 45.9. The token defaults to the `CRM_API_TOKEN` environment variable, or the practice token if it isn't set. It waits and tries again when the API answers **429**, and `raise_for_status()` turns any other error, such as a 401, into an exception.
- `load_leads()` replaces the whole `raw.crm_leads` table with the leads it's given, using Chapter 45's table definition. `CURRENT_DATE` fills `loaded_for` with the day the load ran. It returns the number of rows now in the table.

The first cell

With `ingest.py` read, start the notebook:

```
import os, hashlib
import duckdb, requests, dagster as dg
from dotenv import load_dotenv

load_dotenv()
from reset_ch46 import reset
from apply_day import apply_day
from mock_crm_api import start_server
import ingest

STEP_ORDER = ["raw_orders", "raw_order_items", "raw_dispatch", "raw_crm_leads",
              "daily_flash", "flash_matches_erp", "deliver_flash"]
RUN_LOG = []

def log(step, message):
    RUN_LOG.append((step, message))

def show_log():
    """Print this run's messages in pipeline order, then clear them."""
    for step, message in sorted(RUN_LOG, key=lambda e: STEP_ORDER.index(e[0])):
        print(message)
    RUN_LOG.clear()

reset()
apply_day(1) # the ERP's business day of 2 January
wh = duckdb.connect("warehouse/riverstone_wh.duckdb")
api = start_server()
print("environment ready")
```

```
environment ready
```

How it works, line by line.

- The imports are Chapter 45's, plus `dagster`, imported under the short name `dg`, as its documentation does. `requests` is there for its error type, which section 46.6 uses. `load_dotenv()` reads your `.env` file, if there is one, before the companion files read `RIVERSTONE_SOURCE`.
- The new part is a small **run log**. Each step records a message with `log()`, and `show_log()` prints them in pipeline order. Steps that don't depend on each other can run in any order, so printing messages as they happen would show them in a different sequence each time. `sorted(..., key=lambda e: STEP_ORDER.index(e[0]))` sorts the messages by their step's position in `STEP_ORDER` (the `lambda` from Chapter 18). Dagster keeps its own detailed event log too; this one keeps the book's outputs short.
- `reset()` rebuilds `riverstone_source` and empties the four folders. `apply_day(1)` then plays the ERP's business day of 2 January, as in Chapter 45: two new orders and two status changes. The story starts on the **morning of 3 January**, with 2 January's orders in the ERP and nothing yet in the warehouse.
- `start_server()` starts the practice CRM API in the background, and `api` keeps hold of it so the last cell of section 46.7 can stop it.

Tool note. Outputs in this chapter were produced with Python 3.14.7, Dagster 1.13.23, DuckDB 1.5.6, psycopg2 2.9.13, requests 2.33.1, and PostgreSQL 16. Dagster's API changes between versions; if a name doesn't exist in your version, check its documentation for the current equivalent.

A first asset

A Dagster asset is an ordinary Python function with `@dg.asset` on the line above it. That line is a **decorator** (Chapter 29, section 29.5): it hands the function to Dagster, which from then on runs it for you and records the result. The function's name becomes the asset's name. Start with the two ERP tables:

```
@dg.asset
def raw_orders():
    new, changed, deleted = ingest.sync_table(wh, "orders")
    log("raw_orders", f"raw_orders: {new} new, {changed} changed, {deleted} deleted")

@dg.asset
def raw_order_items():
    new, changed, deleted = ingest.sync_table(wh, "order_items")
    log("raw_order_items", f"raw_order_items: {new} new, {changed} changed, {deleted} deleted")

result = dg.materialize([raw_orders, raw_order_items])
show_log()
print("run succeeded:", result.success)
```

```
raw_orders: 177 new, 0 changed, 0 deleted
raw_order_items: 333 new, 0 changed, 0 deleted
run succeeded: True
```

How it works.

- Each function calls `ingest.sync_table()` and logs the three counts it returns.
- `dg.materialize([...])` runs the listed assets once, now, inside this notebook. **Materialize** is Dagster's word for "build this asset". It returns a result object, and `result.success` is `True` only if every step succeeded.

Reading it. The first run loaded all 177 orders and 333 order lines into the empty warehouse: Chapter 45's 175 orders and 330 lines, plus the two orders and three lines of 2 January.

Below your printed lines, the notebook also shows many lines of Dagster's own event log, one or more for each thing that happened in the run: the step started, the asset was stored, the step succeeded. That's Dagster's record of the run, and the book leaves it out. `materialize()` keeps that record only while Python is running; `dagster dev` (at the end of this section) stores every run permanently.

These two assets always bring the whole table up to date, whichever day the pipeline is building: Chapter 45's hash comparison finds every change, whenever it was made. They don't need to know the day.

One partition per day

Some work does belong to one day. The dispatch file is one day's file, and the Flash reports one day's orders. For those, Dagster needs to know which days exist:

```
daily = dg.DailyPartitionsDefinition(start_date="2026-01-01", end_date="2026-01-06",
    timezone="Asia/Kolkata")
print(daily.get_partition_keys())
```

```
['2026-01-01', '2026-01-02', '2026-01-03', '2026-01-04', '2026-01-05']
```

- `DailyPartitionsDefinition` declares **one partition per day**, from `start_date` 1 January to 5 January 2026 (the `end_date`, 6 January, isn't included), **in IST** (`timezone`). A real pipeline would have no end date; this one stops so the practice data stays small.
- `get_partition_keys()` lists the days. Each **partition key** is the day as text, which is what a step receives when it builds that day.

Now the dispatch file, as a daily asset:

```
@dg.asset(partitions_def=daily)
def raw_dispatch(context):
    day = context.partition_key
    try:
        outcome = ingest.load_dispatch(wh, day)
    except ValueError as e:
        log("raw_dispatch", f"[{day}] raw_dispatch: FAILED, {e}")
        raise
    log("raw_dispatch", f"[{day}] raw_dispatch: {outcome}")

result = dg.materialize([raw_dispatch], partition_key="2026-01-02")
show_log()
print("run succeeded:", result.success)
```

```
[2026-01-02] raw_dispatch: 5 rows (the day's rows replaced)
run succeeded: True
```

How it works, line by line.

- `@dg.asset(partitions_def=daily)` is the same decorator, given a setting: this asset is built one day at a time.
- `context` is passed in by Dagster when it runs the function. It has to be the function's first parameter and be called `context`; that name is how Dagster knows to pass it. `context.partition_key` is the day being built, as text: `"2026-01-02"`. The step uses it instead of reading the clock.
- `try / except ValueError as e` catches the schema-change error from `load_dispatch()` and logs it. The bare **raise** on its own line then raises the **same** error again, so Dagster still marks the step as failed. Log it, then re-raise it: catching an error without re-raising would hide the failure.
- `dg.materialize(..., partition_key="2026-01-02")` builds that one day.

Reading it. The 2 January file's 5 rows loaded, replacing anything that was there for that day.

A step that talks to the CRM

The last ingestion asset calls the CRM API, which can drop a connection. First, two practice settings for it:

```
CRM = {"fail_next": False, "token": os.environ.get("CRM_API_TOKEN", "practice-token-45")}
```

CRM is a dictionary of two settings. `fail_next` lets section 46.6 simulate a dropped connection. `token` is the CRM token, read from `CRM_API_TOKEN` with the practice token as the default, as in Chapter 45, section 45.9; section 46.6 swaps in a wrong one. It's a dictionary rather than two plain variables because the step will **change** `fail_next`, and a function can change what's inside a dictionary defined outside it without any extra keyword. (Assigning to a plain outside variable would need `global`.)

Now the step. Dagster can retry it when it fails:

```
@dg.asset(retry_policy=dg.RetryPolicy(max_retries=3, delay=1))
def raw_crm_leads(context):
    attempt = context.retry_number + 1
    try:
        if CRM["fail_next"]:
            CRM["fail_next"] = False
            raise ConnectionError("connection dropped")
        leads = ingest.fetch_leads(token=CRM["token"])
    except Exception as e:
        log("raw_crm_leads", f"raw_crm_leads: FAILED, {e} (attempt {attempt})")
        raise
    n = ingest.load_leads(wh, leads)
    log("raw_crm_leads", f"raw_crm_leads: {n} leads (attempt {attempt})")

result = dg.materialize([raw_crm_leads])
show_log()
print("run succeeded:", result.success)
```

```
raw_crm_leads: 43 leads (attempt 1)
run succeeded: True
```

How it works, line by line.

- `retry_policy=dg.RetryPolicy(max_retries=3, delay=1)` gives this step a **retry policy**: if it raises an error, Dagster runs it again, up to three more times, waiting one second between attempts.
- `context.retry_number` counts the retries, starting from 0 on the first attempt, so `attempt` is 1, 2, 3, or 4.
- The `try` block does the risky part: the simulated drop, then `fetch_leads()`. The `except` logs whatever went wrong and re-raises it, as in `raw_dispatch`, so Dagster sees the failure and can retry.
- `load_leads()` replaces the table and returns the row count.

Reading it. All 43 leads arrived on the first attempt. The practice API's first answer for page 2 is a 429, as in Chapter 45; `fetch_leads()` waited and asked again inside the same attempt, so the step never failed.

The Flash and its check

The Flash is built by one query, written once with gaps for the table names, so the **same** definition runs against the warehouse and against the ERP:

```
FLASH_SQL = """
    SELECT COUNT(DISTINCT o.order_id) AS orders_booked,
           ROUND(COALESCE(SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)), 0), 2)
             AS revenue_booked
    FROM {orders} AS o
    JOIN {items} AS i ON i.order_id = o.order_id
    WHERE o.order_date = {day}
           AND o.status <> 'Cancelled'"""

INSERT_FLASH = ("INSERT INTO mart.daily_flash SELECT CAST(? AS DATE), * FROM ("
                + FLASH_SQL.format(orders="raw.orders", items="raw.order_items", day="?") + ")")
print(INSERT_FLASH)
```

```
INSERT INTO mart.daily_flash SELECT CAST(? AS DATE), * FROM (
    SELECT COUNT(DISTINCT o.order_id) AS orders_booked,
           ROUND(COALESCE(SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)), 0), 2)
             AS revenue_booked
    FROM raw.orders AS o
    JOIN raw.order_items AS i ON i.order_id = o.order_id
    WHERE o.order_date = ?
           AND o.status <> 'Cancelled')
```

How it works, line by line.

- `FLASH_SQL` counts the day's orders and adds up $\text{quantity} \times \text{unit price} \times (1 - \text{discount}\%)$, leaving out cancelled orders. That's **net bookings**: Chapter 3 showed that bookings can be counted with or without cancelled orders, and its KPI table leaves them out; the Flash does the same, as Chapter 20's Flash did. Chapter 20's Flash had four numbers; this one keeps two, so the outputs stay short.
- `ROUND(..., 2)` rounds revenue to paise. Both sides of the check below will run this same query, so both round the same way. Compare numbers the way they'll be reported: section 45.5 made the same point about hashes, where both sides must build the value identically.
- `{orders}`, `{items}`, and `{day}` are gaps that `.format()` fills with text. It fills in **table names**, which placeholders can't do. The day is different: it's a value, so it becomes a `?` placeholder, and the real day is passed separately, as Chapter 45 did.
- `INSERT_FLASH` puts the day in front of the query's two numbers and inserts the row. The printout shows the finished statement: the table names are filled in, and two `?`s wait for values, the date and the `WHERE` day, so it's run with `[day, day]`. `CAST(? AS DATE)` turns the text `"2026-01-02"` into a date.

Now the asset that runs it:

```
@dg.asset(partitions_def=daily, deps=[raw_orders, raw_order_items])
def daily_flash(context):
    day = context.partition_key
    wh.execute("CREATE SCHEMA IF NOT EXISTS mart")
    wh.execute("""CREATE TABLE IF NOT EXISTS mart.daily_flash (
        flash_date DATE, orders_booked INTEGER, revenue_booked DECIMAL(12,2)""")
    wh.execute("BEGIN")
    try:
        wh.execute("DELETE FROM mart.daily_flash WHERE flash_date = ?", [day])
        wh.execute(INSERT_FLASH, [day, day])
        wh.execute("COMMIT")
    except Exception:
        wh.execute("ROLLBACK")
        raise
    n, rev = wh.execute("SELECT orders_booked, revenue_booked FROM mart.daily_flash WHERE flash_date =
        ?",
                        [day]).fetchone()
    log("daily_flash", f"[{day}] daily_flash: orders={n}, revenue=Rs {rev:,.2f}")

result = dg.materialize([daily_flash], partition_key="2026-01-02")
show_log()
```

```
[2026-01-02] daily_flash: orders=2, revenue=Rs 38,710.00
```

How it works, line by line.

- `deps=[raw_orders, raw_order_items]` declares the Flash's **dependencies**: when they're in the same run, it runs only after both have succeeded.
- The day's row is built with **delete-then-insert inside one transaction** (BEGIN, COMMIT, and ROLLBACK if anything fails, as in Chapter 12, section 12.13). Re-running a day replaces that day's row, and a failure halfway leaves the table as it was. Section 46.4 tests both claims.
- The step then reads its own row back and logs it. `{rev:,.2f}` prints the number with commas and two decimals.

Reading it. The 2 January Flash: 2 orders, ₹38,710.00.

Dialect note: rupees in outputs. The code prints Rs rather than the ₹ symbol, because some terminals and email systems don't display ₹ correctly. The prose uses ₹ as elsewhere in the book. Python's `,` format groups digits in thousands, so an amount of a lakh or more prints as, say, Rs 154,840.00; the prose writes the same amount the Indian way, ₹1,54,840.00.

A **check** is a test attached to an asset. This one recomputes the Flash from the ERP and compares:

```
@dg.asset_check(asset=daily_flash, blocking=True)
def flash_matches_erp(context):
    day = context.partition_key
    src_n, src_rev = ingest.fetch_rows(
        FLASH_SQL.format(orders="orders", items="order_items", day="%s"), [day])[0]
    rows = wh.execute("SELECT orders_booked, revenue_booked FROM mart.daily_flash WHERE flash_date =
        ?",
                        [day]).fetchall()
    passed = len(rows) == 1 and rows[0][0] == src_n and rows[0][1] == src_rev
    if len(rows) == 1:
        wh_text = f"orders={rows[0][0]}, revenue=Rs {rows[0][1]:,.2f}"
    else:
        wh_text = f"{len(rows)} rows"
    log("flash_matches_erp", f"[{day}] check flash_matches_erp: {'PASSED' if passed else 'FAILED'} "
        f"(ERP: orders={src_n}, revenue=Rs {src_rev:,.2f} | warehouse:
        {wh_text})")
    return dg.AssetCheckResult(passed=passed)

result = dg.materialize([daily_flash, flash_matches_erp], partition_key="2026-01-02")
show_log()
```

```
[2026-01-02] daily_flash: orders=2, revenue=Rs 38,710.00
[2026-01-02] check flash_matches_erp: PASSED (ERP: orders=2, revenue=Rs 38,710.00 | warehouse: orders=2,
revenue=Rs 38,710.00)
```

How it works, line by line.

- `@dg.asset_check(asset=daily_flash, blocking=True)` attaches the check to the Flash. `blocking=True` means that if the check fails, **nothing downstream of daily_flash runs**.
- The ERP side runs the same `FLASH_SQL` against the source tables, `orders` and `order_items`. `psycpg2` spells the placeholder `%s` instead of `?`, which is why the day gap is filled with `"%s"` here. `[0]` takes the first (and only) row, which holds the two numbers.
- The warehouse side reads the day's rows. The check passes only if there's **exactly one** row and both numbers match.
- `wh_text` describes the warehouse side for the message: the numbers when there's one row, or the row count when there isn't.
- `dg.AssetCheckResult(passed=passed)` reports the result to Dagster.

Reading it. The check recomputed the numbers from the ERP, and they matched.

Delivery

Delivery writes the report to the `outbox` folder, which stands in for the email that Chapter 20 showed how to send. It must never send the same report twice, so it keeps a **delivery log**: a table recording, for every report sent, a hash of its content. First, the report itself:

```
def flash_body(day):
    n, rev = wh.execute("SELECT orders_booked, revenue_booked FROM mart.daily_flash WHERE flash_date =
        ?",
                        [day]).fetchone()
    body = (f"Riverstone Daily Sales Flash - {day}\n"
           f"Orders booked: {n}\n"
           f"Net bookings (excl. cancelled, net of discounts): Rs {rev:,.2f}\n")
    return rev, body

rev, body = flash_body("2026-01-02")
print(body)
print("content hash:", hashlib.sha256(body.encode()).hexdigest()[:16], "...")
```

```
Riverstone Daily Sales Flash - 2026-01-02
Orders booked: 2
Net bookings (excl. cancelled, net of discounts): Rs 38,710.00

content hash: bb49a92871a565ba ...
```

- `flash_body()` reads the day's row and builds the report's text, returning the revenue and the text.
- `hashlib.sha256(body.encode()).hexdigest()` is the content fingerprint from Chapter 45, section 45.5: any change to the report, even one digit, gives a completely different hash. The print shows only its first 16 characters.

Now the delivery step:

```
@dg.asset(partitions_def=daily, deps=[daily_flash])
def deliver_flash(context):
    day = context.partition_key
    rev, body = flash_body(day)
    digest = hashlib.sha256(body.encode()).hexdigest()
    wh.execute("""CREATE TABLE IF NOT EXISTS mart.delivery_log (
        flash_date DATE, delivery_no INTEGER, content_sha VARCHAR,
        revenue DECIMAL(12,2), file_name VARCHAR)""")
    sent = wh.execute("""SELECT content_sha, revenue FROM mart.delivery_log
        WHERE flash_date = ? ORDER BY delivery_no""", [day]).fetchall()
    if digest in [s[0] for s in sent]:
        log("deliver_flash", f"[{day}] deliver_flash: already delivered, skipped")
        return
    if sent:
        old = sent[-1][1]
        body = (f"CORRECTION to the Flash sent earlier for {day}: "
              f"revenue changed from Rs {old:,.2f} to Rs {rev:,.2f}.\n\n" + body)
        name = f"flash_{day}_correction_{len(sent)}.txt"
    else:
        name = f"flash_{day}.txt"
    with open(f"outbox/{name}", "w") as f:
        f.write(body)
    wh.execute("INSERT INTO mart.delivery_log VALUES (?, ?, ?, ?, ?)", [day, len(sent) + 1, digest, rev,
        name])
    log("deliver_flash", f"[{day}] deliver_flash: wrote outbox/{name}")
```

How it works, line by line.

- `deps=[daily_flash]`: delivery runs only after the Flash has been built, and, because the check is blocking, only after the check has passed.
- `digest` is the report's content hash. `mart.delivery_log` has one row per report sent: the day, a delivery number (1 for the first, 2 for the next), the hash, the revenue, and the file name.
- `sent` is the list of earlier deliveries for this day, oldest first. If this exact content was already delivered (`digest in [...]`), the step logs that and stops: `return` ends the function early.
- If something was sent for the day but with **different** content, this is a **correction**. The report gets a first line saying what changed, from the last revenue sent (`sent[-1][1]`, the last delivery's revenue) to the new one, and a file name that says `correction`.
- The step writes the file, records the delivery, and logs it. The step has no output of its own yet; it runs in the first run below.

Declaring the job and its schedule

```
ALL = [raw_orders, raw_order_items, raw_dispatch, raw_crm_leads, daily_flash, flash_matches_erp, deliver_flash]
flash_job = dg.define_asset_job("daily_flash_job", selection=[raw_orders, raw_order_items, raw_dispatch,
                                                             raw_crm_leads, daily_flash, deliver_flash])

flash_schedule = dg.build_schedule_from_partitioned_job(flash_job, hour_of_day=6, minute_of_hour=30)
defs = dg.Definitions(assets=[raw_orders, raw_order_items, raw_dispatch, raw_crm_leads, daily_flash,
                               deliver_flash],
                      asset_checks=[flash_matches_erp], jobs=[flash_job], schedules=[flash_schedule])
schedule = defs.resolve_schedule_def("daily_flash_job_schedule")
print("job      :", flash_job.name)
print("schedule:", schedule.cron_schedule, schedule.execution_timezone)
```

```
job      : daily_flash_job
schedule: 30 6 * * * Asia/Kolkata
```

How it works, line by line.

- `ALL` lists every step, for running the whole pipeline in this notebook.
- A **job** is a named selection of assets that run together. `define_asset_job` builds one. Checks on the selected assets come along automatically, so the list doesn't name the check, and the job takes its partitions from the assets.
- `build_schedule_from_partitioned_job` creates a schedule for the job: every day at 6:30 (`hour_of_day=6, minute_of_hour=30`), in the time zone of the partitions, `Asia/Kolkata`.
- `Definitions` bundles everything Dagster needs to know about: the assets, the checks (`asset_checks`), the jobs, and the schedules. Dagster checks the graph when it's built, so a missing dependency or a duplicate name fails here instead of at 6:30 a.m.
- `resolve_schedule_def()` fetches the finished schedule by its name, which Dagster made from the job's name plus `_schedule`.

Which day does the 6:30 run build? Before you run the next cell, predict: on 3 January at 6:30, which partition will the schedule ask for?

```

from datetime import datetime
from zoneinfo import ZoneInfo

tick = datetime(2026, 1, 3, 6, 30, tzinfo=ZoneInfo("Asia/Kolkata"))
tick_context = dg.build_schedule_context(scheduled_execution_time=tick,
    repository_def=defs.get_repository_def())
print([r.partition_key for r in schedule.evaluate_tick(tick_context).run_requests])

```

```
['2026-01-02']
```

- `tick` is 6:30 a.m. on 3 January, India time. `tzinfo=ZoneInfo("Asia/Kolkata")` attaches the time zone, as in Chapter 20, so Python knows which 6:30 is meant.
- `build_schedule_context` builds the information the daemon would hand the schedule at that moment: the time of the tick (`scheduled_execution_time`) and the definitions it belongs to (`repository_def`, which `defs.get_repository_def()` supplies), and `evaluate_tick` asks the schedule what it would run. `run_requests` is the list of runs it asks for; the code prints each one's partition.

Reading it. The 6:30 run on 3 January builds **2 January**, the last complete day, not 3 January. That's what a morning Flash about yesterday needs: 3 January has barely started. Manual runs, like the ones in this chapter, can build any day you name.

The first run

It's the morning of 3 January, and the ERP has had its 2 January business day. Run the whole pipeline for 2 January:

```

result = dg.materialize(ALL, partition_key="2026-01-02", raise_on_error=False)
show_log()
print("run succeeded:", result.success)
print()
print(open("outbox/flash_2026-01-02.txt").read())

```

```

raw_orders: 0 new, 0 changed, 0 deleted
raw_order_items: 0 new, 0 changed, 0 deleted
[2026-01-02] raw_dispatch: 5 rows (the day's rows replaced)
raw_crm_leads: 43 leads (attempt 1)
[2026-01-02] daily_flash: orders=2, revenue=Rs 38,710.00
[2026-01-02] check_flash_matches_erp: PASSED (ERP: orders=2, revenue=Rs 38,710.00 | warehouse: orders=2,
    revenue=Rs 38,710.00)
[2026-01-02] deliver_flash: wrote outbox/flash_2026-01-02.txt
run succeeded: True

Riverstone Daily Sales Flash - 2026-01-02
Orders booked: 2
Net bookings (excl. cancelled, net of discounts): Rs 38,710.00

```

- `raise_on_error=False` tells `materialize()` to return the result even when a step fails, instead of stopping the notebook with an error. The result's `success` then says what happened. Every run from here on uses it.

Reading it. The steps ran in dependency order. The ingestion steps found nothing new, because you loaded everything while building them. The dispatch file's rows were replaced, and the leads reloaded. The Flash shows 2 orders booked for ₹38,710.00. The check recomputed the same numbers from the ERP and they matched, so delivery ran and wrote the report.

Notice the dates in the log: the lines that start with [2026-01-02] belong to one day's partition. The ingestion lines have no date, because those steps always bring whole tables up to date.

Running it again

What happens if the scheduler runs the same day twice, or someone presses "run" again?

```
result = dg.materialize(ALL, partition_key="2026-01-02", raise_on_error=False)
show_log()
print("run succeeded:", result.success)
print("flash rows for the day:", wh.execute("SELECT COUNT(*) FROM mart.daily_flash WHERE flash_date = '2026-01-02'").fetchone()[0])
print("files in outbox:", sorted(os.listdir("outbox")))
```

```
raw_orders: 0 new, 0 changed, 0 deleted
raw_order_items: 0 new, 0 changed, 0 deleted
[2026-01-02] raw_dispatch: 5 rows (the day's rows replaced)
raw_crm_leads: 43 leads (attempt 1)
[2026-01-02] daily_flash: orders=2, revenue=Rs 38,710.00
[2026-01-02] check_flash_matches_erp: PASSED (ERP: orders=2, revenue=Rs 38,710.00 | warehouse: orders=2, revenue=Rs 38,710.00)
[2026-01-02] deliver_flash: already delivered, skipped
run succeeded: True
flash rows for the day: 1
files in outbox: ['flash_2026-01-02.txt']
```

Nothing changed: no rows loaded, one Flash row for the day, and delivery recognized that exactly this report was already sent. That's the behavior you want from every scheduled pipeline, and it isn't an accident. The next section shows what it takes.

Running it in Dagster's web interface

`materialize()` is how you build and test a pipeline in a notebook. To see the graph, turn the schedule on, and keep a permanent history of runs, Dagster runs the same definitions from a **file**, with `dagster dev`. The practice folder has that file, `riverstone_pipeline.py`. It holds the same assets, check, job, and schedule, without the practice switches (CRM, CRASH), and with four differences from the notebook:

```
WAREHOUSE = "warehouse/riverstone_wh.duckdb"

def warehouse():
    """Open the warehouse for one step; each step closes it again when it finishes."""
    return duckdb.connect(WAREHOUSE)

@dg.asset
def raw_orders(context):
    with warehouse() as wh:
        new, changed, deleted = ingest.sync_table(wh, "orders")
        context.log.info(f"raw_orders: {new} new, {changed} changed, {deleted} deleted")

# ... the other assets, the check, and the job follow the same pattern ...

defs = dg.Definitions(assets=[raw_orders, raw_order_items, raw_dispatch, raw_crm_leads, daily_flash, deliver_flash],
                      asset_checks=[flash_matches_erp], jobs=[flash_job], schedules=[flash_schedule],
                      sensors=[dispatch_file_sensor, flash_failure_alert],
                      executor=dg.in_process_executor) # one step at a time: DuckDB allows one writer
```

- **Nothing runs when the file is loaded.** No `reset()`, no `apply_day()`, no API server: `dagster dev` loads the file to read the definitions, and it may load it more than once.
- **Each step opens the warehouse itself** and closes it at the end (with `warehouse()` as `wh:`). `dagster dev` runs the pipeline in its own Python process, not in your notebook, so there's no shared `wh` to borrow.
- **Steps run one at a time.** Left to itself, Dagster runs independent steps side by side, each in its own process. DuckDB lets only one process write to a file at a time, so two steps opening the warehouse together would fail with "Could not set lock on file". `executor=dg.in_process_executor` tells Dagster to run the steps one after another, in one process.
- **Steps log with `context.log.info()`**, Dagster's own log, instead of the notebook's `RUN_LOG`. The messages appear on each run's page.
- `Definitions` also lists the two sensors from section 46.6.

To run it:

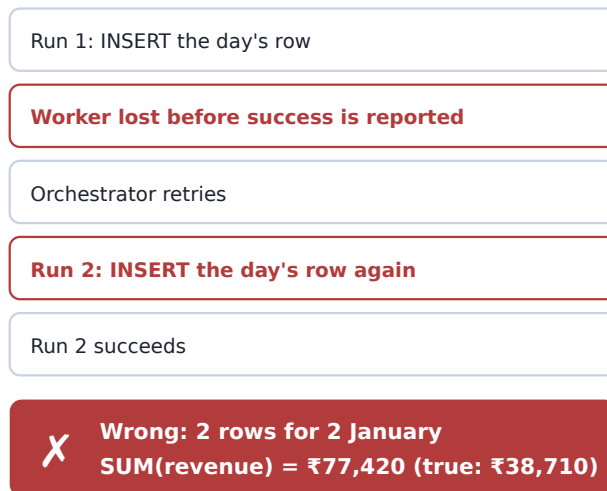
1. Shut down the notebook's kernel, so it releases the warehouse file and port 8045.
2. In one terminal, in `work/ch46` with the environment active, start the practice API:
`python mock_crm_api.py`.
3. In a second terminal, same folder: `dagster dev -f riverstone_pipeline.py`. When it's ready, it prints a line saying `Serving dagster-webserver on http://127.0.0.1:3000`. Leave both terminals running.
4. Open `http://127.0.0.1:3000` in your browser. **Assets** shows the graph, as in Figure 46.1. Choose **Materialize all**; for the daily assets, Dagster asks which partition to build: pick 2026-01-02.
5. Open **Automation** (the schedules and sensors page). Schedules and sensors start **stopped**, so nothing runs by surprise; switch `daily_flash_job_schedule` on to start it, and the two sensors of section 46.6 the same way. The practice partitions end on 5 January, so it won't find a new day to build; in a real pipeline, with no end date, it would run every morning at 6:30.

`dagster dev` starts two programs: the web server that draws the page, and the **daemon** (section 46.2) that fires schedules and sensors. It also keeps every run's history in a folder on your computer, so the runs are still there after you restart. Press Ctrl+C in each terminal to stop.

46.4 Idempotency, proven with a failure test

Chapter 45 defined an **idempotent** step as one you can run again with the same result. In a pipeline, the question that matters is sharper: **if a step fails halfway and is retried, is the result still correct?** The only honest way to answer is to make it fail on purpose.

Naive step: append



Safe step: overwrite in a transaction

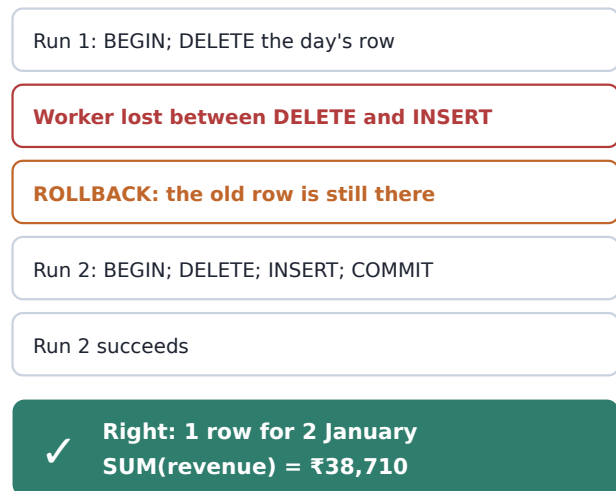


Figure 46.2 – The same kind of failure, two designs. The naive step's retry doubles the day; the safe step's retry is harmless.

A step that isn't idempotent

Here's a common first version of a report step: it **appends** the day's numbers. To test it, the step crashes *after* writing, as happens when a machine is lost or a network call times out after the database has already committed.

```
CRASH = {"now": False}

@dg.asset(partitions_def=daily, deps=[raw_orders, raw_order_items])
def flash_naive(context):
    day = context.partition_key
    wh.execute("""CREATE TABLE IF NOT EXISTS mart.flash_naive (
        flash_date DATE, orders_booked INTEGER, revenue_booked DECIMAL(12,2)""")
    wh.execute(INSERT_FLASH.replace("mart.daily_flash", "mart.flash_naive"), [day, day])
    if CRASH["now"]:
        raise RuntimeError("worker lost after writing (simulated)")

CRASH["now"] = True
first = dg.materialize([flash_naive], partition_key="2026-01-02", raise_on_error=False)
CRASH["now"] = False
second = dg.materialize([flash_naive], partition_key="2026-01-02", raise_on_error=False)
print("first run succeeded :", first.success)
print("second run succeeded:", second.success)
print(wh.execute("""SELECT COUNT(*) AS rows_for_day, SUM(revenue_booked) AS revenue_reported
    FROM mart.flash_naive WHERE flash_date = '2026-01-02'""").fetchall())
```

```
first run succeeded : False
second run succeeded: True
[(2, Decimal('77420.00'))]
```

How it works. `flash_naive` inserts the day's row into its own table, `mart.flash_naive`, with the same insert as the Flash (`.replace()` swaps the table name in the SQL text). There's no delete first and no transaction. `CRASH` is a switch like `CRM`: when it's on, the step raises an error just after the insert. The cell runs the step once with the crash, then once without it, the way a scheduler's retry would.

Reading it. The first run failed, so the retry (here, a second run) did its job and succeeded. But the table now has **two rows** for 2 January, and anything that sums revenue reports **₹77,420.00**, exactly double the true ₹38,710.00. Nothing looks broken until someone compares the report with the ERP.

Try it. In `flash_naive`, move the two lines `if CRASH["now"]:` and `raise ...` above the `INSERT`, re-run the cell that defines it, and run the test again. Does the naive step still double the revenue? What does that tell you about when a non-idempotent step is dangerous?

Delete, then insert: the gap in the middle

The Flash in section 46.3 deletes the day's row and inserts it again, which can't double anything. But there's a moment between the delete and the insert when the day has **no row at all**. First, a small helper to look at the table between steps:

```
def rows_for(day):
    return wh.execute("SELECT COUNT(*), SUM(revenue_booked) FROM mart.daily_flash WHERE flash_date = ?",
                      [day]).fetchall()

print(rows_for("2026-01-02"))
```

```
[(1, Decimal('38710.00'))]
```

`rows_for()` returns the number of Flash rows for a day and their total revenue: right now, one row and ₹38,710.00. Now crash between the delete and the insert, without a transaction:

```
def build_flash_no_transaction(day, crash=False):
    wh.execute("DELETE FROM mart.daily_flash WHERE flash_date = ?", [day])
    if crash:
        raise RuntimeError("worker lost between DELETE and INSERT (simulated)")
    wh.execute(INSERT_FLASH, [day, day])

try:
    build_flash_no_transaction("2026-01-02", crash=True)
except RuntimeError as e:
    print("run failed:", e)
print("after the crash:", rows_for("2026-01-02"))
build_flash_no_transaction("2026-01-02")
print("after the retry:", rows_for("2026-01-02"))
```

```
run failed: worker lost between DELETE and INSERT (simulated)
after the crash: [(0, None)]
after the retry: [(1, Decimal('38710.00'))]
```

`build_flash_no_transaction()` is the Flash's delete-then-insert with no `BEGIN` or `COMMIT`: each statement is kept as soon as it runs. With `crash=True`, it stops between the two.

Reading it. After the crash, 2 January has **0 rows**, and `SUM` of nothing is `None`. If a dashboard refreshed at 6:31, it would show an empty day. The retry put the row back, but only because someone retried.

The same step, made safe

Now the design from section 46.3: the delete and the insert inside one transaction. The same crash, at the same point:

```
def build_flash_safely(day, crash=False):
    wh.execute("BEGIN")
    try:
        wh.execute("DELETE FROM mart.daily_flash WHERE flash_date = ?", [day])
        if crash:
            raise RuntimeError("worker lost between DELETE and INSERT (simulated)")
        wh.execute(INSERT_FLASH, [day, day])
        wh.execute("COMMIT")
    except Exception as e:
        wh.execute("ROLLBACK")
        print("run failed and rolled back:", e)

build_flash_safely("2026-01-02", crash=True)
print("after the crash:", rows_for("2026-01-02"))
build_flash_safely("2026-01-02")
print("after the retry:", rows_for("2026-01-02"))
```

```
run failed and rolled back: worker lost between DELETE and INSERT (simulated)
after the crash: [(1, Decimal('38710.00'))]
after the retry: [(1, Decimal('38710.00'))]
```

Reading it. When the step crashed, the transaction **rolled back**: the delete was undone, so the old row was still there, and anyone reading the table at that moment saw the right number. The retry then committed. There's one row, with the correct ₹38,710.00, before and after.

How it works. A **transaction** groups statements so they succeed or fail together (Chapter 12, section 12.13). `BEGIN` starts one; `COMMIT` makes it permanent; `ROLLBACK` discards everything since `BEGIN`. The combination of **replace the whole partition** and **one transaction** is the standard pattern for idempotent pipeline steps. Chapter 49 covers how warehouses guarantee it, under the name ACID.

Patterns for idempotent steps

Pattern	How it works	Use it for
Partition overwrite	Delete (or overwrite) the partition, then insert, in one transaction	Daily reports, aggregates, anything built per day
Upsert by key	Insert new keys, update existing ones (Chapter 45)	Raw tables copied from sources
Write to a new location, then swap	Build the result as a new table or file; replace the old one in a single step when complete	Large rebuilds, where a half-built table must never be visible
Delivery log	Record what was sent (with a content hash); skip exact repeats	Emails, messages, API calls, CRM updates
Idempotency keys	Send a unique key with each external request so the receiver ignores duplicates	Payments and external APIs that support it (Chapter 51)

Watch out: side effects outside the database can't be rolled back. A transaction can undo a table write. It can't un-send an email, un-post a message, or un-call an API. That's why `deliver_flash` checks a delivery log first, and why delivery should always be the *last* step, after every check has passed.

46.5 Partitions and backfills

A **backfill** runs a pipeline for past partitions: to build history for a new pipeline, to rebuild after fixing a bug, or to fill a gap after an outage.

Backfills bring one important rule: **rebuild history, don't re-deliver it**. Nobody wants five old Sales Flash emails arriving at once. So the backfill below leaves out delivery. It leaves out the ingestion steps too: they bring whole tables up to date, so running them once is enough, and running them again for every day would only re-read the ERP five times.

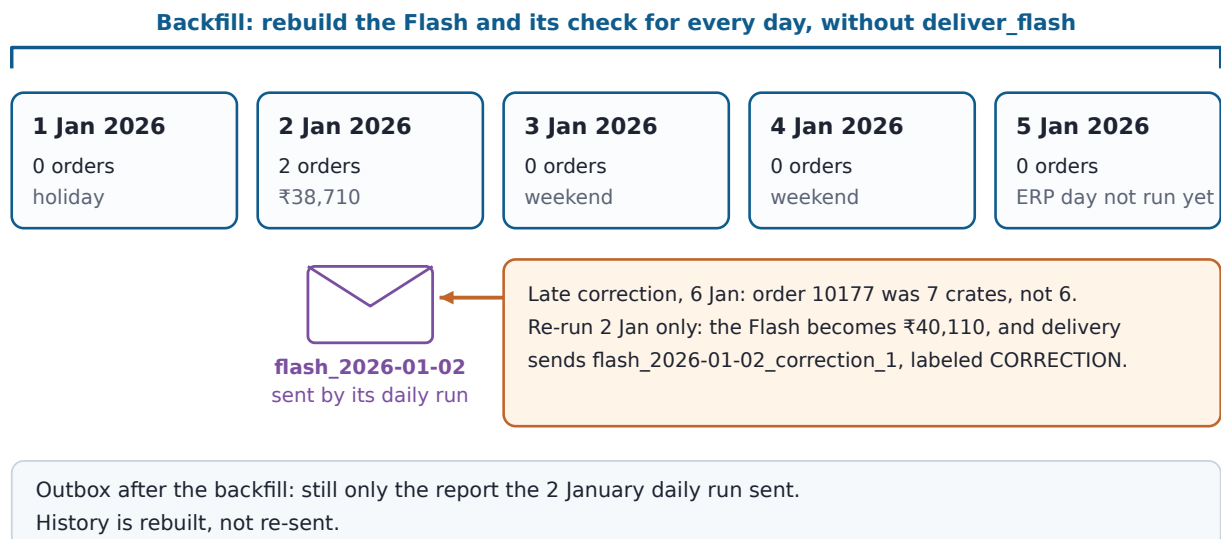


Figure 46.3 – Partitions, a backfill, and a late correction. The backfill fills in history quietly; only real changes to already-delivered numbers produce a new message.

```
BACKFILL = [daily_flash, flash_matches_erp]          # rebuild history, don't email it

for day in daily.get_partition_keys():
    r = dg.materialize(BACKFILL, partition_key=day, raise_on_error=False)
    print(day, "ok" if r.success else "FAILED")
RUN_LOG.clear()
for row in wh.execute("SELECT flash_date, orders_booked, revenue_booked FROM mart.daily_flash ORDER BY
    flash_date").fetchall():
    print(row)
print("files in outbox:", sorted(os.listdir("outbox")))
```

```

2026-01-01 ok
2026-01-02 ok
2026-01-03 ok
2026-01-04 ok
2026-01-05 ok
(datetime.date(2026, 1, 1), 0, Decimal('0.00'))
(datetime.date(2026, 1, 2), 2, Decimal('38710.00'))
(datetime.date(2026, 1, 3), 0, Decimal('0.00'))
(datetime.date(2026, 1, 4), 0, Decimal('0.00'))
(datetime.date(2026, 1, 5), 0, Decimal('0.00'))
files in outbox: ['flash_2026-01-02.txt']

```

How it works. BACKFILL is the Flash and its check, nothing else. The loop builds each day in turn and prints whether that day's run succeeded: a backfill is judged day by day, like any other run.

`RUN_LOG.clear()` throws away the steps' messages, which the per-day lines summarize. In a deployed Dagster instance, you'd select a date range in the web interface, or run `dagster job backfill` from the command line, and Dagster would queue the runs and track each one.

Reading it. Every day's run succeeded, and the Flash now has a row for every day from 1 to 5 January. Three days have no bookings in the practice data: 1 January, a holiday, and the weekend of 3–4 January. 5 January also shows 0, because in the story it's still the morning of 3 January: the ERP's 5 January business day hasn't happened yet (it happens in section 46.7), and the check agreed with the ERP's 0. No new file appeared in the outbox: the only report is the one the 2 January daily run delivered.

Backfill etiquette

- **Check that every step is idempotent first.** A backfill runs every step again for every day. A non-idempotent step turns a one-day bug into a month of doubled numbers.
- **Exclude side effects:** emails, CRM updates, messages, API writes. Run them only for the days that truly need re-sending, with a clear "correction" label.
- **Mind the source.** Thirty days of backfill can mean thirty full reads of the ERP. Run large backfills outside business hours, limit how many partitions run at once, and tell the source system's owner.
- **Keep days independent.** A per-day step that reads "the latest" data instead of "the partition's" data gives every backfilled day today's numbers.
- **Record why.** Note who ran the backfill, which dates, and why, next to the run.

Late-arriving data

Data doesn't always arrive on the day it describes. On 6 January, the sales team discovers that order 10177, booked on 2 January, was for 7 industrial crates, not 6, and corrects it in the ERP. The 2 January Flash is now wrong.

First, find the order line to correct:

```

print(ingest.fetch_rows("SELECT order_item_id, order_id, product_id, quantity, unit_price "
                        "FROM order_items WHERE order_id = %s", [10177]))

```

```

[(333, 10177, 105, 6, Decimal('1400.00'))]

```


Order 10177 has one line, `order_item_id` 333: 6 crates at ₹1,400. Correct it in the ERP, then re-run that one day:

```
ingest.run_sql("UPDATE order_items SET quantity = %s WHERE order_item_id = %s", [7, 333])

result = dg.materialize(ALL, partition_key="2026-01-02", raise_on_error=False)
show_log()
print()
print(open("outbox/flash_2026-01-02_correction_1.txt").read())
```

```
raw_orders: 0 new, 0 changed, 0 deleted
raw_order_items: 0 new, 1 changed, 0 deleted
[2026-01-02] raw_dispatch: 5 rows (the day's rows replaced)
raw_crm_leads: 43 leads (attempt 1)
[2026-01-02] daily_flash: orders=2, revenue=Rs 40,110.00
[2026-01-02] check_flash_matches_erp: PASSED (ERP: orders=2, revenue=Rs 40,110.00 | warehouse: orders=2,
revenue=Rs 40,110.00)
[2026-01-02] deliver_flash: wrote outbox/flash_2026-01-02_correction_1.txt

CORRECTION to the Flash sent earlier for 2026-01-02: revenue changed from Rs 38,710.00 to Rs 40,110.00.

Riverstone Daily Sales Flash - 2026-01-02
Orders booked: 2
Net bookings (excl. cancelled, net of discounts): Rs 40,110.00
```

Reading it. The order-line load found one changed row. The Flash for 2 January was rebuilt at ₹40,110.00, ₹1,400.00 more than before: one more crate at ₹1,400. The check passed against the ERP. Delivery saw that the content differed from what was sent on 2 January, so it wrote a separate, clearly named correction, whose first line says what changed, instead of silently replacing the original.

Deciding how far back to look. A pipeline can't re-run every past day every morning. Common approaches: re-run a fixed **look-back window** each day (for example, the last 7 days), which catches most late changes automatically; re-run specific days when a change is detected (for example, with the hash comparison from Chapter 45 grouped by order date); or accept that some numbers are final after a closing date, as finance teams do with month-end.

46.6 Retries, timeouts, and alerting

Retries: for problems that go away

Some failures are **transient**: a dropped connection, a busy API, a database restarting. Trying again a little later usually works. That's what the retry policy on `raw_crm_leads` is for. Simulate a dropped connection:

```
CRM["fail_next"] = True
result = dg.materialize([raw_crm_leads], raise_on_error=False)
show_log()
print("run succeeded:", result.success)
```

```
raw_crm_leads: FAILED, connection dropped (attempt 1)
raw_crm_leads: 43 leads (attempt 2)
run succeeded: True
```

Reading it. The first attempt failed with a dropped connection. Dagster waited one second, retried, and the second attempt loaded all 43 leads. The run succeeded without anyone being woken up.

Retries have a cost, and a trap. The trap first: a retry policy retries **every** error, including ones that will never go away. Try a wrong token, which the CRM will refuse every time:

```
CRM["token"] = "expired-token"
result = dg.materialize([raw_crm_leads], raise_on_error=False)
show_log()
print("run succeeded:", result.success)
```

```
raw_crm_leads: FAILED, 401 Client Error: Unauthorized for url: http://127.0.0.1:8045/api/leads?
page=1&page_size=20 (attempt 1)
raw_crm_leads: FAILED, 401 Client Error: Unauthorized for url: http://127.0.0.1:8045/api/leads?
page=1&page_size=20 (attempt 2)
raw_crm_leads: FAILED, 401 Client Error: Unauthorized for url: http://127.0.0.1:8045/api/leads?
page=1&page_size=20 (attempt 3)
raw_crm_leads: FAILED, 401 Client Error: Unauthorized for url: http://127.0.0.1:8045/api/leads?
page=1&page_size=20 (attempt 4)
run succeeded: False
```

Reading it. Four attempts, four identical 401s, three seconds of waiting, and the same failure at the end. Here that's only wasted time; against a real system, repeated wrong logins can lock the account.

Chapter 45's rule applies to whole steps too: **retry what might succeed next time, and fail fast on what won't**. Replace the asset with a version that tells Dagster not to retry a refusal:

```
@dg.asset(retry_policy=dg.RetryPolicy(max_retries=3, delay=1))
def raw_crm_leads(context):
    attempt = context.retry_number + 1
    try:
        if CRM["fail_next"]:
            CRM["fail_next"] = False
            raise ConnectionError("connection dropped")
        leads = ingest.fetch_leads(token=CRM["token"])
    except requests.HTTPError as e:
        code = e.response.status_code
        log("raw_crm_leads", f"raw_crm_leads: FAILED, CRM refused the request ({code}) (attempt {attempt})")
        if code in (400, 401, 403, 404):
            raise dg.Failure(description=f"CRM refused the request ({code})", allow_retries=False)
        raise
    except Exception as e:
        log("raw_crm_leads", f"raw_crm_leads: FAILED, {e} (attempt {attempt})")
        raise
    n = ingest.load_leads(wh, leads)
    log("raw_crm_leads", f"raw_crm_leads: {n} leads (attempt {attempt})")

ALL = [raw_orders, raw_order_items, raw_dispatch, raw_crm_leads, daily_flash, flash_matches_erp,
       deliver_flash]
result = dg.materialize([raw_crm_leads], raise_on_error=False)
show_log()
print("run succeeded:", result.success)
```

```
raw_crm_leads: FAILED, CRM refused the request (401) (attempt 1)
run succeeded: False
```

How it works, line by line.

- The first `except` catches only `requests.HTTPError`, the error `raise_for_status()` raises when the API answers with an error code. `e.response.status_code` is that code.
- For 400, 401, 403, and 404, which mean "your request is wrong" rather than "try later", it raises `dg.Failure` with `allow_retries=False`: Dagster marks the step failed and **skips the retry policy**. Any other HTTP error (a 503, say) is re-raised with a bare `raise`, so it's retried.
- The second `except` handles everything else, such as the dropped connection, exactly as before.
- Defining `raw_crm_leads` again replaces the old version in the notebook, so the cell also rebuilds ALL, which still held the old one.

Reading it. One attempt, no retries, a clear message.

Try it. What happens if you change `max_retries=3` to `max_retries=0` in the cell above, re-run it, and then run the dropped-connection cell again? Predict the output first, then run it, and put the setting back afterwards.

Put the right token back before going on:

```
CRM["token"] = os.environ.get("CRM_API_TOKEN", "practice-token-45")
```

Three more things to know about retries:

- **Retries hide bugs.** While building this chapter's pipeline, `raw_crm_leads` first failed because it tried to create a table in a schema that didn't exist yet on a brand-new warehouse. The retry succeeded, because another step had created the schema in the meantime. The run looked healthy; the bug was real. Log every retry, and treat frequent retries as a problem to investigate.
- **Retries at two levels multiply.** `fetch_leads()` already retries a 429 inside each attempt, and Dagster retries the whole step up to three more times. A request that keeps failing can be tried many times over before anyone hears about it. Keep the total waiting time bounded, and know which level retries what.
- **Retries need idempotent steps.** A retry is a re-run. Section 46.4 is what makes retries safe.

Timeouts: for problems that never end

A step that hangs (waiting forever on a network call, or a query that locks) is worse than one that fails, because nothing downstream runs and no alert fires. Set timeouts at every level: on network calls (Chapter 45), on database statements, and on whole runs, so a run that takes far longer than usual is stopped and reported. Dagster supports run-level timeouts through tags such as `dagster/max_runtime` in a deployed instance; check your version's documentation.

A sensor that watches for the dispatch file

Section 46.2 described a **sensor**: a function the daemon calls every few minutes to see whether something has happened. Bhiwandi Main's dispatch file arrives when the warehouse team saves it, not at a fixed time. A sensor can start the dispatch load when the file appears:

```
dispatch_job = dg.define_asset_job("dispatch_job", selection=[raw_dispatch])

@dg.sensor(job=dispatch_job, minimum_interval_seconds=300)
def dispatch_file_sensor(context):
    for name in sorted(os.listdir("exports")):
        if name.startswith("dispatch_"):
            yield dg.RunRequest(run_key=name, partition_key=name[9:19])

for request in dispatch_file_sensor(dg.build_sensor_context()):
    print(request.run_key, "->", request.partition_key)
```

```
dispatch_2026-01-02.csv -> 2026-01-02
dispatch_2026-01-05.csv -> 2026-01-05
```

How it works, line by line.

- `dispatch_job` is a job with just the dispatch load.
- `@dg.sensor(job=dispatch_job, minimum_interval_seconds=300)` turns the function into a sensor for that job, called at most every 300 seconds (five minutes).
- The function looks at every file in `exports`. For each dispatch file, it asks for a run with `yield dg.RunRequest(...)`. **yield** hands back one request and carries on with the loop, so one call can ask for several runs: the function is a generator (Chapter 29, section 29.9).
- `partition_key=name[9:19]` cuts the date out of the file name: characters 9 to 18 of `dispatch_2026-01-02.csv` are `2026-01-02`.
- `run_key=name` is the important part. Dagster remembers every run key it has seen and never starts a second run for the same key. The daemon calls the sensor every five minutes, and the file is there every time, but the load runs **once per file**. That's idempotency again, for triggers.
- `dg.build_sensor_context()` builds what the daemon would pass in, so the notebook can call the sensor once and print the runs it asks for, without starting them.

Reading it. The sensor found both practice files and asked for one run each, for the right days. In `riverstone_pipeline.py`, it's registered in `Definitions`, and under `dagster dev` the daemon calls it every five minutes. It's switched off at first, like the schedule.

Alerting: telling someone what to do

An **alert** is a message sent to a person when something needs attention. A good alert is:

- **Actionable:** it says what failed, what the impact is, and what to do next.
- **Rare:** alerts that fire for things nobody needs to act on teach people to ignore alerts.
- **Routed:** it goes to the person or team that owns the fix, not to everyone.
- **Linked:** it points to the run, the logs, and a **runbook**, a short document that explains how to diagnose and fix a known problem.

Section 46.7 shows an alert from a real failed run. In a deployed Dagster instance, alerts are usually sent by a second kind of sensor, a **run failure sensor**, which the daemon calls whenever a run fails. This one is in `riverstone_pipeline.py`:

```
def send_message(to, subject, body):
    """Stand-in for email, Slack, or Teams: write the message to the alerts folder."""
    with open(f"alerts/{subject[:40].replace(' ', '_').replace(':', '')}.txt", "w") as f:
        f.write(f"To: {to}\nSubject: {subject}\n\n{body}\n")

@dg.run_failure_sensor(monitored_jobs=[flash_job])
def flash_failure_alert(context: dg.RunFailureSensorContext):
    run = context.dagster_run
    send_message(to="data-oncall@riverstone.example",
                 subject=f"Daily Sales Flash failed: run {run.run_id[:8]}",
                 body=f"Failed job: {run.job_name}\nError: {context.failure_event.message}\n"
                      f"Runbook: docs/runbooks/daily_flash.md")
```

- `send_message()` is **your** function, not Dagster's: here a three-line stand-in that writes the message to the alerts folder. In production it would send an email (Chapter 20) or a chat message.
- `@dg.run_failure_sensor(monitored_jobs=[flash_job])` is Dagster's: it calls the function whenever a run of `flash_job` fails. `context.dagster_run` is the failed run, and `context.failure_event.message` is Dagster's description of the failure.
- It works only when it's registered in Definitions (`sensors=[...]`, as in the file) and the daemon is running, so it isn't run in the notebook.

Under `dagster dev`, with this sensor switched on and the practice API **not** started, a run of `daily_flash_job` for 2 January failed at `raw_crm_leads` (after its retries), and shortly afterwards the sensor wrote this file to `alerts/`. The run ID is different every time:

```
To: data-oncall@riverstone.example
Subject: Daily Sales Flash failed: run da07a425

Failed job: daily_flash_job
Error: Execution of run for "daily_flash_job" failed. Steps failed: ['raw_crm_leads'].
Runbook: docs/runbooks/daily_flash.md
```

The same run still delivered the 2 January Flash, because the Flash doesn't depend on the CRM: the run failed, but only the part of it that needed the CRM.

46.7 Delivery only after the data passes its checks

This is the most important design rule in this chapter: **nothing leaves the pipeline until the data has passed its checks**. Reports, emails, dashboard refreshes that people act on, and syncs into the CRM all come *after* the checks, and depend on them.

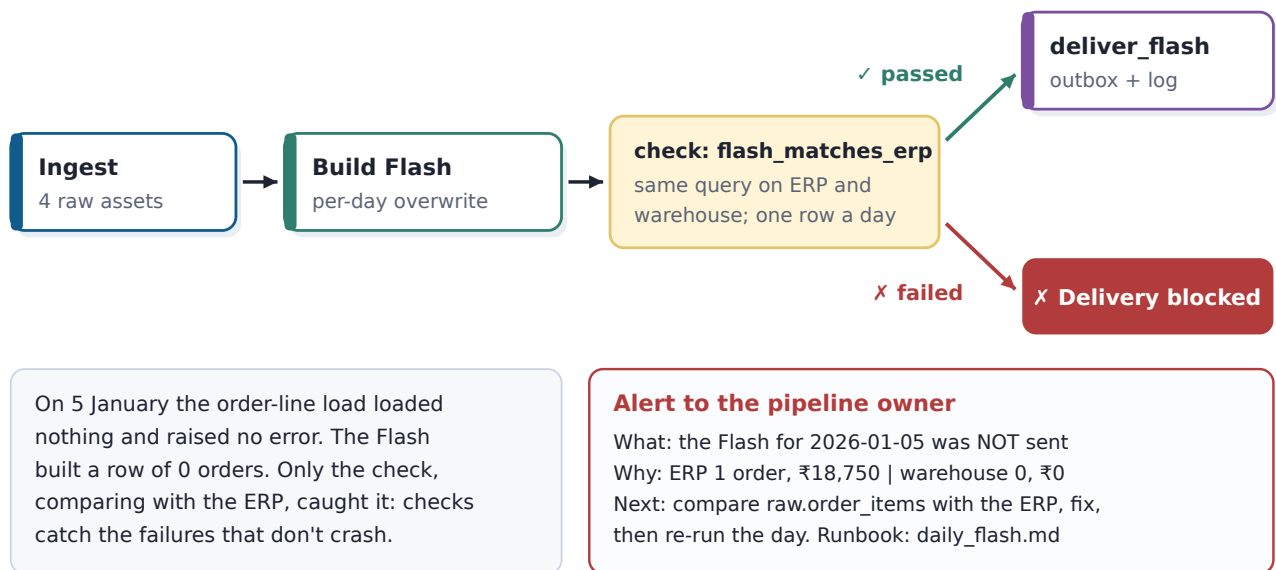


Figure 46.4 – Checks gate delivery. When a check fails, the report isn't sent, and the owner gets an alert that says what to do.

A bad morning

It's the morning of 6 January. The ERP has had its business day of 5 January (`apply_day(2)`), and two things go wrong at once: a bug in the order-line load means it loads nothing, and Bhiwandi Main has changed the columns in its dispatch file (Chapter 45, section 45.8).

```

apply_day(2)
ingest.BROKEN_ITEMS_SYNC = True          # simulate a faulty order-line load

result = dg.materialize(ALL, partition_key="2026-01-05", raise_on_error=False)
show_log()
print("run succeeded:", result.success)
print("files in outbox:", sorted(os.listdir("outbox")))

```

```

raw_orders: 1 new, 1 changed, 0 deleted
raw_order_items: 0 new, 0 changed, 0 deleted
[2026-01-05] raw_dispatch: FAILED, schema change in exports/dispatch_2026-01-05.csv: missing ['qty_units']
raw_crm_leads: 43 leads (attempt 1)
[2026-01-05] daily_flash: orders=0, revenue=Rs 0.00
[2026-01-05] check flash_matches_erp: FAILED (ERP: orders=1, revenue=Rs 18,750.00 | warehouse: orders=0,
revenue=Rs 0.00)
run succeeded: False
files in outbox: ['flash_2026-01-02.txt', 'flash_2026-01-02_correction_1.txt']

```

Reading it, step by step.

- `raw_orders` loaded the new order 10178 and the change to order 10176, correctly.
- `raw_order_items` loaded **nothing**: the simulated bug. Nothing in its own output says it's wrong: "0 new" is a normal message on a quiet day.
- `raw_dispatch` **failed loudly** with a clear message: the expected `qty_units` column is missing. That's the column check from Chapter 45 doing its job.
- `raw_crm_leads` succeeded.
- `daily_flash` **ran**, because both of its dependencies succeeded, and built a row with 0 orders and ₹0.00, because the new order had no lines in the warehouse.
- The check **failed**: the ERP says 1 order for ₹18,750.00; the warehouse says 0.
- `deliver_flash` **didn't run**. The outbox still holds only the 2 January reports. No manager received a Flash saying Riverstone sold nothing on 5 January.

Notice two things. First, the dispatch failure didn't stop the Flash, because the Flash doesn't depend on dispatches: the dependency graph limited the damage. Second, the order-line bug didn't raise any error at all. Only the check, comparing the result against the ERP, caught it. **Checks catch the failures that don't crash.**

Watch out: a blocked delivery isn't a clean table. The check stopped the email, but `mart.daily_flash` now holds a wrong row for 5 January, and anyone querying that table directly would see it. Chapter 47 covers the **write-audit-publish** pattern, which builds new data in a staging area, checks it, and only then makes it visible, so failed data never reaches readers at all.

An alert someone can act on

```
NEXT_STEP = {
    "daily_flash_flash_matches_erp": "compare raw.order_items with the ERP, fix the load, then re-run the
    day",
    "raw_dispatch": "confirm the column change with Bhiwandi Main, add the mapping, then re-run the day",
    "raw_crm_leads": "check the CRM token and the API's status page, then re-run raw_crm_leads",
}

def alert_on_failure(result, day):
    if result.success:
        return None
    failed_checks = [c.check_name for c in result.get_asset_check_evaluations() if not c.passed]
    failed_steps = sorted({e.step_key for e in result.all_events if e.event_type_value == "STEP_FAILURE"})
    lines = [f"Daily Sales Flash for {day} was NOT sent.",
            f"Failed checks: {failed_checks or 'none'}",
            f"Failed steps: {failed_steps}",
            "Next steps:"]
    for step in failed_steps:
        lines.append(f"  - {step}: {NEXT_STEP.get(step, 'read the run log')}")
    lines.append("Runbook: docs/runbooks/daily_flash.md")
    message = "\n".join(lines)
    with open(f>alerts/flash_{day}.txt", "w") as f:
        f.write(message)
    return message

print(alert_on_failure(result, "2026-01-05"))
```



```

Daily Sales Flash for 2026-01-05 was NOT sent.
Failed checks: ['flash_matches_erp']
Failed steps: ['daily_flash_flash_matches_erp', 'raw_dispatch']
Next steps:
- daily_flash_flash_matches_erp: compare raw.order_items with the ERP, fix the load, then re-run the day
- raw_dispatch: confirm the column change with Bhiwandi Main, add the mapping, then re-run the day
Runbook: docs/runbooks/daily_flash.md

```

How it works, line by line.

- `NEXT_STEP` maps each step that can fail to the first thing to do about it. Dagster names a check's step after its asset and its check, so the check's step is `daily_flash_flash_matches_erp`.
- `alert_on_failure()` returns `None` straight away if the run succeeded: no alert for a good run.
- `result.get_asset_check_evaluations()` lists the checks that ran; the code keeps the names of those that didn't pass.
- `result.all_events` is Dagster's event log for the run. Each failed step leaves a `STEP_FAILURE` event, whose `step_key` names the step. The set `{...}` removes repeats, and `sorted` puts them in order.
- For each failed step, the alert adds that step's next step from `NEXT_STEP`. `.get(step, 'read the run log')` gives a default for a step that isn't in the dictionary.
- `"\n".join(lines)` joins the lines into one message, which is written to the `alerts` folder and returned.

Reading it. The alert says what didn't happen (the Flash wasn't sent), what failed, and one next step for each failure, and where the runbook is. A different failure, such as a CRM 401, would get its own next step. In production, the same message would be sent by email or chat, as the run failure sensor in section 46.6 does.

Fixing and re-running

The pipeline owner fixes the order-line bug, and confirms with Bhiwandi Main that the renamed column `units` means the same as `qty_units`. Both fixes are small; the re-run is one command:

```

ingest.BROKEN_ITEMS_SYNC = False
ingest.CONFIRMED_RENAMES = {"units": "qty_units"} # confirmed with Bhiwandi Main
result = dg.materialize(ALL, partition_key="2026-01-05", raise_on_error=False)
show_log()
print("run succeeded:", result.success)
api.shutdown()

```

```

raw_orders: 0 new, 0 changed, 0 deleted
raw_order_items: 1 new, 0 changed, 0 deleted
[2026-01-05] raw_dispatch: 2 rows (the day's rows replaced)
raw_crm_leads: 43 leads (attempt 1)
[2026-01-05] daily_flash: orders=1, revenue=Rs 18,750.00
[2026-01-05] check_flash_matches_erp: PASSED (ERP: orders=1, revenue=Rs 18,750.00 | warehouse: orders=1,
revenue=Rs 18,750.00)
[2026-01-05] deliver_flash: wrote outbox/flash_2026-01-05.txt
run succeeded: True

```

Reading it. The order-line load picked up the missing line for order 10178. The dispatch file loaded with the confirmed column mapping. The Flash was rebuilt at 1 order for ₹18,750.00, the check passed, the report was delivered, and the run as a whole succeeded. Because every step is idempotent, re-running the whole day was safe: `raw_orders` found nothing new to do. The last line, `api.shutdown()`, stops the practice CRM API, which the chapter no longer needs.

What to check before delivery

Check	Catches	Chapter
Reconciliation with the source (counts, totals)	Loads that silently miss or duplicate data	45, this section
Uniqueness of keys (one row per day, per order)	Double-counting from non-idempotent steps	46.4, 47
Freshness: the newest data is from the expected day	Stale sources, stuck loads	47
Volume: row counts within a normal range	Empty loads that don't error	47
Validity: no negative quantities, known statuses only	Bad data from sources	47
Schema: expected columns present	Source changes	45

46.8 The same pipeline in Airflow

Apache Airflow is the orchestrator most often named in data engineering job descriptions. It's task-based: you define a DAG of tasks, and each task runs a function or a command. Here's Riverstone's pipeline written for **Airflow 3**, the current major version (3.0 came out in April 2025), using its TaskFlow style.

```
# dags/daily_flash.py (Airflow 3, TaskFlow API)
import pendulum
from datetime import timedelta
from airflow.sdk import dag, task
from airflow.sdk.exceptions import AirflowFailException
from airflow.timetables.interval import CronDataIntervalTimetable

import ingest, flash                                # the same companion functions

@dag(
    schedule=CronDataIntervalTimetable("30 6 * * *", timezone="Asia/Kolkata"),
    start_date=pendulum.datetime(2026, 1, 1, tz="Asia/Kolkata"),
    catchup=False,                                   # don't run every missed day automatically
    tags=["riverstone", "daily-flash"],
)
def daily_flash_pipeline():

    @task
    def raw_orders():
        return ingest.sync_table(flash.warehouse(), "orders")

    @task
    def raw_order_items():
        return ingest.sync_table(flash.warehouse(), "order_items")
```

```

@task
def raw_dispatch(ds=None):
    # ds = the run's logical date, "YYYY-MM-DD"
    return ingest.load_dispatch(flash.warehouse(), ds)

@task(retries=3, retry_delay=timedelta(seconds=60))
def raw_crm_leads():
    return ingest.load_leads(flash.warehouse(), ingest.fetch_leads())

@task
def build_flash(ds=None):
    flash.build_flash_safely(ds)
    # delete-then-insert in one transaction

@task
def check_flash(ds=None):
    if not flash.matches_erp(ds):
        raise AirflowFailException(f"Flash for {ds} does not match the ERP") # fail, no retry

@task
def deliver_flash(ds=None):
    flash.deliver(ds)
    # skips if already delivered

raw_dispatch()
raw_crm_leads()
[raw_orders(), raw_order_items()] >> build_flash() >> check_flash() >> deliver_flash()

daily_flash_pipeline()

```

How it works.

- `from airflow.sdk import dag, task` brings in the two decorators. `@dag` turns the outer function into a DAG; `@task` turns each inner function into a task.
- `schedule=CronDataIntervalTimetable("30 6 * * *", timezone="Asia/Kolkata")` runs the DAG at 6:30 India time, and makes each run's **logical date** the day it reports on (the Watch-out below explains why that needs saying). `start_date` is the first day the DAG exists; `catchup=False` stops Airflow from running every missed day since then on its own.
- `ds` is the run's logical date as text, "2026-01-02": Airflow's version of `context.partition_key`. A task receives it by having a parameter called `ds`.
- `@task(retries=3, retry_delay=timedelta(seconds=60))` gives **only** the CRM task a retry policy, as the project asks. `AirflowFailException` fails the check task at once, without using any retries.
- The last lines draw the graph. `>>` means "then": the two ERP loads, then the Flash, then the check, then delivery. `raw_dispatch()` and `raw_crm_leads()` are called without any arrow into the Flash, as in Figure 46.1, so their failures don't stop it.
- `flash` is a helper module you would write from this chapter's Dagster code: `warehouse()`, `build_flash_safely()`, `matches_erp()`, and `deliver()`.

How it maps to the Dagster version.

Idea	Dagster (this chapter)	Airflow
Unit of work	Asset (a table or file that should exist)	Task (a function to run)
Dependencies	<code>deps=[...]</code> on the asset	<code>>></code> between tasks
Which day	<code>context.partition_key</code>	<code>ds</code> (the logical date), or <code>data_interval_start</code>

Idea	Dagster (this chapter)	Airflow
Schedule	A schedule built from the job (section 46.3)	<code>schedule=</code> on the DAG
Time zone	On the partitions definition	On the timetable and <code>start_date</code>
Retries	<code>RetryPolicy</code> per asset	<code>retries</code> and <code>retry_delay</code> per task (or for every task, in <code>default_args</code>)
Checks that block delivery	<code>@asset_check(blocking=True)</code>	A check task upstream of delivery; <code>AirflowFailException</code> fails without retrying
Backfill	Select partitions; <code>dagster job backfill</code>	<code>airflow backfill create --dag-id daily_flash_pipeline --from-date 2026-01-01 --to-date 2026-01-05</code>
Run one day for testing	<code>materialize(..., partition_key=...)</code>	<code>airflow dags test daily_flash_pipeline 2026-01-02</code>

Watch out: which day is the logical date? In Airflow 2, a daily DAG's logical date was the **start of the interval** it processes: the run on the morning of 3 January had a `ds` of 2 January. Airflow 3 changed the default for a plain cron string: the logical date is now **the moment the run starts**, so that same run has a `ds` of 3 January, and a Flash built from `ds` would report a day that has barely begun. `CronDataIntervalTimetable` asks for the Airflow 2 behavior, which is what a report about yesterday needs. Dagster's partitioned schedule behaves the same way (section 46.3); Airflow just names it differently. Whichever you use, test with `airflow dags test` and print `ds` before trusting it.

Simplification note. This Airflow code isn't executed in the book: Airflow needs its own installation, a metadata database, and a scheduler process, and it imports the helper module `flash` described above. It was checked against Airflow 3.3.2 with such a helper: `airflow dags test daily_flash_pipeline 2026-01-02` ran every task and delivered the 2 January Flash, 2 orders for ₹38,710.00. To try it yourself, install Airflow in a **separate** virtual environment, because it pins many libraries to exact versions: `python -m pip install "apache-airflow==3.3.2" --constraint https://raw.githubusercontent.com/apache/airflow/constraints-3.3.2/constraints-3.14.txt` (the last part names your Python version), then `airflow standalone` starts everything on your computer. Airflow doesn't run directly on Windows; use WSL (Chapter 34, section 34.1) or a container (Chapter 52). In Airflow 2, which you will still meet at many companies, the decorators are imported from `airflow.decorators` and a backfill is `airflow dags backfill -s 2026-01-01 -e 2026-01-05 daily_flash_pipeline`.

Interview extra point. If an interviewer asks about Airflow and you've used Dagster (or the reverse), map the ideas: DAG and dependencies, logical date or partition, retries, backfills with side effects excluded, checks before delivery, and idempotent tasks. Interviewers care far more about whether you'd design the pipeline safely than about which decorator you'd type. Chapter 77 includes pipeline design questions.

46.9 Choosing an orchestrator

Option	Model	Strengths	Watch for	A good fit when
Cron / Task Scheduler	Time-based commands	Simple, everywhere, nothing to run	No dependencies, retries, history, or backfills; failures are silent unless you add alerts	One or two independent scripts with an owner who checks them
Apache Airflow	Task DAGs	Very widely used; huge library of operators; managed versions on every major cloud	Heavier to run; the logical-date model; asset lineage is less central	Teams with many varied workflows, or where the platform already runs it
Dagster	Asset graphs	Assets, partitions, and checks built in; quick local testing; clear lineage	Smaller community than Airflow; concepts to learn	Data platforms built around tables and models, including dbt projects
Prefect	Python flows	Pythonic and lightweight; good for dynamic workflows	Fewer data-asset features built in	Python-heavy teams wanting minimal ceremony
Cloud schedulers and workflow services	Varies	Managed, integrated with one cloud	Tied to one provider; features vary	Small pipelines already on one cloud (Chapter 52)
Built-in schedulers in tools (dbt Cloud, warehouse tasks, BI refresh schedules)	Per tool	Nothing extra to run	Each tool schedules only itself; cross-tool dependencies get lost	A single tool's work, not a cross-system pipeline

A few questions settle most choices:

1. **How many steps depend on each other, across how many systems?** Few and independent: a scheduler may be enough. Many and connected: an orchestrator.
2. **What does the team already run and know?** A well-run Airflow beats a half-adopted new tool.
3. **Is the work about data assets (tables, models) or about actions (jobs, scripts)?** Assets lean toward Dagster; varied actions lean toward Airflow or Prefect.
4. **Who will operate it?** Self-hosting any orchestrator means upgrades, a metadata database, and monitoring. Managed services cost money and save that effort.

Real-life example: the report that outgrew cron. Many teams start exactly like Riverstone: a handful of cron jobs at 6:00, 6:15, and 6:30, spaced to "leave enough time". It works until month-end volumes make the 6:00 job run long, and the 6:30 report quietly uses half-loaded data. Moving to an orchestrator usually starts with that one incident, and the first win is replacing time gaps with real dependencies.

46.10 Operating pipelines

A pipeline that runs is not the same as a pipeline that's operated well. A few habits make the difference.

- **Every pipeline has an owner.** A named person or team who gets the alerts and decides what happens when it fails.
- **Every important output has a deadline.** A **service-level agreement** (SLA) or objective states when data must be ready, such as "Daily Sales Flash delivered by 7:30 a.m. IST on working days". Alert when the deadline is at risk, not only when a step fails.
- **Every known failure has a runbook.** What the alert means, how to check the cause, how to fix it, how to re-run safely, and whom to tell. The alert in section 46.7 links to one.
- **Every re-run and backfill is recorded.** Who ran what, for which dates, and why.
- **Changes are tested before they're deployed.** Run the pipeline for a known partition in a test environment and compare outputs with the previous version (Chapter 29's testing habits, applied to data).
- **Pipelines are code.** Definitions live in Git, go through review, and are deployed the same way every time (Chapter 52).
- **Downstream readers know the status.** When the Flash is late, tell the sales team before 7:30, rather than letting them discover it.

APPLY

Common mistakes

Mistake	Symptom	Fix
Spacing cron jobs by time instead of dependencies	Reports built on half-loaded data on busy days	Declare dependencies in an orchestrator (46.1)
Leaving schedules in the server's time zone	Runs at the wrong local time; "yesterday" is wrong for hours each night	Set Asia/Kolkata (or your zone) explicitly (46.2)
Steps that read the clock instead of the partition	Backfilled days get today's numbers	Pass the partition date into every step
Appending results in a step that can be retried	Doubled rows and revenue after a failure (46.4)	Partition overwrite in a transaction, or upserts
Assuming idempotency without testing it	Surprise duplicates at the first real failure	Crash a step on purpose and re-run (46.4)
Sending emails or syncs before checks	Wrong numbers reach managers or the CRM	Delivery last, blocked by checks (46.7)
Re-delivering history during backfills	Dozens of old reports arrive at once	Exclude side effects from backfills (46.5)
Silently overwriting delivered numbers after a correction	People act on the old figure without knowing it changed	Send a labeled correction; keep a delivery log (46.5)
Retrying every failure	Bugs hidden by lucky retries; wrong passwords retried until accounts lock	Retry transient errors only; log and review retries (46.6)

Mistake	Symptom	Fix
No timeouts	A stuck run blocks everything and no alert fires	Timeouts on calls, statements, and runs
Alerts without context	People ignore alerts or can't act on them	What failed, impact, next step, runbook link (46.7)
Trusting "0 rows loaded"	Empty loads pass unnoticed	Reconciliation and volume checks (46.7, Ch 47)
Making everything depend on everything	One broken file stops every report	Declare only real dependencies (Figure 46.1)
Pipelines without owners or deadlines	Failures noticed by the business, days later	Owner, SLA, runbook (46.10)

In the real world: the Flash that went out twice

Two months after the pipeline went live, Anita forwarded Meera an email at 7:10 a.m. with one line: "Which one is right?"

Below it were two copies of the Daily Sales Flash for the previous day, sent eleven minutes apart. The first said ₹3,12,400 in bookings. The second said ₹3,24,900.

Meera opened the run history. The 6:30 run had failed at 6:41 when the warehouse connection dropped, and the orchestrator had retried it automatically. Both runs had reached delivery. The difference was easy to explain once she saw it: between the first and second runs, a sales executive had entered a late order from the previous afternoon, so the second run's numbers were more complete. But the email step had no memory. It sent whatever it was asked to send, every time it ran.

"So the second one is right," Anita said when Meera called her. "But the managers don't know that. Two of them have already replied to the first one."

Meera and the data engineer made three changes that week.

First, **delivery got a memory**. Every sent report was recorded with a hash of its content. An identical report wouldn't be sent twice. A report whose numbers changed would be sent with the subject line "CORRECTION: Daily Sales Flash", and a first line saying what changed and why.

Second, **delivery moved behind a cut-off check**. The Flash now waited until the ERP's end-of-day close flag was set before building, so late entries from the previous afternoon were included in the first send, not discovered by a retry.

Third, **retries became visible**. Any run that needed a retry now posted a one-line note in the data team's channel, so a flaky warehouse connection would be noticed and fixed, not silently absorbed.

The next month, the warehouse connection dropped again during a run. The retry succeeded, the content hash matched what had already been prepared, and nothing was sent twice. The only sign was a single line in the data team's channel, which led the engineer to a network setting that had been causing the drops all along.

What made this work.

- **Meera read the run history before guessing.** The orchestrator's record of both runs explained the duplicate in minutes.
- **They fixed the design, not only the incident:** idempotent delivery, a clear correction path, and a real readiness condition instead of a clock time.
- **They made retries visible,** so the underlying problem got fixed instead of hidden.

Project: orchestrate Riverstone's ingestion

Goal: turn your Chapter 45 ingestion job into an orchestrated, partitioned, tested pipeline that delivers the Daily Sales Flash only when the data is right. Chapter 47 will add a fuller set of quality checks and monitoring on top.

Tools you'll need

- **Dagster**, installed in section 46.3 with `python -m pip install dagster==1.13.23 dagster-webserver==1.13.23`. `dagster` is the library; `dagster-webserver` is the local web page that `dagster dev -f riverstone_pipeline.py` opens. `dagster dev` also starts the daemon that runs schedules and sensors.
- **Python** in the book's virtual environment (Chapter 17), with **DuckDB**, **psycopg2**, and **requests**, and **PostgreSQL 16**, as in Chapter 45.
- **The Chapter 46 companion folder** (`companion/ch46/`), copied to `work/ch46`: `reset_ch46.py`, `ingest.py`, `riverstone_pipeline.py`, and the three helpers copied from Chapter 45. Run the notebook from that folder. If your PostgreSQL needs a user or password, copy your `.env` file from `work/ch45` (Chapter 45, section 45.2).
- **Versions used for the outputs shown:** Python 3.14.7, Dagster 1.13.23, DuckDB 1.5.6, psycopg2 2.9.13, requests 2.33.1, PostgreSQL 16.
- **Apache Airflow** (not needed for this chapter): section 46.8's Simplification note gives the install command, in a separate virtual environment; managed versions exist on the major clouds.

Option A: Riverstone. Use the companion environment and your Chapter 45 project.

Option B: your own pipeline. Use a report you currently produce on a schedule, with data you have permission to use. Keep credentials out of code.

Steps

1. **Draw the graph first.** List every asset (raw tables, staging tables, the Flash, delivery) and its real dependencies. Mark which failures should stop which outputs.
2. **Define daily partitions** in Asia/Kolkata (or your time zone). Make every step that works on one day (the dispatch file, the Flash, delivery) use the partition date, never the clock; leave the loads that bring whole tables up to date unpartitioned.
3. **Wrap your Chapter 45 loads as assets.** Add `customers` and `products` as full loads.
4. **Build the Flash** with partition overwrite in a transaction. Add a second measure of your choice, such as orders by segment.

5. **Add at least two blocking checks:** reconciliation with the ERP, and exactly one row per day.
6. **Add delivery** with a delivery log and a correction path.
7. **Add a retry policy** to the API step only, and log every retry.
8. **Write the failure test:** crash the Flash step after writing, re-run, and assert that there's one row and the revenue matches the ERP. Then do the same with an append version and record what goes wrong.
9. **Run a backfill** for 1–5 January without delivery, and show the outbox is unchanged.
10. **Simulate a bad morning:** break one load, confirm the check blocks delivery, and produce an alert with a next step and a runbook link. Write the runbook (half a page).
11. **Schedule it** for 6:30 IST. Put your definitions in one file, as `riverstone_pipeline.py` does (section 46.3), run it with `dagster dev`, switch the schedule on, and take a screenshot of the graph and a successful run.

Stretch goals

- Translate your pipeline to Airflow and run it with `airflow dags test` for 2 January.
- Add a 7-day look-back: each daily run also rebuilds the previous six days, and only sends corrections when numbers change.
- Add a deadline: if the Flash hasn't been delivered by 7:30 IST, write an alert even if nothing has failed.

REVIEW

Recap

- An **orchestrator** runs pipeline steps in dependency order, on schedule, with retries, history, backfills, and alerts. **Cron** is enough for simple, independent jobs.
- A pipeline is a **DAG**. **Task-based** tools (Airflow) describe what to run; **asset-based** tools (Dagster) describe what data should exist.
- **Schedules** use **cron expressions** and need explicit **time zones**. **Partitions** split runs by day, so each run knows exactly what it's building.
- **Idempotency** must be proven with a **failure test**. An append step retried after a crash doubled revenue to ₹77,420.00; **partition overwrite in a transaction** kept it at ₹38,710.00.
- **Side effects** (emails, syncs) can't be rolled back: put them last, behind checks, with a **delivery log**.
- **Backfills** rebuild history; exclude delivery. **Late corrections** re-run one partition and send a labeled correction.
- **Retries** handle transient failures, and can hide bugs; log them, and **fail fast** on errors that won't go away (`allow_retries=False`). **Timeouts** stop runs that never end.
- **Sensors** start runs when something happens, such as a file arriving; a **run key** makes sure each event triggers one run. The **daemon** runs schedules and sensors.
- **Checks that block delivery** catch failures that don't crash: an order-line load that loaded nothing raised no error, and only the reconciliation check stopped a ₹0 Flash reaching managers.

- Good **alerts** are actionable, rare, routed, and linked to a **runbook**.
- Airflow expresses the same design with tasks, ds, retries, and a check task before delivery.
- Operate pipelines with **owners**, **SLAs**, **runbooks**, recorded re-runs, and code review.

Key terms

orchestrator · pipeline · cron · directed acyclic graph (DAG) · cycle · task · asset · materialize · dependency · schedule · cron expression · time zone (UTC, IST) · partition · partition key · sensor · daemon · run key · job · executor · retry policy · fail fast · asset check · blocking check · idempotent · failure test · transaction (BEGIN, COMMIT, ROLLBACK) · partition overwrite · delivery log · idempotency key · side effect · backfill · late-arriving data · look-back window · transient failure · timeout · alert · runbook · run failure sensor · logical date (Airflow ds) · catchup · service-level agreement (SLA) · write-audit-publish

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ You can explain what an orchestrator adds to cron, with at least four examples.
- ☐ You can draw a pipeline as a DAG and explain why only real dependencies belong in it.
- ☐ You can explain the difference between task-based and asset-based orchestration.
- ☐ You can read and write cron expressions, and set time zones explicitly.
- ☐ You can build partitioned assets in Dagster that use the partition date instead of the clock.
- ☐ You can prove a step is idempotent with a deliberate failure test, and explain why the append version doubled revenue.
- ☐ You can list idempotency patterns for tables and for side effects such as emails.
- ☐ You can run a backfill that rebuilds history without re-delivering it.
- ☐ You can handle a late correction by re-running one partition and sending a labeled correction.
- ☐ You can explain when retries help, when they hide bugs, how to fail fast, and why timeouts matter.
- ☐ You can write a sensor that starts one run per new file, and explain what the daemon does.
- ☐ You can make checks block delivery, and explain why the failed order-line load raised no error.
- ☐ You can write an actionable alert with a next step and a runbook link.
- ☐ You can map the pipeline between Dagster and Airflow, including Airflow's logical date.
- ☐ You can recommend an orchestrator for a given team and explain why.

Exercises

Run `reset()` and re-run the definitions blocks before each exercise that uses the companion environment, unless it says otherwise.

Warm-up

1. Write cron expressions for: (a) 7:15 every weekday (Monday to Friday); (b) every 30 minutes; (c) 9:00 on the first day of every quarter (January, April, July, October).
2. A schedule is set to `0 1 * * *` in UTC. At what time does it run in Mumbai (IST)? Which calendar date is it in IST when it runs?
3. For each step, say whether it's idempotent as described, and why: (a) `INSERT INTO mart.daily_flash ...` for the partition day; (b) `INSERT OR REPLACE` by primary key; (c) sending an email every time the step runs; (d) delete the partition's rows, then insert, in one transaction.

Core

4. In section 46.4, the naive table ended with two rows for 2 January. If the naive step had been retried three times after three crashes, and then succeeded, how many rows would there be, and what revenue would a `SUM` report?
5. Add a second blocking check to the pipeline, `one_flash_row_per_day`, that passes only when `mart.daily_flash` has exactly one row for the partition. Why is this check useful even though `flash_matches_erp` already compares the row with the ERP?
6. In section 46.7's bad morning, why did `daily_flash` run at all, and why did `raw_dispatch`'s failure not stop it? Draw (or describe) the dependency change that would make a dispatch failure block the Flash, and argue whether that would be a good idea.
7. After the correction in section 46.5, what would `deliver_flash` do if the ERP were corrected *back* to 6 crates and the 2 January partition re-run? Trace it through the delivery log.
8. Rewrite the alert in section 46.7 for a different failure: the CRM API returned 401 Unauthorized. Include what failed, the impact on the Flash, the next step, and a runbook link.
9. A colleague proposes adding `retry_policy=dg.RetryPolicy(max_retries=5)` to `daily_flash` "to make the pipeline more reliable". Explain what this would and wouldn't help with, using the section 46.7 bad morning as an example.

Stretch

10. Design a 7-day look-back for the Flash: each morning's run rebuilds today's partition and the previous six, but only delivers today's Flash and any *corrections* to earlier days. Describe the assets and selections, and what the delivery log must store.
11. Riverstone's month-end finance pack must use numbers that don't change after the 5th of the following month. Describe how you'd combine partitions, a closing rule, and checks so late corrections after the 5th are reported for review rather than silently changing the pack.
12. Translate the section 46.7 check-then-deliver behavior into Airflow terms precisely: which task fails, which exception, which retries apply, and what state the delivery task ends in.

Think about it (no code needed)

13. Riverstone's data team has six cron jobs today and one part-time engineer. Make the case for staying with cron for another six months, then the case for moving to an orchestrator now. Which would you choose, and what single incident would change your mind?
14. Why is "delivery last, behind checks" a rule for more than emails? Give two examples from Chapter 51's world (writing data back into business systems) where breaking it would be worse than a wrong email.

Answers

1. (a) `15 7 * * 1-5`. (b) `* /30 * * * *`. (c) `0 9 1 1,4,7,10 *`. The common slip in (a) is `* 7 * * 1-5`, which runs every minute from 7:00 to 7:59.
2. 1:00 UTC is **6:30 a.m. IST** (UTC + 5:30), on the **same calendar date**. The reverse trap is a schedule at 20:00 UTC, which runs at 1:30 a.m. IST on the *next* day.
3. (a) **Not idempotent**: each run inserts another row for the day, as section 46.4 showed. (b) **Idempotent**: the same keys end with the same values however often it runs. (c) **Not idempotent**: each run sends another email; the delivery log in section 46.3 is what makes it safe. (d) **Idempotent**: every run leaves exactly the rows for that partition, and a failure rolls back.
4. Each of the three crashed runs inserted a row before crashing, and the successful run inserted one more: **4 rows**. A `SUM` would report $4 \times ₹38,710.00 = ₹1,54,840.00$.
5. A sample check:

```
@dg.asset_check(asset=daily_flash, blocking=True)
def one_flash_row_per_day(context):
    day = context.partition_key
    n = wh.execute("SELECT COUNT(*) FROM mart.daily_flash WHERE flash_date = ?", [day]).fetchone()[0]
    return dg.AssetCheckResult(passed=(n == 1), metadata={"rows": n})
```

Like `flash_matches_erp`, it runs one `COUNT(*)` for the partition's day and returns an `AssetCheckResult`; `metadata={"rows": n}` attaches the count, which Dagster shows beside the result. It's useful because it names the problem precisely. `flash_matches_erp` also fails when there are duplicate rows, but its message only says the numbers don't match; the uniqueness check says *why*, which points straight to a non-idempotent step. Separate checks for separate failure modes make alerts actionable. It also protects readers who query the table directly and sum it.

6. `daily_flash` depends only on `raw_orders` and `raw_order_items`, and both **succeeded** (the order-line load loaded nothing, but didn't raise an error). `raw_dispatch` isn't a dependency of the Flash, so its failure only affects assets downstream of it. To make a dispatch failure block the Flash, you'd add `raw_dispatch` to `daily_flash`'s `deps`. That would usually be a **bad idea**: the Flash reports bookings, which don't use dispatch data, so a warehouse spreadsheet problem would stop the sales team's numbers for no reason. Add a dependency only when the output really uses the input; a dispatch report, by contrast, *should* depend on `raw_dispatch`.

7. The Flash would be rebuilt at ₹38,710.00. `deliver_flash` computes the content hash and finds that **exactly this content was already delivered** (the original 2 January report, recorded on the first run), so it logs "already delivered, skipped" and sends nothing. The managers' latest message is still the correction for ₹40,110.00, which is now wrong. That's a gap in this simple design: "already delivered" should compare with the most recent delivery, not any past delivery. A fix is to compare the new hash with the latest entry for that day, and send a second correction when they differ.

8. A sample alert: "Daily Sales Flash for 2026-01-05: CRM leads were NOT loaded. The CRM API returned 401 Unauthorized (the token is missing, wrong, or expired), so the step failed at once, without retrying. Impact: the Flash itself is unaffected and has been delivered; lead figures in the weekly pipeline report will be stale until this is fixed. Next step: check whether the CRM API token was rotated, update the secret `CRM_API_TOKEN`, then re-run `raw_crm_leads` only. Runbook: docs/runbooks/crm_ingestion.md." A good answer says the Flash wasn't affected, because `raw_crm_leads` isn't one of its dependencies, and notes that a 401 shouldn't be retried at all, which is why the step fails fast (section 46.6).

9. Retries on `daily_flash` would help only if the step itself failed **transiently**, for example a dropped warehouse connection during the build, and only because the step is idempotent. They wouldn't help on the bad morning at all: `daily_flash` **didn't fail**. It succeeded with wrong numbers, because its input was incomplete. The problem was caught by the check, and retrying a check that compares wrong data with the ERP gives the same failure every time. Five retries would also delay the alert. Reliability comes from checks, idempotency, and fixing root causes; retries only cover transient faults.

10. One design: keep the daily partitions. The morning run materializes ingestion once (it isn't per-day), then `daily_flash` and its checks for **seven partition keys**: today and the previous six. Delivery runs for **today's** partition as normal, plus a `deliver_corrections` step that, for each of the six earlier days, compares the new content hash with the **latest** delivery for that day and sends a labeled correction only when it differs. The delivery log must store the day, the content hash, the time sent, and the type (original or correction), so "latest" can be found. Checks must pass for every rebuilt day before any correction is sent.

11. One approach: keep daily (or monthly) partitions and a **closing table** that records when each month was closed (the 5th of the following month). The pipeline still re-runs late partitions when source data changes, but a **blocking check** on the month-end pack compares the new numbers with the closed version: if the month is closed and numbers changed, the check fails *for the pack*, the pack isn't regenerated, and an alert goes to finance with the difference and the orders involved. Finance then decides whether to reopen the month or book an adjustment in the next one. The daily Flash can still show corrected operational numbers; the closed pack stays as reported.

12. In Airflow, the check is a task (`check_flash`) upstream of `deliver_flash`. When the numbers don't match, it raises `AirflowFailException`, which marks the task **failed immediately, without using its retries**. (A normal exception would be retried according to `retries`, which is wasteful for a data mismatch.) Because `deliver_flash` depends on `check_flash` with Airflow's default trigger rule (`all_success`), it ends in the **upstream_failed** state and never runs. The DAG run is marked failed, and a failure callback or alerting integration notifies the owner.

13. *Stay with cron*: six jobs and one part-time engineer means an orchestrator adds a system to install, upgrade, and learn; if the jobs are independent and each has alerts and a runbook, the risk is low, and the engineer's time may be better spent on checks. *Move now*: the Flash already depends on several

loads, cron spacing will fail on a busy day, and adding checks, retries, and backfills by hand in six scripts costs more than adopting a tool once; a managed or simple local deployment keeps the operational burden small. A reasonable choice is to move the Flash's steps to an orchestrator now and leave truly independent jobs on cron. The incident that would change a "stay" decision: any day when a report goes out built on half-loaded or stale data because a job ran long.

14. Because anything that leaves the pipeline changes something outside it, and many of those changes can't be undone. Examples: (1) syncing **credit limits** from the warehouse into the ERP: a failed load that sets limits to zero would block real customers' orders until someone notices; (2) pushing **lead scores** or "customer at risk" flags into the CRM: wrong scores would send sales reps to the wrong customers and trigger automated emails, and the CRM keeps the history. Other good examples include reorder flags that trigger purchase orders, and marketing lists that send messages. In each case, a check that blocks the sync costs minutes; a wrong sync can cost orders, money, or customer trust.

NEXT

Where this leads

- **Chapter 47, Data Quality, Observability & Contracts**, turns this chapter's checks into a full testing and monitoring layer: freshness, volume, and validity checks, write-audit-publish, data contracts, and alert fatigue.
- **Chapter 48, Big Data & Distributed Compute**, runs heavier steps on Spark and DuckDB at scale, inside pipelines like this one.
- **Chapter 49, Storage, Warehouses & Lakehouses**, explains the transactions (ACID) and table formats that make partition overwrites safe at scale.
- **Chapter 50, Streaming & Real-Time**, covers pipelines that run continuously instead of once a day.
- **Chapter 51, Data Activation**, adds CRM and ERP syncs as downstream steps, gated by checks and made idempotent with delivery logs and idempotency keys.
- **Chapter 52, The Cloud, Containers & Infrastructure as Code**, deploys pipelines like this one: containers, secrets, and environments.
- **Chapter 63, Automation Architecture & Governance**, looks at orchestration across a whole company: ownership, governance, and choosing tools.
- **Part 8:** pipeline and orchestration questions appear in the data engineering and system design interview chapters (Chapter 77).

Chapter 47. Data Quality, Observability & Contracts

Chapter at a glance

You will learn to: name the dimensions of data quality and write a test for each · build a small test framework where every test is a query that returns the rows that break a rule · choose severities so that only real problems stop a pipeline · use write-audit-publish so bad data never reaches the tables people read · monitor freshness and volume, and spot unusual days against history · trace lineage to see which reports a broken table affects · write a data contract with a source owner and check it automatically · run a data incident: severity, communication, fix, and review · keep alerts few enough that people still read them · run the same rules in dbt, Great Expectations, and Soda, and know where observability platforms fit.

Before you start: Chapter 45 (ingestion and reconciliation), Chapter 46 (the orchestrated pipeline), Chapter 32 (dbt), and Chapter 14 (finding and fixing data problems as an analyst).

Time needed: 14–18 hours of reading and practice, spread over two to three weeks, in four sittings: sections 47.1–47.3; section 47.4; sections 47.5–47.7; sections 47.8–47.10 and the project.

Tools: Python 3.14 in the virtual environment from Chapter 17, with duckdb and psycopg2 (installed in Chapter 45, section 45.2); PostgreSQL 16; Jupyter; and the Chapter 47 companion folder. Section 47.10 adds two optional libraries, great_expectations and soda-duckdb. Everything runs on your own computer. No data quality product is needed for the main thread: the tests are SQL.

Practice data: the Chapter 45 and 46 environment: riverstone_source (the ERP), the dispatch files, and the warehouse you've been building.

In this chapter

START	Why this matters · In plain English	96
LEARN	47.1 The dimensions of data quality	96
	47.2 Testing data inside the pipeline	97
	47.3 A check library for Riverstone	98
	47.4 Write-audit-publish	107
	47.5 Observability: freshness, volume, and unusual values	112
	47.6 Lineage: what breaks when this breaks	116
	47.7 Data contracts in practice	117
	47.8 Handling a data incident	122
	47.9 Alerts people still read	123
	47.10 The tools landscape	126
APPLY	Common mistakes · In the real world · Project	129
REVIEW	Recap · Key terms · Check yourself	132
PRACTISE	Exercises	134
	Answers	135
NEXT	Where this leads	138

Why this matters

Chapter 1 told you that analysts check data before they trust it, and promised that Chapter 47 would show how data teams do that automatically. Chapter 13 built the habit of reconciling every number against a source. Chapter 46 added one check to a pipeline and used it to stop a wrong report from being sent.

This chapter turns those habits into a system, because as soon as more than one person depends on the data, checking by hand stops working. Somebody is on leave. Somebody assumes somebody else looked. And the failures that matter most are the quiet ones: a load that brings in nothing, a status nobody agreed to, a file that's three days old, a column renamed by a helpful colleague.

There's also a gap left over from Chapter 46. When the check failed on 5 January, the report wasn't sent, which was right. But the wrong row was still sitting in the table, visible to anyone who queried it. This chapter closes that gap, and adds the monitoring, lineage, contracts, and incident habits that make a data platform something people trust.

In plain English

Think about a factory's quality department, because Riverstone has one.

- **Specifications** say what "good" means: this crate weighs between 4.8 and 5.2 kg, this lid fits this base. Data has specifications too, and writing them down is most of the work.
- **Checks on the line** catch problems while the product is being made, not after it ships. That's **testing data inside the pipeline**.
- **Quarantine** holds a doubtful batch until someone inspects it. Nothing doubtful goes to customers. That's **write-audit-publish**.
- **A goods-inward check** confirms what arrived from a supplier matches what was agreed. That's a **data contract**.
- **A gauge that hasn't moved in three days** is more worrying than a gauge reading badly, because nobody noticed. That's **freshness monitoring**.
- **Batch tracing** tells you which customers got product from a faulty batch. That's **lineage**.
- **An alarm that rings all day** gets switched off, and then nobody hears the real one. That's **alert fatigue**.

A data team is the quality department for the numbers. This chapter is its handbook.

47.1 The dimensions of data quality

"Quality" is too vague to test. Break it into dimensions, and each one becomes a rule you can write in SQL.

Dimension	The question it answers	Riverstone example	Typical test
Accuracy	Do the numbers match reality, or a trusted source?	Warehouse revenue for a day equals the ERP's	Reconciliation against the source
Completeness	Is anything missing?	Every order has at least one order line; every order has a customer	Missing values; counts against the source
Validity	Does each value follow its rules?	status is one of Pending, Shipped, Delivered, Cancelled; quantities are positive	Accepted values; ranges
Uniqueness	Is anything counted twice?	One row per order in raw.orders; one Flash row per day	Duplicate key tests
Consistency	Do related things agree?	Every order line points at an order that exists; dispatch rows point at known orders	Relationship tests across tables
Timeliness	Is it here, and recent enough?	Yesterday's orders are loaded by 6:30 a.m.; the dispatch file arrived last night	Freshness checks

Two more properties matter and aren't tested the same way. **Usability** is whether people can find and understand the data: names, definitions, and documentation (Chapter 32 and Chapter 64). **Trust** is whether they believe it, which is earned by the other dimensions holding for months.

Simplification note. Different books and tools name these slightly differently, and some add dimensions such as integrity or conformity. The names matter less than the habit: for every important table, write down what "good" means, then test it.

47.2 Testing data inside the pipeline

A **data test** is a rule about data, expressed so a computer can judge it. The most useful way to write one is this:

Write a query that returns the rows that break the rule. If it returns no rows, the test passes.

That single idea covers almost everything: duplicates, missing values, invalid values, broken relationships, and totals that don't match a source. It also gives you something valuable when a test fails: the offending rows, which is where investigation starts. It's also how dbt's tests work underneath (Chapter 32): each one compiles to exactly this kind of query.

Severity: stop, or warn?

Not every problem should stop a pipeline at 6:30 a.m. Each test gets a **severity**, and this chapter's code writes it as one of two strings, "error" or "warn":

- **Error** ("error"): the data is wrong in a way that would mislead someone. Stop, and don't publish. Example: the warehouse's revenue doesn't match the ERP.
- **Warning** ("warn"): something is odd and worth a look, but publishing is still better than not publishing. Example: one dispatch row points at an order the warehouse doesn't have yet.

Start strict on the numbers people act on, and lenient on the edges. A test whose failures are routinely ignored should either be fixed, downgraded to a warning, or deleted. Tests nobody acts on are worse than no tests, because they train people to ignore red.

Where checks belong

Layer	What to test	Why here
Raw	Structure and load completeness: expected columns, row counts against the source, keys unique	Catches ingestion problems before anything is built on them
Staging	Validity and consistency: types converted, accepted values, relationships between tables	Catches source data problems and conversion mistakes
Mart	Business rules and reconciliation: one row per day, totals match the source, no negative revenue	Catches modeling mistakes, and protects the numbers people read
Before delivery	The blocking checks from Chapter 46	Nothing wrong leaves the platform

47.3 A check library for Riverstone

Setting up

The companion folder `companion/ch47/` is the Chapter 46 environment without Dagster. This chapter doesn't need the orchestrator, only the functions its steps call:

File	What it does
<code>reset_ch47.py</code>	Recreates <code>riverstone_source</code> from <code>riverstone_2025</code> , and empties the warehouse, exports, outbox, and alerts folders, then writes the two dispatch files again
<code>ingest.py</code>	Chapter 46's packaged loads (section 46.3): <code>fetch_rows()</code> (run a query on the ERP), <code>sync_table()</code> (hash comparison and upsert), <code>load_dispatch()</code> (with a column check)
<code>apply_day.py</code> , <code>make_files.py</code> , <code>mock_crm_api.py</code>	Copied from Chapter 45

Before you start, make sure PostgreSQL is running and `riverstone_2025` is loaded (Chapter 45, section 45.2), and activate the book's virtual environment. Then open a Jupyter notebook in `companion/ch47/` (Chapter 17, section 17.0), and run each code block in this chapter as its own cell, top to bottom,

without restarting the kernel. Later cells use names that earlier cells create, such as `wh`, `run_tests`, and `TESTS`.

The first cell resets the practice environment and opens the warehouse:

```
import duckdb
from reset_ch47 import reset
import ingest

reset()
wh = duckdb.connect("warehouse/riverstone_wh.duckdb")
print("ERP orders after reset:", ingest.fetch_rows("SELECT COUNT(*) FROM orders")[0][0])
```

```
ERP orders after reset: 175
```

How it works.

- `from reset_ch47 import reset` and `import ingest` load two of the companion files as modules (Chapter 29, section 29.2).
- `reset()` puts everything back to the start: a fresh copy of the ERP, an empty warehouse, and the two dispatch files. Run this cell again whenever you want to start over.
- `duckdb.connect()` opens the warehouse file, as in Chapter 45. `wh` is the connection every later cell uses.
- `ingest.fetch_rows(sql)` runs a query on the ERP and returns its rows as a list of tuples. `[0][0]` takes the first value of the first row. 175 orders is the one-year database, before any of the January business days.

If this cell fails with a connection error, PostgreSQL isn't running, or `riverstone_2025` isn't loaded.

Next, replay the 2 January business day from Chapter 46 and load it:

```
from apply_day import apply_day

apply_day(1) # the 2 January business day (Chapter 45)
for table in ["orders", "order_items"]:
    ingest.sync_table(wh, table)
print("dispatch:", ingest.load_dispatch(wh, "2026-01-02"))
```

```
dispatch: 5 rows (the day's rows replaced)
```

How it works.

- `apply_day(1)` changes the ERP the way 2 January did in Chapters 45 and 46: two new orders with three lines, one order delivered, one cancelled.
- `ingest.sync_table(wh, table)` is Chapter 45's hash comparison and upsert (section 45.5), packaged as a function in Chapter 46: it makes the warehouse's raw copy of the table match the ERP, inserting, updating, and deleting.
- `ingest.load_dispatch(wh, day)` loads that day's dispatch file into `raw.dispatch`, after checking its columns (Chapter 45, section 45.8), and replaces any rows an earlier load of the same day left. It returns a short message: here, the number of rows loaded.

Now count what arrived:

```
print("rows loaded:", wh.execute("""
    SELECT (SELECT COUNT(*) FROM raw.orders),
           (SELECT COUNT(*) FROM raw.order_items),
           (SELECT COUNT(*) FROM raw.dispatch)""").fetchone())
```

```
rows loaded: (177, 333, 5)
```

The query is three `COUNT(*)` queries in one `SELECT`, each in its own brackets. Each bracketed query returns one number (a **scalar subquery**, Chapter 12, section 12.12), so the result is one row with three values. `.fetchone()` returns that row as a tuple. The warehouse now holds the 2 January state from Chapter 46: 177 orders, 333 order lines, and 5 dispatch rows.

A tiny test framework

Every test is a query. The first function only records a test; it doesn't run anything yet:

```
def data_test(name, sql, severity="error", description=""):
    """A data test is a query that returns the rows that BREAK a rule.
    Zero rows means the test passed."""
    assert severity in ("error", "warn"), "severity must be 'error' or 'warn'"
    return {"name": name, "sql": sql, "severity": severity, "description": description}

print(data_test("demo", "SELECT 1 WHERE FALSE"))
```

```
{'name': 'demo', 'sql': 'SELECT 1 WHERE FALSE', 'severity': 'error', 'description': ''}
```

How it works.

- The text in triple quotes under `def` is a **docstring** (Chapter 17, section 17.8): the function's one-paragraph description.
- `data_test()` returns a dictionary with the test's name, the query that returns *failing* rows, its severity, and a plain-English description of the rule. Writing the description first is a good habit: if you can't say the rule in one sentence, you don't have a rule yet.
- `severity="error"` is a default: a test is an error unless you say otherwise.
- `assert` stops with a message when its condition is false. Here it catches a misspelled severity, which would otherwise be silently ignored.

What happens if you change `"warn"` to the natural spelling, `"warning"`? Try it:

```
data_test("demo", "SELECT 1 WHERE FALSE", severity="warning")
```

```
AssertionError: severity must be 'error' or 'warn'
```

Without the `assert`, this test would be created, and when it failed it would be counted neither as an error nor as a warning, because the runner below only knows those two words.

The second function runs a list of tests and reports:

```
def run_tests(tests, show_rows=2):
    results = []
    for t in tests:
        rows = wh.execute(t["sql"]).fetchall()
        results.append({"name": t["name"], "severity": t["severity"],
                        "failing_rows": len(rows), "sample": rows[:show_rows]})
    width = max(len(r["name"]) for r in results)
    for r in results:
        status = "PASS" if r["failing_rows"] == 0 else r["severity"].upper()
        example = f"  e.g. {r['sample']}" if r["failing_rows"] else ""
        print(f"{r['name']:<{width}} {status:<5} failing rows: {r['failing_rows']}{example}")
    passed = [r for r in results if r["failing_rows"] == 0]
    errors = [r for r in results if r["failing_rows"] and r["severity"] == "error"]
    warnings = [r for r in results if r["failing_rows"] and r["severity"] == "warn"]
    print(f"\n{len(results)} tests, {len(passed)} passed, "
          f"{len(errors)} failed as errors, {len(warnings)} warnings")
    return results

print("test framework ready")
```

```
test framework ready
```

How it works, line by line.

- `wh.execute(t["sql"]).fetchall()` runs the test's query on the warehouse and returns every row as a list of tuples. `wh` is the connection from the setup cell; the function reads it from the notebook. (Passing the connection in as an argument would be cleaner, as Chapter 29 recommends; here it keeps each test call short.)
- `len(rows)` is the number of failing rows. `rows[:show_rows]` keeps the first two as examples, so a test with ten thousand failures doesn't flood the screen.
- `width = max(len(r["name"]) for r in results)` finds the length of the longest test name. The expression inside `max()` works like a list comprehension without the square brackets: it produces each name's length in turn.
- `status` is a **conditional expression** (Chapter 17, section 17.5): "PASS" if nothing failed, otherwise the severity in capitals, ERROR or WARN. `example` is either the sample rows or empty text.
- In the f-string, `<{width}` means "left-align in width characters", and `<5` pads the status to five, so the columns line up (Chapter 17, section 17.7).
- The three list comprehensions sort the results into passed, errors, and warnings, and the last `print` counts them. `"\n"` starts with a blank line.
- `return results` hands the results back, so a pipeline can decide what to do. Section 47.4 uses the count of **errors** to decide whether to publish.

This is a teaching version, in about 20 lines. dbt, Great Expectations, and Soda add scheduling, storage of results, history, and reporting. dbt's tests compile to failing-rows queries like these; Great Expectations and Soda compute counts of failing values and report them, which is the same rule expressed as a measurement. Section 47.10 runs one rule in each.

The tests

```
TESTS = [
    data_test("orders_unique_key", ""
        SELECT order_id, COUNT(*) AS n FROM raw.orders GROUP BY order_id HAVING COUNT(*) > 1"",
        description="uniqueness: one row per order"),
    data_test("orders_customer_not_null", ""
        SELECT order_id FROM raw.orders WHERE customer_id IS NULL"",
        description="completeness: every order has a customer"),
    data_test("orders_status_accepted", ""
        SELECT order_id, status FROM raw.orders
        WHERE status NOT IN ('Pending', 'Shipped', 'Delivered', 'Cancelled')"",
        description="validity: only known statuses"),
    data_test("order_items_have_an_order", ""
        SELECT i.order_item_id, i.order_id FROM raw.order_items AS i
        LEFT JOIN raw.orders AS o ON o.order_id = i.order_id
        WHERE o.order_id IS NULL"",
        description="consistency: every order line belongs to an order"),
    data_test("order_items_quantity_positive", ""
        SELECT order_item_id, quantity FROM raw.order_items WHERE quantity <= 0"",
        description="validity: quantities are positive"),
    data_test("orders_have_at_least_one_line", ""
        SELECT o.order_id FROM raw.orders AS o
        LEFT JOIN raw.order_items AS i ON i.order_id = o.order_id
        WHERE i.order_item_id IS NULL"",
        description="completeness: every order has at least one order line"),
    data_test("dispatch_order_known", ""
        SELECT d.dispatch_id FROM raw.dispatch AS d
        LEFT JOIN raw.orders AS o ON o.order_id = d.order_id
        WHERE o.order_id IS NULL"",
        severity="warn",
        description="consistency: dispatches point at known orders"),
]
results = run_tests(TESTS)
```

```
orders_unique_key           PASS    failing rows: 0
orders_customer_not_null    PASS    failing rows: 0
orders_status_accepted      PASS    failing rows: 0
order_items_have_an_order   PASS    failing rows: 0
order_items_quantity_positive PASS    failing rows: 0
orders_have_at_least_one_line PASS    failing rows: 0
dispatch_order_known        PASS    failing rows: 0
```

```
7 tests, 7 passed, 0 failed as errors, 0 warnings
```

Reading it. Seven rules, covering four of the six dimensions: uniqueness, completeness, validity, and consistency. Accuracy (reconciliation with the ERP) comes in section 47.4, and timeliness (freshness) in section 47.5. All seven pass on today's data. Each query is SQL you already know:

- `GROUP BY ... HAVING COUNT(*) > 1` returns keys that appear more than once (Chapter 12).
- `LEFT JOIN ... WHERE o.order_id IS NULL` is the **anti-join** from Chapter 12, section 12.10: it keeps the rows that found no partner. Two tests use it in each direction: order lines with no order, and orders with no lines.
- `dispatch_order_known` is a **warning**: dispatch rows can legitimately arrive for an order the warehouse hasn't loaded yet, so it's worth seeing but shouldn't stop the Flash.

Notice also what these tests are *not*: they aren't a check that the pipeline ran. Chapter 46's checks answered "did this step produce the right numbers?"; these answer "is the data in the warehouse sane?" Both matter, and both run in the pipeline.

Watching them fail

A test suite nobody has seen fail is a test suite you don't know works. Break things on purpose. First, try to put an order line for an order that doesn't exist into the ERP itself:

```
import psycopg2

conn = psycopg2.connect(ingest.SRC)
try:
    with conn, conn.cursor() as cur:
        cur.execute("""INSERT INTO order_items
                        (order_item_id, order_id, product_id, quantity, unit_price,
                        discount_pct)
                        VALUES (335, 99999, 101, 12, 430.00, 0.00)""")
except psycopg2.Error as e:
    print("the ERP refused it:", str(e).splitlines()[0])
finally:
    conn.close()
```

```
the ERP refused it: insert or update on table "order_items" violates foreign key constraint
"order_items_order_id_fkey"
```

How it works.

- `ingest.SRC` is the ERP's connection string, the same one Chapter 45 set up (section 45.2). `psycopg2.connect()` opens the connection.
- The `INSERT` names its columns, so you can read the values: line 335 claims to belong to order 99999, for 12 units of product 101 at ₹430.00 with no discount. There is no order 99999.
- In `psycopg2`, `with conn` wraps the statements in a **transaction**: it commits if they succeed and rolls back if one fails. It does *not* close the connection. That's why `conn.close()` sits in **finally**, which runs whether the insert worked or raised an error (Chapter 17, section 17.10).
- `except psycopg2.Error as e` catches the database's refusal. `str(e).splitlines()[0]` keeps the first line of the error message; the rest is detail.

The ERP **refused** the orphan order line: the source database has a **foreign key** (Chapter 12) preventing a line from pointing at an order that doesn't exist. Source systems often have constraints like this, which is why many warehouse problems come from the loading, not from the source.

So break the *warehouse* instead, the way a half-finished load, a bad manual fix, or a bug in a transformation would. Before you run the next cell, predict which of the seven tests will fail:

```
wh.execute("""INSERT INTO raw.order_items
            (order_item_id, order_id, product_id, quantity, unit_price, discount_pct)
            VALUES (335, 99999, 101, 12, 430.00, 0.00)""")    # an order that isn't there
wh.execute("""INSERT INTO raw.order_items
            (order_item_id, order_id, product_id, quantity, unit_price, discount_pct)
            VALUES (336, 10176, 103, -8, 115.00, 0.00)""")    # a negative quantity
wh.execute("UPDATE raw.orders SET status = 'On Hold' WHERE order_id = 10177")    # a status nobody
agreed
results = run_tests(TESTS)
```

```
orders_unique_key          PASS    failing rows: 0
orders_customer_not_null   PASS    failing rows: 0
orders_status_accepted     ERROR   failing rows: 1   e.g. [(10177, 'On Hold')]
order_items_have_an_order  ERROR   failing rows: 1   e.g. [(335, 99999)]
order_items_quantity_positive ERROR   failing rows: 1   e.g. [(336, -8)]
orders_have_at_least_one_line PASS    failing rows: 0
dispatch_order_known       PASS    failing rows: 0

7 tests, 4 passed, 3 failed as errors, 0 warnings
```

Reading it. The warehouse has no foreign key, so it accepted all three changes. Three tests caught three different problems: an unknown status, an order line with no order, and a negative quantity. Each failure shows the rows involved, so the investigation starts with facts.

Self-healing loads

Because Chapter 45's hash comparison makes the warehouse match the source (inserting, updating, and deleting), re-running the load repairs damage of this kind:

```
ingest.sync_table(wh, "orders")
ingest.sync_table(wh, "order_items")
results = run_tests(TESTS)
```

```
orders_unique_key          PASS    failing rows: 0
orders_customer_not_null   PASS    failing rows: 0
orders_status_accepted     PASS    failing rows: 0
order_items_have_an_order  PASS    failing rows: 0
order_items_quantity_positive PASS    failing rows: 0
orders_have_at_least_one_line PASS    failing rows: 0
dispatch_order_known       PASS    failing rows: 0

7 tests, 7 passed, 0 failed as errors, 0 warnings
```

The cell re-runs `sync_table()` for both tables, then the same `run_tests(TESTS)`. The load deleted the two invented order lines, which exist nowhere in the ERP, and set order 10177's status back to the ERP's value. That's worth knowing during an incident: for tables loaded by full comparison, "re-run the load" is often the fix, and it's safe because the load is idempotent (Chapter 46).

The same tests as dbt tests

If your transformations live in dbt (Chapter 32), most of these tests are one or two lines of YAML, and dbt writes the SQL. In your Chapter 32 project, `stg_orders` and `stg_order_items` are the staging models over the ERP's two tables. Written in Chapter 32's style (`data_tests:`, with a test's settings under `arguments:`), the rules in `models/staging/schema.yml` look like this:

```
# models/staging/schema.yml (the stg_orders and stg_order_items entries)
version: 2
models:
  - name: stg_orders
    description: "One row per order, from the ERP"
    columns:
      - name: order_id
        data_tests: [unique, not_null]
      - name: customer_id
        data_tests: [not_null]
      - name: status
        data_tests:
          - accepted_values:
              arguments:
                values: ["Pending", "Shipped", "Delivered", "Cancelled"]
  - name: stg_order_items
    columns:
      - name: order_id
        data_tests:
          - relationships:
              arguments:
                to: ref('stg_orders')
                field: order_id
      - name: quantity
        data_tests:
          - dbt_utils.accepted_range:
              arguments:
                min_value: 1
```

- `unique`, `not_null`, `accepted_values`, and `relationships` are the generic tests Chapter 32 used (section 32.7). `relationships` is the anti-join: every `order_id` in the order lines must exist in `stg_orders`.
- `dbt_utils.accepted_range` comes from the `dbt_utils` package. It must be listed in `packages.yml` and downloaded with `dbt deps`, as Chapter 32 showed (section 32.11). `min_value: 1` means quantities of 1 or more pass.

The rule "every order has at least one line" has no generic test, so it's a **singular test** (Chapter 32): a SQL file in the project's `tests/` folder that returns the failing rows, the counterpart of `data_test()`:

```
-- tests/assert_orders_have_lines.sql
-- Every order has at least one order line. A dbt test passes when it returns no rows.
select o.order_id
from {{ ref('stg_orders') }} as o
left join {{ ref('stg_order_items') }} as i on i.order_id = o.order_id
where i.order_item_id is null
```

Run just these models' tests (the Chapter 32 project reads `riverstone_2025`, so there are no January orders here):

```
# terminal, in riverstone_dbt
$ dbt test --threads 1 --select stg_orders stg_order_items
04:11:22 Running with dbt=1.12.5
04:11:23 Found 10 models, 27 data tests, 1 snapshot, 5 sources, 594 macros
04:11:23 1 of 7 PASS accepted_values_stg_orders_status__Pending__Shipped__Delivered__Cancelled [PASS in 0.06s]
04:11:23 2 of 7 PASS assert_orders_have_lines ..... [PASS in 0.02s]
04:11:23 3 of 7 PASS dbt_utils_accepted_range_stg_order_items_quantity__1 ..... [PASS in 0.02s]
04:11:23 4 of 7 PASS not_null_stg_orders_customer_id ..... [PASS in 0.02s]
04:11:23 5 of 7 PASS not_null_stg_orders_order_id ..... [PASS in 0.02s]
04:11:23 6 of 7 PASS relationships_stg_order_items_order_id_order_id_ref_stg_orders_ .. [PASS in 0.03s]
04:11:23 7 of 7 PASS unique_stg_orders_order_id ..... [PASS in 0.02s]
04:11:23 Done. PASS=7 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=7
```

(Trimmed: `dbt` also prints a "START" line before each test, and a few setup lines.) `--threads 1` runs the tests one at a time, so they print in a steady order; `--select` picks the two models, and `dbt` runs every test attached to them. `dbt` names each generic test after the rule, the model, and the column.

To see that `dbt` writes the same kind of query as `data_test()`, open the compiled SQL for one test, in `target/compiled/riverstone_dbt/models/staging/schema.yml/unique_stg_orders_order_id.sql`:

```
select
  order_id as unique_field,
  count(*) as n_records

from "riverstone_2025"."dbt_dev_staging"."stg_orders"
where order_id is not null
group by order_id
having count(*) > 1
```

It's `orders_unique_key` with different names: `dbt` calls the key `unique_field` and the count `n_records`, and names the model in full as database, schema, and view, `"riverstone_2025"."dbt_dev_staging"."stg_orders"` (Chapter 32's development schema). Severity works the same way too: a test that should warn rather than stop gets one more setting. In `dbt`, `config: {severity: warn}` under a test is the counterpart of `severity="warn"`:

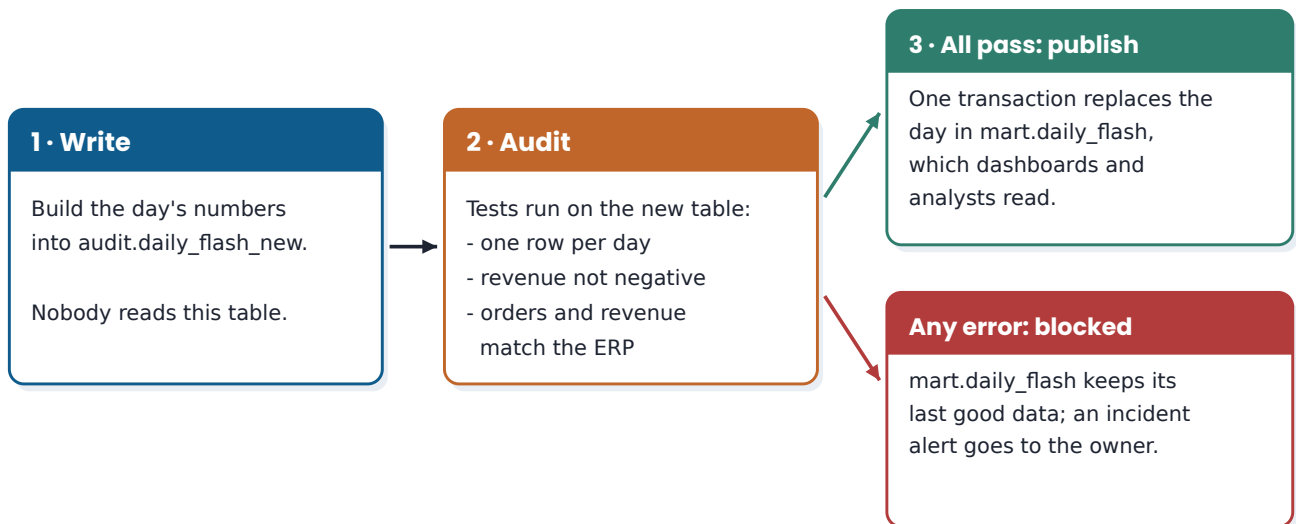
```
- name: order_id
  data_tests:
    - relationships:
        arguments:
          to: ref('stg_orders')
          field: order_id
        config:
          severity: warn
```

`dbt build` then reports that test as `WARN` instead of `ERROR` when it finds rows, and doesn't stop the models downstream of it.

47.4 Write-audit-publish

Chapter 46 left a gap. The check stopped the email, but the wrong row stayed in `mart.daily_flash`, where any dashboard or analyst could read it. **Write-audit-publish** (WAP) closes it:

1. **Write** the new data somewhere readers don't look (an audit or staging schema).
2. **Audit** it: run the tests against the new data.
3. **Publish** only if the tests pass: swap it into the table people read, in one transaction.



Readers of the published table only ever see data that passed its tests.

In Chapter 46 the same failure left a wrong row where anyone could query it.

Figure 47.1 – Write-audit-publish. Readers of the published table only ever see data that passed its tests.

The code comes in four small steps, one for each part of the figure and one to put them together.

Write: build the day where nobody reads it

```
def build_flash_staging(day):
    """Write: build one day's Flash into the audit schema, where nobody reads it."""
    wh.execute("CREATE SCHEMA IF NOT EXISTS audit")
    wh.execute("""
        CREATE OR REPLACE TABLE audit.daily_flash_new AS
        SELECT CAST(? AS DATE) AS flash_date,
               COUNT(DISTINCT o.order_id) AS orders_booked,
               CAST(ROUND(COALESCE(SUM(i.quantity * i.unit_price
                                     * (1 - i.discount_pct / 100)), 0), 2)
                  AS DECIMAL(12,2)) AS revenue_booked
        FROM raw.orders AS o
        JOIN raw.order_items AS i ON i.order_id = o.order_id
        WHERE o.order_date = CAST(? AS DATE) AND o.status <> 'Cancelled'""", [day, day])

    build_flash_staging("2026-01-02")
    print(wh.execute("SELECT * FROM audit.daily_flash_new").fetchall())
```

```
[(datetime.date(2026, 1, 2), 2, Decimal('38710.00'))]
```

How it works.

- **CREATE SCHEMA IF NOT EXISTS audit** makes a separate schema, a named folder of tables inside the warehouse (Chapter 45 used `raw`; Chapter 46 used `mart`). Dashboards and analysts are pointed at `mart`, never at `audit`, so whatever is built here is invisible to them until it's published.
- **CREATE OR REPLACE TABLE ... AS SELECT** builds a table from a query's result, replacing any earlier version. Each run starts from a clean audit table.
- The query is Chapter 46's Flash: orders booked and revenue net of discounts, excluding cancelled orders, for one day.
- The two `?` marks are **parameters**, filled in order from the list `[day, day]`. The day's text never becomes part of the SQL itself (Chapter 45, section 45.3). `CAST(? AS DATE)` turns the text `"2026-01-02"` into a date.
- **ROUND(..., 2)** and **CAST(... AS DECIMAL(12,2))** store revenue as an exact amount in rupees and paise. The sum by itself is a floating-point number, because dividing by 100 produces one. The next step explains why that matters.

The result is one row: 2 January, 2 orders, ₹38,710.00, exactly as in Chapter 46.

The ERP's own answer

The audit needs something to compare with. This is the same Flash query, run on the ERP:

```
ERP_FLASH_SQL = """
    SELECT COUNT(DISTINCT o.order_id),
           ROUND(COALESCE(SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)), 0), 2)
    FROM orders AS o JOIN order_items AS i ON i.order_id = o.order_id
    WHERE o.order_date = %s AND o.status <> 'Cancelled'"""

print(ingest.fetch_rows(ERP_FLASH_SQL, ["2026-01-02"]))
```

```
[(2, Decimal('38710.00'))]
```

How it works. `ingest.fetch_rows(sql, params)` runs the query on the ERP and returns its rows. `%s` is `psycopg2`'s placeholder, where DuckDB uses `?`: two libraries, two placeholder styles, as Chapter 45's placeholder table showed (section 45.3), and both keep the value out of the SQL text. The ERP's tables have no `raw.` prefix, because they're the source itself. PostgreSQL returns money as a `Decimal`, an exact number, and `ROUND(..., 2)` keeps it to the paise.

Why round both sides? Because comparing floating-point numbers exactly is a trap. Before you run this, predict the two answers:

```
print(wh.execute("""
    SELECT 0.1::DOUBLE + 0.2::DOUBLE = 0.3::DOUBLE,
           0.1::DECIMAL(12,2) + 0.2::DECIMAL(12,2) = 0.3::DECIMAL(12,2)""").fetchone())
```

```
(False, True)
```

A **DOUBLE** is a floating-point number, which stores most decimals only approximately (Chapter 17, section 17.3): $0.1 + 0.2$ comes out a hair above 0.3, so the comparison is false. `x::DOUBLE` is shorthand for `CAST(x AS DOUBLE)`, borrowed from PostgreSQL. A **DECIMAL(12,2)** stores exact paise, so the same sum equals 0.3. Money is compared at two decimal places, as exact decimals. Without the rounding, a day whose revenue had a fraction of a paisa in the middle of the arithmetic (3 units at ₹1,234.50 with 7.5% off is ₹3,425.7375) could fail the reconciliation even when both systems agree.

Audit: run the tests on the new data

```
AUDIT_TESTS = [
    data_test("flash_one_row", """
        SELECT flash_date, COUNT(*) FROM audit.daily_flash_new
        GROUP BY flash_date HAVING COUNT(*) <> 1""",
        description="uniqueness: exactly one row for the day"),
    data_test("flash_revenue_not_negative", """
        SELECT flash_date, revenue_booked FROM audit.daily_flash_new WHERE revenue_booked < 0""",
        description="validity: revenue is never negative"),
    data_test("flash_matches_erp", """
        SELECT n.orders_booked AS warehouse_orders, e.orders_booked AS erp_orders,
               n.revenue_booked AS warehouse_revenue, e.revenue_booked AS erp_revenue
        FROM audit.daily_flash_new AS n, audit.erp_expected AS e
        WHERE n.orders_booked <> e.orders_booked OR n.revenue_booked <> e.revenue_booked""",
        description="accuracy: orders and revenue equal the ERP's"),
]

def audit_flash(day):
    """Audit: store the ERP's answer next to the new data, run the audit tests,
    and return True only if no error-severity test failed."""
    erp_orders, erp_revenue = ingest.fetch_rows(ERP_FLASH_SQL, [day])[0]
    wh.execute("""CREATE OR REPLACE TABLE audit.erp_expected AS
        SELECT CAST(? AS INTEGER) AS orders_booked,
               CAST(? AS DECIMAL(12,2)) AS revenue_booked""", [erp_orders, erp_revenue])
    results = run_tests(AUDIT_TESTS)
    return not any(r["failing_rows"] and r["severity"] == "error" for r in results)

print("safe to publish:", audit_flash("2026-01-02"))
```

```
flash_one_row           PASS    failing rows: 0
flash_revenue_not_negative PASS    failing rows: 0
flash_matches_erp        PASS    failing rows: 0

3 tests, 3 passed, 0 failed as errors, 0 warnings
safe to publish: True
```

How it works.

- The three audit tests are ordinary data tests, pointed at the audit table: one row for the day, no negative revenue, and a reconciliation with the ERP. Together they add the sixth dimension, **accuracy**.
- `audit_flash()` first fetches the ERP's answer, and **unpacks** the one row into two names (Chapter 17, section 17.4). It stores them as a one-row table, `audit.erp_expected`, so the reconciliation can be a query like every other test.
- `FROM audit.daily_flash_new AS n, audit.erp_expected AS e` lists two tables with a comma and no **ON**. That pairs every row of the first with every row of the second (a **cross join**, the comma form

Chapter 32's reconciliation test used). Both tables have exactly one row, so the result is exactly one pair, and the `WHERE` keeps it only if the numbers differ.

- `any(...)` is `True` if at least one test failed as an error (the same pattern as `run_tests()`'s list comprehensions, without the brackets). `not any(...)` is therefore "safe to publish".

Publish: swap it in, in one transaction

```
def publish_flash(day):
    """Write, audit, and publish one day of the Flash. Returns True if it was published."""
    print(f"--- write-audit-publish for {day} ---")
    build_flash_staging(day)
    if not audit_flash(day):
        print("PUBLISH BLOCKED: mart.daily_flash left untouched")
        return False
    wh.execute("CREATE SCHEMA IF NOT EXISTS mart")
    wh.execute("""CREATE TABLE IF NOT EXISTS mart.daily_flash (
        flash_date DATE, orders_booked INTEGER, revenue_booked DECIMAL(12,2)""")
    wh.execute("BEGIN")
    try:
        wh.execute("DELETE FROM mart.daily_flash WHERE flash_date = ?", [day])
        wh.execute("""INSERT INTO mart.daily_flash (flash_date, orders_booked, revenue_booked)
            SELECT flash_date, orders_booked, revenue_booked FROM audit.daily_flash_new""")
        wh.execute("COMMIT")
    except Exception:
        wh.execute("ROLLBACK")
        raise
    print("PUBLISHED")
    return True

publish_flash("2026-01-02")
print("mart.daily_flash:", wh.execute("SELECT * FROM mart.daily_flash ORDER BY
    flash_date").fetchall())
```

```
--- write-audit-publish for 2026-01-02 ---
flash_one_row          PASS    failing rows: 0
flash_revenue_not_negative PASS    failing rows: 0
flash_matches_erp      PASS    failing rows: 0

3 tests, 3 passed, 0 failed as errors, 0 warnings
PUBLISHED
mart.daily_flash: [(datetime.date(2026, 1, 2), 2, Decimal('38710.00'))]
```

How it works.

- `publish_flash()` runs the three stages in order: write, audit, and, only if the audit passed, publish. If any error-severity test failed, it prints why and returns `False` without touching `mart.daily_flash`.
- `CREATE TABLE IF NOT EXISTS` makes the published table the first time only.
- **BEGIN ... COMMIT** is a transaction: the delete and the insert happen together or not at all, so a reader never sees the day missing halfway through a swap.
- Deleting the day before inserting it makes the step **idempotent**, exactly as in Chapter 46 (section 46.4): publishing the same day twice leaves one row, not two.
- The `INSERT` names its columns in both lists, so it still works if someone later adds a column to either table in a different position.

- **try / except** with **ROLLBACK**: if the delete or insert fails, the transaction is undone, so the connection isn't left half-way through it, and **raise** passes the original error on so it isn't hidden.

The same bad morning, with WAP

Now repeat Chapter 46's 5 January failure: the order-line load silently loads nothing.

```
apply_day(2)                                # the 5 January business day (Chapter 45)
ingest.BROKEN_ITEMS_SYNC = True              # Chapter 46's bug: the order-line load loads nothing
ingest.sync_table(wh, "orders")
ingest.sync_table(wh, "order_items")

publish_flash("2026-01-05")
print("mart.daily_flash:", wh.execute("SELECT * FROM mart.daily_flash ORDER BY flash_date").fetchall())
print("readers of mart.daily_flash never saw the 5 January row:",
      wh.execute("SELECT COUNT(*) FROM mart.daily_flash WHERE flash_date = '2026-01-05'").fetchone()[0] ==
      0)
```

```
--- write-audit-publish for 2026-01-05 ---
flash_one_row          PASS    failing rows: 0
flash_revenue_not_negative PASS    failing rows: 0
flash_matches_erp      ERROR    failing rows: 1  e.g. [(0, 1, Decimal('0.00'),
      Decimal('18750.00'))]

3 tests, 2 passed, 1 failed as errors, 0 warnings
PUBLISH BLOCKED: mart.daily_flash left untouched
mart.daily_flash: [(datetime.date(2026, 1, 2), 2, Decimal('38710.00'))]
readers of mart.daily_flash never saw the 5 January row: True
```

Reading it. `ingest.BROKEN_ITEMS_SYNC = True` switches on the simulated bug in the companion module, the same switch Chapter 46 used (section 46.7). The reconciliation test failed: the ERP has 1 order for ₹18,750.00, the new data has 0 orders and ₹0.00. Publishing was blocked, and `mart.daily_flash` still holds only the good 2 January row. In Chapter 46, the same failure left a wrong row in the published table. Now readers never see it: they see the last good data, and the freshness check in the next section is what tells them (and the on-call engineer) that today's is missing.

The bug stays switched on until section 47.9 fixes it and publishes the day, so sections 47.5 to 47.9 find the problem the way a real team would.

Watch out: "no data" is also a state people can misread. A missing day can look like a zero day on a chart. Publish a status alongside the data (for example, a `load_status` table saying which days are published, late, or failed), and show it on dashboards, so "we don't know yet" never looks like "we sold nothing".

Variations

- **Blue/green tables:** build a whole new table, test it, then swap names. Good for full rebuilds.
- **Table snapshots or time travel:** warehouses such as Snowflake, BigQuery, and Delta or Iceberg tables (Chapter 49) can keep previous versions, so publishing is a metadata change and rolling back is quick.
- **Views over the last good partition:** readers query a view that points at published partitions only.

47.5 Observability: freshness, volume, and unusual values

Tests answer "is this data right?" **Observability** answers "is the platform behaving normally?" Three signals carry most of the value.

Freshness

Freshness is the age of the newest data. Stale data is one of the most common data incidents, and the hardest to notice, because nothing failed: a report keeps showing last Tuesday's numbers.

A freshness check needs a date column that says which day each row belongs to. For orders it's `order_date`. For the dispatch file, look at what `raw.dispatch` holds:

```
for row in wh.execute("DESCRIBE raw.dispatch").fetchall():
    print(row[:2])
```

```
('dispatch_id', 'VARCHAR')
('order_id', 'INTEGER')
('dispatch_date', 'VARCHAR')
('qty_units', 'VARCHAR')
('file_day', 'DATE')
```

`DESCRIBE` lists a table's columns and types (Chapter 45, section 45.7), and `row[:2]` keeps the name and the type. The last column, **file_day**, isn't in the file: `load_dispatch()` (Chapter 46, section 46.3) stamps every row with the date in the file's name, `dispatch_2026-01-02.csv`, which is also how it replaces a day's rows when the same file is loaded again. `dispatch_date` stays as text, because the file writes dates as dd-mm-yyyy (Chapter 45 types it in staging). So `file_day` is the column that says how fresh the dispatch data is.

How fresh is fresh enough? Riverstone's Flash is promised on **working days**, Monday to Friday. At the 6:30 a.m. run, the newest data should be from the previous working day. That day is easy to work out:

```
from datetime import date, datetime, time, timedelta

def previous_working_day(day):
    """The last Monday-to-Friday date before `day`."""
    day = day - timedelta(days=1)
    while day.weekday() >= 5:          # Monday is 0 ... Saturday is 5, Sunday is 6
        day = day - timedelta(days=1)
    return day

print("the run on Tuesday 6 January expects:", previous_working_day(date(2026, 1, 6)))
print("the run on Monday 5 January expects: ", previous_working_day(date(2026, 1, 5)))
```

```
the run on Tuesday 6 January expects: 2026-01-05
the run on Monday 5 January expects:  2026-01-02
```

How it works. `timedelta(days=1)` is one day (Chapter 17, section 17.11), so `day - timedelta(days=1)` is the day before. `.weekday()` numbers the days of the week from Monday, 0, to Sunday, 6. The `while` loop keeps stepping back over Saturday and Sunday, so on a Monday the answer is the Friday before. (Public holidays need a list of their own; the loop would skip those dates too.)

Now the check itself:

```
NOW = datetime(2026, 1, 6, 6, 30)          # the 6:30 a.m. run on Tuesday 6 January, as a fixed clock

def freshness_check(table, date_column, now=NOW):
    newest = wh.execute(f"SELECT MAX({date_column}) FROM {table}").fetchone()[0]
    expected = previous_working_day(now.date())
    age_hours = (now - datetime.combine(newest, time(0, 0))).total_seconds() / 3600
    status = "PASS" if newest >= expected else "STALE"
    print(f"{table:<17} newest {newest} expected {expected} age {age_hours:5.1f} h {status}")
    return (table, age_hours, status)

freshness = [freshness_check("raw.orders", "order_date"),
              freshness_check("raw.dispatch", "file_day"),
              freshness_check("mart.daily_flash", "flash_date")]
```

raw.orders	newest	2026-01-05	expected	2026-01-05	age	30.5 h	PASS
raw.dispatch	newest	2026-01-02	expected	2026-01-05	age	102.5 h	STALE
mart.daily_flash	newest	2026-01-02	expected	2026-01-05	age	102.5 h	STALE

How it works, line by line.

- **NOW** is a fixed clock: 6:30 a.m. on Tuesday 6 January. With a fixed time, the output is the same every time you run it. In production, `now` would be `datetime.now()`.
- `f"SELECT MAX({date_column}) FROM {table}"` puts the table and column **names** into the SQL text. Parameters (?) can only stand for values, not names, so names have to be pasted in. Only pass names you control, never text typed by a user (Chapter 45).
- `expected` is the previous working day, from the helper above. The status is **PASS** if the newest date is that day or later, and **STALE** if it's older.
- `age_hours` is for people reading the report. A date has no time of day, so `datetime.combine(newest, time(0, 0))` treats it as midnight at the start of that day. `now - ...` is a `timedelta`, `.total_seconds()` turns it into seconds, and dividing by 3,600 gives hours. So yesterday's orders are already $24 + 6.5 = 30.5$ hours old at 6:30 this morning.
- `{age_hours:5.1f}` prints the hours with one decimal place in five characters, and `{table:<17}` lines up the names.
- The function returns the table, its age, and its status as a tuple, so section 47.9 can build its alert from these real results.

Reading it. Orders are fresh: the newest is from 5 January, the previous working day. Two things are stale. `raw.dispatch` hasn't been loaded since 2 January: the 5 January file did arrive, but with changed columns (section 47.7), so it wasn't loaded. A load that fails or never runs looks exactly like a file that never came, and freshness catches both. And `mart.daily_flash` has no 5 January row, because publishing was blocked. Both are real problems that no test on the *contents* would find.

Setting limits. Base each limit on the promise made to users, not on how the pipeline usually behaves. Riverstone's Flash promise is "sent by 7:30 a.m. IST on working days", so at the 6:30 run the newest day must be the previous working day. Why not a limit in hours? Because the hours move with the calendar:

```
tuesday_age = datetime(2026, 1, 6, 6, 30) - datetime(2026, 1, 5)      # Tuesday's run, Monday's data
monday_age = datetime(2026, 1, 5, 6, 30) - datetime(2026, 1, 2)      # Monday's run, Friday's data
print("Monday's data at Tuesday's run:", tuesday_age.total_seconds() / 3600, "hours")
print("Friday's data at Monday's run: ", monday_age.total_seconds() / 3600, "hours")
```

```
Monday's data at Tuesday's run: 30.5 hours
Friday's data at Monday's run: 78.5 hours
```

Perfectly fresh data is 30.5 hours old on a Tuesday morning, and 78.5 hours old on a Monday morning, because 2 January 2026 is a Friday. A limit of "under 24 hours" would call good data stale every morning; a limit of 48 hours would call it stale every Monday and after every holiday. If you must use hours, you need two limits (about 32 hours on Tuesday to Friday, 80 on Mondays). Comparing with the previous working day says the promise directly. Check freshness *before* using a table, and monitor it separately from the pipeline, so a pipeline that never starts is noticed too.

Volume and unusual values

Volume checks compare the size of today's data with what's normal: rows loaded, orders booked, revenue. The classic silent failure is a load that brings in far less than usual, or nothing at all.

Comparing against history needs a baseline. A **median** is a good one, because a single huge day doesn't drag it around the way an average does (Chapter 21). The warehouse holds all of 2025 as well as the January days, so first get each 2025 day's orders and revenue:

```
import statistics

daily = wh.execute("""
    SELECT o.order_date, COUNT(DISTINCT o.order_id) AS orders,
           SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)) AS revenue
    FROM raw.orders AS o JOIN raw.order_items AS i ON i.order_id = o.order_id
    WHERE o.status <> 'Cancelled' AND o.order_date BETWEEN DATE '2025-01-01' AND DATE '2025-12-31'
    GROUP BY o.order_date""").fetchall()
median = statistics.median(rev for d, n, rev in daily)
unusual = [(str(d), rev) for d, n, rev in daily if rev > 3 * median]
print("days with orders in 2025:", len(daily))
print("median day's revenue      : Rs", f"{median:,.2f}")
print("days above 3x the median:", len(unusual))
for day, rev in sorted(unusual, key=lambda pair: pair[1], reverse=True)[:3]:
    print(f" {day} Rs {rev:,.2f}")
```

```
days with orders in 2025: 131
median day's revenue      : Rs 27,442.50
days above 3x the median: 10
2025-11-16 Rs 175,895.00
2025-11-09 Rs 115,847.50
2025-09-13 Rs 108,735.00
```

How it works.

- The query returns one row per 2025 day with orders: the date, the number of orders, and the revenue. `.fetchall()` gives a list of those three-item tuples.
- `for d, n, rev in daily` **unpacks** each tuple into three names (Chapter 17, section 17.4). The median only needs `rev`.
- `statistics.median()` (Chapter 17, section 17.11) sorts the values and takes the middle one, or the average of the two middle ones when the count is even. Here 131 days means the 66th value.
- `unusual` keeps the days above three times the median. `str(d)` turns each date into text for printing. The revenue here is a floating-point number, which is fine for a rough comparison like this one; only published money is kept exact.
- `sorted(..., key=lambda pair: pair[1], reverse=True)` sorts by the second item of each pair, the revenue, largest first. `lambda` is a one-line function without a name (Chapter 18). `[:3]` keeps the top three.
- The program prints `Rs` rather than `₹` so the output reads correctly in any terminal, and Python's `,` format groups digits in threes; in the text, amounts use the book's lakh grouping.

Reading it. Across 2025, Riverstone had orders on 131 days, and a typical day brought ₹27,442.50. Ten days were more than three times that, the biggest being ₹1,75,895.00 on 16 November. Those aren't errors: they're large orders and seasonal peaks. That's the lesson of anomaly detection on business data: **unusual is not the same as wrong**. Use these signals to prompt a look, not to block a pipeline.

Sensible volume rules for Riverstone:

- Error if a load that should bring new rows brings **zero** on a working day.
- Error if a published day's revenue is **negative**.
- Warn if a day's revenue is **outside a wide band** around the median of recent days (for example, below a quarter or above five times).
- Warn if row counts in a raw table **fall** compared with the previous load, since sources rarely lose rows.

Watch out: don't learn your baseline from broken days. If the baseline is the average of recent days and last week was broken, "normal" quietly moves toward broken. Exclude days with failed loads from the baseline, or use a longer window and a median.

The three questions to monitor

1. **Did it run?** Pipeline success or failure, and run duration (a run that suddenly takes three times longer is a warning).
2. **Is it fresh?** Age of the newest data in each important table.
3. **Does it look normal?** Row counts and key totals against recent history.

Chapter 46's orchestrator answers the first. This section answers the other two. Together they're what "observability" means for data.

47.6 Lineage: what breaks when this breaks

Lineage is the record of where data comes from and where it goes: which sources feed a table, and which tables, dashboards, and reports depend on it. It answers two questions that come up in every incident:

- **Impact:** this table is wrong. What else is wrong, and who should be told?
- **Cause:** this number is wrong. Which upstream table or source could explain it?

Chapter 46's pipeline already declares its dependencies (each asset's `deps`), and Dagster's web interface draws them as a graph. Here the graph is copied by hand from Chapter 46's asset definitions into a dictionary, with two more reports added to show the point:

```
GRAPH = {                                     # asset -> what it is built from (Chapter 46's
    pipeline)
    "raw_orders": [], "raw_order_items": [], "raw_dispatch": [], "raw_crm_leads": [],
    "daily_flash": ["raw_orders", "raw_order_items"],
    "deliver_flash": ["daily_flash"],
    "dispatch_report": ["raw_dispatch", "raw_orders"],
    "weekly_pipeline_report": ["raw_crm_leads", "raw_orders"],
}

def downstream_of(asset, graph=GRAPH):
    """Every asset built, directly or indirectly, from `asset`."""
    affected, queue = set(), [asset]
    while queue:
        current = queue.pop()
        for name, parents in graph.items():
            if current in parents and name not in affected:
                affected.add(name)
                queue.append(name)
    return sorted(affected)

for broken in ["raw_order_items", "raw_dispatch"]:
    print(f"{broken} is broken -> affected: {downstream_of(broken)}")
```

```
raw_order_items is broken -> affected: ['daily_flash', 'deliver_flash']
raw_dispatch is broken -> affected: ['dispatch_report']
```

How it works, line by line.

- `GRAPH` maps each asset to the list of assets it's built from: its **parents**. The four raw tables have none.
- `affected` is a **set** (Chapter 17, section 17.7) of assets found so far; a set can't hold the same name twice. `queue` is a list of assets still to look at, starting with the broken one.
- `while queue:` repeats until the queue is empty. `queue.pop()` takes the last name off the list.
- The `for` loop looks through every asset and its parents. If the current asset is one of its parents, that asset is affected: add it to the set, and put it on the queue so its children are found too.
- By hand, for `raw_order_items`: the queue starts as `["raw_order_items"]`. Taking it off finds `daily_flash`, which lists it as a parent; `daily_flash` goes on the queue. Taking `daily_flash` off finds `deliver_flash`. Taking `deliver_flash` off finds nothing new. The queue is empty, so the answer is `['daily_flash', 'deliver_flash']`.

The other question, **cause**, walks the same graph the other way, from a wrong output back to everything it's built from:

```
def upstream_of(asset, graph=GRAPH):
    """Every asset that `asset` is built from, directly or indirectly."""
    found, queue = set(), [asset]
    while queue:
        current = queue.pop()
        for parent in graph[current]:
            if parent not in found:
                found.add(parent)
                queue.append(parent)
    return sorted(found)

print("deliver_flash is built from:", upstream_of("deliver_flash"))
```

```
deliver_flash is built from: ['daily_flash', 'raw_order_items', 'raw_orders']
```

The loop is the same; only the direction changes. Instead of searching for assets that list the current one as a parent, it reads the current asset's own parents, `graph[current]`.

Reading it. Broken order lines affect the Flash and its delivery. A broken dispatch file affects only the dispatch report. That's the difference between an alert to the sales managers and a quiet note to the warehouse team, and it's why the Flash isn't wired to depend on dispatches. And if the Flash's number is wrong, only three things could explain it: `daily_flash` itself, `raw_order_items`, or `raw_orders`. The CRM and the dispatch file can be ruled out without looking.

In a real platform, lineage comes from the orchestrator's graph (Dagster's asset graph, Airflow datasets), from dbt's model graph (Chapter 32, section 32.8), and from tools that read query logs to see which dashboards touched which tables. **Column-level lineage** goes further, tracing which source column feeds which report field, which helps when a single measure looks wrong.

Interview extra point. "A dashboard shows wrong numbers. Walk me through it." A strong answer starts with lineage and freshness: which tables feed it, when were they last loaded, did their tests pass, and what changed upstream today. Chapter 77 has more of these.

47.7 Data contracts in practice

Chapter 45 ended with Bhiwandi Main renaming a column and breaking a load. A **data contract** is a written agreement between a data producer and the data team about what will be delivered: the fields, their types and rules, when it arrives, who owns it, and how much notice you get before it changes.

Contracts work best when they're small, written in plain language, and **checked automatically**, so a violation is caught by the pipeline, not by an argument in a meeting. First, the contract itself, written as a Python dictionary so the pipeline can read it:

```
DISPATCH_CONTRACT = {
    "dataset": "bhiwandi_dispatch_sheet",
    "owner": "Bhiwandi Main warehouse supervisor",
    "consumer": "data team (raw.dispatch)",
    "delivered_by": "20:00 IST on the day of dispatch",
    "change_notice_days": 10,
    "columns": {
        "dispatch_id": {"type": "text", "unique": True, "nullable": False},
        "order_id": {"type": "integer", "nullable": False},
        "dispatch_date": {"type": "date dd-mm-yyyy", "nullable": False},
        "warehouse": {"type": "text", "nullable": False},
        "qty_units": {"type": "integer", "nullable": True, "note": "blank allowed while a count is pending"},
        "remarks": {"type": "text", "nullable": True},
    },
}
print(len(DISPATCH_CONTRACT["columns"]), "columns promised by", DISPATCH_CONTRACT["owner"])
```

```
6 columns promised by Bhiwandi Main warehouse supervisor
```

Each column has a type, whether it may be blank (nullable), and, for `dispatch_id`, a promise that no value repeats (unique). The first check compares the file's columns with the contract's:

```
import os

def check_contract(path, contract):
    header = wh.execute(f"DESCRIBE SELECT * FROM read_csv('{path}', all_varchar = true)").fetchall()
    found = [r[0] for r in header]
    expected = list(contract["columns"])
    violations = [f"missing column: {c}" for c in expected if c not in found]
    violations += [f"undeclared column: {c}" for c in found if c not in expected]
    return violations

for f in ["exports/dispatch_2026-01-02.csv", "exports/dispatch_2026-01-05.csv"]:
    v = check_contract(f, DISPATCH_CONTRACT)
    print(f"{os.path.basename(f)}: {'columns match the contract' if not v else v}")
```

```
dispatch_2026-01-02.csv: columns match the contract
dispatch_2026-01-05.csv: ['missing column: qty_units', 'undeclared column: units', 'undeclared column: vehicle_no']
```

How it works.

- `read_csv(path, all_varchar = true)` reads the file with every column as text (Chapter 45, section 45.8), so DuckDB's guesses about types can't fail or hide anything. Only the header matters here.
- `DESCRIBE SELECT ...` returns one row per column the query would produce; `r[0]` is the column's name. The list comprehension collects the names into `found`.
- `list(contract["columns"])` lists the dictionary's keys: the six promised names.
- The two comprehensions find names that are promised but missing, and names that appear without being promised. An empty list means the columns match.
- `os.path.basename(f)` keeps the file name without its folder, for a shorter line.

Reading it. The 2 January file has the agreed columns. The 5 January file breaks the contract in three ways: the agreed `qty_units` column is gone, and two columns nobody agreed to have appeared. The pipeline stops, the owner is told exactly what changed, and nobody has to guess whether `units` means the same thing.

Matching column names is only half of what the contract promises. The rules about **values** can be checked too, with the failing-rows idea from section 47.2. This function checks three of them for every column: no blanks where blanks aren't allowed, no repeats where values must be unique, and whole numbers where the type is an integer:

```
def check_contract_values(path, contract):
    source = f"read_csv('{path}', header = true, all_varchar = true)"
    violations = []
    for col, rules in contract["columns"].items():
        if not rules["nullable"]:
            blanks = wh.execute(f"SELECT COUNT(*) FROM {source} WHERE {col} IS NULL").fetchone()[0]
            if blanks:
                violations.append(f"{col}: {blanks} blank value(s)")
        if rules.get("unique"):
            repeats = wh.execute(f"SELECT COUNT(*) - COUNT(DISTINCT {col}) FROM {source}").fetchone()[0]
            if repeats:
                violations.append(f"{col}: {repeats} repeated value(s)")
        if rules["type"] == "integer":
            bad = wh.execute(f"""SELECT {col} FROM {source}
                                WHERE {col} IS NOT NULL
                                AND TRY_CAST(replace({col}, ',', '' ) AS INTEGER) IS
                                NULL""").fetchall()
            if bad:
                violations.append(f"{col}: not a whole number: {[b[0] for b in bad]}")
    return violations

print(check_contract_values("exports/dispatch_2026-01-02.csv", DISPATCH_CONTRACT))
```

```
["qty_units: not a whole number: ['N/A']"]
```


How it works.

- `.items()` gives each column's name and its rules together (Chapter 17, section 17.7).
- **Blanks:** for a column with `"nullable": False`, count the rows where it's empty. Any count above zero is a violation.
- **Repeats:** `COUNT(*) - COUNT(DISTINCT dispatch_id)` is the number of rows whose value has appeared before; zero means every value is unique. `rules.get("unique")` returns `None` for columns without that rule, so they're skipped.
- **Whole numbers:** Chapter 45's pattern (section 45.7). `replace(qty_units, ',', '')` removes the thousands separator from `"1,200"`, and `TRY_CAST(... AS INTEGER)` returns `NULL` instead of an error when the text isn't a number. A value that isn't blank but can't be converted breaks the rule.
- The function only makes sense on a file whose columns already match; the 5 January file has no `qty_units` column to check.

Reading it. The 2 January file matches the contract's columns, but not its values: one row says `N/A` where the contract says a pending count is left **blank**. Chapter 45 quietly turned `N/A` into a missing value in staging. The contract turns it into a conversation: either the sheet leaves the cell empty, or the contract adds `N/A` as an agreed marker. Either answer is fine; what matters is that it's written down.

The contract's other promises need other tools. The **date format** could be checked the same way, with DuckDB's `TRY_STRPTIME`. The **delivery deadline**, 20:00 IST, is a freshness promise: the freshness monitor from section 47.5 checks it, by asking whether that day's file has been loaded.

What a good contract contains

Part	Riverstone's dispatch sheet
Dataset and purpose	Daily dispatch sheet; feeds <code>raw.dispatch</code> and the dispatch report
Producer and consumer	Bhiwandi Main warehouse supervisor → data team
Fields	Names, types, whether blanks are allowed, and what each means
Rules	<code>dispatch_id</code> unique; <code>order_id</code> exists in the ERP; quantity a whole number
Delivery	One file per working day by 20:00 IST, named <code>dispatch_YYYY-MM-DD.csv</code>
Change process	10 working days' notice; new columns may be added, existing ones not renamed or removed without agreement
What happens on a breach	Load stops; alert to both sides; previous data stays published
Review	Revisited every six months, or when the process changes

Contracts inside the warehouse

Contracts aren't only for files from outside. A dbt model that other models and dashboards read is a promise too, and dbt can enforce it. A **model contract** lists the model's columns and their types, and dbt refuses to build the model if its SQL no longer produces exactly those:

```
- name: stg_orders
  config:
    contract: {enforced: true}
  columns:
    - {name: order_id, data_type: integer}
    - {name: customer_id, data_type: integer}
    - {name: sales_rep_id, data_type: integer}
    - {name: order_date, data_type: date}
    - {name: status, data_type: varchar}
    - {name: is_counted, data_type: boolean}
```

{...} is YAML's way of writing a small map on one line: {name: order_id, data_type: integer} means the same as two lines, name: order_id and data_type: integer. With enforced: true, every column the model produces must be listed with its data_type. If someone renames status to order_status in stg_orders.sql, dbt run stops before replacing the model, with a table of the differences:

```
# terminal, in riverstone_dbt
$ dbt run --threads 1 --select stg_orders
04:16:15 1 of 1 ERROR creating sql view model dbt_dev_staging.stg_orders ..... [ERROR in 0.09s]
04:16:16 Compilation Error in model stg_orders (models/staging/stg_orders.sql)
  This model has an enforced contract that failed.
  Please ensure the name, data_type, and number of columns in your contract match the columns in your model's
  definition.
```

column_name	definition_type	contract_type	mismatch_reason
order_status	TEXT		missing in contract
status		TEXT	missing in definition

(Trimmed: dbt also prints its setup lines, and the list of macros that raised the error.) The rename is caught before any dashboard reads the new column name, which is exactly where a contract should catch it.

Making contracts work with people

- **Write it with the producer, not at them.** The warehouse supervisor isn't trying to break your pipeline; they added a column because a manager asked for vehicle numbers.
- **Explain the cost of a surprise change** in their terms: "when the column changed, the dispatch report was unavailable for a day."
- **Make the safe path the obvious one:** adding columns is always allowed; renaming needs a note. Most breakages come from renames and type changes.
- **Version the contract,** and keep it in Git next to the pipeline code.
- **Contracts go both ways.** When your team publishes data others depend on (Chapter 51), you owe them the same promises.

47.8 Handling a data incident

When a check fails at 6:30 a.m., what happens next should be routine, not improvised.

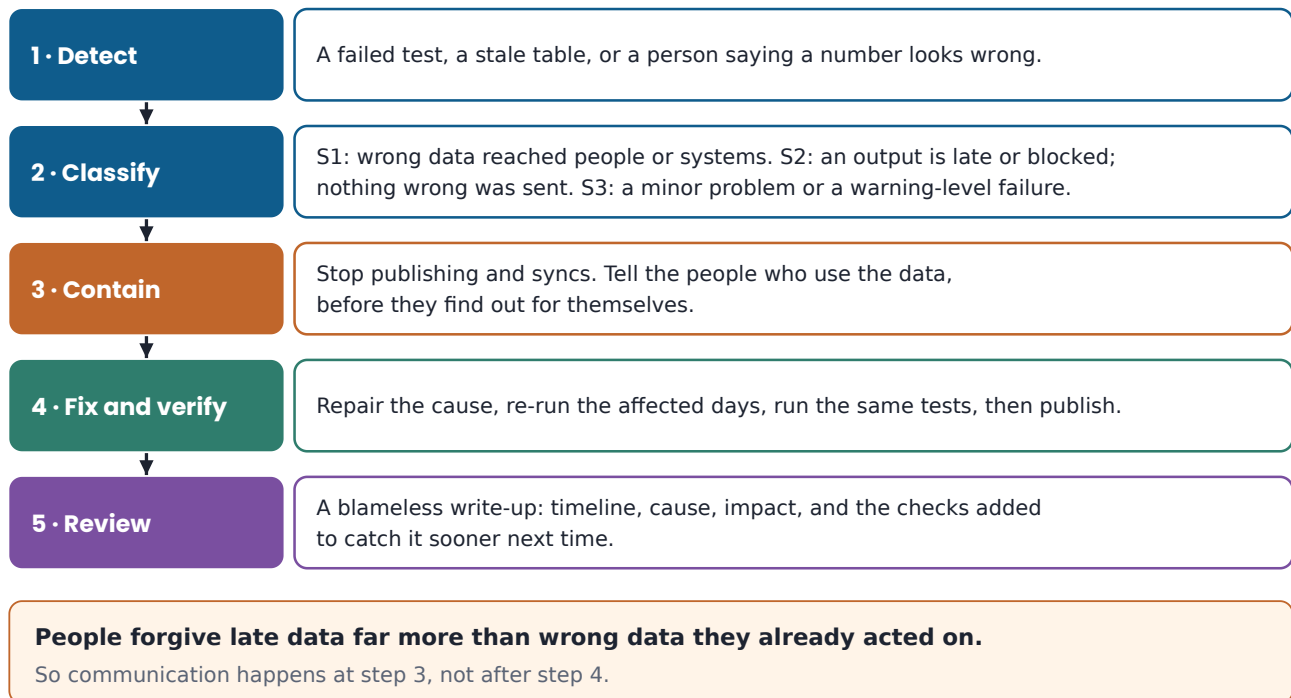


Figure 47.2 – A data incident, step by step. Communication happens before the fix, not after.

1. Detect. From a failed test, a freshness alert, or a person. Treat "a manager says the number looks wrong" as an incident: it means detection failed.

2. Classify. Severity decides how much noise to make.

Severity	Meaning	Riverstone example	Response
S1	Wrong data has reached people or another system	Wrong credit limits synced into the ERP; the Flash sent with wrong numbers	Immediate: tell recipients, correct, then fix
S2	A daily output is late or blocked; no wrong numbers went out	5 January's Flash not published	Same morning; tell users the report is late
S3	A non-critical problem or a warning-level test failure	One dispatch row points at an unknown order	Next working day; fix in normal work

3. Contain. Stop publishing and stop side effects: the blocked publish in section 47.4 is containment. Then tell the people who use the data, in their words: *"The Daily Sales Flash for 5 January is delayed. The order lines didn't load, so the numbers would have shown zero sales. Nothing has been sent. We expect it by 11:00."* People forgive late data far more than wrong data they acted on.

4. Fix and verify. Repair the cause, re-run the affected days (Chapter 46's partitions and idempotency), run the same tests, and publish. Section 47.9 does exactly this for 5 January, after the alert goes out. If wrong numbers did go out, send a labeled correction.

5. Review. A short write-up, within a few days, blameless and factual: what happened, the timeline, the cause, the impact, what was done, and what will stop it or catch it faster next time. For the 5 January incident, the review adds `orders_have_at_least_one_line` to the blocking audit before publishing (it existed as a data test, but didn't gate the Flash), plus a volume check for "zero new order lines on a day with new orders".

Real-life example: the missing piece in most incidents. Teams usually fix the data and forget step 5. Six weeks later the same failure happens, and nobody remembers the fix. Ten minutes of writing, kept next to the runbook, is what turns an incident into an improvement.

47.9 Alerts people still read

Alerting fails in two directions: too few, and too many. **Alert fatigue** is what happens when a channel fills with messages nobody acts on, and the real one scrolls past.

Grouping one incident into one message

A single failure usually trips several signals. Sending five messages makes it look like five problems. This function gathers the morning's real results, from the tests, the freshness checks, and the contract check, into one message:

```
SEVERITY_MEANING = {"S1": "wrong data reached people or systems",
                    "S2": "a daily report is late or blocked; no wrong numbers were sent",
                    "S3": "a minor problem or a warning"}

def group_into_one_alert(day, test_results, freshness_results, contract_violations, severity="S2"):
    errors = [r for r in test_results if r["failing_rows"] and r["severity"] == "error"]
    warnings = [r for r in test_results if r["failing_rows"] and r["severity"] == "warn"]
    stale = [f"{table} ({hours:.0f} h old)"
             for table, hours, status in freshness_results if status == "STALE"]
    if not (errors or warnings or stale or contract_violations):
        return None
    lines = [f"DATA INCIDENT - {day} - Daily Sales Flash not published",
             f"Severity: {severity} ({SEVERITY_MEANING[severity]})", ""]
    if errors:
        lines.append("Failed tests (errors):")
        lines += [f"  - {r['name']}: {r['failing_rows']} failing row(s)" for r in errors]
    if warnings:
        lines.append("Warnings (not blocking):")
        lines += [f"  - {r['name']}: {r['failing_rows']} row(s)" for r in warnings]
    if stale:
        lines.append("Stale data: " + ", ".join(stale))
    if contract_violations:
        lines.append("Contract violations: " + "; ".join(contract_violations))
    lines += ["", "Impact: sales managers have no Flash for this day; mart.daily_flash is
              unchanged.",
              "Next step: fix the order-line load, re-run the day, and publish.",
              "Owner: data team on-call. Runbook: docs/runbooks/daily_flash.md"]
    return "\n".join(lines)

print(group_into_one_alert("2026-01-05", [], [], []))
```

None

The function is long, but it only collects and formats; there's no new idea in it. Called with nothing to report, as above, it returns `None`: no message at all is the right message on a good morning.

How it works.

- `SEVERITY_MEANING` is a dictionary from each severity to its meaning, so the message always explains itself. `severity="S2"` is the default, because a blocked Flash with nothing wrong sent is an S2 (section 47.8); an S1 would pass `severity="S1"`.
- The first three assignments sort the inputs: failed error tests, failed warnings, and the tables whose freshness status is `STALE`. for `table, hours, status in freshness_results` unpacks each tuple that `freshness_check()` returned in section 47.5, and `{hours: .0f}` rounds the age to whole hours.
- If there is something to report, it builds the message as a list of lines, adding each part only if it has something in it, and `"\n".join(lines)` joins them with line breaks into one text.

Now call it with the morning's real results:

```
test_results = run_tests(TESTS) + run_tests(AUDIT_TESTS)
alert = group_into_one_alert("2026-01-05", test_results, freshness,
                             check_contract("exports/dispatch_2026-01-05.csv", DISPATCH_CONTRACT))
print()
print(alert)
```

```
orders_unique_key          PASS    failing rows: 0
orders_customer_not_null   PASS    failing rows: 0
orders_status_accepted     PASS    failing rows: 0
order_items_have_an_order  PASS    failing rows: 0
order_items_quantity_positive PASS    failing rows: 0
orders_have_at_least_one_line ERROR    failing rows: 1  e.g. [(10178,)]
dispatch_order_known       PASS    failing rows: 0

7 tests, 6 passed, 1 failed as errors, 0 warnings
flash_one_row              PASS    failing rows: 0
flash_revenue_not_negative PASS    failing rows: 0
flash_matches_erp          ERROR    failing rows: 1  e.g. [(0, 1, Decimal('0.00'),
    Decimal('18750.00'))]

3 tests, 2 passed, 1 failed as errors, 0 warnings

DATA INCIDENT - 2026-01-05 - Daily Sales Flash not published
Severity: S2 (a daily report is late or blocked; no wrong numbers were sent)

Failed tests (errors):
- orders_have_at_least_one_line: 1 failing row(s)
- flash_matches_erp: 1 failing row(s)
Stale data: raw.dispatch (102 h old), mart.daily_flash (102 h old)
Contract violations: missing column: qty_units; undeclared column: units; undeclared column:
    vehicle_no

Impact: sales managers have no Flash for this day; mart.daily_flash is unchanged.
Next step: fix the order-line load, re-run the day, and publish.
Owner: data team on-call. Runbook: docs/runbooks/daily_flash.md
```

How it works. `run_tests(TESTS) + run_tests(AUDIT_TESTS)` runs both suites against the warehouse as it stands on 5 January and adds the two lists of results together. `freshness` is the list of results from section 47.5, and the contract violations come straight from `check_contract()` in section 47.7.

Reading it. One message: both failed tests, the two stale tables, the contract violation, the impact, the next step, the owner, and the runbook. `flash_matches_erp` is the test that blocked publishing; `orders_have_at_least_one_line` points at the cause, order 10178 with no lines. In a real deployment this goes to the on-call channel and, for an S1, to the people who received wrong numbers.

The run groups everything into one message, but not everything in it has the same owner. The contract breach is also sent, as a separate message, to its owner, `DISPATCH_CONTRACT["owner"]`, the Bhiwandi Main supervisor: it happened on the same morning as the Flash incident, but it has a different cause.

Closing the incident: fix, verify, publish

The alert's next step is the fix. The order-line bug is repaired (in the companion, by switching the simulated bug off), the day is re-run, and write-audit-publish decides whether it's now safe to publish:

```
ingest.BROKEN_ITEMS_SYNC = False          # the order-line bug is fixed
print("order lines (new, changed, deleted):", ingest.sync_table(wh, "order_items"))
publish_flash("2026-01-05")
print("mart.daily_flash:", wh.execute("SELECT * FROM mart.daily_flash ORDER BY
    flash_date").fetchall())
flash_freshness = freshness_check("mart.daily_flash", "flash_date")
```

```
order lines (new, changed, deleted): (1, 0, 0)
--- write-audit-publish for 2026-01-05 ---
flash_one_row          PASS    failing rows: 0
flash_revenue_not_negative PASS    failing rows: 0
flash_matches_erp      PASS    failing rows: 0

3 tests, 3 passed, 0 failed as errors, 0 warnings
PUBLISHED
mart.daily_flash: [(datetime.date(2026, 1, 2), 2, Decimal('38710.00')), (datetime.date(2026, 1, 5), 1,
    Decimal('18750.00'))]
mart.daily_flash newest 2026-01-05 expected 2026-01-05 age 30.5 h PASS
```

Reading it. The re-run load found the one missing order line (order 10178's line 334). The same three audit tests now pass, so the day was published: 1 order for ₹18,750.00, the same numbers Chapter 46's fixed re-run produced. The published table's freshness check passes again. That's step 4 of section 47.8: repair, re-run, verify with the same tests, publish. The dispatch file is still waiting for the warehouse supervisor to confirm the rename, which is the contract conversation, not the data team's fix.

Rules that keep alerts useful

- **One incident, one alert.** Group by run and by root cause, as above.
- **Only alert on things someone will do something about.** Everything else goes on a dashboard and is reviewed weekly.
- **Route by owner.** The dispatch contract breach belongs to the warehouse team, not the whole data channel.
- **Say what to do.** Every alert names the next step and links a runbook.
- **Suppress repeats.** While an incident is open, don't re-send the same alert every 15 minutes.
- **Review noisy alerts monthly.** Any alert that fired more than a few times without action is fixed, downgraded, or deleted.
- **Alert on the absence of runs,** not only on failures, or a pipeline that never starts goes unnoticed.

47.10 The tools landscape

Everything in this chapter is SQL, Python, and habits, which is why it works anywhere. Tools add scheduling, history, reporting, and someone else's maintenance.

Tool	What it is	Strengths	Watch for
dbt tests	Tests declared next to your models (47.3)	Free with dbt; runs where your models run; version-controlled; most tests in one line	Only covers data dbt builds; results need somewhere to live
Great Expectations	A Python library of "expectations" with data docs	Large library of checks; profiling; readable reports	Heavier to set up; the API has changed a lot between versions
Soda	Checks written as YAML contracts, run by a command-line tool or a Python library	Readable checks; works across warehouses; hosted option	Another service to run; advanced features are paid
Observability platforms (for example Monte Carlo, Metaplane, Elementary)	Monitor freshness, volume, schema, and lineage automatically	Little to write; anomaly detection and lineage out of the box	Cost; alerts you didn't design; still needs owners
Orchestrator checks (Dagster asset checks, Airflow tasks)	Checks attached to pipeline steps (Chapter 46)	Naturally blocking; visible with the run	Not a full test library on their own
Warehouse constraints	Primary keys, not-null, foreign keys in the database	Stops bad data being written at all	Support varies by warehouse; some are declarative only

A practical combination for a team Riverstone's size: **dbt tests** for the models, **orchestrator checks** for the blocking ones before delivery, **a few freshness and volume monitors**, and **contracts** with the two or three sources that break most often. Add a platform when the number of tables outgrows the team's attention.

Optional: the same rule in Great Expectations and Soda

To see how the two libraries differ from this chapter's failing-rows tests, run one rule, "status is one of the four agreed values", in each. Both are optional installs, into the same virtual environment:

```
# terminal
$ python -m pip install great_expectations==1.23.2 soda-duckdb==4.25.0
```

`great_expectations` is Great Expectations; `soda-duckdb` is Soda's library with its DuckDB connector. The versions are the ones used for the outputs below. Both change often, and Great Expectations' current API (version 1) differs a lot from the version 0 code you'll still find online, so check the documentation for your version. Add both to `requirements.txt` if you keep them.

First, break the rule again, so there's something to find:

```
wh.execute("UPDATE raw.orders SET status = 'On Hold' WHERE order_id = 10177") # a status nobody
agreed
on_hold = wh.execute("SELECT COUNT(*) FROM raw.orders WHERE status = 'On Hold'").fetchone()[0]
print("orders on hold:", on_hold)
```

```
orders on hold: 1
```

Great Expectations checks a table you hand it, here as a pandas DataFrame (Chapter 18):

```
import great_expectations as gx

orders = wh.execute("SELECT order_id, status FROM raw.orders").df()
context = gx.get_context(mode="ephemeral")
batch = (context.data_sources.add_pandas("warehouse")
         .add_dataframe_asset("raw_orders")
         .add_batch_definition_whole_dataframe("all rows")
         .get_batch(batch_parameters={"dataframe": orders}))
rule = gx.expectations.ExpectColumnValuesToBeInSet(
    column="status", value_set=["Pending", "Shipped", "Delivered", "Cancelled"])
result = batch.validate(rule)
print("success:", result.success)
print("rows checked:", result.result["element_count"],
      "| unexpected:", result.result["unexpected_count"],
      f"({result.result['unexpected_percent']:.2f}%)",
      "| examples:", result.result["partial_unexpected_list"])
```

```
success: False
rows checked: 178 | unexpected: 1 (0.56%) | examples: ['On Hold']
```

How it works.

- `.df()` turns a DuckDB result into a pandas DataFrame.
- `gx.get_context(mode="ephemeral")` starts Great Expectations without creating any project files; its settings live only in memory.
- The chain of four calls tells it where the data comes from: a pandas **data source**, a DataFrame **asset**, a **batch definition** that takes the whole DataFrame, and a **batch**, the actual rows for this run, passed in as `batch_parameters`.
- `ExpectColumnValuesToBeInSet` is an **expectation**, Great Expectations' word for a rule: every value in `status` should be in `value_set`.
- `batch.validate(rule)` runs it. `result.success` is the verdict, and `result.result` holds the measurements. While it works, Great Expectations shows a progress bar ("Calculating Metrics"); that isn't part of the printed output.

Soda reads a contract written in YAML, and connects to the warehouse itself. The companion folder has two small files. `soda/warehouse.yml` says where the data is:

```
type: duckdb
name: riverstone_wh
connection:
  database: warehouse/riverstone_wh.duckdb
```

and `soda/orders_contract.yml` says what the table promises:

```
dataset: riverstone_wh/raw/orders
columns:
  - name: order_id
    checks:
      - missing:
      - duplicate:
  - name: status
    checks:
      - invalid:
        valid_values: [Pending, Shipped, Delivered, Cancelled]
```

`dataset:` names the table as data source / schema / table. Each column lists its **checks**: `missing` (no blanks), `duplicate` (no repeats), and `invalid` with the allowed values. Now verify the contract:

```
from soda_core.contracts import verify_contract_locally

check = verify_contract_locally(data_source_file_path="soda/warehouse.yml",
                               contract_file_path="soda/orders_contract.yml")
for r in check.contract_verification_results[0].check_results:
    counts = {k: v for k, v in r.diagnostic_metric_values.items() if k.endswith("_count")}
    print(f"{r.check.name:<20} {r.outcome.name:<7} {counts}")
```

```
No missing values    PASSED  {'missing_count': 0}
No duplicate values  PASSED  {'duplicate_count': 0, 'missing_count': 0}
No invalid values    FAILED   {'invalid_count': 1, 'missing_count': 0}
```

How it works. `verify_contract_locally()` connects to the warehouse file named in `data_source_file_path`, runs every check in the contract file named in `contract_file_path`, and returns the results. "Locally" means on your computer, rather than on Soda's hosted service. There's one contract here, so `contract_verification_results[0]` is its result, and `check_results` has one entry per check. Each has a name, an outcome (PASSED or FAILED), and the measurements behind it in `diagnostic_metric_values`, a dictionary. The dictionary comprehension (Chapter 17, section 17.7) keeps only the counts; the full dictionary also has percentages and the number of rows tested, 178. (Soda sends anonymous usage data by default; setting the environment variable `SODA_CORE_TELEMETRY_ENABLED` to `false` before starting Jupyter turns that off.)

Reading it. Both tools found the one order on hold. Notice *how* they report it. `data_test()` returns the failing rows. Great Expectations and Soda return **measurements**: a count and a percentage of values that break the rule, with a few examples. It's the same rule, expressed as a metric, and a metric is what lets these tools keep a history, draw a trend, or pass a check while a small percentage fails. Which style to use is a team's choice; the rule itself, written down and tested on every load, is what matters.

Put the order back before you go on:

```
print("orders (new, changed, deleted):", ingest.sync_table(wh, "orders"))
```

```
orders (new, changed, deleted): (0, 1, 0)
```

The hash comparison found the one changed order and set its status back to the ERP's value.

APPLY

Common mistakes

Mistake	Symptom	Fix
Testing only that the pipeline ran	Green runs, wrong numbers	Test the data, not only the job (47.2)
Publishing first and checking after	Wrong rows visible while you investigate	Write-audit-publish (47.4)
Every test an error	Pipelines blocked by trivia; people disable tests	Severity per test; warnings for edges (47.2)
Tests that have never failed	False confidence	Break things on purpose and watch them fail (47.3)
No freshness monitoring	Dashboards quietly show last week	Monitor age of newest data per table (47.5)
Baselines learned from broken days	"Normal" drifts toward broken	Exclude failed days; use medians over longer windows
Treating unusual as wrong	Blocked pipelines on genuine big sales days	Anomalies warn; only hard rules block (47.5)
A missing day that looks like a zero day	Charts show a dip that never happened	Publish load status alongside data (47.4)

Mistake	Symptom	Fix
No lineage	"Is anything else affected?" takes hours	Declare dependencies; use the orchestrator graph (47.6)
Contracts as documents nobody checks	Same breakage every quarter	Check the contract in the pipeline (47.7)
Blaming the source's owner	Cooperation stops; surprises continue	Make the safe path the obvious one; explain impact in their terms
Fixing without telling users	People act on numbers they didn't know were wrong	Communicate at containment, before the fix (47.8)
Skipping the review	The same incident twice	Ten-minute blameless write-up, kept with the runbook
One alert per failed signal	Five messages for one problem; fatigue	Group by incident; route by owner (47.9)
Alerting on everything	The channel becomes noise	Alert only what someone will act on; review monthly

In the real world: the dashboard that was right but late

Riverstone's sales dashboard had been correct for months, so when Vikram opened it on a Thursday to prepare for a customer call, he didn't check the date on it. He told the customer their last order had been in November.

It had been in January. The dashboard was showing data that stopped on 4 January, four days earlier.

Meera traced it in twenty minutes. On the Monday, a change in the warehouse had renamed a column in the staging model. The nightly pipeline's transformation step had failed at 2:10 a.m. That failure had sent an alert, exactly as designed, into the data team's channel, where it sat among the eleven other messages the pipeline sent every night: three warnings about dispatch rows with unknown orders, a note about API retries, a long "run finished" message, and a nightly summary nobody read. The failure was message seven.

Meanwhile, every test that ran was green, because the tests all ran against tables that had stopped being updated. Their data was perfectly consistent, and three days old.

The fixes were small, and they're the ones in this chapter.

- **Freshness checks on the tables people read**, not only tests on their contents. The dashboard's own table now fails a check if its newest day is older than the previous working day.
- **Fewer alerts, better routed**. Successes stopped being announced. Warnings moved to a weekly digest. Failures went to the on-call person with a subject line naming the table and the impact.
- **A status banner on the dashboard**, driven by the load status table: "Data current to 7 January, 6:35 a.m." Vikram's habit of glancing at the top-right corner now catches in one second what took three days.
- **A contract conversation** with the analytics engineer who had renamed the column, agreeing that renames in shared models need a note and a deprecation window, exactly as for external sources.

At the next sales review, Anita asked whether the data was reliable now. "It fails more often than it used to," Meera said. "That's the point. When it's wrong, it stops and tells us, instead of looking right."

What made this work.

- **They monitored freshness, not only correctness.** The data was correct; that was never the problem.
- **They treated alert noise as a real defect,** because it's what hid the failure.
- **They put the status where the users look,** so trust didn't depend on the data team noticing first.
- **They applied contracts internally,** where most surprise changes actually come from.

Project: quality, monitoring, and a contract for the Riverstone pipeline

Goal: make the Chapter 46 pipeline trustworthy: nothing wrong gets published, staleness is noticed, one incident produces one useful alert, and the dispatch file has a contract that's checked automatically.

Tools you'll need

- **Python 3.14** in the virtual environment from Chapter 17, with `duckdb` and `psycopg2` (Chapter 45, section 45.2); **PostgreSQL 16**; **Jupyter**; the **Chapter 47 companion folder** (`companion/ch47/`), which is Chapter 46's environment without Dagster, plus `reset_ch47.py` and the two Soda files. Open your notebook in that folder.
- **No data quality product needed:** every test in the main thread is SQL run from Python.
- **Optional:** `great_expectations` and `soda-duckdb` (section 47.10), and your Chapter 32 dbt project for the dbt tests and the model contract.
- **Versions used for the outputs shown:** Python 3.11.15, DuckDB 1.5.6, psycopg2 2.9.13, PostgreSQL 16.13, Great Expectations 1.23.2, Soda Core 4.25.0, dbt Core 1.12.5 with dbt-postgres 1.11.0 and dbt_utils 1.4.1, on 29 September 2026. Your Python 3.14 environment gives the same outputs.

Option A: Riverstone. Extend your Chapter 46 pipeline.

Option B: your own pipeline. Apply the same layers to a report you produce, using data you're allowed to use.

Steps

1. **Write the specification** for two tables: what "good" means, in sentences, for at least four dimensions from section 47.1.
2. **Build a test suite** of at least ten tests across raw, staging, and mart, each with a severity and a one-line description.
3. **Break each rule on purpose** and confirm every test fails when it should. Note any test that didn't catch what you expected.
4. **Convert the pipeline to write-audit-publish** for the Flash, and prove a failing day leaves the published table untouched.

5. **Add freshness checks** for every table people read, with limits based on the promise to users, and a `load_status` table the dashboard can show.
6. **Add two volume checks**: zero rows on a working day (error), and a wide band around the median of recent days (warning).
7. **Add lineage**: list, for each source, which outputs it affects, and use it in your alert text.
8. **Write a data contract** for the dispatch file with the warehouse team's view in mind, and check it in the pipeline.
9. **Run an incident**: break the pipeline, produce one grouped alert, write the user-facing message, fix, re-run, and write the review (half a page).
10. **Count your alerts**. Over a week of runs, how many messages would a person receive? If it's more than a handful, cut them.

Stretch goals

- Store every test result in a `test_results` table with the run and day, and chart pass rates over time.
- Add a check that compares this week's revenue by segment with last week's and warns on large shifts.
- Write the same tests as dbt tests and run both, comparing effort and output.

REVIEW

Recap

- Data quality has **dimensions**: accuracy, completeness, validity, uniqueness, consistency, and timeliness. Each becomes a test.
- A **data test** is a query that returns the rows that break a rule; zero rows means it passed. dbt's tests compile to exactly these queries; Great Expectations and Soda report the same rule as a measurement, a count of the values that break it.
- **Severity** decides what stops a pipeline. Warnings are for edges; errors are for numbers people act on. Tests nobody acts on should be fixed, downgraded, or deleted.
- Tests belong at every layer: structure in **raw**, validity and consistency in **staging**, business rules and reconciliation in **mart**, and blocking checks **before delivery**.
- **Break your tests on purpose**. A suite that has never failed hasn't been tested.
- **Write-audit-publish** keeps failed data out of the tables people read; the same 5 January failure that left a wrong row in Chapter 46 now leaves the published table untouched.
- **Observability** answers three questions: did it run, is it **fresh**, does its **volume** look normal. Stale data is one of the most common incidents, and the least noticed. Freshness is judged against the promise to users, such as "yesterday's working day is loaded by 6:30", not a fixed number of hours.
- **Unusual is not wrong**. Anomalies warn; only hard rules block.
- **Lineage** tells you what a broken table affects, and where a wrong number could come from.

- **Data contracts** state fields, rules, delivery, change notice, and what happens on a breach, and are checked in the pipeline: columns and values. They apply to internal producers too, where dbt's model contracts enforce them.
- **Incidents** follow detect, classify, contain, fix and verify, review. Tell users at containment, before the fix.
- **Alert fatigue** hides real failures: one incident, one alert; route by owner; alert only what someone will act on.

Key terms

data quality dimensions (accuracy, completeness, validity, uniqueness, consistency, timeliness) · data test · failing rows · severity (error, warning) · raw, staging, mart layers · reconciliation · anti-join · foreign key · self-healing load · dbt test · singular test · write-audit-publish (WAP) · audit schema · cross join · floating-point comparison · publish swap · blue/green tables · load status · observability · freshness · previous working day · staleness limit · volume check · baseline · median · anomaly · lineage · column-level lineage · impact analysis · data contract · model contract · change notice · breach · data incident · severity levels (S1, S2, S3) · containment · blameless review · alert fatigue · alert grouping · runbook · expectation (Great Expectations) · check (Soda)

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ You can name the six data quality dimensions and write a test for each on Riverstone data.
- ☐ You can explain why a data test is "a query that returns the rows that break the rule".
- ☐ You can choose error or warning severity for a test, and defend the choice.
- ☐ You can say which checks belong in raw, staging, mart, and before delivery.
- ☐ You can break a rule on purpose and show the test catching it.
- ☐ You can implement write-audit-publish and prove readers never see failed data.
- ☐ You can explain why a re-run repairs a table loaded by hash comparison.
- ☐ You can set freshness limits from a promise to users, and explain why stale data is the hardest failure to notice.
- ☐ You can build a volume baseline with a median, and explain why unusual is not the same as wrong.
- ☐ You can use lineage to say who is affected by a broken table.
- ☐ You can write a data contract and check it automatically.
- ☐ You can classify an incident's severity, write the user-facing message, and run the review.
- ☐ You can group one incident into one alert and explain three rules that prevent alert fatigue.
- ☐ You can write the same rule as a dbt test, a Great Expectations expectation, and a Soda check, and say how their results differ.

- ☐ You can recommend a starting set of tools for a small data team.

PRACTISE

Exercises

Run `reset()` and re-run the setup and test blocks before each exercise that uses the companion environment.

Warm-up

1. Name the data quality dimension each rule belongs to: (a) every order line points at an order that exists; (b) `status` is one of four values; (c) yesterday's orders are loaded by 6:30 a.m.; (d) the warehouse's revenue for a day equals the ERP's; (e) one row per order.
2. Write the failing-rows query for: (a) duplicate `dispatch_id` values in `raw.dispatch`; (b) orders with a `sales_rep_id` that doesn't exist in a `raw.employees` table.
3. For each, choose error or warning and give a reason: (a) revenue doesn't match the ERP; (b) a dispatch row points at an unknown order; (c) a customer's email is missing; (d) `mart.daily_flash` has two rows for one day.

Core

4. Add a test `flash_revenue_matches_lines` that fails when a published Flash row's revenue doesn't equal the revenue recomputed from `raw.orders` and `raw.order_items` for that day. Would this have caught the 5 January failure? Would it have caught a wrong `discount` in the source?
5. In section 47.3 the ERP refused the orphan order line, and the warehouse accepted it. Explain why, and say what you'd add to the warehouse to make it refuse too. What's the trade-off of adding it?
6. After the `resync` block, all seven tests pass again. Explain, in terms of Chapter 45's hash comparison, why re-running the load repaired the warehouse. Which of the three broken things would a load with an ID watermark have repaired?
7. The freshness check flagged `mart.daily_flash` as stale at 6:30 a.m. on 6 January. Write the user-facing message the sales team should receive, in three sentences.
8. Set freshness limits for: (a) `raw.orders`; (b) `raw.dispatch`; (c) `raw.crm_leads`, given the Flash is promised by 7:30 a.m. IST on working days, the dispatch file is due at 20:00 IST daily, and lead figures are used in a Monday report. State your assumption about weekends.
9. Using the 2025 numbers in section 47.5 (median ₹27,442.50, 131 days with orders), design two volume rules: one error and one warning. Say what each would have done on 16 November 2025 (₹175,895.00) and on a day the load brought in nothing.

Stretch

10. Design the `load_status` table mentioned in section 47.4: columns, one row per what, and how a dashboard would use it to show "data current to...". Include how it distinguishes "not published yet", "published", and "failed".
11. Write the full dispatch data contract as a document the warehouse supervisor would accept: fields, rules, delivery, change process, breach handling, review. Then list the three checks you'd automate first, and why those three.
12. An analytics engineer renames a column in a shared staging model, breaking two downstream reports. Design an internal contract process for shared models: what's promised, how changes are announced, how long the old name survives, and how the pipeline enforces it.

Think about it (no code needed)

13. Riverstone's pipeline now fails more often than it used to, because it stops when data is wrong. Write the paragraph you'd use to explain to Anita why that's an improvement, and what you'd show her each month to prove it.
14. Which is worse: a wrong number delivered on time, or no number at all? Argue both sides using examples from this chapter, then say what design choice follows from your answer.

Answers

1. (a) Consistency (a relationship between tables). (b) Validity. (c) Timeliness. (d) Accuracy. (e) Uniqueness. Completeness would cover "every order has a customer".

2. Sample answers:

```
-- (a) duplicate dispatch_id
SELECT dispatch_id, COUNT(*) AS n
FROM raw.dispatch
GROUP BY dispatch_id
HAVING COUNT(*) > 1;

-- (b) orders whose sales rep is unknown
SELECT o.order_id, o.sales_rep_id
FROM raw.orders AS o
LEFT JOIN raw.employees AS e ON e.employee_id = o.sales_rep_id
WHERE o.sales_rep_id IS NOT NULL AND e.employee_id IS NULL;
```

Both follow the rule: return the rows that break the rule, so zero rows means the test passed. In (b), the `IS NOT NULL` keeps "no rep assigned" out of this test; that's a separate completeness rule.

3. (a) **Error**: the Flash would mislead the sales team. (b) **Warning**: a dispatch may legitimately arrive before its order is loaded, and holding the Flash for it would be worse than publishing. (c) **Warning** in most cases: a missing email doesn't make the sales numbers wrong, though it would be an error for a table that feeds an email campaign. (d) **Error**: duplicate rows double revenue, and it signals a non-idempotent step.

4. A sample test:

```
data_test("flash_revenue_matches_lines", ""
SELECT f.flash_date, f.revenue_booked, COALESCE(c.revenue, 0) AS recomputed
FROM mart.daily_flash AS f
LEFT JOIN (
  SELECT o.order_date, SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)) AS revenue
  FROM raw.orders AS o JOIN raw.order_items AS i ON i.order_id = o.order_id
  WHERE o.status <> 'Cancelled' GROUP BY o.order_date) AS c
ON c.order_date = f.flash_date
WHERE f.revenue_booked <> CAST(ROUND(COALESCE(c.revenue, 0), 2) AS DECIMAL(12,2))""")
```

The recomputed revenue is rounded to the paisa and made an exact decimal before the comparison, for the reason in section 47.4: the raw sum is a floating-point number. It would **not** have caught the 5 January failure on its own: the Flash was built *from* those same warehouse tables, so both sides would have agreed on ₹0. That's why the reconciliation test compares with the **ERP**, not with the warehouse. (The `LEFT JOIN` keeps a published day even if no lines are found for it, and `COALESCE` turns that missing revenue into 0, so such a day fails the test.) It also would **not** catch a wrong discount in the source, for the same reason: the warehouse faithfully copies the source's mistake. It's still useful, because it catches a wrong transformation, publishing the wrong day, or a stale Flash row.

5. The ERP has a **foreign key** on `order_items.order_id`, so the database itself refuses a line whose order doesn't exist. The warehouse table has no such constraint, so it accepted the row. You could add one in DuckDB or your warehouse where supported. In DuckDB a foreign key must be declared when the table is created (`order_id INTEGER REFERENCES raw.orders(order_id)`); it can't be added to an existing table with `ALTER TABLE`, so the raw tables would have to be created again. The trade-offs: constraints are checked on every write, which slows large loads; they force load *order* (orders before lines) and make partial loads fail rather than land; and many cloud warehouses don't enforce them at all, which is why data tests exist. A common compromise is a primary key on raw tables (Chapter 45's idempotency needs it) plus relationship tests instead of enforced foreign keys.

6. The hash comparison builds the full picture of what the source contains and makes the warehouse match it: rows in the warehouse but not the source are **deleted**, and rows whose values differ are **updated**. The two invented order lines existed nowhere in the source, so they were deleted; order 10177's status differed, so it was updated back to the source's value. An **ID watermark** load would have repaired **none of them**: it only inserts rows with IDs above the watermark, so it never deletes and never updates. That's Chapter 45's lesson again, now visible as a data quality property.

7. A sample message: *"The Daily Sales Flash for 5 January hasn't been published. Yesterday's order lines didn't load, so today's figures would have shown no sales, which would be wrong. We're fixing the load now and expect the Flash by 11:00; the dashboard still shows data up to 2 January."* Three things make it good: no jargon, the reason the report was held, and when to expect it.

8. Assumption: orders can arrive on any day (2025 had big Sunday orders: 16 and 9 November), but the Flash is promised only on working days, and the checks run at 6:30 a.m. (a) `raw.orders`: on a working-day run, the newest `order_date` must be on or after the previous working day. In hours, that's 30.5 hours on Tuesday to Friday (a date counts from midnight) and 78.5 hours on a Monday, so an hour limit needs about 32 and 80. (b) `raw.dispatch`: the file for day D is due at 20:00 on D, so at the next 6:30 run the newest `file_day` must be on or after D, the previous working day; check it only after

working days. (c) `raw.crm_leads`: the Monday report needs last Friday's leads by Monday at 6:30 (78.5 hours, so about 80); make it an error on Monday mornings and a warning on other days. Any answer is acceptable if the limit comes from a promise and the weekend assumption is stated.

9. Error rule: on a working day, the day's booked revenue is **zero** while the ERP has orders for that day, or the published revenue is negative. On 16 November this does nothing (₹175,895.00 is not zero); on the day the load brought nothing, it fires and blocks publishing. **Warning rule:** the day's revenue is below **a quarter** of the recent median (₹6,860) or above **five times** it (₹137,212). On 16 November this warns, and a human confirms that a genuinely large order arrived; on the empty-load day the error rule has already blocked it. Using three times the median as an *error* would have blocked ten legitimate days in 2025, which is exactly the mistake section 47.5 warns against.

10. A sample design: one row per **table and day** (or per published partition): `table_name`, `business_date`, `status` (pending, published, failed, late), `run_id`, `published_at`, `rows`, `failed_tests`, `note`. The pipeline inserts `pending` when a run starts, updates to `published` after a successful publish, and `failed` when the audit blocks it; a separate freshness monitor marks a `pending` row `late` when it passes its deadline. A dashboard reads the latest `published` row for its table to show "Data current to 7 January, 6:35 a.m.", and shows an amber banner when today's status is `late` or `failed`. The important part is that "we have no data for today" is a *recorded state*, not an absence, so charts can show it instead of drawing a zero.

11. A good answer covers: **dataset and purpose** (daily dispatch sheet, feeds the dispatch report used by customer service); **fields** with types, meanings, and whether blanks are allowed, including that `qty_units` may be blank while a count is pending; **rules** (`dispatch_id` unique, `order_id` exists in the ERP, quantities whole numbers, date as dd-mm-yyyy); **delivery** (one file per working day by 20:00 IST, named `dispatch_YYYY-MM-DD.csv`, in the shared folder); **change process** (new columns allowed any time; renames, removals, and type changes need 10 working days' notice and a quick call); **breach handling** (load stops, both sides alerted, previous data stays published, the day is loaded once corrected); **review** every six months; **contacts** on both sides. The three checks to automate first: (1) **columns match the contract**, because renames caused the last two breakages; (2) **file arrived by the deadline**, because a missing file is the most common and least visible failure; (3) **`dispatch_id` unique**, because duplicates silently double dispatched quantities. Each is cheap and catches a failure that has already happened.

12. A workable internal process: shared models are **owned** by a named person, and their column names are treated as an interface. Changes are announced in the data channel and in the model's documentation, with the reason. A rename is done as **add, then deprecate**: the new column is added alongside the old one, the old one is marked deprecated with a removal date at least two weeks out, consumers are told which reports use it (from lineage), and only then is it removed. Enforcement lives in the pipeline: a test asserts that every column listed in the model's contract file still exists, so a removal without a contract change fails in the pull request, not at 6:30 a.m. dbt's model contracts (section 47.7) and Chapter 29's code review make this routine rather than personal.

13. A sample paragraph: *"The pipeline stops more often because we added checks that compare our numbers with the ERP before anything is published. When it stops, the old numbers stay up and you get a message telling you what's late and when to expect it. Before, the same problems still happened; we published them, and someone found out in a meeting. What I'd like to show you each month: how many*

times we stopped publication and why, how many wrong numbers reached anyone (the number I want at zero), and how long it took us to publish after a problem. If the first number is high and the second is zero, the system is working." The key move is to reframe visible failures as caught failures, with metrics that separate the two.

14. A *wrong number on time* is worse in most business settings: people act on it, decisions are made, and the error spreads into other systems and into trust, as the Chapter 46 story of duplicate emails showed. No *number* is worse when a decision must be made at a fixed time and a rough figure would be better than nothing, for example a dispatch cut-off, or where the absence itself misleads (a chart showing zero sales). The design that follows from the first view is this chapter's: block publication, keep the last good data, tell people, and fix. The design that follows from the second is to publish with an explicit, visible warning about what's missing or provisional, so people can decide for themselves. A good answer picks one and names the conditions where it would switch.

NEXT

Where this leads

- **Chapter 48, Big Data & Distributed Compute**, scales the pipeline's heavy steps, where tests must run on far more data than fits in memory.
- **Chapter 49, Storage, Warehouses & Lakehouses**, provides the transactions, snapshots, and time travel that make publishing and rollback quick.
- **Chapter 50, Streaming & Real-Time**, asks how to check data that never stops arriving.
- **Chapter 51, Data Activation**, sends warehouse data into the CRM and ERP, where an S1 incident means wrong data in a business system, not only a wrong chart.
- **Chapter 32** covers dbt models, tests, and documentation; **Chapter 14** covers the analyst-level version of this work.
- **Chapter 64, Security, Privacy, Governance & Responsible AI**, covers ownership, stewardship, and the governance that data contracts sit inside.
- **Part 8:** data quality and incident questions appear in the data engineering interview chapters, and "walk me through a wrong dashboard" is a standard system design question (Chapter 77).

Chapter 48. Big Data & Distributed Compute

Chapter at a glance

You will learn to: install Spark and Polars and start your first Spark session · judge for yourself whether a dataset needs distributed computing at all · explain partitions, workers, the driver, and why moving data between machines is what costs · write PySpark that reads, filters, groups, joins, and writes Riverstone's sensor data · read a physical query plan with `explain()` and find the shuffle in it · use partition pruning, column pruning, and broadcast joins, and see each one change the plan · recognize skew and know the usual fixes · use Polars for the first time · compare DuckDB, Polars, and Spark on the same question and interpret the timings · avoid the common ways a Spark job surprises you · place Databricks, EMR, Dataproc, Fabric, Trino, and Ray on the map.

Before you start: Chapter 17 (the virtual environment and Jupyter). Chapter 18 (pandas, and Parquet). Chapter 26 section 26.0 (the terminal). Chapter 29 (Python as software). Chapter 45 (DuckDB from Python, and Parquet). Chapter 46 (pipelines and idempotent re-runs). Chapter 14's regular expressions help with one optional helper in section 48.4.

Time needed: 14–18 hours of reading and practice, spread over two to three weeks. Plan four sittings: section 48.0, installing Java, Spark, and Polars and starting your first Spark session (1–2 hours, more if Java fights you); sections 48.1–48.4; sections 48.5–48.6 (making jobs faster, Polars, three engines); section 48.7 onwards and the project.

Tools: Python in the book's virtual environment with `pyspark` and `polars` (installed in section 48.0), `duckdb` (Chapter 45), and `pandas` and `pyarrow` (Chapter 18); a Java runtime (Java 17, 21, or 25) for Spark. Everything runs on one machine.

Practice data: Riverstone's plant sensor readings, generated by the Chapter 48 companion script: 25 machines across two plants, one reading every 10 seconds from 1 October to 31 December 2025. That's **19,872,000 readings**, about 114 MB as Parquet. It's a larger, separate dataset from Chapter 40's sensor practice data.

In this chapter

START	Why this matters · In plain English	140
LEARN	48.0 Setting up Spark, Polars, and the data	141
	48.1 Most "big data" isn't	145
	48.2 What "distributed" actually means	146
	48.3 PySpark on the sensor data	147
	48.4 Reading the plan, and finding the shuffle	150
	48.5 Making a job faster	153
	48.6 The same question in three tools	158
	48.7 Writing Spark that doesn't surprise you	164
	48.8 The wider landscape in one page	166
APPLY	Common mistakes · In the real world · Project	166
REVIEW	Recap · Key terms · Check yourself	169
PRACTISE	Exercises	170
	Answers	171
NEXT	Where this leads	174

START

Why this matters

Somewhere in every data engineer's first year, a dataset stops fitting comfortably in memory, a job that took two minutes starts taking two hours, and somebody says the word "Spark".

Two mistakes follow. The first is reaching for a cluster far too early, which is expensive, slow to develop on, and harder to debug than the problem it replaced. The second is refusing to learn distributed computing at all, and then writing single-machine code that quietly can't grow, or Spark code that works but costs ten times what it should because of one line.

This chapter gives you the judgment and the mechanics. You'll write real PySpark on 20 million sensor readings, read the plans Spark makes, watch a shuffle appear and then make it disappear, and time the same question in three tools. By the end you should be able to answer, with reasons and numbers, the question every interviewer eventually asks: *"How would you process this, and why not something simpler?"*

In plain English

Think about counting the votes in an election.

- One counting officer can handle a small booth. That's a **single machine**, and it's usually enough.
- For a whole state, you divide ballots into boxes and hand them to many officers. Each box is a **partition**; each officer is a **worker**.
- A supervisor tells everyone what to count and collects the totals at the end. That's the **driver**.
- Counting inside one box is fast. The expensive part is when ballots must be **moved between officers**, for example to put all the votes from one constituency together. That's a **shuffle**, and it's the single most important idea in this chapter.

- If every officer gets a small box except one who gets a truckload, everyone waits for that one person. That's **skew**.
- If you only need votes from one district, you shouldn't open every box. That's **partition pruning**.
- If you only need the party name and not the whole ballot, you shouldn't read the whole ballot. That's **column pruning**.
- Hiring a thousand officers to count a village's votes is slower than one person doing it, because organizing them takes longer than the counting. That's Spark on small data.

LEARN

48.0 Setting up Spark, Polars, and the data

This chapter uses two new tools. **Spark** is a distributed engine: it splits work across many machines, or, on your laptop, across your processor's cores. **PySpark** is the Python library you use to drive it. Spark itself runs on **Java**, so Java must be installed too. **Polars** is a DataFrame library like pandas, built for speed; section 48.6 introduces it.

Setting up takes one sitting. Java is the step most likely to cause trouble, so the troubleshooting box at the end of this section lists the usual errors.

Step 1. Your practice folder

Work the way Chapter 17 did. In your file manager, copy the folder `companion/ch48` and paste it inside `work`, so you have `work/ch48` to practise in. It holds two files:

File	What it does
<code>make_sensor_data.py</code>	Generates the sensor dataset (step 4).
<code>time_three_tools.py</code>	Times the same question in DuckDB, Polars, and Spark (section 48.6).

Step 2. Install Java

Spark 4.2 runs on Java 17, 21, or 25. Install **Java 21**, a long-term-support release, for your system:

- **Windows:** download the **Eclipse Temurin 21** installer (the `.msi` file) from adoptium.net. On the installer's *Custom Setup* page, turn on **Set JAVA_HOME variable**, then finish the install. `JAVA_HOME` is an environment variable (Chapter 34) that tells programs such as Spark where Java lives.
- **macOS:** download the **Eclipse Temurin 21** `.pkg` installer from adoptium.net and run it. (With Homebrew, `brew install --cask temurin@21` does the same.)
- **Linux, and WSL on Windows:** `sudo apt install openjdk-21-jre-headless` on Ubuntu or Debian. "Headless" means without the parts for desktop windows, which Spark doesn't need.

Open a **new** terminal (so it sees the new settings) and check:

```
# terminal
$ java -version
openjdk version "21.0.10" 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
```

java -version prints the version of the Java that your terminal finds. The first line is the one that matters: it must start with 17, 21, or 25. The build details on the next two lines depend on who made your Java; Temurin's say **Temurin**.

Step 3. Install PySpark and Polars

In the terminal, go to your practice folder and activate the book's virtual environment (Chapter 17), then install:

```
# terminal
$ cd work/ch48
$ source ../../.venv/bin/activate
$ python -m pip install pyspark==4.2.0 polars==1.44.2
```

- **source ../../.venv/bin/activate** activates the environment, two levels up; on Windows PowerShell it's `..\..\venv\Scripts\Activate.ps1`.
- **pyspark** is a large download (about 450 MB), because it carries Spark itself inside it. **polars** is Polars.
- `==4.2.0` and `==1.44.2` ask for the exact versions used for this chapter's outputs. The pins matter more than usual here: the text of Spark's plans changes a little between versions. **duckdb** is already installed from Chapter 45, and **pandas** and **pyarrow** from Chapter 18.

Add **pyspark** and **polars** to your `requirements.txt`, one per line (Chapter 26). Then check that both import:

```
# terminal
$ python -c "import pyspark, polars; print(pyspark.__version__, polars.__version__)"
4.2.0 1.44.2
```

Step 4. Generate the data

```
# terminal
$ python make_sensor_data.py
sensor_readings: 19,872,000 readings, 114.3 MB
$ python make_sensor_data.py --small
sensor_readings_small: 216,000 readings, 1.2 MB
```

The first command builds the full dataset in the folder `sensor_readings`; it takes from a few seconds to a minute or two, depending on your computer. The second builds one day, `sensor_readings_small`, which section 48.6 uses for timing. What the generator makes:

- **25 machines**, `M-01` to `M-25`. Machines 1–15 are at **Bhiwandi Main** and 16–25 at **Chakan Pune**. Each machine has a **line type**, Injection, Extrusion, or Blow, taken in turn: Bhiwandi Main has 5 of each,

and Chakan Pune has 3 Blow, 4 Extrusion, and 3 Injection.

- **One reading every 10 seconds**, 8,640 a day, from 1 October to 31 December 2025 (92 days). $25 \times 8,640 \times 92 = 19,872,000$.
- Each reading has a **temperature** around a daily base between 180 and 212 °C, with random noise, and now and then a machine that drifts upwards through the day; a **pressure** that follows the temperature; **units made** (about 3 per reading); and a **scrap flag** that is true for about 1.2% of readings, and for 6% when the machine runs hot.
- The random numbers come from a fixed **seed** (48), so your data is identical to the book's, number for number.

The data is written as Parquet (Chapter 45), **partitioned by reading_date**: one folder per day, named in a special way. See for yourself. From here on, work in a Jupyter notebook: create `ch48.ipynb` in `work/ch48`, choose the `.venv` kernel (Chapter 17, section 17.0), and run each code block as its own cell, top to bottom, without restarting the kernel.

```
import os

days = sorted(os.listdir("sensor_readings"))
print(len(days), "folders, from", days[0], "to", days[-1])
print("inside the first:", os.listdir(os.path.join("sensor_readings", days[0])))

sizes = [os.path.getsize(os.path.join("sensor_readings", d, "part-0.parquet")) for d in days]
print(f"total {sum(sizes) / 1e6:.1f} MB; one day about {sizes[0] / 1e6:.1f} MB")
```

```
92 folders, from reading_date=2025-10-01 to reading_date=2025-12-31
inside the first: ['part-0.parquet']
total 114.3 MB; one day about 1.2 MB
```

- `os.listdir()` lists a folder's contents; `sorted()` puts the names in order.
- `os.path.join()` builds a path with the right separator for your system (`/` or `\`).
- The list comprehension (Chapter 17) collects each day's file size in bytes; dividing by `1e6` (one million) gives megabytes.

Reading it. Each folder is named `reading_date=2025-10-01`: the column name, an equals sign, and the value. This is **Hive-style partitioning** (named after the Hadoop tool that made it popular), and every engine in this chapter understands it. The files inside don't contain a `reading_date` column at all; the engines take it from the folder names. That's what lets them skip whole days they don't need (section 48.5).

Each day's folder is only about 1.2 MB. That's far below the file sizes this chapter recommends later (128–512 MB), and at this size daily folders are still fine, because there are only 92 files. With years of data you'd partition by month instead, or keep days and merge small files regularly (Chapter 49).

Step 5. Your first Spark session

A Spark program starts by creating a **SparkSession**, the entry point to everything Spark does. This cell is also your test that Java and PySpark work together.

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
        .appName("riverstone-sensors")
        .master("local[2]")
        .config("spark.ui.showConsoleProgress", "false")
        .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")
print("Spark", spark.version, "is running")
```

Spark 4.2.0 is running

How it works, line by line.

- The whole builder sits inside **round brackets** so that it can run over several lines, one setting per line. Python treats everything between the brackets as one expression.
- **SparkSession.builder** starts describing the session you want.
- **.appName("riverstone-sensors")** names the job. The name appears in Spark's web interface (section 48.5).
- **.master("local[2]")** says where the work runs: **local** means **local mode**, one process on this computer, and **[2]** means two worker threads, so two cores work in parallel. **local[*]** would use every core. This chapter uses two, so that your computer stays responsive and section 48.6 can give all three engines the same two cores. On a real cluster, this is where the cluster's address goes instead.
- **.config("spark.ui.showConsoleProgress", "false")** turns off the progress bar Spark would otherwise draw under each cell.
- **.getOrCreate()** creates the session, or, if one is already running, returns that one. That matters in Jupyter: re-running the cell doesn't start a second Spark.
- **setLogLevel("ERROR")** hides Spark's stream of information messages and warnings, so only real errors show.
- The first run takes several seconds, because Java starts up behind the scenes. Later cells are faster.

When the first Spark cell fails.

- **JAVA_GATEWAY_EXITED** or "Java gateway process exited before sending its port number": Spark couldn't start Java. Java isn't installed, or `JAVA_HOME` isn't set. Check `java -version` in a new terminal, then restart VS Code so it sees the change.
- **UnsupportedClassVersionError**: your Java is too old (Java 8 or 11). Install Java 21 and make sure `java -version` shows it.
- A warning that **port 4040 is in use**: another Spark session is already running (perhaps in another notebook). Spark picks port 4041 instead. It's harmless.
- On Windows, a warning about `winutils.exe` or **HADOOP_HOME**: harmless for reading data. Writing files with Spark on Windows (section 48.7) needs one extra download; the simplest route is to run this chapter in WSL (Chapter 34), or search for "Spark winutils HADOOP_HOME" for the Windows steps.
- A warning that **PySpark "does not yet fully support pandas 3"**: nothing in this chapter depends on it. Ignore it.

Tool note. The outputs in this chapter were produced with Python 3.11.15, PySpark 4.2.0, Polars 1.44.2, DuckDB 1.5.6, PyArrow 25.0.1, pandas 3.0.6, and OpenJDK 21.0.10. Newer versions may format a few outputs slightly differently.

48.1 Most "big data" isn't

A claim worth making early, because it has only become more true: **most datasets described as "big data" fit on one computer.**

A modern laptop has 8 to 32 GB of memory and 8 or more cores. Tools such as DuckDB and Polars are built to use them, to work directly on Parquet files, and to spill to disk when data doesn't fit in memory. In practice that means:

Rough size	What usually works
Up to a few million rows	Spreadsheets struggle; SQL in a database, pandas, DuckDB, or Polars are all comfortable
Tens of millions of rows	DuckDB or Polars on a laptop; a warehouse if it's shared
Hundreds of millions to billions	A cloud warehouse (Chapter 49), or Spark, usually on a cluster
Continuous, never-ending data	Streaming (Chapter 50), which is a different problem

Riverstone's entire order history is a few hundred thousand rows: a spreadsheet's worth. Its sensor data is far bigger, and this chapter's 19.87 million readings are three months from 25 machines. A year across more machines reaches hundreds of millions, which is where the question becomes real.

Before choosing a distributed tool, ask four questions.

1. **How much data does the job actually touch?** Not the size of the source; the size after filtering to the days and columns you need. Partitioned Parquet often turns "a billion rows" into "40 million".
2. **How often does it run, and how long can it take?** A nightly job with a four-hour window is a different problem from a query someone waits for.
3. **Does it fit in memory after the filter?** If yes, one machine is almost always faster and simpler.
4. **What's the cost of being wrong?** Starting simple and moving to Spark later is cheap. Starting on a cluster and discovering you never needed it costs months of complexity.

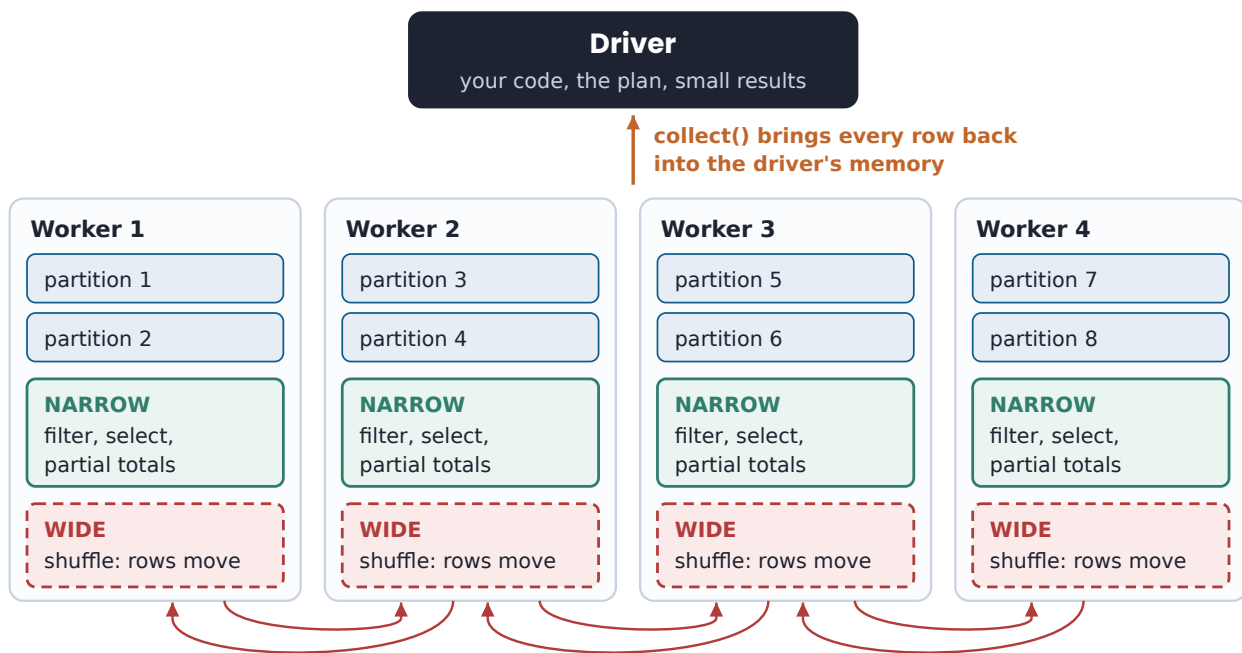
Watch out: "we might need it later" is not a reason. Later is when to build it. A pipeline written in SQL or Polars can be rewritten in Spark in days, and most never need to be.

48.2 What "distributed" actually means

A distributed engine splits data into **partitions**, sends work on each partition to a **worker** (also called an executor), and coordinates everything from the **driver**, which is the process running your code.

Three consequences shape everything you write.

- **Work that stays inside a partition is cheap.** Filtering, selecting columns, arithmetic, and per-row work run in parallel with no coordination. These are **narrow** operations.
- **Work that needs data from other partitions is expensive.** Grouping, joining, sorting, and deduplication must first bring matching rows together, which means writing data to disk, sending it over the network, and reading it back. That movement is the **shuffle**, and these are **wide** operations.
- **Bringing results back to the driver is a bottleneck.** `collect()` pulls every row into the memory of one process. It's how many first Spark jobs die.



NARROW (solid boxes): each worker uses only its own partitions. Cheap, fully parallel.
WIDE (dashed boxes, arrows underneath): rows for the same key must meet, so they cross between workers, through shuffle files and, on a cluster, the network. This is the cost.

Figure 48.1 – Driver, workers, partitions, and the shuffle. The crossing arrows are what you pay for.

Lazy execution ties it together. Spark doesn't run anything as you build a query. It records what you asked for, plans it as a whole, and runs it when you ask for a result (an **action** such as `count()`, `show()`, `collect()`, or writing files). That's why Spark can rearrange your work, and why the plan is worth reading.

Simplification note. Everything here runs Spark in **local mode**, one Java process on one machine, using your cores as workers. That's the standard way to develop and test, and every idea, plan, and optimization transfers to a cluster. What local mode can't show you is network cost and cluster tuning; section 48.6 is honest about what that means for the timings.

48.3 PySpark on the sensor data

Spark cuts the data into a number of pieces after every shuffle, set by `spark.sql.shuffle.partitions`. The default is 200, which is far too many for one machine and a common cause of slow local jobs. Set it once, then read the data:

```
spark.conf.set("spark.sql.shuffle.partitions", 8)

readings = spark.read.parquet("sensor_readings")
print(f"rows: {readings.count():,}")
readings.printSchema()
```

```
rows: 19,872,000
root
 |-- machine_id: string (nullable = true)
 |-- plant: string (nullable = true)
 |-- line_type: string (nullable = true)
 |-- reading_ts: timestamp_ntz (nullable = true)
 |-- temperature_c: double (nullable = true)
 |-- pressure_bar: double (nullable = true)
 |-- units_made: integer (nullable = true)
 |-- scrap_flag: boolean (nullable = true)
 |-- reading_date: date (nullable = true)
```

- **spark.conf.set(name, value)** changes a setting on the running session. Eight pieces suits two cores; on a cluster, aim for a few times the total number of cores.
- **spark.read.parquet("sensor_readings")** reads the whole folder. Spark discovers the `reading_date` partition column from the folder names, which is why it appears in the schema without being in any file. The result, `readings`, is a Spark **DataFrame**: like a pandas DataFrame, but split into partitions and computed lazily.
- **count()** is an action: it runs a job and returns the number of rows.
- **printSchema()** prints each column's name and type. `timestamp_ntz` is a timestamp with **no time zone** ("ntz"); `nullable = true` means the column may hold missing values.

A first real question

"How many readings does each plant have?" The simplest group-by:

```
readings.groupBy("plant").count().orderBy("plant").show()
```

```
+-----+-----+
|      plant|  count|
+-----+-----+
|Bhiwandi Main|11923200|
|  Chakan Pune| 7948800|
+-----+-----+
```

- **groupBy("plant")** groups rows by plant, as `GROUP BY` does in SQL and `groupby` in pandas.
- **.count()** after a `groupBy` counts rows per group, in a column called `count`.
- **.orderBy("plant")** sorts the result. Without it, groups can come back in any order.
- **.show()** is an action: it runs the job and prints up to 20 rows as a table.

Now the full question: "What's the average temperature and scrap rate for each plant and line type?"

```
from pyspark.sql import functions as F

by_plant = (readings
    .groupBy("plant", "line_type")
    .agg(F.count("*").alias("readings"),
        F.round(F.avg("temperature_c"), 2).alias("avg_temp_c"),
        F.round(F.avg(F.col("scrap_flag").cast("int")) * 100, 2).alias("scrap_pct"))
    .orderBy("plant", "line_type"))
by_plant.show(truncate=False)
```

```

+-----+-----+-----+-----+-----+
|plant      |line_type|readings|avg_temp_c|scrap_pct|
+-----+-----+-----+-----+-----+
|Bhiwandi Main|Blow      |3974400 |196.34    |1.3      |
|Bhiwandi Main|Extrusion |3974400 |196.35    |1.29     |
|Bhiwandi Main|Injection |3974400 |196.35    |1.29     |
|Chakan Pune  |Blow      |2384640 |196.35    |1.3      |
|Chakan Pune  |Extrusion |3179520 |196.32    |1.3      |
|Chakan Pune  |Injection |2384640 |196.38    |1.29     |
+-----+-----+-----+-----+-----+

```

How it works, line by line.

- `from pyspark.sql import functions as F` imports Spark's built-in functions under the short name `F`, the convention everyone uses. Don't write `from pyspark.sql.functions import *`: that would replace Python's own `sum`, `round`, and `max` with Spark's, and plain Python code later in the notebook would break in confusing ways.
- `.agg(...)` computes several results per group at once, one per argument.
- `F.count("*")` counts rows. `.alias("readings")` names the result column.
- `F.avg("temperature_c")` is the mean. `F.round(x, 2)` rounds it to two decimal places.
- `F.col("scrap_flag")` refers to a column. In most functions, `F.col("x")` and the plain string `"x"` mean the same thing; you need `F.col` when you do something to the column, like here.
- `.cast("int")` turns `True` and `False` into `1` and `0`. The average of a 0/1 column is the **share** of ones: the values `1, 1, 1, 0` average `0.75`, and three of the four are ones. Times `100` makes it a percentage.
- `show(truncate=False)` prints full values; by default `show()` cuts any value longer than `20` characters.

Reading it. Three line types at each plant, with scrap around 1.3% and average temperature near 196 °C. Bhiwandi Main has more readings because it has more machines (15 of the 25). This is a **wide** operation: the group-by needed a shuffle, which section 48.4 makes visible.

The PySpark DataFrame API will feel familiar after pandas (Chapter 18): `select`, `filter`, `groupBy`, `agg`, `join`, `orderBy`, and `withColumn` (which adds or replaces a column) all do what their names say. Spark also accepts SQL directly (`readings.createOrReplaceTempView("readings"); spark.sql("SELECT ...")`), and the two are the same engine underneath. Use whichever makes the logic clearer.

Lazy execution, seen

```

import time

start = time.perf_counter()
hot = (readings.filter(F.col("temperature_c") > 200)
       .select("machine_id", "reading_ts", "temperature_c"))
build_seconds = time.perf_counter() - start

start = time.perf_counter()
hot_count = hot.count() # the action: this runs the job
count_seconds = time.perf_counter() - start

print(f"readings above 200 °C: {hot_count:,}")
print("counting took longer than building:", count_seconds > build_seconds)

```

```
readings above 200 °C: 8,143,256
counting took longer than building: True
```

- `time.perf_counter()` reads a precise clock, in seconds. The difference between two readings is how long the code between them took.
- `filter` keeps the rows where the condition is true; `select` keeps the named columns. The round brackets around `hot = (...)` let the chain run over two lines, as with the builder in section 48.0.

Reading it. Building `hot` touched no data: Spark only checked that the columns exist and wrote down the plan. `count()` is an action, so Spark planned and ran the job, scanning 19.87 million readings to find 8.14 million above 200 °C. On the book's test machine, building took between 0.03 and 0.16 seconds and counting between 0.6 and 1.3 seconds; yours will differ, which is why the cell prints a comparison rather than the times. On a cluster, the same code would run the same way across machines.

Four actions come up all the time: `count()`, `show()`, `collect()` (section 48.7), and **writing files** (also section 48.7). Everything else only adds to the plan.

48.4 Reading the plan, and finding the shuffle

A **query plan** is Spark's description of how it will do the work. Reading it is the difference between guessing at performance and knowing. Every `DataFrame` has an `explain()` method that prints its plan without running it.

```
units = readings.groupBy("machine_id").agg(F.sum("units_made").alias("units"))

spark.conf.set("spark.sql.maxMetadataStringLength", 40) # shorten long values, such as file paths
units.explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[machine_id#0], functions=[sum(units_made#6)])
   +- Exchange hashpartitioning(machine_id#0, 8), ENSURE_REQUIREMENTS, [plan_id=234]
      +- HashAggregate(keys=[machine_id#0], functions=[partial_sum(units_made#6)])
         +- Project [machine_id#0, units_made#6]
            +- FileScan parquet [machine_id#0,units_made#6,reading_date#8] Batched: true, DataFilters: [],
               Format: Parquet, Location: InMemoryFileIndex(1 paths)[file:/home..., PartitionFilters: [],
               PushedFilters: [], ReadSchema: struct<machine_id:string,units_made:int>
```

- `F.sum("units_made")` totals a column, per group.
- `spark.sql.maxMetadataStringLength` limits how long some values in the plan may be before Spark cuts them with `....`. The folder path is the long one: set to 40, your plan shows only the start of it, `[file:/home...` here, or something like `[file:/C:/Us...` on Windows.
- `explain()` prints the **physical plan**: the steps Spark will actually run.

The plan is a tree, printed upside down: each `+-` line is a step whose input is the line below it. Three kinds of noise make it look harder than it is:

- `#0`, `#6`, `#8` after each column name are Spark's internal ids for columns. They change from run to run, and yours may differ from the book's if you ran cells in a different order. Ignore them.
- `[plan_id=...]` is an internal number for the shuffle step, also different every run.
- The `FileScan` line is very long: it lists everything about the file read, separated by commas.

Try it. Run `units.explain("formatted")`. The same plan comes out as a short numbered tree followed by one block of details per step, which is easier to read for long plans. `explain()` alone is what you'll see in most answers online and in interviews.

Optional: a tidier printout

The rest of this chapter prints plans through a small helper, `plan()`, that removes the ids and puts each part of the `FileScan` line on its own line, so the plans fit the page and match what you'll see. It uses only `explain()` underneath. You can skip the helper and use `explain()` everywhere; the plans are the same.

```
import io, re, contextlib

def plan(df, only=None):
    """Print df's physical plan without the ids, one FileScan detail per line."""
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):
        df.explain()
    text = re.sub(r"#\d+L?", "", buffer.getvalue()) # drop column ids like #6 or #56L
    text = re.sub(r", \[plan_id=\d+\]", "", text) # drop plan ids
    for line in text.splitlines()[1:]: # skip "== Physical Plan =="
        parts = [line]
        if "FileScan" in line: # cut before each "Name: " field
            head, *fields = re.split(r", (?=[A-Za-z]+: )", line)
            parts = [head] + [" " + f for f in fields
                              if f.startswith(("PartitionFilters", "PushedFilters", "ReadSchema"))]
        for part in parts:
            if part.strip() and (only is None or any(word in part for word in only)):
                print(part)

spark.conf.unset("spark.sql.maxMetadataStringLength") # back to the default
plan(units)
```

```
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[machine_id], functions=[sum(units_made)])
   +- Exchange hashpartitioning(machine_id, 8), ENSURE_REQUIREMENTS
      +- HashAggregate(keys=[machine_id], functions=[partial_sum(units_made)])
         +- Project [machine_id, units_made]
            +- FileScan parquet [machine_id,units_made,reading_date] Batched: true
PartitionFilters: []
PushedFilters: []
ReadSchema: struct<machine_id:string,units_made:int>
```


How it works, line by line.

- `io.StringIO()` is a text buffer that behaves like a file in memory.
`contextlib.redirect_stdout(buffer)` sends everything printed inside the `with` block into the buffer instead of the screen, so `explain()`'s output ends up in `buffer.getvalue()` as a string.
- `re.sub(pattern, replacement, text)` replaces every match of a regular expression (Chapter 14, section 14.2's "Patterns in ten minutes"). `#\d+L?` means a `#`, one or more digits, and an optional `L`: the column ids. `, \[plan_id=\d+\]` matches the plan ids; the square brackets need a backslash because on their own they mean "one of these characters".
- `text.splitlines()[1:]` splits the text into lines and skips the first, the `== Physical Plan ==` title.
- `re.split(r", (?=[A-Za-z]+: )", line)` cuts the `FileScan` line at every comma-and-space that is followed by a word and a colon, such as `, ReadSchema: .` The `(?= ... )` part means "followed by", without cutting that text off. Commas inside a list, such as `[IsNull(plan_id), EqualTo(...)]`, aren't followed by `Word: ,` so the lists stay whole.
- `head, *fields = ...` puts the first piece in `head` and all the others in the list `fields`.
- `f.startswith((..., ..., ...))` is true if `f` starts with any of the three strings in the tuple. The helper keeps only the three fields this chapter reads, indented by eight spaces.
- `only` is an optional list of words: when you pass it, only lines containing one of them are printed.
`spark.conf.unset()` puts the setting from the previous cell back to its default.

How to read it

From the bottom up, because that's the order things happen.

- `FileScan parquet` reads the files. `ReadSchema` shows Spark only reads two columns, `machine_id` and `units_made`. That's **column pruning**, free with Parquet. `reading_date` is listed in the brackets too, but it comes from the folder names, so it costs nothing to read.
- `Project` keeps only the columns needed.
- `HashAggregate ... partial_sum` is a partial total computed inside each partition, with no communication. Doing this before the shuffle is why grouping is efficient: far less data has to move.
- `Exchange hashpartitioning(machine_id, 8)` is the **shuffle**: rows are redistributed so that every row with the same `machine_id` ends up in the same one of the 8 partitions. `ENSURE_REQUIREMENTS` means Spark added it because the next step needs it.
- `HashAggregate` at the top combines the partial totals into final ones.
- `AdaptiveSparkPlan` means adaptive query execution is on: Spark may change parts of the plan at runtime once it sees the real data sizes (section 48.5).

Every Exchange is a shuffle, and every shuffle costs. When a job is slow, count the exchanges first.

Group by machine: one Exchange, one shuffle

```
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[machine_id], functions=[sum(units_made)])
  +- Exchange hashpartitioning(machine_id, 8), ENSURE_REQUIREMENTS
    +- HashAggregate(keys=[machine_id], functions=[partial_sum(units_made)])
      +- Project [machine_id, units_made]
        +- FileScan parquet [machine_id,units_made,reading_date] Batched: true
          PartitionFilters: []
          PushedFilters: []
          ReadSchema: struct<machine_id:string,units_made:int>
```

← the shuffle

One day at one plant: no Exchange, and pruning

```
*(1) Project [machine_id, temperature_c]
+- *(1) Filter (isnotnull(plant) AND (plant = Bhiwandi Main))
  +- *(1) ColumnarToRow
    +- FileScan parquet [machine_id,plant,temperature_c,reading_date] Batched: true
      PartitionFilters: [isnotnull(reading_date), (reading_date = 2025-12-01)]
      PushedFilters: [IsNotNull(plant), EqualTo(plant,Bhiwandi Main)]
      ReadSchema: struct<machine_id:string,plant:string,temperature_c:double>
```

← 1 folder of 92

← 3 columns read

Read a plan from the bottom up. Count the Exchange lines: each one is a shuffle, the cost.

Then check the scan: PartitionFilters skips whole folders; ReadSchema lists the only columns read.

Figure 48.2 – The two things to look for in a plan: exchanges (cost) and pruning (work avoided).

48.5 Making a job faster

Partition pruning: don't open boxes you don't need

```
december_1 = readings.filter((F.col("reading_date") == "2025-12-01") &
                             (F.col("plant") == "Bhiwandi Main"))
print(f"rows for 1 December at Bhiwandi Main: {december_1.count():,}")
plan(december_1.select("machine_id", "temperature_c"))
```

```
rows for 1 December at Bhiwandi Main: 129,600
*(1) Project [machine_id, temperature_c]
+- *(1) Filter (isnotnull(plant) AND (plant = Bhiwandi Main))
  +- *(1) ColumnarToRow
    +- FileScan parquet [machine_id,plant,temperature_c,reading_date] Batched: true
      PartitionFilters: [isnotnull(reading_date), (reading_date = 2025-12-01)]
      PushedFilters: [IsNotNull(plant), EqualTo(plant,Bhiwandi Main)]
      ReadSchema: struct<machine_id:string,plant:string,temperature_c:double>
```

- **&** combines two conditions: both must be true, as **AND** does in SQL. Each condition needs its own round brackets, exactly as in pandas (Chapter 18).

Reading it. The plan has **no Exchange at all** (this query needs no shuffle), and two lines matter:

- **PartitionFilters:** `[isnotnull(reading_date), (reading_date = 2025-12-01)]` means Spark reads one day's folder out of 92. It never opens the other 91.
- **PushedFilters:** `[IsNotNull(plant), EqualTo(plant,Bhiwandi Main)]` means the plant filter is handed to the Parquet reader, which can skip chunks of a file (row groups) whose statistics rule the value out. That only helps when the values are clustered, for example if the file were sorted by plant (Chapter 49). Here each day's file mixes both plants, so the filter is applied as the rows are read.

Between them, a query over 19.87 million readings touched 216,000. **Partitioning your storage by the column people filter on** (usually a date) is the single biggest performance decision in a data platform.

Pruning works only when Spark can see that a filter is about the partition column. Compare two filters that you know mean nearly the same thing:

```
plan(readings.filter(F.month("reading_date") == 12).select("machine_id"), only=
  ["PartitionFilters"])
plan(readings.filter(F.year("reading_ts") == 2025).select("machine_id"), only=["PartitionFilters"])
```

```
PartitionFilters: [isnotnull(reading_date), (month(reading_date) = 12)]
PartitionFilters: []
```

- **F.month()** and **F.year()** take the month and year out of a date or timestamp.
- **only=["PartitionFilters"]** prints just that line of each plan.

Reading it. An expression on the partition column itself, `month(reading_date) = 12`, still prunes: Spark checks it against each folder name. A filter on `reading_ts` doesn't, even though you know `reading_ts` and `reading_date` always agree. Spark doesn't know that, so `PartitionFilters` is empty and it opens every folder.

Watch out: a filter Spark can't see doesn't prune. Filtering with a Python function (a user-defined function, section 48.7), or on a different column that carries the same information (`F.year("reading_ts") == 2025`), hides the filter from the planner, and Spark reads everything. Filter on the partition column, and check `PartitionFilters` in the plan.

Joins: the shuffle you can often avoid

Suppose the plant team wants readings per supervisor. The supervisors live in a tiny table. `createDataFrame` makes a small Spark DataFrame from Python data:

```
machines = spark.createDataFrame(
    [("M-01", "Anil Deshmukh", 2019), ("M-02", "Sunita Rao", 2021), ("M-03", "Imran Shaikh",
    2017)],
    ["machine_id", "supervisor", "installed_year"])
machines.show()
```

```

+-----+-----+-----+
|machine_id| supervisor|installed_year|
+-----+-----+-----+
|      M-01|Anil Deshmukh|      2019|
|      M-02|  Sunita Rao|      2021|
|      M-03|Imran Shaikh|      2017|
+-----+-----+-----+

```

- The first argument is a **list of tuples** (Chapter 17), one tuple per row; the second is the list of column names, in the same order.

Joining the readings to this table needs matching rows together, which normally means shuffling **both** sides. Spark would normally notice that `machines` is tiny and avoid that (as you'll see next), so the first line switches that off, to show the expensive plan:

```

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1) # don't broadcast automatically
joined = (readings.join(machines, on="machine_id", how="inner")
          .groupBy("supervisor").agg(F.count("*").alias("readings")))
plan(joined, only=["Join", "Exchange"])
joined.orderBy("supervisor").show()

```

```

+- Exchange hashpartitioning(supervisor, 8), ENSURE_REQUIREMENTS
    +- SortMergeJoin [machine_id], [machine_id], Inner
      :   +- Exchange hashpartitioning(machine_id, 8), ENSURE_REQUIREMENTS
      :   +- Exchange hashpartitioning(machine_id, 8), ENSURE_REQUIREMENTS
+- -----+-----+
| supervisor|readings|
+- -----+-----+
|Anil Deshmukh| 794880|
|Imran Shaikh| 794880|
|  Sunita Rao| 794880|
+- -----+-----+

```

- `join(other, on="machine_id", how="inner")` joins on the `machine_id` column. `how="inner"` keeps only readings whose machine is in both tables, as `INNER JOIN` does (Chapter 12). That's why only 3 of the 25 machines appear in the result.
- `only=["Join", "Exchange"]` prints only the join and shuffle lines of a longer plan.

Reading it. `SortMergeJoin` with two exchanges below it: both sides are redistributed by `machine_id`. That means 19.87 million rows are written to shuffle files (and, on a cluster, sent across the network) to be matched with three rows. The results are right (794,880 readings for each machine: 92 days × 8,640), and the work is wasteful.

The fix is a **broadcast join**: send the small table to every worker, so the big table never moves.

```

broadcast_join = (readings.join(F.broadcast(machines), on="machine_id")
                  .groupBy("supervisor").agg(F.count("*").alias("readings")))
plan(broadcast_join, only=["Join", "Exchange"])
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10 * 1024 * 1024) # back to the default

```

```

+- Exchange hashpartitioning(supervisor, 8), ENSURE_REQUIREMENTS
    +- BroadcastHashJoin [machine_id], [machine_id], Inner, BuildRight, false, false
      +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true]),false)

```

- **F.broadcast(machines)** marks the small table to be copied to every worker. `how` is left out, and defaults to `"inner"`.
- The last line puts the automatic threshold back to its default, 10 MB ($10 \times 1,024 \times 1,024$ bytes).

Reading it. `BroadcastHashJoin` with a `BroadcastExchange` for the small side, and **no exchange on the big side**. The big data stays where it is. Spark does this automatically when it can tell a table is small (`spark.sql.autoBroadcastJoinThreshold`, 10 MB by default); `F.broadcast()` tells it explicitly, which matters when the size estimate is wrong, for example after a complex sub-query.

The remaining `Exchange hashpartitioning(supervisor, 8)` is the group-by, which still needs a shuffle. One shuffle instead of three.

Skew: when one worker gets a truckload

Skew is uneven partitions. If half the readings come from one machine, the worker holding that machine's rows does half the work while others idle. Symptoms: a stage where 199 tasks finish in seconds and one runs for an hour, and memory errors on a single task.

The place to see skew is **Spark's web interface**. While a session is running, open `http://localhost:4040` in your browser. The *Stages* tab lists each stage of your jobs; click one and compare the **maximum** task duration with the **median**. A maximum many times the median is skew.

Usual fixes:

- **Adaptive query execution** (AQE, on by default since Spark 3.2) detects skewed join partitions and splits them. Often this is enough.
- **Salting**: add a random number to the join key on the big side and expand the small side to match, spreading the hot key across workers.
- **Handle the hot key separately**: filter it out, process it on its own, and union the results.
- **Filter earlier**, so the skewed rows never reach the join.

Salting is for joins where **both** sides are large. With a three-row table you'd simply broadcast, as above; the example below uses `machines` only to keep it small enough to see. First the small side: every machine row is copied once per salt value.

```
SALTS = 8
small = machines.withColumn("salt", F.explode(F.array([F.lit(i) for i in range(SALTS)])))
print("small side:", small.count(), "rows")
small.filter(F.col("machine_id") == "M-01").select("machine_id", "salt").show()
```

```

small side: 24 rows
+-----+-----+
|machine_id|salt|
+-----+-----+
|      M-01|  0|
|      M-01|  1|
|      M-01|  2|
|      M-01|  3|
|      M-01|  4|
|      M-01|  5|
|      M-01|  6|
|      M-01|  7|
+-----+-----+

```

- **withColumn(name, value)** adds a column (or replaces one with the same name).
- **F.lit(i)** turns a plain Python value into a Spark column holding that value in every row. The list comprehension makes eight of them, for 0 to 7.
- **F.array([...])** puts those eight values into one array (a list) per row, and **F.explode()** turns each element of an array into its own row. So each machine row becomes eight rows, with salt 0 to 7: $3 \times 8 = 24$.

Then the big side: each reading gets one random salt, and the join uses both columns.

```

big = readings.withColumn("salt", (F.rand(seed=42) * SALTS).cast("int"))
big.groupBy("salt").count().orderBy("salt").show()

salted = big.join(small, on=["machine_id", "salt"])
plain = readings.join(machines, on="machine_id")
print(f"salted join: {salted.count():,} rows; plain join: {plain.count():,} rows")

```

```

+----+-----+
|salt| count|
+----+-----+
|  0|2485538|
|  1|2486457|
|  2|2481887|
|  3|2483628|
|  4|2483236|
|  5|2483089|
|  6|2482425|
|  7|2485740|
+----+-----+

```

```
salted join: 2,384,640 rows; plain join: 2,384,640 rows
```

- **F.rand(seed=42)** gives each row a random number between 0 and 1. The **seed** makes the numbers repeatable, so your counts match the book's.
- Times **SALTS** and **.cast("int")**, which drops the decimal part, gives a whole number from 0 to 7.
- **on=["machine_id", "salt"]** joins on both columns: a reading with salt 5 meets only the copy of its machine with salt 5.

Reading it. The salts spread the readings evenly, about 2.48 million each, so a hot machine's rows would now land in eight partitions instead of one. The last line is the check that matters: the salted join returns exactly as many rows as the plain one, 2,384,640 ($3 \text{ machines} \times 794,880$). Salting changed

how the work is split, not the answer.

A short list of what actually helps

Change	Why it helps	Look for in the plan
Partition storage by date	Whole folders are skipped	<code>PartitionFilters</code>
Select only needed columns	Less data read and moved	<code>ReadSchema</code>
Filter before joining and grouping	Less data shuffled	Filters below the Exchange
Broadcast small tables	The big side never moves	<code>BroadcastHashJoin</code> , no Exchange on the big side
Set <code>shuffle.partitions</code> sensibly	Avoids thousands of tiny tasks	Number in <code>hashpartitioning(..., n)</code>
Write files of a sensible size (128–512 MB, once the data is that large)	Fewer, bigger reads	Number of output files
Cache only what's reused several times	Avoids recomputation; costs memory	<code>InMemoryTableScan</code>

48.6 The same question in three tools

Now the comparison that matters in practice: one question, three engines. Two of them you've met, DuckDB (Chapter 45) and Spark. The third, Polars, needs a page of its own first.

Polars in one page

Polars is a DataFrame library, like pandas, written for speed: it uses every core it's allowed and can plan a whole query before running it, as Spark does. To compare the engines fairly, this chapter gives Polars the same two cores as Spark. That setting must come before Polars is first imported:

```
os.environ["POLARS_MAX_THREADS"] = "2"
import polars as pl

day = pl.read_parquet("sensor_readings/reading_date=2025-12-01/part-0.parquet")
print(day.shape)
print(day.select("machine_id", "reading_ts", "temperature_c", "units_made", "scrap_flag").head(3))
```

```
(216000, 8)
shape: (3, 5)
```

machine_id	reading_ts	temperature_c	units_made	scrap_flag
---	---	---	---	---
str	datetime[ms]	f64	i32	bool
M-01	2025-12-01 00:00:00	187.28	5	false
M-01	2025-12-01 00:00:10	187.11	2	false
M-01	2025-12-01 00:00:20	185.26	2	false

- `os.environ[...] = "2"` sets an environment variable for this notebook's process; Polars reads `POLARS_MAX_THREADS` once, when it's imported.

- `import polars as pl` is the usual short name.
- `pl.read_parquet()` reads one file, straight away, into a Polars DataFrame. `.shape` is (rows, columns), as in pandas. There is no `reading_date` column: this is one file, read without its folder name.
- `select("machine_id", ...)` with column names keeps those five columns, so the table fits the page, and `head(3)` shows the first three rows. Polars prints each column's **type** under its name: `str` is text, `datetime[ms]` a timestamp stored to the millisecond, `f64` a decimal number, `i32` a 32-bit whole number, and `bool` True/False.

Polars works through **expressions**: small recipes such as "the `temperature_c` column, averaged", which you hand to a method like `select` or `filter`:

```
print(day.select(pl.col("temperature_c").mean().round(2)))
print(day.filter(pl.col("temperature_c") > 200).height, "readings above 200 °C that day")
```

```
shape: (1, 1)
```

temperature_c

f64
196.49

```
86879 readings above 200 °C that day
```

- `pl.col("temperature_c")` means "the `temperature_c` column", like Spark's `F.col`. Methods chain onto it: `.mean()`, then `.round(2)`.
- `select(...)` returns a new DataFrame with the expressions' results. `filter(...)` keeps the rows where the expression is true; `.height` is the number of rows.

Why expressions, instead of pandas-style `day["temperature_c"].mean()`? Because an expression describes the work without doing it, Polars can see a whole query at once and plan it, as Spark does. That's what **lazy** mode uses. `pl.scan_parquet()` doesn't read anything; it starts a plan. `.collect()` runs it, and `.explain()` shows the optimized plan first:

```
lazy = (pl.scan_parquet("sensor_readings/**/*.parquet")
        .filter(pl.col("temperature_c") > 200)
        .select(pl.len()))
print(lazy.explain())
print(lazy.collect())
```

```
SELECT [len()]
  simple π 0/0 []
    Parquet SCAN [sensor_readings/reading_date=2025-10-01/part-0.parquet, ... 91 other sources]
    PROJECT 1/8 COLUMNS
    SELECTION: col("temperature_c") > 200.0
    ESTIMATED ROWS: 19872614
shape: (1, 1)
```

len

u32
8143256

- **"sensor_readings/**/*.parquet"** is a pattern: every Parquet file in every day folder (* matches any name, as in the terminal, Chapter 34).
- **pl.len()** counts rows.

Reading it. Read it from the bottom up, as in Spark. The **Parquet SCAN** covers all 92 files, with a **SELECTION** (the filter, pushed into the read) and **PROJECT 1/8 COLUMNS**: of the eight columns in each file, only **temperature_c** is read. That's column pruning and a pushed filter, the same ideas you saw in Spark's plan. **ESTIMATED ROWS** is Polars' guess before reading, not a count. The count itself, 8,143,256, matches Spark's.

One question, three engines

"For readings above 190 °C, what's the average temperature, scrap rate, and total output per plant?" First DuckDB, in SQL:

```
import duckdb

QUESTION = """
    SELECT plant,
           ROUND(AVG(temperature_c), 2)           AS avg_temp_c,
           ROUND(AVG(CAST(scrap_flag AS INT)) * 100, 2) AS scrap_pct,
           SUM(units_made)::BIGINT                AS units
    FROM readings
    WHERE temperature_c > 190
    GROUP BY plant
    ORDER BY plant"""

con = duckdb.connect()
con.execute("SET threads = 2")
con.execute("CREATE VIEW readings AS SELECT * FROM read_parquet('sensor_readings/**/*.parquet')")
print(con.execute(QUESTION).df().to_string(index=False))
```

plant	avg_temp_c	scrap_pct	units
Bhiwandi Main	203.43	1.33	23234149
Chakan Pune	203.44	1.33	15491402

- **duckdb.connect()** with no file name opens a database in memory. **SET threads = 2** gives it two cores, like the other two engines.
- **read_parquet('sensor_readings/**/*.parquet')** reads every file; DuckDB recognizes the **reading_date=** folder names and adds **reading_date** as a column automatically. The **view** lets the

question say FROM readings.

- `.df()` returns the result as a pandas DataFrame (Chapter 18), and `to_string(index=False)` prints it without the row numbers.

Then Polars, one method per line:

```
polars_answer = (pl.scan_parquet("sensor_readings/**/*.parquet")
    .filter(pl.col("temperature_c") > 190)
    .group_by("plant")
    .agg(pl.col("temperature_c").mean().round(2).alias("avg_temp_c"),
        (pl.col("scrap_flag").cast(pl.Int8).mean() *
         100).round(2).alias("scrap_pct"),
        pl.col("units_made").sum().alias("units"))
    .sort("plant")
    .collect())
print(polars_answer)
```

shape: (2, 4)

plant	avg_temp_c	scrap_pct	units
---	---	---	---
str	f64	f64	i32
Bhiwandi Main	203.43	1.33	23234149
Chakan Pune	203.44	1.33	15491402

- `group_by("plant")` then `agg(...)` computes one result per expression for each plant, like Spark's `groupBy().agg()`. `.alias()` names each result.
- `.cast(pl.Int8)` turns True/False into 1/0 (as a small 8-bit whole number), so that `.mean()` gives the share of scrap readings, the same trick as in Spark.
- `.sort("plant")` orders the rows; `.collect()` runs the whole plan.
- Polars keeps the input's type for the sum, 32-bit whole numbers (`i32`), which hold values up to about 2.1 billion. The stretch goal's ten-times-larger data is still safe; for bigger totals, add `.cast(pl.Int64)` before `.sum()`.

And Spark, with the functions from section 48.3:

```
(readings.filter(F.col("temperature_c") > 190)
    .groupBy("plant")
    .agg(F.round(F.avg("temperature_c"), 2).alias("avg_temp_c"),
        F.round(F.avg(F.col("scrap_flag").cast("int")) * 100, 2).alias("scrap_pct"),
        F.sum("units_made").alias("units"))
    .orderBy("plant")
    .show())
```

```
+-----+-----+-----+-----+
|      plant|avg_temp_c|scrap_pct|  units|
+-----+-----+-----+-----+
|Bhiwandi Main|    203.43|     1.33|23234149|
|  Chakan Pune|    203.44|     1.33|15491402|
+-----+-----+-----+-----+
```

Reading it. All three agree to the last unit: 203.43 °C and 1.33% scrap at Bhiwandi Main, 23,234,149 units. That's the first thing to check when comparing tools, and it's reassuring how often people skip it.

Timings

Speed depends on the machine, so these numbers are **measurements from one specific setup, not a benchmark**. The timing code is short. It runs each engine three times and keeps the fastest run:

```
def best_of_3(job):
    """Run job three times; return the fastest time, in milliseconds."""
    times = []
    for _ in range(3):
        start = time.perf_counter()
        job()
        times.append(time.perf_counter() - start)
    return min(times) * 1000

duckdb_job = lambda: con.execute(QUESTION).fetchall()
polars_job = lambda: (pl.scan_parquet("sensor_readings/*.parquet")
    .filter(pl.col("temperature_c") > 190).group_by("plant")
    .agg(pl.col("temperature_c").mean(),
        pl.col("scrap_flag").cast(pl.Int8).mean(),
        pl.col("units_made").sum())
    .collect())
spark_job = lambda: (readings.filter(F.col("temperature_c") > 190).groupBy("plant")
    .agg(F.avg("temperature_c"), F.avg(F.col("scrap_flag").cast("int")),
        F.sum("units_made"))
    .collect())

for name, job in [("DuckDB", duckdb_job), ("Polars", polars_job), ("Spark", spark_job)]:
    print(f"{name:<7}{best_of_3(job):8.1f} ms")
```

DuckDB	231.7 ms
Polars	273.2 ms
Spark	1137.0 ms

How it works, line by line.

- **best_of_3(job)** takes a **function** as its argument and calls it three times, timing each call with `time.perf_counter()`. `for _ in range(3)` repeats three times; `_` is the usual name for a loop variable you don't use.
- It returns the **fastest** of the three. The first run is usually the slowest, because the files aren't yet in the operating system's memory cache and, for Spark, Java is still warming up. The fastest run measures the engine, not the disk.
- **Lambda:** ... (Chapter 18, "Functions without names") makes a small function with no arguments. `duckdb_job` doesn't run the query; calling `duckdb_job()` does, which is what `best_of_3` needs.
- Each job is the whole question, including bringing the two result rows back to Python: `fetchall()` for DuckDB, `collect()` for Polars and Spark. The Spark job reads through the `readings` DataFrame from section 48.3, so Spark's start-up time isn't counted.
- The `for` loop takes each (name, function) pair in turn. `{name:<7}` pads the name to 7 characters so the numbers line up, and `:8.1f` prints the time 8 characters wide with one decimal place.

Your numbers will differ, and should: they depend on your processor, your disk, and what else is running. The companion script `time_three_tools.py` runs the same timings on both the one-day folder and the full data. On the book's test machine, a shared computer with 4 cores, with each engine given two of them, it measured:

Data	DuckDB	Polars	Spark (local)
216,000 readings (one day)	6.8 ms	5.8 ms	252.3 ms
19,872,000 readings (92 days)	213.0 ms	262.3 ms	1,211.8 ms

What this does and doesn't show.

- On a single machine, at this size, **DuckDB and Polars are several times faster than Spark**, and far simpler to run. On the small file, Spark's overhead dominates completely.
- The gap is not a flaw in Spark. Spark is paying for machinery that lets the same code run over hundreds of machines, and here it has only two cores, so its parallelism has almost nowhere to go.
- What these numbers *can't* show is the crossover: a dataset large enough that one machine runs out of memory or time, where Spark keeps going by adding machines. You'll reach that point with hundreds of millions to billions of rows, or with jobs that need more memory than a machine can hold.

Try it. Run `python time_three_tools.py` in your practice folder and compare. On 8 cores, all three get faster: DuckDB and Polars use every core too. Spark's fixed start-up cost stays, so on the one-day file it will still be slowest by far. The gap on the full data may narrow, but on one machine it rarely reverses. Spark's advantage appears only when the data or the work outgrows one machine.

Choosing

Choose	When
SQL in your warehouse	The data is already there and the query is a query (most of the time)
DuckDB	Files on disk or in cloud storage, SQL, one machine, up to tens of millions of rows and beyond
Polars	The same, expressed as Python DataFrame code, or inside an existing Python pipeline
Spark	Data too large for one machine, jobs that must scale with the business, streaming at volume, or a platform (Databricks, EMR) your company already runs
A cluster of any kind	Only when a single machine has been tried and can't do it

48.7 Writing Spark that doesn't surprise you

Don't collect() big results. `collect()` is the action that brings rows back into Python, as a list, and it pulls every one of them into the driver's memory. Bring back aggregates, samples, or filtered slices:

```
one_machine_day = readings.filter((F.col("reading_date") == "2025-12-01") &
                                  (F.col("machine_id") == "M-07"))
rows = one_machine_day.select("reading_ts", "temperature_c").limit(3).collect()
print("three rows brought back to Python:", [(str(r[0]), r[1]) for r in rows])
print(f"rows that would come back without the filter: {readings.count():,}")
```

```
three rows brought back to Python: [('2025-12-01 00:00:00', 195.78), ('2025-12-01 00:00:10', 191.9), ('2025-12-01 00:00:20', 195.91)]
rows that would come back without the filter: 19,872,000
```

- `limit(3)` keeps only the first three rows, before anything leaves Spark.
- `collect()` returns a list of `Row` objects. A `Row` works like a tuple: `r[0]` is its first value (the timestamp) and `r[1]` its second. `str()` turns the timestamp into readable text.

Write full results to files, and watch how many. Spark writes one file per partition. Straight after a shuffle that can be up to `spark.sql.shuffle.partitions` files: 200 by default. Adaptive execution often merges small partitions, but don't rely on it for file sizes. Thousands of small files make every later read slow (the "small files problem", Chapter 49). This writes a small weekly summary as one file:

```
weekly = (readings.groupBy("plant", F.weekofyear("reading_date").alias("week"))
          .agg(F.sum("units_made").alias("units"),
              F.round(F.avg(F.col("scrap_flag").cast("int")) * 100,
                      2).alias("scrap_pct")))
weekly.coalesce(1).write.mode("overwrite").parquet("out/weekly_scrap")

files = sorted(f for f in os.listdir("out/weekly_scrap") if not f.startswith("."))
print(len(files), "files:", [f if f == "_SUCCESS" else f[:10] + "..." for f in files])
print("rows written:", spark.read.parquet("out/weekly_scrap").count())
```

```
2 files: ['_SUCCESS', 'part-00000...']
rows written: 28
```

- `F.weekofyear()` gives the week number of a date, so the result has one row per plant per week. The `agg(...)` with `F.sum`, `F.avg`, `.cast("int")`, `F.round`, and `.alias` works exactly as in section 48.3.
- `coalesce(1)` merges the partitions into one without a shuffle, so one file is written. `repartition(n)` does a full shuffle into exactly `n` evenly sized partitions; use it when you need more files, or balanced ones.
- `.write.mode("overwrite").parquet(path)` writes the result as Parquet. "overwrite" replaces whatever is there, so running the cell again gives the same result: the idempotent re-run of Chapter 46. The default mode, "error", refuses to write if the folder exists.
- `sorted(f for f in ... if not f.startswith("."))` lists the output folder, leaving out hidden files. In the print, `f if f == "_SUCCESS" else f[:10] + "..."` is Python's one-line *if/else*: it keeps `_SUCCESS` whole and shortens the long data-file name to its first 10 characters.

- Spark writes a **folder**, not a file. The listing skips hidden files (names starting with a dot, small checksum files Spark adds). What's left is one data file, whose full name looks like `part-00000-` followed by a long random id and `.snappy.parquet`, and an empty file called `_SUCCESS`, which Spark writes last so that a later job can tell the output is complete.
- For output partitioned into folders, as the sensor archive is, add `.partitionBy("week")` before `.parquet()`.

Aim for files of roughly 128–512 MB **once your data is that large**. This summary is a few kilobytes, so one file is right.

The rest of the rules:

- **Cache deliberately.** `df.cache()` keeps a DataFrame in memory for reuse. It helps when the same intermediate result is used several times; otherwise it wastes memory you'd rather give to the shuffle.
- **Prefer built-in functions over Python UDFs.** A Python **user-defined function** (UDF) is your own Python function applied to every row. It moves every row out of Java into Python and back, and blocks the optimizer from pushing filters down. Built-ins in `pyspark.sql.functions` stay in the engine.
- **Test on a small partition.** Run your job on one day before running it on a year: same code, one folder.
- **Keep work off the driver, and keep the job idempotent.** Bring back only small results, and don't loop over rows in Python. Chapter 46's rules apply: writes replace partitions, and a re-run is safe.

The settings in this chapter

Setting	What it does	What happens if you change it
<code>.master("local[2]")</code>	Runs Spark on this computer with two worker threads	<code>local[*]</code> uses every core: faster, but the rest of the computer slows down while a job runs
<code>spark.sql.shuffle.partitions</code> (set to 8)	How many pieces the data is cut into after each shuffle	Left at 200 on a laptop: hundreds of tiny tasks and slower jobs. Too few on a cluster: most cores sit idle
<code>spark.sql.autoBroadcastJoinThreshold</code> (10 MB)	Tables smaller than this are broadcast automatically	<code>-1</code> switches automatic broadcasting off (section 48.5). Much larger: Spark may copy a big table to every worker and run out of memory
<code>spark.sql.maxMetadataStringLength</code> (100)	How long a value in a printed plan may be before it's cut	Smaller hides detail, such as long filter lists; it changes only the printout, never the job
<code>spark.ui.showConsoleProgress</code>	Draws a progress bar under each cell	<code>"true"</code> brings the bar back: handy for long jobs, noisy in a notebook

48.8 The wider landscape in one page

Thing	What it is	Where you'll meet it
Apache Spark	The distributed engine in this chapter	Almost everywhere; the default for large batch work
Databricks	A managed platform built around Spark, with notebooks, Delta Lake, and job scheduling	Large enterprises; very common in job descriptions
Amazon EMR, Google Dataproc, Azure Synapse/Fabric	Cloud-managed Spark (and friends)	Companies standardized on one cloud
Delta Lake, Apache Iceberg, Apache Hudi	Table formats that add transactions and time travel to files	Chapter 49
Trino / Presto	A query engine that runs SQL across many sources without moving the data	Ad-hoc analytics over a lake
Ray, Dask	Distributed Python for machine learning and general workloads	Data science teams scaling Python
Hadoop and MapReduce	The original distributed platform; HDFS still appears	Older systems; mostly historical now
Snowflake, BigQuery, Redshift	Cloud warehouses that scale SQL without you managing machines	Chapter 49; often the simpler answer

The honest summary for a career: learn **SQL and one single-machine engine** deeply, learn **Spark's model** well enough to read plans and write correct jobs, and learn a **cloud platform** when a job needs one. Interviews for data engineering roles ask about Spark far more often than most day jobs use it, which is a reason to understand it, not a reason to build on it.

When you're done for the day, `spark.stop()` shuts the session down and frees its memory.

APPLY

Common mistakes

Mistake	Symptom	Fix
Java missing or too old	JAVA_GATEWAY_EXITED or <code>UnsupportedClassVersionError</code> on the first cell	Install Java 21; check <code>java -version</code> (48.0)
Reaching for Spark on small data	A five-second job takes two minutes	Measure first; DuckDB or Polars (48.1, 48.6)
Leaving <code>shuffle.partitions</code> at 200 locally	Hundreds of tiny tasks, slow jobs	Set it to a few times your core count
Not looking at the plan	Guessing at why a job is slow	<code>explain()</code> ; count the exchanges (48.4)
Filtering on <code>reading_ts</code> instead of the partition column	<code>PartitionFilters</code> empty; whole dataset read	Filter on the partition column (48.5)

Mistake	Symptom	Fix
Shuffling a big table to join a tiny one	SortMergeJoin with two exchanges	Broadcast the small side (48.5)
Ignoring skew	One task runs for hours; memory errors	AQE, salting, or handle the hot key separately
<code>collect()</code> on a large DataFrame	Driver runs out of memory	Aggregate, sample, or write to files (48.7)
Python UDFs in hot paths	Slow stages; optimizer can't push filters	Use built-in functions
Caching everything	Memory pressure, slower shuffles	Cache only what's reused
Thousands of tiny output files	Later reads get slower every week	<code>coalesce()</code> or <code>repartition()</code> before writing (48.7)
Comparing tools without comparing answers	A "fast" query that's wrong	Check the numbers match first (48.6)
Treating one machine's benchmark as universal	Wrong tool chosen for a bigger job	State the hardware; know where the crossover is

In the real world: the cluster that was slower than the laptop

Riverstone's plant team wanted a weekly report: for each machine, how long it ran above its safe temperature, and how that lined up with scrap. By then sensors covered about 200 machines across both plants, and the archive held six months of readings: about 300 million.

The contractor who took the job had done large Spark work before, and quoted a managed Spark cluster. Within three weeks there was a cluster, a notebook, and a job that took 26 minutes and cost more per month than the warehouse.

Meera asked the one question that changed the project: *"How much data does the report actually read?"*

The answer was a week at a time: about 12 million readings, filtered to one condition, grouped by machine. The cluster was reading the whole archive every run, because the files were partitioned by machine rather than by date, and the report's date filter couldn't prune anything.

Two changes followed. The archive was rewritten partitioned by date, with machine as a sorted column inside each day's files. Then the same report was written in DuckDB, reading the week's folders directly from cloud storage, and it finished in under a minute on a small virtual machine.

The cluster wasn't wrong in principle. It was solving a problem the storage layout had created. Riverstone kept Spark for one genuinely large job, the yearly rebuild across the full archive, and ran everything else on a single machine.

At the review, the contractor made the point that mattered most. "The 26 minutes weren't Spark's fault," he said. "We were reading 300 million rows to answer a question about 12 million. No engine fixes that."

What made this work.

- **Meera asked about the data the job touches**, not the size of the archive.
- **The storage layout was the real problem**; the engine was a symptom.
- **They kept Spark where it earned its place** instead of treating the choice as ideological.

Project: three questions, three engines

Goal: answer the plant team's questions on the sensor data, choose an engine with evidence, and be able to defend the choice.

Tools you'll need

- **Python** in the book's virtual environment with `pyspark` and `polars` (section 48.0), `duckdb` (Chapter 45), and `pandas` and `pyarrow` (Chapter 18); **Java 17, 21, or 25** (check with `java -version`).
- **The Chapter 48 companion folder** (`companion/ch48/`), copied to `work/ch48`: `make_sensor_data.py` (generates the dataset) and `time_three_tools.py` (the timing script from section 48.6). Run Python from that folder.
- **Versions used for the outputs shown:** Python 3.11.15, PySpark 4.2.0, Polars 1.44.2, DuckDB 1.5.6, PyArrow 25.0.1, pandas 3.0.6, OpenJDK 21.0.10. The timings came from a shared test machine with 4 cores, with each engine given two.
- **Spark's web interface** at `http://localhost:4040` while a session runs (section 48.5).

Option A: Riverstone. Use the generated sensor dataset.

Option B: your own data. Any dataset of at least a few million rows that you're allowed to use, stored as Parquet.

Steps

1. **Generate the data** and record its size on disk and its row count.
2. **Answer three questions** in DuckDB, Polars, and PySpark, and confirm all three give identical numbers: a. For each machine, the share of readings above 200 °C in December. b. For each plant and week, total units made and scrap percentage. c. The ten machine-days with the highest scrap rate, with at least 5,000 readings that day.
3. **Time each engine** on each question, three runs, and report the fastest. State your hardware.
4. **Read the Spark plan** for question (c). Write down every exchange and what causes it.
5. **Make one query faster** by pruning, broadcasting, or reducing shuffles. Show the plan before and after, and the timing change.
6. **Break something on purpose:** filter on `F.year(F.col("reading_ts"))` instead of the partition column, and show what happens to `PartitionFilters` and to the timing.
7. **Write a recommendation** of half a page: which engine for this report, at what data size you'd change your mind, and what it would cost to be wrong.

Stretch goals

- Repeat with the dataset ten times larger (change `DAYS` in the generator) and see whether the ranking changes.
- Write the output of question (b) as Parquet partitioned by week, and measure how much faster the next read gets.
- Introduce skew by duplicating one machine's rows, and show what adaptive query execution does about it.

REVIEW

Recap

- **Most "big data" isn't.** Ask what the job actually reads, after filtering.
- Spark needs **Java** and the `pyspark` library; a **SparkSession** in **local mode** runs it on your own cores.
- A distributed engine splits data into **partitions**, runs work on **workers**, and coordinates from the **driver**.
- **Narrow** operations stay inside a partition and are cheap; **wide** operations need a **shuffle** and are expensive. Every `Exchange` in a plan is a shuffle, and `explain()` shows the plan.
- **Lazy execution** means nothing runs until an **action**; the plan is built from the whole query.
- **Partition pruning** (folders skipped) and **column pruning** (columns not read) remove work before it starts; check `PartitionFilters` and `ReadSchema`.
- A **broadcast join** sends a small table to every worker so the big one never moves.
- **Skew** makes one worker do most of the work; adaptive execution, salting, or special-casing the hot key fix it.
- **Polars** builds queries from expressions, and its lazy mode plans them as Spark does.
- On one machine, at 19.87 million rows, **DuckDB (213 ms) and Polars (262 ms) beat Spark (1,212 ms)**; the numbers are machine-specific, and Spark's advantage appears when data outgrows one machine.
- Avoid `collect()` on large results, thousands of tiny files, careless caching, and Python UDFs in hot paths.
- Learn SQL and one single-machine engine deeply, understand Spark's model, and add a platform when a job needs it.

Key terms

distributed computing · Java runtime · `JAVA_HOME` · `SparkSession` · partition · worker (executor) · driver · narrow operation · wide operation · shuffle · exchange · lazy execution · action · query plan · physical plan · `explain()` · adaptive query execution (AQE) · Hive-style partitioning · partition pruning · column pruning · pushed filter · broadcast join · sort-merge join · skew · salting · cache · `collect` · small files problem · user-defined function (UDF) · local mode · Spark web interface · DuckDB · Polars · expression · lazy frame · PySpark · Databricks · EMR · Dataproc · Trino · Ray

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ You can install Java and PySpark, start a SparkSession, and fix the usual start-up errors.
- ☐ You can argue, with numbers, whether a dataset needs a distributed engine.
- ☐ You can explain partitions, workers, the driver, and narrow versus wide operations.
- ☐ You can say what a shuffle is and why it dominates cost.
- ☐ You can write PySpark to read, filter, group, join, and write Parquet.
- ☐ You can explain lazy execution and name three actions.
- ☐ You can read a physical plan with `explain()`, find every exchange, and say what caused each one.
- ☐ You can prove partition pruning and column pruning are happening.
- ☐ You can turn a sort-merge join into a broadcast join, and explain when not to.
- ☐ You can describe skew, its symptoms, and three fixes.
- ☐ You can write a simple Polars query with expressions, eagerly and lazily.
- ☐ You can compare engines on correctness first, then speed, and state your hardware.
- ☐ You can list five ways a Spark job surprises people, including `collect()` and small files.
- ☐ You can place Databricks, EMR, Trino, and Ray on the map.

PRACTISE

Exercises

Warm-up

1. Classify each operation as narrow or wide: (a) `filter`; (b) `groupBy(...).agg(...)`; (c) `select`; (d) `join` on a key; (e) `withColumn` with arithmetic; (f) `orderBy`.
2. In the plan in section 48.4, name every line that causes data to move between workers, and say which column it partitions by.
3. A colleague's job on 4 million rows takes 90 seconds in Spark on their laptop and 3 seconds in DuckDB. Give two reasons, and say what you'd recommend.

Core

4. Write the PySpark for: for each machine in December, the number of readings, the share above 200 °C as a percentage, and the total units made, ordered by that share, highest first.
5. Using the plan from section 48.5's pruning example, explain the difference between `PartitionFilters` and `PushedFilters`. Which one would disappear if the data weren't partitioned by date, and what would the query then cost?
6. The join in section 48.5 shuffled 19.87 million rows to match them with 3. Explain why Spark did that when broadcasting was switched off, and why `F.broadcast()` is still worth writing even

when the automatic threshold would catch it.

7. `spark.sql.shuffle.partitions` is 200 by default. Explain what goes wrong with 200 partitions on a one-core machine, and what goes wrong with 8 partitions on a 200-core cluster.
8. From the timings in section 48.6, calculate how many times slower Spark was than DuckDB at each data size. What does the change between the two sizes suggest about where the crossover might be?
9. A job writes its output with `df.write.parquet(...)` after a group-by, and produces 200 files of 400 KB each. Explain the problem this creates for tomorrow's reader, and write the one line that fixes it.

Stretch

10. A join between readings and a 2 GB table of machine settings is slow, and the plan shows `SortMergeJoin` with two exchanges. Broadcasting isn't possible. List three other things you'd try, in order, and what you'd check in the plan after each.
11. Half of Riverstone's readings come from one high-volume machine, and a join on `machine_id` has one task that never finishes. Describe the salting fix in detail, including what happens to the small side, and what you'd check afterwards.
12. Design the storage layout for the sensor archive so that the weekly report in "In the real world" prunes correctly: partition columns, file sizes, sort order inside files, and what you'd do about the existing files.

Think about it (no code needed)

13. Spark appears in far more job descriptions than most teams' daily work needs. How would you prepare for those interviews without building a platform you don't need? Be specific about what you'd learn and what you'd build.
14. "The cluster wasn't wrong; the storage layout was." Explain to a non-technical manager why moving the same data into a bigger machine can fail to fix a slow report, using the votes-and-boxes analogy from the start of the chapter.

Answers

1. Narrow: (a) `filter`, (c) `select`, (e) `withColumn` with arithmetic. Wide: (b) `groupBy().agg()`, (d) join on a key (unless it's broadcast, in which case the big side isn't shuffled), (f) `orderBy` (a global sort needs data moved so ranges are in order).

2. One line: `Exchange hashpartitioning(machine_id, 8)`. It partitions by `machine_id`: it redistributes rows by a hash of `machine_id` into 8 partitions, so every row for a machine lands in the same partition. `ENSURE_REQUIREMENTS` means the aggregate above it needs that arrangement. The `HashAggregate ... partial_sum` below it is what keeps the shuffle small: only partial totals cross, not raw rows.

3. Two reasons: (1) **Overhead**: Spark starts a Java process, plans, and coordinates tasks; at 4 million rows that setup is most of the runtime, while DuckDB runs in the same process. (2) **Too many shuffle partitions**: with the default 200, the job creates hundreds of tiny tasks on a machine with a handful of

cores. Recommendation: keep it in DuckDB (or Polars), and revisit only if the data grows past what the machine can hold. If Spark must be used, set `spark.sql.shuffle.partitions` to a few times the core count.

4. A sample answer:

```
december = readings.filter(F.col("reading_date").between("2025-12-01", "2025-12-31"))

answer = (december.groupBy("machine_id")
    .agg(F.count("*").alias("readings"),
        F.round(F.avg((F.col("temperature_c") > 200).cast("int")) * 100,
            2).alias("hot_pct"),
        F.sum("units_made").alias("units"))
    .orderBy(F.col("hot_pct").desc(), "machine_id"))
answer.show(5)
```

```
+-----+-----+-----+-----+
|machine_id|readings|hot_pct| units|
+-----+-----+-----+-----+
|      M-14|   267840|   43.88|803351|
|      M-24|   267840|   43.88|803690|
|      M-19|   267840|   43.82|803423|
|      M-23|   267840|   43.73|804166|
|      M-18|   267840|   43.71|803467|
+-----+-----+-----+-----+
only showing top 5 rows
```

The filter is on the partition column, so only December's folders are read; `(condition).cast("int")` then averaged is the usual way to turn a share into a percentage. `between(a, b)` includes both ends, `.desc()` sorts from highest to lowest, and `"machine_id"` breaks ties, so machines with the same share always come out in the same order.

5. PartitionFilters are applied to the *folder structure*: Spark decides which `reading_date=` folders to open at all, so unmatched files are never read. **PushedFilters** are applied *inside* the Parquet files as they're read, using each file's own row-group statistics to skip chunks where the values are clustered, and then row by row. Without date partitioning, **PartitionFilters** would be empty; the date condition would become an ordinary pushed filter, Spark would open all 92 days of files, and the query would read roughly 92 times as much data (though Parquet statistics would still let it skip some row groups if the files were sorted by time).

6. With broadcasting switched off, Spark's only general way to match rows is to put equal keys in the same partition, which means hash-partitioning **both** sides by `machine_id`: the 19.87 million-row side gets written to shuffle files (on a cluster, also sent over the network) and read back. `F.broadcast()` is still worth writing because Spark's automatic decision rests on an **estimate** of the small side's size, and estimates go wrong after filters, joins, or aggregations, especially with no table statistics. Being explicit also documents the intent for the next reader. The caution: broadcasting a table that isn't actually small can run the driver and every worker out of memory.

7. With **200 partitions on one core**, every shuffle produces 200 small tasks that run more or less one after another, each with scheduling overhead and a tiny file; the job spends most of its time coordinating. With **8 partitions on 200 cores**, only 8 tasks exist, so at most 8 cores do anything and

the other 192 sit idle, and each task handles a large slice that may not fit comfortably in memory. The rule of thumb is a few times the total core count, adjusted so each partition holds roughly 100–200 MB.

8. At 216,000 rows: $252.3 \div 6.8 \approx 37$ **times slower**. At 19.87 million rows: $1,211.8 \div 213.0 \approx 5.7$ **times slower**. The ratio fell sharply as the data grew, which is what you'd expect if a fixed overhead is being spread over more work. (Work from the milliseconds, not rounded seconds: at this speed, rounding to 0.01 s would turn 6.8 ms into 0.01 s and the ratio into nonsense.) Extrapolating a ratio isn't a prediction, but the direction suggests the crossover lies well beyond this size, and on one machine it may never arrive: DuckDB and Polars speed up with more cores just as Spark does, and Spark's real advantage is adding machines, which this setup can't do.

9. 200 files of 400 KB means the next reader opens 200 files to read 80 MB, and each open costs a round trip, especially on cloud storage: the "small files problem". Metadata handling and task scheduling then dominate the read. The fix is to reduce the number of output partitions before writing, for example:

```
df.coalesce(1).write.mode("overwrite").parquet("mart/weekly_scrap")
```

`coalesce(n)` merges partitions without a shuffle (cheaper); `repartition(n)` reshuffles to exactly `n` and balances them. For larger outputs, aim for files of 128–512 MB rather than a single file.

10. In order: (1) **Filter and select earlier**, so less of the 2 GB reaches the join, then check the plan shows the filters below the exchanges and a smaller `ReadSchema`. (2) **Pre-partition or bucket both tables on the join key** in storage, so the join can happen without a shuffle; check for the disappearance of one or both exchanges. (3) **Reduce the join itself**: aggregate the settings table to one row per key first, or join on a narrower key, then check the estimated sizes in the plan. If it's still slow, look for **skew** (one long task in the Spark UI) and apply the fixes in 48.5, and only then consider more workers.

11. Add a **salt** to spread the hot key. On the **big** side, add a column `salt = floor(rand() * N)` for some `N`, say 8. On the **small** side, replicate every row `N` times, once per salt value (`explode` over an array of `0...N-1`), so that whatever salt a big row got, there's a matching small row. Join on `[machine_id, salt]`. The hot machine's rows now spread across 8 partitions instead of one, so 8 workers share the work, at the cost of the small side being 8 times larger. Afterwards, check in the Spark UI that the longest task in that stage is close to the median task, and confirm row counts are unchanged, as section 48.5 did, since a mistake in replicating the small side silently duplicates or drops rows.

12. Partition by **date** (`reading_date=YYYY-MM-DD`), because every report filters by time; that alone lets the weekly report read 7 folders out of thousands. Inside each day, **sort by machine_id** (and write with statistics), so Parquet row-group pruning can skip most of a file when a report asks about a few machines. At Riverstone's volume a day is only about 10 MB (about 1.7 million readings), so aim for **one file per day**, never one per machine; if the file count grows into the thousands, partition by month instead, or rely on regular compaction (Chapter 49). The 128–512 MB target applies once a partition is that large. For the existing archive, run a one-off **rewrite job**: read the old layout, write the new one partitioned by date, verify counts and totals per day match the old files (Chapter 47's reconciliation),

then switch readers over and keep the old files until the next cycle has proved the new layout works. Machine-based partitioning should be dropped: it produced thousands of small files and prunes nothing for time-based reports.

13. A specific answer: learn Spark's **model** rather than its operations catalogue: partitions, narrow versus wide, shuffles, joins, skew, and how to read a plan. Build **one** honest project: a few tens of millions of rows in local mode, with a plan you can explain, a broadcast join before and after, and measured timings against DuckDB. Read the Spark UI for a real job. Then be ready to say, in an interview, when you would *not* use Spark, and why, with your own numbers. That combination (model, one real project, and judgment about when it's the wrong tool) answers most interview questions, and it's honest about what you've done. Anything beyond that is best learned on a job that actually has the data.

14. Using the analogy: the report is asking a question about one district's votes, but the ballots were sorted into boxes by candidate rather than by district. So whichever way you organize the counting officers, every box still has to be opened to find that district's ballots. Hiring more officers (a bigger machine or a cluster) means the opening happens in parallel, but the total work stays enormous, and coordination adds its own cost. If instead the ballots are stored in one box per district, a single officer answers the question in minutes by opening one box. The lesson for the manager: the way the data is organized decides how much work a question needs; the size of the machine only decides how fast that work gets done.

NEXT

Where this leads

- **Chapter 49, Storage, Warehouses & Lakehouses**, explains the layouts this chapter depends on: partitioning, file sizes, table formats with transactions and time travel, and cloud warehouse costs.
- **Chapter 50, Streaming & Real-Time**, uses the same sensor data as a never-ending stream, with Spark Structured Streaming.
- **Chapter 47** applies: tests and reconciliation matter more at this scale, not less, and they must be cheap to run.
- **Chapter 52, The Cloud, Containers & Infrastructure as Code**, covers packaging and running jobs like these in containers and in the cloud, instead of on your laptop.
- **Part 8:** "how would you process X" design cases, and questions on partitioning and scale, come up in the data engineering bank, Chapter 77 (section 77.5).

Chapter 49. Storage, Warehouses & Lakehouses

Chapter at a glance

You will learn to: say where each kind of data belongs: files, object storage, databases, warehouses, lakes, lakehouses · explain row versus columnar storage, and show the difference in file size, read time, and what a query has to read · read a Parquet file's internals: row groups, column chunks, encodings, compression, statistics · explain ACID properly, and why plain files don't have it · turn a folder of Parquet into a real table with a transaction log, then overwrite one machine's data atomically, travel back to an earlier version, restore it, change the schema safely, and compact small files · choose partition columns and file sizes on purpose · explain how cloud warehouses separate storage from compute, and what actually drives their bills.

Before you start: Chapter 12 (transactions: `BEGIN`, `COMMIT`, `ROLLBACK`), Chapter 17 (the virtual environment and Jupyter), Chapter 45 (DuckDB from Python, files and Parquet), Chapter 46 (idempotent writes), Chapter 47 (write-audit-publish), and Chapter 48 (partitions, pruning, and the sensor dataset).

Time needed: 14–17 hours of reading and practice, spread over two to three weeks. Plan three sittings: sections 49.0–49.4 (setting up, where data sits, row versus columnar, inside a Parquet file, ACID); section 49.5 (table formats, the longest section); sections 49.6 onwards and the project.

Tools: Python in the book's virtual environment with `delta lake` (installed in section 49.0), `duckdb` (Chapter 45), and `pyarrow` and `pandas` (Chapter 18). No Spark, no Java, no cloud account.

Practice data: the Chapter 48 sensor archive. The companion script rebuilds one day of it (2025-12-01, 216,000 readings) as CSV, Parquet, and a row-store database table, so the three can be compared directly.

In this chapter

START	Why this matters · In plain English	176
LEARN	49.0 Setting up	177
	49.1 Where data sits	178
	49.2 Row versus columnar, measured	179
	49.3 Inside a Parquet file	183
	49.4 ACID, properly	186
	49.5 Table formats: a folder of files that behaves like a table	187
	49.6 Partitioning and file sizing	197
	49.7 Warehouses: storage and compute, separated	198
	49.8 What it costs	199
	49.9 Choosing a layout	201
APPLY	Common mistakes · In the real world · Project	202
REVIEW	Recap · Key terms · Check yourself	204
PRACTISE	Exercises	206
	Answers	207
NEXT	Where this leads	209

START

Why this matters

Chapter 48 ended with a story where the engine wasn't the problem: the storage layout was. That's the normal case. Most "we need a bigger cluster" conversations are really "we stored this badly" conversations, and the fix costs a day rather than a monthly bill.

Storage decisions also outlive everything else. A tool can be swapped in a quarter. A format and layout, once a year of data is in it and forty reports read it, is what you live with. Getting it right means understanding four things that this chapter demonstrates rather than asserts: **how columnar files work, what ACID actually guarantees, what a table format adds to a folder of files, and what you're paying for in a warehouse.**

This is also where Part 5's promises land. Chapter 46 needed writes that replace a partition atomically. Chapter 47 needed to publish only after checks pass, and to roll back when something slips through. Both of those are storage features, and this chapter is where you get them.

In plain English

Think about how a warehouse stores stock, which Riverstone's does.

- **Loose boxes on a floor** are files in a folder. Anyone can put one down or take one away. Nothing stops two people leaving half-finished piles.
- **Racking with labelled bays** is partitioning: when someone asks for the October stock, you walk to the October bay instead of searching the floor.
- **A stock register** is a **transaction log**: a written record of what's officially in the warehouse right now, and what changed when. The boxes matter, but the register decides what counts.

- **Never counting a half-finished delivery** is **atomicity**: a delivery is either registered in full or not at all.
- **Two people counting at once and both getting the right answer** is **isolation**.
- **The register surviving a power cut** is **durability**.
- **Being able to say what the stock was last Tuesday** is **time travel**.
- **A thousand tiny parcels instead of twenty pallets** is the small files problem: same goods, far slower to handle.

A **lakehouse** is exactly this: cheap floor space (object storage, section 49.1) plus a stock register (a table format, section 49.5) that makes it behave like a proper warehouse.

LEARN

49.0 Setting up

This chapter needs one new library, **deltalake**, and the sensor data you generated in Chapter 48.

Step 1. Your practice folder

Work the way Chapter 48 did. In your file manager, copy the folder `companion/ch49` and paste it inside `work`, so you have `work/ch49` next to `work/ch48`. It holds two files:

File	What it does
<code>setup_ch49.py</code>	Rebuilds one day of the Chapter 48 sensor data (2025-12-01) as CSV, Parquet, and SQLite, in a folder called <code>storage</code> .
<code>format_test.py</code>	Saves the same day in five formats and times reading each one back (section 49.2).

`setup_ch49.py` reads the day from `work/ch48/sensor_readings`, so the Chapter 48 dataset must exist (Chapter 48, section 48.0, step 4). If you deleted it, run `python make_sensor_data.py` in `work/ch48` again: the seed makes it identical.

Step 2. Install deltalake

Open a terminal (Chapter 26, section 26.0), go to your practice folder, activate the book's virtual environment (Chapter 17), and install:

```
# terminal
$ cd work/ch49
$ source ../../.venv/bin/activate
$ python -m pip install deltalake==1.6.6
```

- **`source ../../.venv/bin/activate`** activates the environment, two levels up; on Windows PowerShell it's `..\..\venv\Scripts\Activate.ps1`.
- **`deltalake`** is the Python package for **Delta Lake** tables (section 49.5). It needs no Spark and no Java, unlike Chapter 48.
- **`==1.6.6`** asks for the exact version used for this chapter's outputs. `duckdb` is already installed from Chapter 45 (this chapter needs version 1.4 or later), and `pyarrow` and `pandas` from Chapter 18.

Add `deltalake` to your `requirements.txt` (Chapter 26). Then check that it imports:

```
# terminal
$ python -c "import deltalake; print(deltalake.__version__)"
1.6.6
```

Step 3. The notebook and the first cell

In VS Code, create a notebook `ch49.ipynb` in `work/ch49` and choose the `.venv` kernel (Chapter 17, section 17.0). Its working folder is then `work/ch49`, where `setup_ch49.py` sits, which is what lets the notebook import it. Run each code block in this chapter as its own cell, top to bottom, without restarting the kernel: later cells use names that earlier ones create, such as `con` and `dt`.

```
import os
import duckdb
from setup_ch49 import build

con = duckdb.connect()
print("ready")
```

```
ready
```

- `import os` brings in Python's module for files and folders (Chapter 17), used here to list folders and read file sizes.
- `from setup_ch49 import build` imports the function `build` from the companion file `setup_ch49.py`. Python finds the file because it sits in the notebook's working folder.
- `duckdb.connect()`, with nothing in the brackets, opens a DuckDB database **in memory**: nothing is saved to a file, and it disappears when the kernel stops. That's all this chapter needs, because every query reads files directly. (Chapter 45 passed a file name, to keep a warehouse.)

Each later cell imports what it needs the first time it needs it.

Tool note. The outputs in this chapter were produced with Python 3.11.15, `deltalake` 1.6.6, DuckDB 1.5.6, PyArrow 25.0.1, pandas 3.0.6, and SQLite 3.45.1. Newer versions may format a few outputs slightly differently.

49.1 Where data sits

Object storage in five sentences

One place in the table below is new in this book, and the rest of the chapter depends on it.

1. **Object storage** is cloud storage for files, organised into **buckets**: a bucket is a named container, such as `riverstone-sensors`.
2. Each file, called an **object**, is stored under a **key**, its full name inside the bucket, such as `sensor/reading_date=2025-12-01/part-0.parquet`. The slashes look like folders, but they're only part of the name.

3. You can put, get, list, and delete whole objects, but you can't change bytes inside one: to edit an object, you write a new one in its place.
4. Every request is a call across the network, so 10,000 tiny objects cost 10,000 calls, and listing a bucket with millions of keys is slow.
5. **Amazon S3**, **Google Cloud Storage (GCS)**, and **Azure Blob Storage** are the three big services; Chapter 52 sets one up.

Place	What it is	Good at	Poor at
Files on a disk or share	CSV, Excel, Parquet in folders	Simple, universal, cheap	No transactions, no access control, no schema guarantees
Object storage (S3, GCS, Azure Blob)	Files (objects) in buckets, addressed by key, in the cloud	Very cheap, endlessly scalable, durable	Objects can't be edited in place; listing is slow; no transactions by itself
Operational database (PostgreSQL, MySQL)	Row-oriented, transactional (Chapter 12)	Many small reads and writes; running the business	Large scans; analytics over billions of rows
Data warehouse (Snowflake, BigQuery, Redshift)	Columnar, SQL, managed compute	Analytics at scale; concurrency; governance	Cost if used carelessly; less suited to non-tabular data
Data lake	Files in object storage, usually Parquet	Cheap, any format, any engine	Without discipline, a swamp: no transactions, no schema, no history
Lakehouse	A lake plus a table format (Delta, Iceberg, Hudi; section 49.5)	Warehouse behavior on lake storage; many engines	More moving parts to understand

Riverstone's layout by the end of Part 5 is typical of a mid-sized company: orders and customers in the ERP's PostgreSQL (the system of record), a warehouse for analytics (raw, staging, mart), and the sensor archive as Parquet in object storage, exposed as a lakehouse table so it can be corrected, versioned, and read by whichever engine suits the job.

Watch out: "the warehouse" means two things. In this book, Riverstone's physical warehouse at Bhiwandi Main stores crates; the data warehouse stores tables. The chapter names the physical one explicitly when it means crates.

49.2 Row versus columnar, measured

The most important storage idea is also the easiest to demonstrate: **how the bytes are arranged decides what a query has to read.**

- **Row-oriented** storage keeps each record's fields together. Reading one whole record is cheap; reading one field of every record means touching everything. Operational databases and CSV are row-oriented.
- **Columnar** storage keeps each column's values together. Reading one column of every record is cheap, and values of the same type sit side by side, so they compress extremely well. Parquet, and

every analytics warehouse, is columnar.

Row oriented (CSV, an operational database)

id	plant	line	ts	temp	pres	units	scrap	reading 1
id	plant	line	ts	temp	pres	units	scrap	reading 2
id	plant	line	ts	temp	pres	units	scrap	reading 3
id	plant	line	ts	temp	pres	units	scrap	reading 4

... 216,000 readings, each one's 8 fields stored side by side

Key: id = machine_id · ts = reading_ts · temp = temperature_c · pres = pressure_bar

❌ **Average temperature: reads every box, all 8 fields of every reading.**

Columnar (Parquet, every analytics warehouse)

machine_id	216,000 values, stored together
plant	216,000 values, stored together
line_type	216,000 values, stored together
reading_ts	216,000 values, stored together
temperature_c	the only stripe read
pressure_bar	216,000 values, stored together
units_made	216,000 values, stored together
scrap_flag	216,000 values, stored together

The same 216,000 readings

CSV	14.6 MB
SQLite table	16.3 MB
Parquet	1.3 MB

Values of one kind sit together, so Parquet is about 11 times smaller.

✅ **Average temperature: reads one stripe; the other seven are never touched.**

Figure 49.1 – The same 216,000 readings, two arrangements. The query is the same; the work isn't.

The companion function `build()` writes one day of sensor readings three ways, into a folder called `storage`, and returns how many readings it wrote:

```
rows = build()
print(f"{rows:,} readings, one day, 25 machines")
```

```
216,000 readings, one day, 25 machines
```

`build()` reads the day's Parquet file from the Chapter 48 archive with DuckDB, then writes `storage/day.csv` (text, row by row), `storage/day.parquet` (columnar, compressed with Snappy, section 49.3), and `storage/day.sqlite`, a small database file holding one table called `readings`. It deletes and rebuilds the folder on every run, so running the cell twice is safe. Now compare the three files' sizes:

```
for f in sorted(os.listdir("storage")):
    size_mb = os.path.getsize(os.path.join("storage", f)) / 1e6
    print(f"{f:<12} {size_mb:>7.1f} MB")
```

```
day.csv          14.6 MB
day.parquet      1.3 MB
day.sqlite       16.3 MB
```

- `os.listdir("storage")` lists the names of the files in the folder; `sorted()` puts them in alphabetical order.
- `os.path.getsize(...)` returns a file's size in bytes, and `os.path.join` builds the path with the right separator for your system. Dividing by `1e6` (one million, written in scientific notation) turns bytes into megabytes.
- In the f-string, `:<12` pads the name to 12 characters, lined up on the left, and `:>7.1f` prints the size right-aligned in 7 characters with one decimal place (Chapter 17).

Reading it. Identical data: **14.6 MB as CSV**, **16.3 MB in a row-store database table**, and **1.3 MB as Parquet**. Parquet is about **11 times smaller than the CSV** and **12 times smaller than the row store**, because each column is stored on its own and shrinks on its own: the plant name repeats, the timestamps follow a pattern, and the machine ids come from a set of 25. Section 49.3 shows exactly how.

All three give the same answer, of course. First Parquet and CSV, through DuckDB:

```
CSV      = "read_csv('storage/day.csv')"
PARQUET = "'storage/day.parquet'"

print("average temperature, from Parquet:",
      con.execute(f"SELECT ROUND(AVG(temperature_c), 2) FROM {PARQUET}").fetchone()[0])
print("average temperature, from CSV      :",
      con.execute(f"SELECT ROUND(AVG(temperature_c), 2) FROM {CSV}").fetchone()[0])
```

```
average temperature, from Parquet: 196.49
average temperature, from CSV      : 196.49
```

- **DuckDB lets you put a file where a table name would go.** For a CSV you call `read_csv('...')`, which reads the file and guesses the column types; for a Parquet file the quoted path alone is enough, because the file carries its own types. That's why `PARQUET` holds a path *inside* single quotes: the single quotes belong to the SQL, the double quotes to Python.
- Keeping each `FROM` clause in a variable means the query text is otherwise identical. The f-string pastes it in, so the first query that actually runs is `SELECT ROUND(AVG(temperature_c), 2) FROM 'storage/day.parquet'`.
- `.fetchone()[0]` takes the first row of the result as a tuple, and `[0]` its first value (Chapter 45).

Then the row store. **SQLite** is a small row-oriented database that lives in a single file, with no server (Chapter 12 named it, and Chapter 18 wrote to one through pandas). Python comes with a module, `sqlite3`, to talk to it directly. Here it plays the part of an operational database:

```
import sqlite3

lite = sqlite3.connect("storage/day.sqlite")
avg = lite.execute("SELECT AVG(temperature_c) FROM readings").fetchone()[0]
print("average temperature, from SQLite :", round(avg, 2))
lite.close()
```

```
average temperature, from SQLite : 196.49
```


- `sqlite3.connect(path)` opens the database file (and would create it if it didn't exist).
- `lite.execute(sql)` runs one SQL statement; `readings` is the table `build()` created. `fetchone()` returns the first row as a tuple, and `[0]` takes its first value.
- `round(avg, 2)` rounds in Python, because this query doesn't round in SQL. `lite.close()` releases the file, so nothing else is blocked from using it.

What differs is the work underneath. To average one column, the CSV reader must read and parse every byte of all eight fields; the row store must walk every row; Parquet reads one column chunk and skips the rest, which Chapter 48's `ReadSchema` line made visible in Spark's plans.

The same test across formats, with timing

Chapter 2 introduced Parquet as column-by-column storage and promised the full measurement. Time the two DuckDB queries first. **Before you run it, predict:** how many times faster will the Parquet read be?

```
import time

def seconds(sql):
    start = time.perf_counter()
    con.execute(sql).fetchone()
    return time.perf_counter() - start

csv_seconds = seconds(f"SELECT AVG(temperature_c) FROM {CSV}")
parquet_seconds = seconds(f"SELECT AVG(temperature_c) FROM {PARQUET}")
print("Parquet read faster than CSV:", parquet_seconds < csv_seconds)
```

```
Parquet read faster than CSV: True
```

- `time.perf_counter()` reads a precise clock, in seconds (Chapter 48). The difference between two readings is how long the code between them took.
- `seconds(sql)` is a small function that runs one query and returns how long it took.
- The cell prints a comparison rather than the times, because your times will differ from the book's. Print `csv_seconds / parquet_seconds` to see your ratio: on the book's test machine it was between 7 and 12 on repeated runs, with the CSV read taking about a tenth of a second.

The companion script `format_test.py` does the same for five formats: it saves the day as CSV, as CSV compressed with **gzip** (the usual tool for zipping a single file, the `.gz` you meet on downloads), as JSON with one reading per line, as Excel, and as Parquet, then reads each back with pandas (Chapter 18), timing each read three times and keeping the fastest. On the book's test machine (4 shared processor cores) it printed:

Format	File size	Time to read everything	Time to read one column
CSV	14.6 MB	0.175 s	0.075 s
CSV, compressed with gzip	1.8 MB	0.202 s	0.106 s
JSON	39.8 MB	0.844 s	—
Excel (.xlsx)	8.5 MB	12.765 s	—
Parquet	1.3 MB	0.019 s	0.003 s

The 216,000 sensor readings of 2025-12-01 (8 columns), read with pandas 3.0.6. pandas can't read one column of a JSON or Excel file without reading the rest, hence the dashes.

Exact times depend on the computer, but the pattern holds everywhere. **Parquet was the smallest file, and about nine times faster than CSV to read in full**, and reading one column of it took three thousandths of a second. Excel took about seventy times longer than CSV, because every cell has to be unpacked from zipped XML. JSON was the largest, because it repeats all eight column names on every one of the 216,000 readings. The gzip CSV is small, but it must still be unzipped and parsed in full before anything can be read. Run `python format_test.py` in your practice folder to see your own numbers; it takes a minute or two, most of it spent on Excel.

When row storage is right. Fetching one order and its lines by id, writing a new order, updating a status: row storage wins, which is why Riverstone's ERP is PostgreSQL and not a pile of Parquet. The rule of thumb: **row for running the business, columnar for understanding it.**

49.3 Inside a Parquet file

Parquet isn't only "columnar". Its internal structure is what lets engines skip work, and what makes the file small.

Two layers of shrinking

Parquet shrinks each column in two steps. Work them by hand first, on eight values.

Layer 1: encoding rewrites the values in a shorter form, using what's known about the column.

- **Dictionary encoding.** Eight values of `plant` are all `Bhiwandi Main`. Store the string once in a dictionary, `{0: "Bhiwandi Main"}`, and then a small code per row: `0 0 0 0 0 0 0 0`.
- **Run-length encoding.** Those eight codes are one value repeated, so write them as a run: `"8 × 0"`. Eight machine ids `M-01 M-01 M-01 M-02 M-02 M-02 M-02 M-03` become a dictionary `{0: M-01, 1: M-02, 2: M-03}` and three runs: `(3 × 0) (4 × 1) (1 × 2)`.
- **Bit-packing.** A code only needs as many bits as the dictionary is big: 3 values fit in 2 bits, 25 in 5. Eight true/false values fit in one byte, one bit each.

Layer 2: a compression codec then squeezes the encoded bytes further. A **codec** (compressor-decompressor) is a general-purpose method for writing repeated patterns more efficiently, the same idea that zipping a file uses: gzip cut the CSV above from 14.6 MB to 1.8 MB, because columns like `plant` and `line_type` repeat the same few words 216,000 times. Parquet's usual codecs are **Snappy** (fast, and the default in most tools), **zstd** (smaller files, a little more work), and **gzip** (small, but slowest to read). Parquet's codecs are all lossless (Chapter 2): you get back exactly the bytes that went in.

Now read the real file. PyArrow (Chapter 18) can open a Parquet file's **metadata**, the description of the file that Parquet keeps in a **footer** at the end of the file. Reading it touches a few kilobytes, not the data:

```
import pyarrow.parquet as pq

meta = pq.ParquetFile("storage/day.parquet").metadata
print("rows          :", f"{meta.num_rows:,}")
print("row groups   :", meta.num_row_groups)
print("columns      :", meta.num_columns)
```

```
rows          : 216,000
row groups   : 2
columns      : 8
```

- `pq.ParquetFile(path)` opens the file without reading its data; `.metadata` reads the footer.
- `num_rows`, `num_row_groups`, and `num_columns` are three facts from the footer. A **row group** is a horizontal slice of the file: a block of rows stored together.

The file has two row groups. Look at the first:

```
group = meta.row_group(0)
on_disk = sum(group.column(i).total_compressed_size for i in range(meta.num_columns))
print(f"row group 0 : {group.num_rows:,} rows")
print(f"  before compression: {group.total_byte_size / 1e6:.2f} MB")
print(f"  on disk           : {on_disk / 1e6:.2f} MB")
```

```
row group 0 : 122,880 rows
  before compression: 0.80 MB
  on disk           : 0.69 MB
```

- `meta.row_group(0)` picks the first row group (counting from 0, like a Python list).
- `group.total_byte_size` is the size of the row group's data **after encoding but before compression**: 0.80 MB.
- `group.column(i).total_compressed_size` is the size of column `i` as it sits in the file. `sum(... for i in range(meta.num_columns))` adds it up over all eight columns: **0.69 MB on disk**.
- **122,880** is DuckDB's standard row-group size (it writes groups of that many rows), which is why the day's 216,000 readings made two groups: 122,880 and 93,120.

Now each column of that row group, with the encoding the file used and the size before and after compression:

```
for i in range(meta.num_columns):
    col = group.column(i)
    before = col.total_uncompressed_size / 1e3
    after = col.total_compressed_size / 1e3
    print(f"{col.path_in_schema:<14} {col.encodings[0]:<17} {before:>6.1f} KB {after:>6.1f} KB")
```

machine_id	PLAIN_DICTIONARY	0.2 KB	0.2 KB
plant	PLAIN_DICTIONARY	0.1 KB	0.1 KB
line_type	PLAIN_DICTIONARY	0.1 KB	0.1 KB
reading_ts	PLAIN_DICTIONARY	284.7 KB	209.3 KB
temperature_c	PLAIN_DICTIONARY	238.0 KB	219.4 KB
pressure_bar	PLAIN_DICTIONARY	202.3 KB	193.8 KB
units_made	PLAIN_DICTIONARY	62.0 KB	62.0 KB
scrap_flag	PLAIN	15.4 KB	4.1 KB

- `group.column(i)` is one **column chunk**: one column's values inside one row group.
- `col.path_in_schema` is the column's name. `col.encodings` lists the encodings used; `[0]` takes the first. `total_uncompressed_size` and `total_compressed_size` are the chunk's size before and after the codec, divided by `1e3` to give kilobytes.

How to read it.

- **Every column except `scrap_flag` is dictionary encoded** (`PLAIN_DICTIONARY`). `plant` has one value in this row group, so it needs a one-entry dictionary and one long run: 0.1 KB for 122,880 values.
- **`temperature_c` takes 219 KB because it has thousands of different values**: every 0.01 °C between about 170 and 223. The dictionary is big, and each row still needs a 13-bit code into it (the next cell counts them). Few distinct values means a small column; many means a big one.
- **`scrap_flag` isn't dictionary encoded**: 122,880 true/false values are simply bit-packed, 8 to a byte, which is the 15.4 KB. They're almost all false, so Snappy then squeezes the long runs of zero bits down to 4.1 KB.
- **For most columns, the size before and after compression is close**. The encoding did most of the work; the codec adds a little.

Each column chunk also carries **statistics**: its minimum and maximum, and, in files DuckDB writes, a count of distinct values:

```
for i in range(meta.num_columns):
    stats = group.column(i).statistics
    print(f"{group.column(i).path_in_schema:<14} min={stats.min} max={stats.max} "
          f"distinct={stats.distinct_count}")
```

```

machine_id    min=M-01 max=M-15 distinct=15
plant         min=Bhiwandi Main max=Bhiwandi Main distinct=1
line_type     min=Blow max=Injection distinct=3
reading_ts    min=2025-12-01 00:00:00 max=2025-12-01 23:59:50 distinct=8640
temperature_c min=169.88 max=223.06 distinct=4729
pressure_bar  min=81.36 max=111.66 distinct=2184
units_made    min=0 max=14 distinct=15
scrap_flag    min=False max=True distinct=None

```

- `.statistics` holds the chunk's statistics; `stats.min`, `stats.max`, and `stats.distinct_count` are three of them. `None` means the writer didn't record that one.
- The second `print` argument continues on a new line inside the brackets; Python joins the two f-strings into one.

Reading it.

- `temperature_c` has **4,729 distinct values** in this row group. Codes for 4,729 values need 13 bits each (12 bits only reach 4,096), and $122,880 \times 13$ bits is about 200 KB: most of its size.
- The statistics are what make Chapter 48's pushed filters work. A query for readings above 230 °C can skip this row group entirely, because its maximum is 223.06.
- **This file happens to be sorted by machine.** Row group 0 holds M-01 to M-15 only (the last one partly), and row group 1 holds M-15 to M-25. So a query for M-20 skips row group 0 without reading it. If the rows were shuffled, both groups would say `min=M-01 max=M-25`, and nothing could be skipped.

What this means in practice.

- **Sort data by the column people filter on** before writing. Statistics only help when values are grouped: if every row group contains machines M-01 to M-25, no row group can ever be skipped by machine.
- **Choose row group size deliberately.** Tiny row groups mean lots of metadata; huge ones mean less skipping. 128 MB per row group is a common target. You set it in rows: in DuckDB, `COPY ... TO 'x.parquet' (FORMAT parquet, ROW_GROUP_SIZE 500000)`; in PyArrow, `pq.write_table(table, 'x.parquet', row_group_size=500_000)`. Rows translate to MB through your row width: here about 6 bytes a reading, so 128 MB is roughly 20 million readings.
- **Pick a codec every engine you use can read.** Snappy is fast and universal, zstd compresses more at some CPU cost, and gzip is slowest to read. Inside Parquet, each column chunk is compressed on its own, so several workers can still read different row groups at once. (A whole-file gzip, such as a `.csv.gz`, is different: one worker must read it from the start, which is one reason Chapter 48's engines prefer Parquet.)

49.4 ACID, properly

ACID is four guarantees, and they matter more the moment two processes touch the same data.

Letter	Guarantee	What goes wrong without it
Atomicity	A change happens completely or not at all	A job dies after deleting yesterday's files and before writing today's: the day is gone
Consistency	Every committed change leaves the data obeying its declared rules: types, required columns, keys, checks	A job writes a reading with no <code>machine_id</code> , or <code>units_made = -3</code> , and nothing stops it
Isolation	Concurrent readers and writers don't see each other's half-done work	A dashboard runs while a table is being replaced and shows 40% of the numbers
Durability	Once committed, it survives crashes and restarts	A "successful" write lost on a machine restart

Chapter 12 showed the first of these with `BEGIN`, `COMMIT`, and `ROLLBACK`: the two open purchase orders vanished inside the transaction and came back after `ROLLBACK`. That was atomicity. PostgreSQL gives you all four by default. On plain files, **you don't have any of them**. A folder of Parquet files has no notion of a commit: `rm` then `write` is two separate acts, and anyone reading in between sees whatever happens to be on disk.

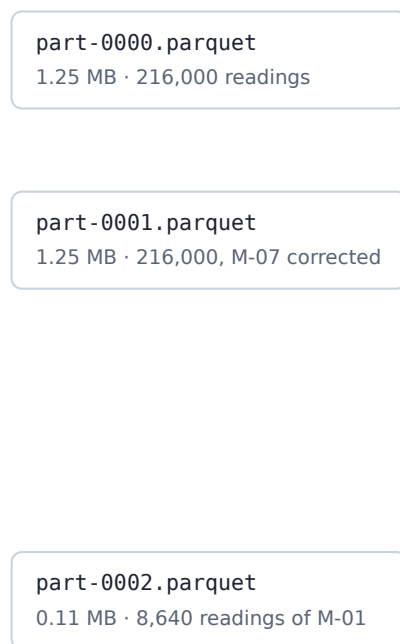
That's exactly the gap Chapter 46 worked around with careful ordering, and Chapter 47 worked around with write-audit-publish. A **table format** closes most of it properly: it gives you atomicity, isolation, and durability, and it enforces the schema, so a write with the wrong columns or types is rejected (section 49.5 shows one). Business rules, such as "units made is never negative", are still Chapter 47's tests.

49.5 Table formats: a folder of files that behaves like a table

Delta Lake, **Apache Iceberg**, and **Apache Hudi** all do the same core thing: they keep a **transaction log** beside the data files that records which files make up the table right now, and what changed in each version. Readers consult the log, not the folder. That single move buys atomicity, isolation, time travel, schema evolution, and safe compaction.

This chapter uses Delta through the `deltalake` package from section 49.0. The ideas transfer directly to Iceberg (a one-page tour closes this section) and Hudi.

The folder: ordinary Parquet



The log: what each version contains



Each arrow points at the file a version adds. File names are shortened.

Readers follow the log, not the folder, so a commit becomes visible all at once.
Time travel reads an older version's list. Vacuum deletes files no kept version needs.

Figure 49.2 — A table format is a folder of ordinary Parquet plus a log that says which files each version contains. These are the four versions this section builds.

Creating a table

A Delta table is written from data held in memory, so first read the day into memory. DuckDB can hand a result over as an **Arrow table**: the in-memory columnar form that DuckDB, pandas, Polars, and `deltalake` all share, so data passes between them without being converted.

```
day1 = con.execute("SELECT * FROM 'storage/day.parquet']").arrow().read_all()
print(f"{day1.num_rows:,} rows")
print(day1.schema.names)
```

```
216,000 rows
['machine_id', 'plant', 'line_type', 'reading_ts', 'temperature_c', 'pressure_bar', 'units_made',
 'scrap_flag']
```

- `.arrow()` returns the query's result as a stream of Arrow batches (a *reader*), so a huge result doesn't have to fit in memory at once. `.read_all()` collects the whole stream into one Arrow table. (DuckDB versions before 1.4 returned the table directly.)
- `day1.num_rows` is the row count, and `day1.schema.names` the column names.

Now write it as a Delta table:

```
import shutil
from deltalake import DeltaTable, write_deltalake

shutil.rmtree("delta_readings", ignore_errors=True)
write_deltalake("delta_readings", day1, mode="overwrite")
for name in sorted(os.listdir("delta_readings")):
    print(name[:10])
```

```
_delta_log
part-000000
```

- **shutil.rmtree(folder, ignore_errors=True)** deletes a folder and everything in it; **ignore_errors=True** means "and don't complain if it isn't there". It gives the cell a clean start on every run. (shutil comes with Python.)
- **write_deltalake(path, data, mode=...)** writes data as a Delta table in the folder **path**, creating the folder if needed. **mode="overwrite"** means "replace whatever the table holds"; on a folder that doesn't exist yet, it simply creates the table. The other common mode, **"append"**, adds rows.
- The loop prints the first 10 characters of each name in the folder. The data file's full name continues with a random id, like **part-000000-a9ea...-c000.snappy.parquet**, which would differ on your computer.

Reading it. The folder holds one ordinary Parquet file and a folder called **_delta_log**. Any engine can read the Parquet file; the log is what makes it a table. Open the table and look at it the way readers do:

```
dt = DeltaTable("delta_readings")
con.register("readings_delta", dt.to_pyarrow_dataset())
print("version      :", dt.version())
print("rows         :", f"{con.execute('SELECT COUNT(*) FROM readings_delta').fetchone()[0]:,}")
print("data files  :", len(dt.file_uris()))
```

```
version      : 0
rows         : 216,000
data files   : 1
```

- **DeltaTable(path)** opens the table by reading its log. **dt.version()** is its current version number: a new table starts at 0.
- **dt.to_pyarrow_dataset()** describes the files that make up the current version, and **con.register(name, dataset)** gives DuckDB a table name for them, so SQL can say **FROM readings_delta**. A registered dataset is a **snapshot** of one version: after every write, open the table again and register it again, as the later cells do.
- **dt.file_uris()** lists the data files the current version uses; **len()** counts them.

What's in the log

The log folder holds one JSON file per version, numbered from `00000000000000000000000000000000.json`. Each line of the file is one JSON object, called an **action**. Look at the raw text first:

```
log_file = "delta_readings/_delta_log/00000000000000000000000000000000.json"
with open(log_file, encoding="utf-8") as f:
    lines = f.read().splitlines()
print(len(lines), "actions")
print(lines[1])
```

```
4 actions
{"protocol":{"minReaderVersion":3,"minWriterVersion":7,"readerFeatures":["timestampNtz"],"writerFeatures":
  ["timestampNtz"]}}
```

- `with open(...) as f` opens the file and closes it afterwards (Chapter 17); `f.read().splitlines()` reads the text and splits it into a list of lines.
- `lines[1]` is the second line, printed whole. It's the `protocol` action: which versions of the Delta rules a reader and a writer must understand. `timestampNtz` means the table uses timestamps with no time zone, like `reading_ts`.

The other three lines are longer, and they contain a timestamp and random ids that would differ on your computer. Read the useful part of each with `json.loads` (Chapter 17):

```
import json

for line in lines:
    action = json.loads(line)
    kind = list(action)[0]
    if kind == "add":
        stats = json.loads(action["add"]["stats"])
        print(f"add      : {action['add']['size'] / 1e6:.2f} MB, {stats['numRecords']:,} records")
    elif kind == "metaData":
        schema = json.loads(action["metaData"]["schemaString"])
        print("metaData  :", len(schema["fields"]), "columns")
    elif kind == "commitInfo":
        print("commitInfo:", action["commitInfo"]["operation"])
    else:
        print("protocol  : reader version", action["protocol"]["minReaderVersion"])
```

```
commitInfo: WRITE
protocol   : reader version 3
metaData   : 8 columns
add        : 1.25 MB, 216,000 records
```

- `json.loads(line)` turns one line into a dictionary. Each action's dictionary has a single key, the action's kind, so `list(action)[0]` (the dictionary's keys as a list, first item) is that kind.
- Two values are **JSON text stored inside JSON**: the file's statistics (`stats`) and the table's schema (`schemaString`). They are strings, so each needs a second `json.loads` before you can look inside.
- The `if/elif` chain prints one line per kind of action.

Reading it. Four kinds of action in one commit: `commitInfo` (what operation ran), `protocol` (which reader versions can understand this table), `metaData` (the schema, 8 columns), and one `add` per data file, with its size and row count. A later commit will contain `remove` actions for files that are no longer part of the table. **Nothing is edited in place**; each version is a new log file describing the change.

An atomic correction

The plant team reports that machine M-07's temperature probe reads 1.5 °C low. That's a correction to one machine's readings, in the middle of a day's data. First, M-07's average as recorded:

```
before = con.execute("""
    SELECT ROUND(AVG(temperature_c), 2) FROM 'storage/day.parquet'
    WHERE machine_id = 'M-07'""").fetchone()[0]
print("M-07 average before:", before)
```

```
M-07 average before: 196.0
```

Then the corrected rows, M-07's readings with 1.5 °C added:

```
corrected = con.execute("""
    SELECT * REPLACE (temperature_c + 1.5 AS temperature_c)
    FROM 'storage/day.parquet'
    WHERE machine_id = 'M-07'""").arrow().read_all()
print(f"{corrected.num_rows:,} corrected rows")
```

```
8,640 corrected rows
```

- **`SELECT * REPLACE (expression AS column)`** is DuckDB's shorthand for "all the columns, but with this column swapped for a new expression". Standard SQL would list all eight columns and write `temperature_c + 1.5 AS temperature_c` in the middle.
- 8,640 rows is one machine's day: one reading every 10 seconds.

Now the write that matters:

```
write_deltalake("delta_readings", corrected, mode="overwrite",
                predicate="machine_id = 'M-07'")
dt = DeltaTable("delta_readings")
print("version now:", dt.version())
```

```
version now: 1
```

- **`predicate="machine_id = 'M-07'"`** turns the overwrite into a partial one: delete the rows that match the predicate, and add the new rows, **in one commit**. Rows for the other 24 machines are kept.
- Reopening the table shows the new version: 1.

Check the result through a fresh snapshot. The M-07 query goes in a variable, because later cells reuse it:

```
con.register("readings_delta", dt.to_pyarrow_dataset())
m07_avg = """SELECT ROUND(AVG(temperature_c), 2) FROM readings_delta
              WHERE machine_id = 'M-07'"""
print("rows now      :", f"{con.execute('SELECT COUNT(*) FROM readings_delta').fetchone()[0]:,}")
print("M-07 average now :", con.execute(m07_avg).fetchone()[0])
print("data files      :", len(dt.file_uris()))
```

```
rows now      : 216,000
M-07 average now : 197.5
data files      : 1
```

Reading it. The write replaced only the rows matching `machine_id = 'M-07'`, and the table went from version 0 to version 1. M-07's average moved from 196.0 °C to 197.5 °C, and the table still holds all 216,000 rows. Because the table had a single file, `delta` wrote a new file with the other machines' rows plus the corrected ones, and the commit swapped one file for the other. At no point could a reader see a table missing M-07, because the change became visible only when the commit landed. With plain Parquet files, the same correction means deleting and rewriting files, with a window where the data is wrong or missing.

Watch out: the log is the table. Don't add, delete, or "tidy up" Parquet files inside a table's folder by hand. Readers follow the log, so a file you add is invisible and a file you delete makes the table unreadable. Use the table's own API (append, overwrite, delete, optimize, vacuum).

Two writers at once. Each writer prepares its data files, then tries to create the next log file, say `00000000000000000002.json`. Only one can create it; the other finds it already exists, re-reads the log, and retries, or fails if the two changes clash (both rewrote M-07, say). This is called **optimistic concurrency**: writers don't lock the table, they check at the moment of committing. Readers are never affected: they see version 1 or version 2, never a mixture. That's isolation. (On object storage, "only one can create it" needs support from the storage service, or a small extra service that hands out locks; each table format's documentation says what it relies on.)

Time travel

Every version stays readable. Open version 0 by number and compare:

```
con.register("v0", DeltaTable("delta_readings", version=0).to_pyarrow_dataset())
v0_avg = con.execute("""SELECT ROUND(AVG(temperature_c), 2) FROM v0
                          WHERE machine_id = 'M-07'""").fetchone()[0]
print("version 0, M-07 average:", v0_avg)
print("version 1, M-07 average:", con.execute(m07_avg).fetchone()[0])
for entry in dt.history():
    print("version", entry["version"], "-", entry["operation"], "-",
          entry["operationParameters"].get("predicate", "whole table"))
```

```
version 0, M-07 average: 196.0
version 1, M-07 average: 197.5
version 1 - WRITE - machine_id = 'M-07'
version 0 - WRITE - whole table
```

- `DeltaTable(path, version=0)` opens the table as it was at version 0, by reading the log only up to that version.
- `dt.history()` returns a list of dictionaries, newest first, one per commit. `operationParameters` holds the write's settings; `.get("predicate", "whole table")` returns the predicate, or "whole table" for a write that had none (Chapter 17's `get` with a default).

Reading it. Version 0 still shows the original 196.0 °C, and version 1 the corrected 197.5 °C. The history lists both commits and what each one touched. It's also how you answer "what did this report say last Tuesday?"

Restoring a version

Time travel is what makes Chapter 47's promise of a quick rollback real: if a bad load publishes, you **restore** the previous version instead of rebuilding from source. Suppose the plant team now says the probe was fine after all, and the correction must be undone:

```
dt.restore(0)
dt = DeltaTable("delta_readings")
con.register("readings_delta", dt.to_pyarrow_dataset())
print("version now      :", dt.version())
print("M-07 average now :", con.execute(m07_avg).fetchone()[0])
```

```
version now      : 2
M-07 average now : 196.0
```

- `dt.restore(0)` makes the table's content match version 0 again. Then the cell reopens and re-registers the table, as after every write.
- **The restore is itself a new commit, version 2.** It removes version 1's file and adds version 0's file back. Nothing is erased from the history: versions 0 and 1 can still be read, and the log records that someone restored.

Vacuum: deleting old files

Old versions cost storage, because their files stay in the folder. **Vacuum** deletes data files that no version inside a **retention window** still needs. The window is 7 days (168 hours) by default in Delta. `dry_run=True` lists what vacuum *would* delete, without deleting anything:

```
print(dt.vacuum(dry_run=True))
```

```
[]
```


Nothing: every file here is minutes old, so all of them are inside the 7-day window. Ask for a one-hour window instead:

```
dt.vacuum(retention_hours=1, dry_run=True)
```

```
_internal.DeltaError: Generic error: Invalid retention period, minimum retention for vacuum is configured to be greater than 168 hours, got 1 hours
```

retention_hours=1 sets the window to one hour, and **deltalake** refuses. Running vacuum with a short retention destroys your ability to time travel, and can break long-running readers that are still reading older files. The check can be switched off with **enforce_retention_duration=False**, which is exactly the setting to be suspicious of when you see it in someone's code.

Schema changes

The plant team wants a **shift** column: A before noon, B after. Build one machine's readings with the new column, and append them the way you would any other rows:

```
import traceback

with_shift = con.execute("""
    SELECT *, CASE WHEN EXTRACT(hour FROM reading_ts) < 12 THEN 'A' ELSE 'B' END AS shift
    FROM 'storage/day.parquet' WHERE machine_id = 'M-01'""").arrow().read_all()
try:
    write_deltalake("delta_readings", with_shift, mode="append")
except Exception:
    print(traceback.format_exc().splitlines()[-1])
```

```
_internal.SchemaMismatchError: Cannot cast schema, number of fields does not match: 9 vs 8
```

- **CASE WHEN ... THEN 'A' ELSE 'B' END AS shift** (Chapter 13) makes the new column, and **EXTRACT(hour FROM reading_ts)** takes the hour from the timestamp.
- The append is rejected: the data has 9 columns and the table 8. That's the schema enforcement from section 49.4, and it's what you want by default.
- **try/except** (Chapter 17) catches the error, and **traceback.format_exc().splitlines()[-1]** prints just its last line, so the notebook doesn't fill with a long error message.

To grow the schema on purpose, say so:

```
write_deltalake("delta_readings", with_shift, mode="append", schema_mode="merge")
dt = DeltaTable("delta_readings")
con.register("readings_delta", dt.to_pyarrow_dataset())
print("version:", dt.version())
print("columns:", dt.schema().to_arrow().names)
```

```
version: 3
columns: ['machine_id', 'plant', 'line_type', 'reading_ts', 'temperature_c', 'pressure_bar', 'units_made', 'scrap_flag', 'shift']
```

- `schema_mode="merge"` tells the write to add any new columns to the table's schema.
`dt.schema().to_arrow().names` lists the table's columns.

What do the existing rows hold in the new column?

```
print(con.execute("SELECT shift, COUNT(*) FROM readings_delta GROUP BY shift ORDER BY
shift").fetchall())
```

```
[('A', 4320), ('B', 4320), (None, 216000)]
```

Reading it. The 8,640 appended rows have a shift, half A and half B. The 216,000 existing rows have shift as NULL (None in Python): nothing was rewritten, the table simply knows the old files lack the column. Adding a column is safe. Renaming or removing one, or changing a type, is not: those need a new version of the table's contract with the people reading it (Chapter 47).

Compaction, and the small files problem

Streaming and frequent appends produce many small files. Every reader then pays for opening each one, which on object storage means a network round trip apiece (section 49.1). Make the problem: 24 appends, one per hour of the day, as an hourly job would.

```
shutil.rmtree("delta_small", ignore_errors=True)
for hour in range(24):
    # 24 hourly appends
    batch = con.execute(f"""SELECT * FROM 'storage/day.parquet'
                        WHERE EXTRACT(hour FROM reading_ts) = {hour}""").arrow().read_all()
    write_deltalake("delta_small", batch, mode="append")
print("version:", DeltaTable("delta_small").version())
```

```
version: 23
```

- Each pass of the loop selects one hour's readings, 9,000 of them (25 machines × 360), and appends them. The first append creates the table, as version 0, so 24 appends end at version 23.

A small helper reports a table's files. It reads each file's size from the log itself, through `get_add_actions`, one row per data file:

```
import pyarrow as pa

def file_report(path, label):
    dt = DeltaTable(path)
    sizes = pa.table(dt.get_add_actions(flatten=True)).column("size_bytes").to_pylist()
    files = "file" if len(sizes) == 1 else "files"
    print(f"{label}: {len(sizes)} {files}, average {sum(sizes) / len(sizes) / 1e6:.2f} MB, "
          f"total {sum(sizes) / 1e6:.2f} MB")

file_report("delta_small", "before")
```

```
before: 24 files, average 0.06 MB, total 1.41 MB
```

- `dt.get_add_actions(flatten=True)` returns the current version's add actions as a table, one row per file, with columns such as `path`, `size_bytes`, and `num_records`. `pa.table(...)` turns it into a

PyArrow table, whose `.column("size_bytes").to_pylist()` gives the sizes as a Python list.

- Reading sizes from the log, rather than asking the disk, works the same on Windows, on a Mac, and on object storage, and it's one more example of "the log is the table".
- `files` is a conditional expression (Chapter 17), so one file prints as "1 file".

Before you run the next cell, predict: after merging the 24 files into one, will the total be the same 1.41 MB, larger, or smaller?

```
DeltaTable("delta_small").optimize.compact()
file_report("delta_small", "after ")
con.register("small_delta", DeltaTable("delta_small").to_pyarrow_dataset())
print("rows unchanged:", f"{con.execute('SELECT COUNT(*) FROM small_delta').fetchone()[0]:,}")
print("table version :", DeltaTable("delta_small").version())
```

```
after : 1 file, average 0.82 MB, total 0.82 MB
rows unchanged: 216,000
table version : 24
```

- `.optimize.compact()` is **compaction**: it merges small files into larger ones and commits the swap as a new version. (In Spark and SQL engines the same operation is called `OPTIMIZE`.)

Reading it. Twenty-four appends left **24 files averaging 0.06 MB** (about 59 KB); compaction merged them into **1 file of 0.82 MB**, and the total shrank from 1.41 MB to 0.82 MB, about 40% less. Two things did that. Each small file carried its own footer, schema, and dictionaries, repeated 24 times. And one large file encodes better: a column like `reading_ts` or `temperature_c` gets one dictionary and long runs across the whole day instead of 24 short ones. The row count is unchanged, and the compaction is itself a new version, so readers switch over atomically. (The merged file's exact size can differ by a kilobyte or two between runs, because compaction may interleave the small files' rows in a slightly different order; that's why the report rounds to 0.01 MB.) What happens if you change the loop to 96 appends, one per 15 minutes? Try it: the files get smaller, and compaction saves more.

Real tables aim for files of **128 MB to 512 MB**. Run compaction as a scheduled maintenance job (Chapter 46), not by hand, and pair it with vacuum to remove the files it replaced.

Iceberg in one page

Apache Iceberg solves the same problem with a deeper tree of metadata instead of one log folder:

- **Data files** are ordinary Parquet, as in Delta.
- **Manifests** list data files, with each file's partition values and column statistics.
- A **manifest list** names the manifests that make up one **snapshot**, Iceberg's word for a version.
- A **metadata file** (JSON) holds the schema, the partition rules, and the list of snapshots, including which one is current.
- A **catalog** is a small service or database that stores one thing per table: where its current metadata file is. A commit writes new metadata and then swaps the catalog's pointer, and only one writer can win that swap, which is Iceberg's atomic commit. A catalog is the one extra piece Delta doesn't need; in Python, the Pylceberg library can keep one in a local SQLite file.

Two features follow from this design. **Hidden partitioning:** the table records how partitions derive from a column, such as "the day of `reading_ts`", so a query that filters on `reading_ts` is pruned without the person writing it knowing the layout (section 49.6). **Column ids:** every column has a number as well as a name, and data files refer to the number, so renaming a column changes only metadata. (Newer Delta tables can do the same, with a feature called column mapping.) Spark, Trino, Flink, and several cloud warehouses can read and write Iceberg tables.

49.6 Partitioning and file sizing

Chapter 48 showed partition pruning saving 91 folders out of 92. Choosing the partition column is therefore one of the highest-leverage decisions in a platform.

Choose the column that most queries filter on. For events, readings, orders, and logs, that's almost always a **date**. Secondary partitioning (date and plant) helps only if queries regularly filter on both.

Don't over-partition. A partition per machine per hour across a year is 219,000 folders, each with a tiny file. Listing them takes longer than reading the data, and every engine slows down. Rules of thumb:

Guide	Why
Aim for partitions of at least a few hundred MB	Small partitions waste more time in overhead than they save in skipping
Keep total partitions in the thousands, not millions	Listing and metadata dominate above that
Target files of 128–512 MB	Fewer, larger reads; fewer round trips on object storage
Sort within files by a common filter column	Lets row-group statistics skip work (section 49.3)
Never partition by a high-cardinality id	One folder per customer is the classic disaster

Rules of thumb are for big tables. Riverstone's 25 machines produce about 1.3 MB of Parquet a day, so a whole year is under half a gigabyte. At that size, partition by **month** (or not at all) and let row-group statistics do the rest; switch to daily partitions when a day reaches a few hundred MB.

If queries don't filter on your partition column, partitioning buys nothing. That was the Chapter 48 story: an archive partitioned by machine, queried by date.

A Delta table is partitioned when you write it, with **partition_by**. The day's file has no date column (in Chapter 48 the date lived in the folder name), so derive one:

```
by_date = con.execute("""
    SELECT *, CAST(reading_ts AS DATE) AS reading_date
    FROM 'storage/day.parquet'""").arrow().read_all()
shutil.rmtree("delta_by_date", ignore_errors=True)
write_deltalake("delta_by_date", by_date, partition_by=["reading_date"], mode="overwrite")
print(sorted(os.listdir("delta_by_date")))
```

```
['_delta_log', 'reading_date=2025-12-01']
```

- `CAST(reading_ts AS DATE)` keeps only the date part of the timestamp, as a new column `reading_date`.
- `partition_by=["reading_date"]` writes one folder per distinct value, named `column=value`: the Hive-style naming from Chapter 48, which is what lets engines skip folders (the `PartitionFilters` in Chapter 48's plans). The log records each file's partition value too, so a reader can prune without listing any folders.
- For Riverstone's real archive, you'd use `DATE_TRUNC('month', reading_ts)` instead of `CAST(reading_ts AS DATE)`, which gives one partition per month.

Iceberg's **hidden partitioning** (section 49.5) and Delta's **liquid clustering**, an alternative to fixed partition columns in which the table keeps data clustered by the columns you name and can change them later, both exist because fixed partition columns chosen early are hard to change later.

49.7 Warehouses: storage and compute, separated

Cloud warehouses (Snowflake, BigQuery, Redshift, Databricks SQL, Microsoft Fabric) share one architectural idea: **storage and compute are separate**. Your tables live in cheap object storage; queries run on **compute**, processors and memory you rent while queries run, often billed by the second, sized independently, and several teams can run at once without fighting for the same machine. It's the same split you used in section 49.2, where DuckDB in your notebook (the compute) read files in a folder (the storage).

That's why a warehouse can be idle at 3 a.m. and cost almost nothing, and why a badly written query at 10 a.m. can cost real money.

Idea	Snowflake	BigQuery	Redshift
Compute unit	Virtual warehouse (sized XS to 6XL)	Slots (on-demand or reserved)	Cluster or serverless workgroup
Data organization	Micro-partitions, automatic	Columnar storage, partitioning and clustering	Sort keys and distribution styles
You tune by	Clustering keys, warehouse size, auto-suspend	Partitioning, clustering, avoiding <code>SELECT *</code>	Sort/dist keys, vacuum, WLM
Typical billing	Per second of compute, per TB stored	Per TB scanned (on-demand) or per slot-hour	Per node-hour or per RPU-second

The product names and units are here so you recognise them, not to memorise. In plain words:

- **Virtual warehouse** (Snowflake): a named block of compute you start, resize, and stop; nothing to do with a physical warehouse. **Auto-suspend** stops it after a few idle minutes.
- **Slot** (BigQuery): its unit of query compute; think of one slot as one worker.
- **Micro-partitions** (Snowflake): small blocks of columnar data, with min/max statistics like a Parquet row group, made automatically. **Clustering keys** tell it which column to keep sorted, so the statistics can skip blocks (section 49.3).
- **Sort keys** and **distribution styles** (Redshift): which column each table is sorted by, and how rows are spread across the cluster's machines.
- **WLM**, workload management (Redshift): queues that stop one heavy query from starving the rest.
- **Node-hour** and **RPU** (Redshift): a cluster is billed per machine (node) per hour; serverless Redshift is billed in Redshift Processing Units per second instead.

Chapter 65 treats billing models properly. Two habits matter in all of them.

- **Prune before you compute.** Filter on the partitioning or clustering column, select only needed columns, and avoid `SELECT *` on wide tables. On a per-TB-scanned model this is literally the bill.
- **Let compute sleep.** Auto-suspend idle warehouses, and don't leave a large cluster running for a dashboard that refreshes hourly.

49.8 What it costs

Cloud bills have four parts.

You pay for	Roughly	Habits that cut it
Storage	Cheap: cents per GB-month	Compress (Parquet), expire old versions, vacuum
Compute	The main cost	Prune, size compute to the job, auto-suspend, avoid rerunning the same query
Data scanned	On some services, the whole bill	Partition, cluster, select fewer columns
Data transfer out (egress)	Small but surprising	Keep processing next to the data; don't export whole tables to laptops

A worked estimate for Riverstone

Start from what the chapter measured: 1.3 MB of Parquet for 216,000 readings. Today's archive, 25 machines at one reading every 10 seconds, is tiny. So suppose instead that a plant-wide rollout puts **1,460 sensors** on the lines, each reporting **once a second**:

```
bytes_per_reading = 1.3e6 / 216_000          # measured in section 49.2
today = 25 * 8_640 * 365                     # readings a year now
rollout_per_day = 1_460 * 86_400             # 1,460 sensors, one reading a second
year_gb = rollout_per_day * 365 * bytes_per_reading / 1e9

print(f"bytes a reading      : {bytes_per_reading:.1f}")
print(f"today, a year       : {today:,} readings, {today * bytes_per_reading / 1e9:.2f} GB")
print(f"rollout, a day      : {rollout_per_day:,} readings")
print(f"rollout, a year     : {year_gb:.0f} GB")
```

```
bytes a reading      : 6.0
today, a year       : 78,840,000 readings, 0.47 GB
rollout, a day      : 126,144,000 readings
rollout, a year     : 277 GB
```

- `1.3e6 / 216_000` is bytes per reading: 1.3 million bytes shared among 216,000 readings. Python lets you write underscores inside a number, as in `216_000`, to make it easier to read.
- 8,640 is a day's readings from one machine (one every 10 seconds), and 86,400 the seconds in a day.
- `year_gb` multiplies readings a day by 365 days and by bytes per reading, then divides by `1e9` (a thousand million) to get gigabytes.

Reading it. At about **6 bytes a reading**, today's 25 machines produce 78.8 million readings a year: under half a gigabyte, whose storage costs almost nothing. The rollout produces **126 million readings a day**, which is **277 GB a year**. Now price it. Two list prices, as published in September 2026: **Amazon S3 Standard** storage costs **\$0.023 per GB-month** (for the first 50 TB, in AWS's US East region), and **BigQuery's** on-demand queries cost **\$6.25 per TiB scanned**, where a TiB (tebibyte, 2 to the power 40 bytes) is about 1,099.5 GB, with the first TiB each month free.

```
GB_PER_TIB = 1_099.5
storage_month = year_gb * 0.023              # dollars a month
week_gb = year_gb / 52
week_scan = week_gb / GB_PER_TIB * 6.25     # dollars a run
year_scan = year_gb / GB_PER_TIB * 6.25

print(f"storage           : ${storage_month:.2f} a month")
print(f"weekly report, 1 wk : {week_gb:.1f} GB scanned, ${week_scan:.3f} a run")
print(f"weekly report, all  : {year_gb:.0f} GB scanned, ${year_scan:.2f} a run")
print(f"ratio              : {year_scan / week_scan:.0f}")
```

```
storage           : $6.37 a month
weekly report, 1 wk : 5.3 GB scanned, $0.030 a run
weekly report, all  : 277 GB scanned, $1.58 a run
ratio              : 52
```

- `GB_PER_TIB` holds the conversion, written in capitals because it's a fixed value, not something the code changes. `storage_month` is the year's gigabytes times the monthly price per gigabyte. `week_gb`

is one week's share of the year; dividing it by `GB_PER_TIB` gives tebibytes, and multiplying by 6.25 gives dollars, which is `week_scan · year_scan` does the same for a report that reads the whole year.

- The `print` lines format the dollars with 2 or 3 decimal places (`:.2f`, `:.3f`), and the last line divides one cost by the other.

Reading it.

- **Storage:** 277 GB at \$0.023 per GB-month is about **\$6.37 a month**, roughly ₹530 at ₹83 to the dollar. Storage is almost never the problem.
- **Query cost on a per-TB-scanned service:** the weekly plant report reads one week, $277 \text{ GB} \div 52 \approx 5.3 \text{ GB}$, which costs about **\$0.03 a run**, a few rupees. The same report written to read the whole archive scans 277 GB, about **\$1.58 a run: 52 times** as much, for the same answer. The ratio is simply the ratio of data scanned.
- **The lesson in one line:** you don't optimize a data platform's cost by buying cheaper storage; you optimize it by not reading data you don't need.

Simplification note. Prices, units, and free tiers change and differ by region and provider, and this estimate ignores BigQuery's free first TiB each month. It also assumes a query scans as many bytes as the Parquet files hold; BigQuery actually counts the uncompressed size of the columns a query reads, which can be several times more. Treat these as an illustration of the *shape* of a bill, not as a quote: check your provider's calculator. Chapter 52 adds compute to this estimate, and Chapter 65 (FinOps) covers cost management properly.

49.9 Choosing a layout

Situation	Store it as	Why
The business's live records (orders, customers)	Operational database, row-oriented	Many small reads and writes; ACID by default
Analytics tables that people query all day	Warehouse tables, columnar	Managed, concurrent, governed
Large event or sensor archives	Parquet in object storage, as a table format	Cheap, any engine, correctable, versioned
Data you receive from partners	Raw files, kept as they arrived, plus a typed table	Reprocessing without asking again (Chapter 45)
Intermediate results in a pipeline	The warehouse, or Parquet	Rebuildable; don't over-engineer
A one-off analysis	A local Parquet file	No platform needed

For Riverstone, the honest answer is: keep orders in the warehouse (a few hundred thousand rows will never be a problem), and keep sensor readings as a Delta table in object storage, partitioned by month, sorted by machine and time within each file, compacted weekly, with a year's retention for time travel.

Common mistakes

Mistake	Symptom	Fix
CSV as a storage format for analytics	Slow queries, huge files, type guessing	Parquet, with declared types (49.2)
Editing files inside a table's folder	Readers see stale or broken tables	Use the table API; the log is the table (49.5)
Treating a file overwrite as atomic	A failed job leaves a day missing or half-written	A table format, or write-audit-publish (49.4–49.5)
Never compacting	Reads get slower every week; thousands of tiny files	Scheduled compaction plus vacuum (49.5)
Vacuuming with a short retention	Time travel gone; long-running readers fail	Keep a retention window that matches your rollback needs
Partitioning by a high-cardinality column	Hundreds of thousands of folders; listing dominates	Partition by date or month; sort within files (49.6)
Partitioning by a column nobody filters on	Pruning never happens	Match partitions to the queries (49.6, Ch 48 story)
SELECT * on wide tables	Big bills on per-TB services	Select what you need
Leaving compute running	A bill with nothing to show for it	Auto-suspend; size to the job (49.7)
Optimizing storage price instead of scan volume	Savings of rupees while scans cost hundreds	Reduce what's read (49.8)
Renaming or dropping columns silently	Downstream reports break	Additive changes; contracts for the rest (49.5, Ch 47)

In the real world: the table nobody could fix

Riverstone's sensor archive lived as plain Parquet in object storage, one folder per day, written by a nightly job.

One Tuesday the plant team asked for a correction: machine M-07's probe had been reading low for six weeks, and every report using it was wrong. The data engineer wrote a job to rewrite those 42 days with corrected temperatures.

It ran for 20 minutes and then failed, because the machine it ran on was restarted during a deployment. By that time it had deleted the files for 19 days and written replacements for 11 of them.

The archive was now in three states: 11 days corrected, 8 days **missing entirely**, and 23 days uncorrected. The nightly job had already run, so some reports had been built from the half-repaired archive. Nobody could say which reports had used which version, because plain files have no history.

Recovering took two days: restoring the missing days from object storage's own versioning (which happened to be enabled), rebuilding the reports, and a difficult conversation with the plant manager about which of last week's numbers had been right.

The fix afterwards was the content of this chapter. The archive became a Delta table: corrections are one atomic commit, a failed job leaves the table exactly as it was, every version is recorded with its timestamp, and "what did this report see on Tuesday?" is a query rather than a guess. Compaction and vacuum run weekly, with retention set to a year.

Six months later a similar correction was needed for a pressure sensor. It was one statement, it took four minutes, and the only sign anything had happened was a new row in the table's history.

What made this work.

- **The failure wasn't unusual;** a deployment restarted a machine. The design assumed that would never happen mid-job.
- **Plain files gave no atomicity and no history,** so neither recovery nor investigation was possible.
- **A transaction log turned a two-day incident into a routine commit.**

Project: turn the sensor archive into a table

Goal: convert the Chapter 48 archive into a versioned, correctable table, and prove the properties this chapter claims.

Tools you'll need

- **Python** in the book's virtual environment, with `deltalake` (section 49.0: `python -m pip install deltalake==1.6.6`), `duckdb` (Chapter 45), and `pyarrow` and `pandas` (Chapter 18). No Spark, no Java, no cloud account.
- **The Chapter 49 companion folder** (`companion/ch49/`): `setup_ch49.py`, which rebuilds one day of the Chapter 48 sensor data as CSV, Parquet, and SQLite, and `format_test.py`, the five-format timing test. Generate the Chapter 48 dataset first.
- **Versions used for the outputs shown:** Python 3.11.15, `deltalake` 1.6.6, `DuckDB` 1.5.6, `PyArrow` 25.0.1, `pandas` 3.0.6, `SQLite` 3.45.1, on 29 September 2026. The timings came from a shared test machine with 4 cores.
- **Worth knowing about:** Apache Iceberg (`PyIceberg`), Apache Hudi, and the warehouse products in section 49.7.

Option A: Riverstone. Use the sensor archive: all 92 days, 19,872,000 readings.

Option B: your own data. Any Parquet dataset you're allowed to use, with at least a few million rows.

Steps

1. **Measure the starting point:** rows, file count, total size, and the time to answer one question.
2. **Create a Delta table** from the archive, partitioned by month (section 49.6: derive a column with `DATE_TRUNC('month', reading_ts)`), and confirm the row count matches.
3. **Correct one machine's readings** for one month with an atomic overwrite, then show the row count is unchanged and the average moved.
4. **Break a write on purpose** (raise an exception between two writes) and show the table is still at the previous version and fully readable.

5. **Time travel:** answer the same question at the version before the correction and after it. Then restore the earlier version and show the history.
6. **Make small files:** append 50 small batches, measure the file count and average size, compact, and measure again, including query time before and after.
7. **Add a column** with `schema_mode="merge"` and show old rows get NULLs.
8. **Write a retention policy:** how long you keep old versions, when vacuum runs, and what would be lost if it ran with a one-hour retention. Show a `dry_run` of your vacuum.
9. **Estimate the monthly cost** of your table at ten times its current size, using your provider's published prices, for storage and for one weekly query, partitioned and unpartitioned.

Stretch goals

- Repeat step 6 with an Iceberg table via PyIceberg and compare the workflow.
- Sort each month's data by `machine_id` before writing, and measure whether queries filtering by machine get faster (check the row group statistics, section 49.3).
- Add the Chapter 47 tests to run before each commit, so a failing correction never becomes a version.

REVIEW

Recap

- **Object storage** keeps whole files (objects) under keys in buckets: cheap and durable, but no edits in place and a network call per request.
- **Row storage** suits running the business; **columnar** suits analyzing it. The same day of readings is 14.6 MB as CSV, 16.3 MB in a row store, and **1.3 MB as Parquet**, and reading it in full was about nine times faster than CSV.
- Parquet shrinks each column twice: **encoding** (dictionary, run-length, bit-packing) and then a **compression codec** (Snappy, zstd, gzip). Its **row groups**, **column chunks**, and **statistics** let engines skip work; sort by the column people filter on so the statistics can help.
- **ACID** means atomicity, consistency, isolation, durability. Plain files in a folder have none of them.
- A **table format** (Delta, Iceberg, Hudi) adds a **transaction log** that lists the files making up each version. The log is the table: never edit files inside it by hand.
- Table formats give **atomic partial overwrites**, **time travel** and **restore**, **schema enforcement and evolution**, and **compaction**. A correction to one machine moved its average from 196.0 °C to 197.5 °C as a single new version, and a restore put it back as another.
- **Compaction** turned 24 hourly files averaging 0.06 MB into 1 file of 0.82 MB with the same rows, 40% smaller in total; target 128–512 MB files.
- **Partition by what queries filter on** (usually a date), at a grain that gives partitions of a few hundred MB: for Riverstone, a month. Never partition by a high-cardinality id.
- Warehouses **separate storage from compute**; bills are driven by compute time and data scanned, not by storage price.

- The cheapest optimization is **not reading data you don't need**: a weekly report that scans a week instead of a year costs 52 times less.

Key terms

row-oriented storage · columnar storage · Parquet · footer (metadata) · row group · column chunk · encoding · dictionary encoding · run-length encoding · bit-packing · compression codec (Snappy, zstd, gzip) · statistics (min/max, distinct count) · ACID (atomicity, consistency, isolation, durability) · schema enforcement · transaction log · action (add, remove, metaData, protocol, commitInfo) · commit · version · table format · Delta Lake · Apache Iceberg · Apache Hudi · Arrow table · predicate overwrite · optimistic concurrency · time travel · restore · vacuum · retention · dry run · schema evolution · schema merge · compaction (OPTIMIZE) · small files problem · manifest · snapshot · catalog · partitioning · over-partitioning · hidden partitioning · liquid clustering · object storage · bucket · key (object storage) · data lake · lakehouse · separation of storage and compute · compute · data scanned · egress

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ You can name six places data can sit and say what each is good and bad at, including what a bucket and a key are.
- ☐ You can explain row versus columnar storage, and quote a real size and speed comparison.
- ☐ You can work dictionary encoding, run-length encoding, and bit-packing by hand, and say what a codec adds.
- ☐ You can read Parquet metadata: row groups, column chunks, encodings, statistics, compressed and uncompressed sizes.
- ☐ You can explain each letter of ACID and what goes wrong without it on plain files.
- ☐ You can create a table format table, read its log, and say what `add`, `remove`, and `metaData` mean.
- ☐ You can correct part of a table atomically, and explain what a reader sees during the change and what happens when two writers commit at once.
- ☐ You can use time travel to answer "what did it say before?", restore a version, and say what vacuum would destroy.
- ☐ You can add a column safely and explain why renaming one isn't safe.
- ☐ You can explain the small files problem and fix it with compaction.
- ☐ You can choose a partition column, a partition grain, and a file size, and defend all three.
- ☐ You can explain how warehouses separate storage from compute and what drives the bill.
- ☐ You can estimate a monthly cost and show why scan volume matters more than storage price.

Exercises

Warm-up

1. For each, say whether row or columnar storage fits better, and why: (a) fetching one order and its lines by id; (b) the average temperature by machine for a year; (c) inserting a new order; (d) counting scrap readings per plant per month.
2. Using the sizes in section 49.2, how many times smaller is the Parquet file than the CSV, and than the SQLite table? Give both to one decimal place.
3. Name the four ACID properties, and for each give one thing that can go wrong on plain files in a folder.

Core

4. From the row-group output in section 49.3, explain why `plant` takes 0.1 KB while `temperature_c` takes 219.4 KB for the same number of values. What would happen to `machine_id`'s size if machine ids were random 36-character identifiers instead of M-01 to M-25?
5. A query asks for readings above 230 °C. Using the statistics shown in section 49.3, explain what the engine does with row group 0, and what it would have to do if the file had no statistics.
6. In section 49.5's correction, the table stayed at 216,000 rows and moved to version 1. Describe precisely what a reader querying the table halfway through that write would see, and why.
7. After compaction, the 24 hourly files (1.41 MB in total) became one file of 0.82 MB with the same rows. Give two reasons the total shrank, and say which rows of section 49.3's column output you'd expect to shrink most.
8. Riverstone keeps 30 days of time travel. Someone runs vacuum with a retention of 1 hour to free space. List three things that break, and say what you'd have done instead.
9. Using the cost estimate in section 49.8, calculate the monthly cost of scanning the full year (277 GB) once a day for 30 days at \$6.25 per TiB (1,099.5 GB), and compare it with scanning one week's data daily. Ignore the free tier. State the ratio.

Stretch

10. Design the storage for Riverstone's sensor archive at 40 machines and five years, one reading every 10 seconds: partition columns, sort order within files, target file size, compaction and vacuum schedule, and retention. Justify each choice with numbers, and say what you'd change if most queries were per-machine rather than per-period.
11. The plant team wants to correct six weeks of readings for one machine, as in the real-world story. Write the sequence of steps you'd follow with a table format, including what you'd test before committing (Chapter 47) and how you'd let people know.
12. Using section 49.5, including "Iceberg in one page", compare Delta and Iceberg for a company whose data is read by Spark and by a cloud warehouse, on: what extra pieces each needs, how each partitions, how each handles renaming a column, and how each commits. Say which you'd choose for Riverstone and why the answer might differ for a larger company.

Think about it (no code needed)

13. "The log is the table." Explain what that sentence means to a colleague who has been tidying up Parquet files in a table folder to save space, and what they should do instead.
14. Storage is cheap and compute is expensive, yet most teams spend their optimization effort on compression settings. Why does that happen, and what would you measure first instead?

Answers

1. (a) **Row**: all the fields of one record sit together, so it's one read. (b) **Columnar**: one column across many rows, and it compresses well. (c) **Row**: inserting a whole record touches one place; columnar formats are written in bulk, not row by row. (d) **Columnar**: two columns (plant, scrap flag) over many rows, with date pruning.
2. $14.6 \div 1.3 = 11.2$ times smaller than the CSV. $16.3 \div 1.3 = 12.5$ times smaller than the SQLite table. (Exact ratios depend on rounding of the printed sizes; anything close is right.)
3. **Atomicity**: a job dies between deleting old files and writing new ones, leaving the day missing. **Consistency**: nothing checks what's written, so a file with a missing `machine_id` or a negative `units_made` is accepted like any other. **Isolation**: a dashboard queries the folder while files are being replaced and averages a mixture of old and new. **Durability**: a write reported as finished is lost when the machine restarts before the data is flushed.
4. `plant` has only one distinct value in that row group ("Bhiwandi Main"), so **dictionary encoding** stores the value once, and **run-length encoding** turns 122,880 identical codes into a single run. `temperature_c` has 4,729 distinct values (every 0.01 °C between about 170 and 223), so the dictionary is large and every row still needs its own 13-bit code, about 200 KB; Snappy can't squeeze that much further. If machine ids were random 36-character identifiers, `machine_id` would grow dramatically: tens of thousands of distinct values, so a huge dictionary and long codes, and no runs to fold away. It could easily become the largest column in the file.
5. Row group 0's statistics say `temperature_c` has a maximum of 223.06, which is below 230, so **no row in that row group can match**: the engine skips the whole row group without reading its column chunks. Without statistics, the engine would have to read and decompress the temperature column for every row group and test each value, which is the difference between reading kilobytes of metadata and megabytes of data.
6. A reader sees **version 0, complete and consistent**, until the commit lands, and **version 1, complete and consistent**, afterwards. It never sees a mixture. That's because a reader resolves the table by reading the log: until the new log file is written, the old entry defines the table, and the new data file sitting in the folder is simply not part of it yet. The commit is a single atomic act of adding one log file.
7. (1) **Less per-file overhead**: each of the 24 files carried its own footer, schema, and statistics; one file carries them once. Measured with section 49.3's metadata code, the footers alone fell from 45 KB to 2 KB. (2) **Better encoding across more rows**: in one file, a column gets one dictionary for the whole day, instead of 24 dictionaries rebuilt every hour with mostly the same values in them. So the columns

with thousands of distinct values shrink most: `temperature_c` fell from 692 KB across the 24 files to about 364 KB, and `pressure_bar` from 453 KB to about 316 KB. Columns such as `plant`, with one value, were tiny either way.

8. Breaks: (1) **Time travel beyond one hour is gone**, so a rollback after a bad publish is impossible; (2) **long-running readers fail**, because files they're still reading are deleted underneath them; (3) **audits and "what did the report say last week?" questions can't be answered**, and any consumer pinned to an older version breaks. `deltalake` refuses such a vacuum by default (section 49.5); getting past it takes `enforce_retention_duration=False`, a deliberate act. Better: keep retention at least as long as your rollback need (30 days here), run vacuum on a schedule with that retention, and if space is genuinely the issue, compact first and check what the old versions actually cost, which is usually small compared with the risk.

9. Full year daily: $277 \text{ GB} \div 1,099.5 = 0.252 \text{ TiB} \times \$6.25 = \text{\$1.58 a run}$, $\times 30 = \text{\$47.26 a month}$. One week's data daily: $5.3 \text{ GB} \div 1,099.5 \times \$6.25 \approx \text{\$0.03 a run}$, $\times 30 \approx \text{\$0.91 a month}$. The ratio is **52 times**, which is exactly the ratio of data scanned (52 weeks in a year). The point: the query's cost is set by how much it reads, and partitioning is what decides that.

10. Size first: $40 \text{ machines} \times 8,640 \text{ readings} \times \text{about } 6 \text{ bytes}$ is about **2.1 MB a day**, 62 MB a month, and 3.8 GB over five years. A defensible design: **partition by month** (60 partitions over five years), because reports are time-based and a day is far too small to be a partition; **sort by machine_id, then reading_ts** within each month's files, so row-group statistics can skip machines; **one file per month** after compaction (62 MB is below the usual 128–512 MB target, which is fine: the target is a ceiling on the number of files, and one file per partition is already the minimum); **compact weekly**, because daily appends leave small files in the current month; **vacuum weekly with a retention of 30 days; retain raw data for five years in the table**, with cold storage for older years if costs justify it. If most queries were per-machine rather than per-period, the cleanest answer is still to partition by month but sort primarily by machine (so statistics prune machines), or to use Iceberg's hidden partitioning or Delta's liquid clustering to cluster by both; partitioning by machine directly would create 40 partitions a month, 2,400 over five years, each holding about 1.6 MB a month.

11. A sensible sequence: (1) **Reproduce the problem:** confirm the probe offset with the plant team and agree the exact machine, date range, and correction. (2) **Build the corrected data into a staging table**, not the live one. (3) **Test it** (Chapter 47): row counts match the original range, the correction moved values by the expected amount and no other machine changed, and reconciliation against the source still holds. (4) **Commit one atomic overwrite** restricted to that machine and date range, and note the version number. (5) **Verify** by querying both versions (time travel) and comparing; if something is wrong, **restore** the earlier version. (6) **Rebuild the affected reports** for the six weeks (Chapter 46's backfill, without re-sending the daily emails), and (7) **tell the people who used the old numbers** which reports changed, by how much, and that a corrected version is available: a labeled correction, exactly as in Chapter 46's story.

12. Extra pieces: Delta needs only the folder and its `_delta_log`; Iceberg also needs a **catalog** that points at each table's current metadata file. **Partitioning:** Iceberg has **hidden partitioning**, so the table knows how partitions derive from a column and queries filter on the column itself; Delta uses partition columns you write (section 49.6) or, as an alternative, liquid clustering. **Renaming a column:** Iceberg identifies columns by **id**, so a rename changes only metadata; newer Delta tables can do the

same with column mapping. **Committing**: Delta writes the next numbered log file, and only one writer can create it; Iceberg swaps the catalog's pointer to new metadata, and only one writer can win the swap. For Riverstone: **Delta**, because the team already uses Python and Spark, `deltalake` needs no extra services, and the tables are read mostly by DuckDB and Spark. For a larger company whose data is read by several engines and warehouses, **Iceberg** is often the safer bet, because a shared catalog lets every engine find the same tables and no single engine is the centre of gravity. Check each engine's current documentation before deciding: support for both formats keeps changing.

13. It means the folder of Parquet files isn't the table; the log is, and the log lists exactly which files belong to which version. Deleting a file the log still references doesn't save space in any useful sense: it makes the table unreadable, because a reader will look for a file that no longer exists. Adding a file by hand does nothing either, since no reader will see it. If space is the problem, the right tools are **compaction** (merge small files into large ones, as a new version) and **vacuum** (delete files no version within the retention window needs). Both are operations the table supports and records, so readers never break.

14. Compression settings are visible, quick to change, and feel like engineering; scan volume is invisible unless someone measures it, and fixing it means changing layouts and rewriting queries, which touches other people's work. There's also an anchoring effect: storage bills come with a clear GB number, while compute bills are diffuse. What to measure first: **bytes scanned per query** (or slot/compute seconds), broken down by the top ten most frequent and most expensive queries, and how much of what they scan they actually use after filtering. That measurement almost always points at a missing filter, a wrong partition column, or `SELECT *`, each of which is worth more than any compression codec choice.

NEXT

Where this leads

- **Chapter 50, Streaming & Real-Time**, writes continuously into tables like these, which is where compaction and small files stop being theoretical.
- **Chapter 51, Data Activation**, reads from the published tables to push data into business systems.
- **Chapter 52, The Cloud, Containers & Infrastructure as Code**, sets up object storage and permissions in practice, and adds compute to section 49.8's cost estimate.
- **Chapter 47** gains its rollback: a failed publish can now be reversed by restoring a version.
- **Chapter 62, Data Architecture Patterns**, organizes a lakehouse into the medallion layers, and **Chapter 65, FinOps: The Economics of Data Platforms**, builds on section 49.8's cost arithmetic.
- **Part 8**: storage and file format questions appear in the data engineering interview chapters, and layout design cases in Chapter 77.

Chapter 50. Streaming & Real-Time

Chapter at a glance

You will learn to: tell the difference between "real-time" as a business word and as an engineering commitment · explain topics, partitions, offsets, consumer groups, and why order is only guaranteed inside a partition · produce and consume events, commit offsets, and handle the duplicates that at-least-once delivery creates · run a real Kafka broker on your own computer, and read and write the Kafka client code you'll meet at work · count events in tumbling, sliding, and session windows, by hand and in Spark · build a Spark Structured Streaming job with event-time windows and a watermark · show what happens to late data, both when it's accepted and when it's dropped · keep a streaming sink from drowning in small files · monitor a stream: lag, watermark, state size, and dropped rows · decide when streaming is worth its cost, using Riverstone's plant monitoring case.

Before you start: Chapter 45 (ingestion, and its incremental-load watermark), Chapter 46 (idempotency and the delivery log), Chapter 47 (quality and alerts), Chapter 48 (Spark, and the sensor data this chapter replays), and Chapter 49 (table formats and compaction). Section 50.3's optional broker uses the terminal from Chapter 34.

Time needed: 15–19 hours of reading and practice, spread over two to three weeks. Plan four sittings: sections 50.0–50.2; section 50.3, plus 30–45 minutes if you run the optional Kafka broker; sections 50.4–50.5, the heart of the chapter; section 50.6 onwards and the project.

Tools: Python 3.14 in the virtual environment from Chapter 17, with `pyspark` (Chapter 48), `deltalake` (Chapter 49), `duckdb` (Chapter 45), and `pyarrow` (Chapter 18), plus Java 21 (Chapter 48). Nothing new is needed for the main thread. The optional Kafka broker in section 50.3 is a free download.

Practice data: the Chapter 48 sensor archive, replayed as a stream of events by the companion scripts.

In this chapter

START	Why this matters · In plain English	211
LEARN	50.0 Setting up	212
	50.1 What "real-time" actually means	212
	50.2 The log: topics, partitions, offsets, groups	213
	50.3 Delivery guarantees, and the duplicates you will get	218
	50.4 Streaming with Spark: the same code, running forever	224
	50.5 Event time, windows, and watermarks	226
	50.6 Writing a stream into a table	233
	50.7 Operating a stream	235
	50.8 Riverstone's plant monitoring case	235
APPLY	Common mistakes · In the real world · Project	236
REVIEW	Recap · Key terms · Check yourself	239
PRACTISE	Exercises	241
	Answers	242
NEXT	Where this leads	244

A note on Kafka. Kafka is the industry's standard event log, and this chapter teaches its model. Kafka runs as a **broker**, a server program that stores the events and hands them out. So that every reader can run every idea, the main examples use `mini_log.py`, a small append-only log in the companion folder that works the way Kafka does: topics, partitions, keys, offsets, consumer groups, committed offsets, and at-least-once delivery. Section 50.3 then shows how to run a real single-server Kafka broker on your own computer, and runs a real producer and consumer against it. That part is optional. Every output in this chapter comes from a real run.

START

Why this matters

Everything in Part 5 so far has been **batch**: a pipeline runs at 6:30, does a day's work, and stops. Batch is the right answer far more often than people admit. But some questions genuinely can't wait for tomorrow morning.

Riverstone's plant is one of them. A machine running 12 °C hot produces scrap continuously, and the loss is per minute, not per day. By the time the nightly pipeline notices, a shift's output is ruined. The same shape appears in fraud detection, delivery tracking, stock levels during a sale, and any alert where the value of the answer decays in minutes.

Streaming is how you answer those questions. It's also where a set of new problems arrive: events that come out of order, events that come twice, events that come late, state that has to be kept between events, and jobs that must run forever rather than finish. This chapter meets all of them on Riverstone's sensor data, and ends with the question that matters more than any of the technology: *is this worth doing in real time at all?*

In plain English

Think about the order counter at a busy restaurant.

- Orders arrive one after another and go onto a **spike** (the old metal spike receipts are pushed onto). Nothing is ever removed or edited; new slips go on top. That's a **log**, and it's what Kafka is.
- The kitchen has several spikes, one per section. A table's orders always go to the same spike, so that table's courses stay in order. That's **partitioning by key**: order is guaranteed within a partition, not across the restaurant.
- Each cook remembers how far down their spike they've got. That's an **offset**. Two teams can read the same spike at their own speed: **consumer groups**.
- If a cook is interrupted before writing down where they got to, they redo the last few slips: some dishes get made twice. That's **at-least-once delivery**, and the fix is to recognize a repeat.
- "How many dishes in the last 15 minutes?" needs a **window**, and the answer depends on whether you mean when the order was *placed* or when the slip *reached* the kitchen: **event time** versus **processing time**.

- A slip found on the floor at 9pm, written at 7pm, is **late data**. You decide how long to hold a window open for stragglers: that's a **watermark**.

LEARN

50.0 Setting up

Work the way Chapter 48 did. In your file manager, copy the folder `companion/ch50` and paste it inside `work`, so you have `work/ch50` next to `work/ch48`. It holds two small Python files, which this chapter imports like any library:

File	What it does	First used
<code>sensor_events.py</code>	Reads the Chapter 48 sensor archive and hands readings back as events, so you can replay them as a stream	Section 50.2
<code>mini_log.py</code>	A small append-only message log that works the way Kafka does, kept in ordinary files	Section 50.2

The events come from the sensor data you generated in Chapter 48 (section 48.0, step 4). `sensor_events.py` looks for it in `../ch48/sensor_readings`, which is where it is if your two practice folders sit side by side. If you deleted it, run `python make_sensor_data.py` in `work/ch48` again: the seed makes it identical.

There's nothing new to install for the main thread. `pyspark` came in Chapter 48, `deltalake` in Chapter 49, `duckdb` in Chapter 45, and `pyarrow` in Chapter 18. On **Windows**, run this chapter in **WSL** (Chapter 34): from section 50.4 on, Spark writes files, which on Windows needs an extra download (Chapter 48's troubleshooting box), and the optional Kafka broker in section 50.3 needs a Linux or macOS terminal anyway.

Create a notebook `ch50.ipynb` in `work/ch50`, choose the `.venv` kernel (Chapter 17, section 17.0), and run each code block as its own cell, top to bottom, without restarting the kernel.

Tool note. The outputs in this chapter were produced with Python 3.11.15, PySpark 4.2.0, deltalake 1.6.6, DuckDB 1.5.6, PyArrow 25.0.1, OpenJDK 21.0.10, Apache Kafka 4.3.1, and kafka-python 3.0.11. Your Python 3.14 environment gives the same outputs.

50.1 What "real-time" actually means

"Real-time" in a business conversation usually means "sooner than now". Before building anything, turn it into a number and a consequence.

Latency	What it takes	Example
Daily	Batch (Chapters 45–47)	Riverstone's Daily Sales Flash
Hourly	Batch, run more often	Stock levels for Riverstone's warehouse team
Minutes	Micro-batch streaming	Riverstone's machine temperature alerts
Seconds	Streaming with a short trigger interval	Flagging suspicious account activity for review
Milliseconds	Not a data pipeline at all: application code	Blocking a card transaction

Micro-batch streaming, in the third row, processes a stream as a series of small batches, one every few seconds to minutes, each holding whatever has arrived since the last. It's how most streaming jobs run, including the Spark jobs in this chapter.

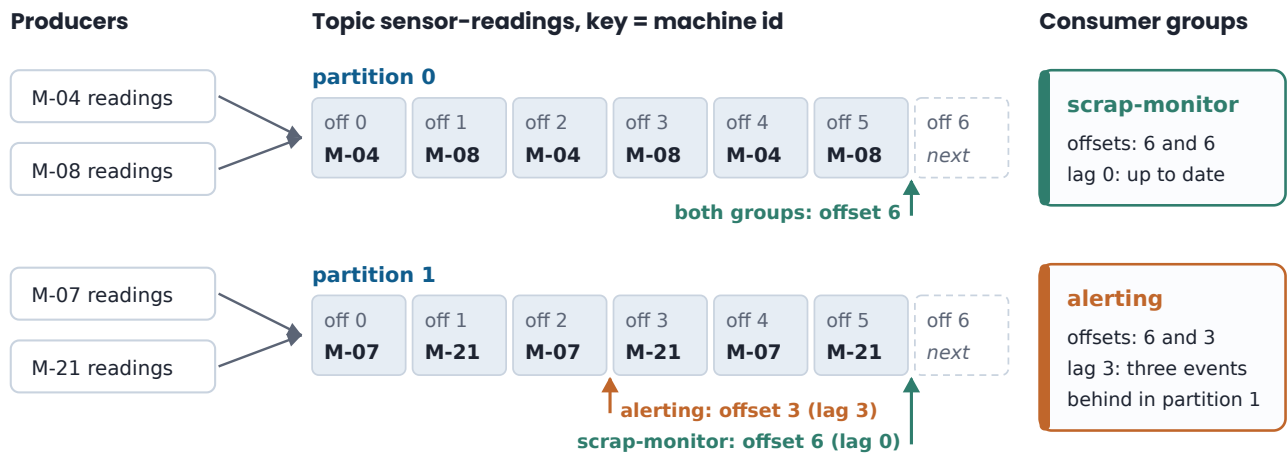
Three questions settle most cases.

1. **What decision changes because the answer arrives sooner?** If nobody acts until the morning meeting, minute-level data buys nothing.
2. **What's the cost of being wrong or late by an hour?** For Riverstone: a shift's worth of scrap from one machine.
3. **Who watches it?** A stream that alerts at 3 a.m. needs someone on call. Streaming moves work from the pipeline to the operations rota.

Watch out: "real-time dashboard" is usually a batch job with a shorter schedule. Refreshing a dashboard every 10 minutes is far simpler than a streaming pipeline, and often indistinguishable to users. Try that first.

50.2 The log: topics, partitions, offsets, groups

An event log is an **append-only** sequence of records. **Producers** add to the end; **consumers** read forwards and remember where they've got to. Nothing is updated in place, which is what makes the model simple to reason about and safe to replay.



Order is guaranteed inside a partition: every M-07 reading is in partition 1, in the order it was sent.

Across partitions there is no order: that is the price of reading them in parallel.

A committed offset is the next position the group will read, so offset 3 means offsets 0, 1 and 2 are done.

Figure 50.1 – A topic is a log split into partitions. A key always lands in the same partition. Offsets are kept per group and per partition, which is why two teams can read the same events at different speeds.

Term	What it means	Why it matters
Topic	A named stream of events (sensor-readings)	The unit producers and consumers agree on
Partition	One append-only log within a topic	Parallelism: more partitions, more consumers
Key	A value that decides the partition (the machine id)	All events for one key stay in order
Offset	A record's position in its partition, counting from 0	Where a consumer has got to
Consumer group	A set of consumers sharing the work of a topic	Two teams read the same data independently
Lag	How far behind the end of the log a group is	The single most important streaming metric
Retention	How long events are kept	Decides how far back you can replay
Broker	A Kafka server: it stores partitions and serves producers and consumers	Production Kafka runs as a cluster of several brokers
Replica	A copy of a partition kept on another broker	If a broker dies, a replica takes over and nothing is lost

The two companion helpers

The events come from `sensor_events.py`. It has two functions:

Call	What it returns or does
<code>se.readings(minute_from, minute_to)</code>	Every reading taken on 1 December 2025 from minute <code>minute_from</code> up to, but not including, minute <code>minute_to</code> , counted in minutes after midnight, like <code>range()</code> . 25 machines send 6 readings a minute (one every 10 seconds).
<code>se.readings(..., machine="M-07")</code>	The same, for one machine only
<code>se.write_batch(folder, name, events)</code>	Writes the events into one file of JSON lines, one event per line, the way a source system would drop a file of new events into a folder (used from section 50.4)

Start with the first two minutes of the day:

```
import sensor_events as se

first = se.readings(0, 2)
print("events:", len(first))
print(first[0])
```

```
events: 300
{'machine_id': 'M-01', 'plant': 'Bhiwandi Main', 'event_time': '2025-12-01 00:00:00', 'temperature_c': 187.28, 'units_made': 5, 'scrap': False}
```

- `import sensor_events as se` imports the companion file under the short name `se`, the way `pandas` becomes `pd`.
- `se.readings(0, 2)` returns minutes 0 and 1: every reading from 00:00:00 to 00:01:50. That's $25 \text{ machines} \times 6 \text{ readings} \times 2 \text{ minutes} = 300$ events.
- `first[0]` is the first event: a dictionary with the machine, its plant, the **event time** (when the reading was taken), the temperature, the units made in those 10 seconds, and whether the reading was flagged as scrap.

The companion's log

`mini_log.py` defines `MessageLog`, a log kept in a folder of ordinary files: one text file per partition, one message per line, and one small JSON file per group holding its committed offsets. It's about 80 lines; open it in VS Code if you're curious. These are the calls this chapter uses, with the Kafka equivalent of each:

Call	What it does	In Kafka
<code>MessageLog("stream_log", partitions=2, reset=True)</code>	Opens a log in the folder <code>stream_log</code> with 2 partitions; <code>reset=True</code> empties it first	Creating a topic with 2 partitions
<code>log.produce(key, value)</code>	Appends <code>value</code> to the end of the key's partition	<code>producer.send(topic, key=..., value=...)</code>
<code>log.partition_for(key)</code>	Which partition a key goes to: a hash of the key	Kafka's partitioner
<code>log.end_offsets()</code>	How many messages each partition holds	<code>consumer.end_offsets(...)</code>

Call	What it does	In Kafka
<code>log.consume(group)</code>	Every message after the group's committed offsets, plus the offsets to commit once the work is done	<code>consumer.poll(...)</code>
<code>log.commit(group, offsets)</code>	Stores the group's position	<code>consumer.commit()</code>
<code>log.committed(group)</code>	The group's stored position, or <code>{}</code> if it has never committed	<code>consumer.committed(...)</code>

Produce the 300 events into a log with two partitions, keyed by machine:

```
from mini_log import MessageLog

log = MessageLog("stream_log", partitions=2, reset=True)
for e in first:
    log.produce(e["machine_id"], e)

print("end offsets:", log.end_offsets())
print("M-07 always goes to partition", log.partition_for("M-07"))
```

```
end offsets: {0: 144, 1: 156}
M-07 always goes to partition 1
```

- `MessageLog("stream_log", partitions=2, reset=True)` creates the folder `stream_log` inside `work/ch50`. `reset=True` means every run of the cell starts from an empty log.
- `log.produce(e["machine_id"], e)` sends each event with its machine id as the **key**. The key isn't stored separately from the event by accident: it's what decides the partition.
- `log.end_offsets()` returns `{partition: number of messages}`. The next message in a partition gets that number as its offset, because offsets count from 0.

Reading it. 300 events went into two partitions, 144 and 156, split by a hash of the machine id. Every M-07 event lands in partition 1, so M-07's readings are in order relative to each other; readings from *different* machines have no guaranteed order between them. That's the trade every log makes: order within a key, parallelism across keys. (The hash here is simple: the sum of the key's bytes. Kafka uses a stronger one, so the same machine can land in a different partition there, but the rule is the same.)

Consuming, and committing

A **committed offset** is a group's saved position: the offset of the **next** message it will read, in each partition. Committing `{0: 144, 1: 156}` means "everything before these has been handled". A group that has never committed starts from the beginning.

```
def hot_readings(messages, limit=205.0):
    return sum(1 for m in messages if m["value"]["temperature_c"] > limit)

print("committed:", log.committed("scrap-monitor") or "nothing yet")
messages, new_offsets = log.consume("scrap-monitor")
print("read      :", len(messages), "messages")
print(messages[0])
```

```
committed: nothing yet
read      : 300 messages
{'partition': 0, 'offset': 0, 'key': 'M-02', 'value': {'machine_id': 'M-02', 'plant': 'Bhiwandi Main',
  'event_time': '2025-12-01 00:00:00', 'temperature_c': 198.4, 'units_made': 6, 'scrap': False}}
```

- **hot_readings** counts the messages whose temperature is above a limit, 205 °C by default. **sum(1 for m in messages if ...)** adds up a 1 for every message that passes the test, which is a count.
- **log.committed("scrap-monitor")** or **"nothing yet"**: **committed** returns {} for a group that has never committed, and **or** gives back its right-hand side when the left is empty. So the line prints the offsets, or "nothing yet".
- **log.consume("scrap-monitor")** returns two things, which the line unpacks into two names (Chapter 17): the messages, and the offsets to commit once the work has succeeded. It doesn't commit anything itself.
- **messages[0]** shows what a message looks like: its partition, its offset, its key, and the event as its **value**. The first message is from partition 0, because **consume** reads partition 0 first.

Now do the work, commit, and read again:

```
print("above 205 C:", hot_readings(messages))
log.commit("scrap-monitor", new_offsets)
print("committed  :", log.committed("scrap-monitor"))

again, _ = log.consume("scrap-monitor")
print("read again :", len(again), "- a committed group does not re-read")
```

```
above 205 C: 82
committed  : {0: 144, 1: 156}
read again : 0 - a committed group does not re-read
```

- **log.commit("scrap-monitor", new_offsets)** saves the group's position, *after* the work (the count) has been done.
- **again, _ = ...** keeps the messages and throws the offsets away: **_** is the usual Python name for "a value I don't need".

Reading it. The consumer counted 82 readings above 205 °C, and then **committed** its offsets. A second read returns nothing, because the group's position is stored. A different group, with its own committed offsets, would read the same 300 events again: that's how an alerting service and a reporting service share one topic without interfering.

50.3 Delivery guarantees, and the duplicates you will get

Three guarantees are worth naming.

- **At-most-once:** commit the offset first, then do the work. If you crash in between, the event is never processed. Nothing is duplicated, some things are lost.
- **At-least-once:** do the work, then commit. If you crash in between, the event is processed again. Nothing is lost, some things are duplicated. **This is the default in practice.**
- **Exactly-once:** achieved only when the processing and the offset commit happen in one transaction, which requires support on both sides: Kafka transactions, or Spark's checkpoint (section 50.4) plus an idempotent sink. A **sink** is wherever a stream's results are written: files, a table, an alert channel. **Idempotent** means what it meant in Chapter 46: writing the same batch twice leaves the same result as writing it once. Exactly-once is real, it's narrower than it sounds, and it's slower.

By hand first: a crash at the wrong moment

Take a log with five messages, offsets 0 to 4, where the readings at offsets 1 and 3 are hot and should each send one alert. The consumer reads all five, and crashes at the worst possible moment. Follow both orders:

Step	At-least-once: work, then commit	At-most-once: commit, then work
1	Read offsets 0–4	Read offsets 0–4
2	Send the alerts for offsets 1 and 3	Commit offset 5
3	Crash before committing	Crash before sending
4	Restart from the committed offset: 0	Restart from the committed offset: 5
5	Read 0–4 again, send alerts 1 and 3 again, commit 5	Nothing left to read
Result	Nothing lost; 2 alerts duplicated	Nothing duplicated; 2 alerts lost

For alerts, a lost one is worse than a repeated one, so choose at-least-once and make the repeat harmless. Here's the duplicate happening for real. A second group, `alerting`, has never committed, so it starts at the beginning:

```
messages, new_offsets = log.consume("alerting")
print("alerts sent      :", hot_readings(messages))
print("... crash before committing")

messages, new_offsets = log.consume("alerting")
print("re-read         :", len(messages), "messages")
print("alerts sent again:", hot_readings(messages), "- the same alerts go out twice")
```

```
alerts sent      : 82
... crash before committing
re-read         : 300 messages
alerts sent again: 82 - the same alerts go out twice
```

- The first `consume` reads all 300 messages and "sends" 82 alerts (here, `hot_readings` counts them instead of sending them).
- The crash is only a printed line: the point is that **`log.commit` never ran**.
- The second `consume` is the restarted consumer. With no committed offsets, it starts from the beginning again.

Reading it. The crash before committing meant all 82 alerts were sent a second time. A plant supervisor's phone buzzing twice for the same reading is exactly how people learn to ignore alerts (Chapter 47).

Remembering what was sent

The fix is Chapter 46's delivery log (section 46.3): before sending, check a record of what has already been sent. The record must be kept **on disk**, not in a Python variable, because a real crash ends the Python process and empties its memory. Here it's a one-column DuckDB table in a file, keyed by a **de-duplication key**: the machine plus the event time, which names one reading exactly.

```
import duckdb

sent_log = duckdb.connect("alerts_sent.duckdb")
sent_log.execute("CREATE OR REPLACE TABLE sent (alert_key VARCHAR PRIMARY KEY)")

def send_once(messages):
    sent = 0
    for m in messages:
        if m["value"]["temperature_c"] > 205.0:
            key = m["key"] + " " + m["value"]["event_time"]
            found = sent_log.execute("SELECT count(*) FROM sent WHERE alert_key = ?",
                                     [key]).fetchone()[0]

            if found == 0:
                sent += 1                                # send the alert here
                sent_log.execute("INSERT INTO sent VALUES (?)", [key])
    return sent

print("first pass:", send_once(messages))
```

```
first pass: 82
```

- **`duckdb.connect("alerts_sent.duckdb")`** opens (or creates) a DuckDB database in a file, as in Chapter 45. **`CREATE OR REPLACE TABLE`** starts the table empty each time you run the cell; in production you'd create it once and keep it.
- **`PRIMARY KEY`** makes the database refuse a second row with the same key, a safety net under the check.
- **`key = m["key"] + " " + ...`** builds the de-duplication key, for example `M-07 2025-12-01 00:00:10`.
- **`SELECT count(*) ... WHERE alert_key = ?`** asks whether this key was sent before; `?` is filled from the list `[key]` (Chapter 45). **`.fetchone()[0]`** takes the single number out of the one-row result.
- Only a key that isn't there yet is sent, then recorded with **`INSERT`**. Sending and recording are two steps: a crash exactly between them can still repeat one alert. Systems that must never repeat one also give the receiving system an **idempotency key** it checks itself (Chapter 51).

Now a real restart: close the database, as a crashing consumer would, and open it again, as the restarted one would.

```
sent_log.close()
sent_log = duckdb.connect("alerts_sent.duckdb")
print("after a restart, second pass:", send_once(messages))
log.commit("alerting", new_offsets)
```

```
after a restart, second pass: 0
```

- **sent_log.close()** drops the connection; the table survives in the file.
- The new connection sees all 82 keys, so the second pass sends nothing. Then, at last, the consumer commits.

Reading it. The first pass sent 82 alerts; after the restart, the same 300 messages sent none, because the keys were on disk. A Python `set()` of sent keys would have printed 0 here too, but only because nothing really restarted: in a real crash it would be empty again and all 82 alerts would go out twice.

Why machine plus event time, and not the message's partition plus offset? Partition and offset catch a consumer re-reading the same record. They don't catch a producer sending the same reading twice, as two records with two offsets. A **business key** like machine plus event time catches both.

The rule to remember: assume duplicates, and make the effect of a repeat harmless. Upserts (Chapter 45), delivery logs (Chapter 46), and idempotency keys (Chapter 51) are the three tools.

Optional: a real Kafka broker on your computer

Everything above works the same way against real Kafka. If you want to see it, you can run a single Kafka broker on your own computer in about half an hour. It's optional: if you skip it, read the two Kafka cells after it as the code you'll meet at work, and carry on at section 50.4.

You need Java 17 or later (Java 21 from Chapter 48 is fine) and a Linux or macOS terminal; on Windows, use WSL (Chapter 34). Kafka's download page, kafka.apache.org/downloads, lists the current release; this chapter used 4.3.1. In the terminal, in `work/ch50`:

```
# terminal
$ curl --silent --remote-name https://downloads.apache.org/kafka/4.3.1/kafka_2.13-4.3.1.tgz
$ tar -xzf kafka_2.13-4.3.1.tgz
$ cd kafka_2.13-4.3.1
$ KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
$ bin/kafka-storage.sh format --standalone -t "$KAFKA_CLUSTER_ID" -c config/server.properties | tail -n 1
Formatting dynamic metadata voter directory /tmp/kraft-combined-logs with metadata.version 4.3-IV0.
$ bin/kafka-server-start.sh config/server.properties > kafka.log 2>&1 &
$ sleep 10
$ bin/kafka-topics.sh --create --topic sensor-readings --partitions 2 --bootstrap-server localhost:9092
Created topic sensor-readings.
```

- **curl --silent --remote-name URL** downloads the file and saves it under its own name (Chapter 34). Older releases move to archive.apache.org/dist/kafka/, so if this link stops working, take the newest release from the download page and change the version in the next two lines.

- **tar -xzf file.tgz** unpacks a compressed archive: x extract, z it's gzip-compressed, f the file name follows. It makes the folder `kafka_2.13-4.3.1` (2.13 is the version of Scala, the language Kafka is built with).
- **KAFKA_CLUSTER_ID="\$(...)"** runs Kafka's tool to make a random cluster id and stores it in a shell variable (`$()` from Chapter 34).
- **kafka-storage.sh format** prepares the folder where the broker will keep its log, named in `config/server.properties: /tmp/kraft-combined-logs`. `--standalone` means one server acting as both broker and controller, the part that keeps track of the cluster. You run this once. `/tmp` may be emptied when your computer restarts, which is fine for practice: format again if it is.
- **kafka-server-start.sh ... > kafka.log 2>&1 &** starts the broker in the background, with its messages going to `kafka.log`, exactly like Chapter 34's file server. `sleep 10` gives it time to start.
- **kafka-topics.sh --create** creates the topic `sensor-readings` with 2 partitions. **--bootstrap-server localhost:9092** is the broker's address: this computer, port 9092.

Then install the Python client, with the book's virtual environment active, and add `kafka-python` to your `requirements.txt`:

```
# terminal
$ cd ..
$ python -m pip install kafka-python==3.0.11
```

When you've finished with this section, stop the broker from inside its folder with `bin/kafka-server-stop.sh`. To start it again another day, repeat only the `kafka-server-start.sh` and `sleep` lines.

The same thing in Kafka

These two cells are the producer and consumer from section 50.2, against the real broker. They need the broker above to be running; without one, the first cell waits about a minute and then stops with `KafkaTimeoutError: Unable to bootstrap`. First the producer, sending the same 300 events:

```
from kafka import KafkaProducer
from kafka.serializer import DefaultSerializer, JsonSerializer

producer = KafkaProducer(
    bootstrap_servers=["localhost:9092"],
    key_serializer=DefaultSerializer(),
    value_serializer=JsonSerializer(),
    acks="all",
    retries=5,
    enable_idempotence=True,
)
events = first
for event in events:
    producer.send("sensor-readings", key=event["machine_id"], value=event)
producer.flush()
print("sent:", len(events))
```

```
sent: 300
```

Argument	Value here	What it does	If you change it
<code>bootstrap_servers</code>	<code>["localhost:9092"]</code>	Where to find the cluster. One broker's address is enough; the client learns the rest from it	A wrong address: the client can't connect and stops with <code>KafkaTimeoutError</code>
<code>key_serializer</code>	<code>DefaultSerializer()</code>	Turns the text key into bytes, which is all Kafka stores	Leave it out and <code>send</code> needs bytes: a text key fails
<code>value_serializer</code>	<code>JsonSerializer()</code>	Turns the dictionary into JSON text, then bytes	Leave it out and a dictionary can't be sent
<code>acks</code>	<code>"all"</code>	The broker confirms a record only when every in-sync replica has it: durability over speed	<code>1</code> : faster, but a record can be lost if a broker dies before copying it; <code>0</code> : no confirmation at all
<code>retries</code>	<code>5</code>	Resends up to 5 times after a temporary failure	<code>0</code> : one network blip loses the record
<code>enable_idempotence</code>	<code>True</code>	The broker drops copies created by this producer's own retries	<code>False</code> : a retry can store the same record twice

- `events = first` reuses the 300 readings from section 50.2.
- `producer.send("sensor-readings", key=..., value=...)` hands one event to the producer, which sends it in the background, grouped with others for speed. `send` returns at once.
- `producer.flush()` waits until everything handed over has been sent and confirmed.

Now the consumer. Creating it only connects; nothing is read yet:

```
from kafka import KafkaConsumer

consumer = KafkaConsumer(
    "sensor-readings",
    bootstrap_servers=["localhost:9092"],
    group_id="scrap-monitor",
    auto_offset_reset="earliest",
    enable_auto_commit=False,
    key_deserializer=DefaultSerializer(),
    value_deserializer=JsonSerializer(),
    max_poll_records=500,
)
```

Argument	Value here	What it does	If you change it
<code>"sensor-readings"</code>	the topic	Which topic to read	
<code>group_id</code>	<code>"scrap-monitor"</code>	The consumer group; Kafka keeps its committed offsets on the broker	A new group name starts again, as a second team would
<code>auto_offset_reset</code>	<code>"earliest"</code>	Where to start when the group has no committed offset: the oldest event kept	<code>"latest"</code> : only events that arrive from now on

Argument	Value here	What it does	If you change it
<code>enable_auto_commit</code>	False	Offsets move only when the code calls <code>commit()</code> , after the work succeeded: at-least-once	True: offsets are committed every few seconds, whether the work is done or not (see Common mistakes)
<code>key_deserializer</code> , <code>value_deserializer</code>	the same two helpers	Turn the stored bytes back into text and a dictionary	Leave them out and you get raw bytes
<code>max_poll_records</code>	500	At most 500 records per poll	Smaller: more, smaller batches

Then the reading loop:

```
def handle(value):
    return 1 if value["temperature_c"] > 205.0 else 0

read, hot = {0: 0, 1: 0}, 0
while True:
    batch = consumer.poll(timeout_ms=5000)
    if not batch:
        break
    for partition, records in batch.items():
        read[partition.partition] += len(records)
        for record in records:
            hot += handle(record.value)
    consumer.commit()
consumer.close()

print("read per partition:", read)
print("above 205 C      :", hot)
```

```
read per partition: {0: 96, 1: 204}
above 205 C      : 82
```

- **handle** is the work done for each event; here it returns 1 for a hot reading, so `hot` counts them. At work it would send an alert or write a row.
- **consumer.poll(timeout_ms=5000)** waits up to 5 seconds for new records and returns a dictionary, `{partition: [records]}`. It's empty if nothing arrived. The first poll can take a few seconds while the broker registers the group.
- **if not batch: break** stops the loop when nothing new arrived for 5 seconds. A real consumer is a service that never stops: it would leave this line out and keep polling.
- Each key of `batch` describes a partition; **partition.partition** is its number. Each **record** has `.key`, `.value`, `.partition`, and `.offset`.
- **consumer.commit()** comes only after every record in the batch was handled: at-least-once. **consumer.close()** leaves the group cleanly.

Reading it. The consumer read the same 300 readings and found the same 82 hot ones. The split between partitions differs from `mini_log`'s 144 and 156, because Kafka's partitioner uses a different hash, but each machine still sits in exactly one partition. Run the loop cell again and it reads nothing: the group's offsets are committed on the broker.

The settings that matter most are `acks`, `enable_idempotence`, `auto_offset_reset`, and turning **off** auto-commit so offsets move only after the work has succeeded. Managed services (Amazon MSK, Confluent Cloud, Azure Event Hubs, Google Pub/Sub) differ in operations, not in this model.

50.4 Streaming with Spark: the same code, running forever

A streaming query in Spark looks almost identical to a batch one. The difference is that it never finishes: it processes what has arrived, records where it got to, and waits. Spark's picture of a stream is a table that keeps growing, an **unbounded table**: each micro-batch processes the rows that arrived since the last one.

Start a Spark session as in Chapter 48 (section 48.0, step 5):

```
from pyspark.sql import SparkSession, functions as F, types as T

spark = (SparkSession.builder
        .appName("riverstone-stream")
        .master("local[2]")
        .config("spark.sql.shuffle.partitions", "4")
        .config("spark.ui.showConsoleProgress", "false")
        .config("spark.sql.streaming.schemaInference", "false")
        .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")
print("Spark", spark.version, "is running")
```

Spark 4.2.0 is running

`appName`, `master("local[2]")`, `showConsoleProgress`, `getOrCreate()`, and `setLogLevel("ERROR")` are Chapter 48's. Two settings are new here, and one import:

Setting	Value here	What it does
<code>spark.sql.shuffle.partitions</code>	4	How many pieces Spark cuts the data into after a shuffle (Chapter 48, section 48.3). Grouping by window is a shuffle, and each piece becomes an output file, so it matters for section 50.6
<code>spark.sql.streaming.schemaInference</code>	false	Never guess a stream's columns from its files. It's already Spark's default; setting it says so on purpose

- **types as T** imports Spark's data types under the short name `T`, the way `functions` became `F`.

A stream must be told its columns in advance, as a **schema**. A schema is a `StructType`, a list of columns; each column is a `StructField` with a name and a type:

```
SCHEMA = T.StructType([
    T.StructField("machine_id", T.StringType()),
    T.StructField("plant", T.StringType()),
    T.StructField("event_time", T.TimestampType()),
    T.StructField("temperature_c", T.DoubleType()),
    T.StructField("units_made", T.IntegerType()),
    T.StructField("scrap", T.BooleanType()),
])
print(SCHEMA.fieldNames())
```

```
['machine_id', 'plant', 'event_time', 'temperature_c', 'units_made', 'scrap']
```

- **StringType**, **DoubleType** (a decimal number), **IntegerType**, and **BooleanType** match the event's text, temperature, count, and True/False values.
- **TimestampType** turns the text "2025-12-01 00:00:00" into a real date and time. Windows need it: Spark can't cut text into 5-minute windows.
- Why not let Spark guess? On a stream, a later file with different columns would silently change the meaning of the job. Declaring the schema makes that file fail loudly instead.

Now the source. A folder of JSON-lines files is the simplest streaming source, and enough to teach every idea in this chapter: the job reads each new file that appears. Write the first file, ten minutes of readings, and point a stream at the folder:

```
import os, shutil

for folder in ["events_in", "stream_out", "checkpoints"]:
    shutil.rmtree(folder, ignore_errors=True)
os.makedirs("events_in")

se.write_batch("events_in", "batch-01.json", se.readings(0, 10))
stream = spark.readStream.schema(SCHEMA).json("events_in")
print("streaming?", stream.isStreaming)
stream.printSchema()
```

```
streaming? True
root
|-- machine_id: string (nullable = true)
|-- plant: string (nullable = true)
|-- event_time: timestamp (nullable = true)
|-- temperature_c: double (nullable = true)
|-- units_made: integer (nullable = true)
|-- scrap: boolean (nullable = true)
```

- The loop deletes the three folders this section uses, so every run of the notebook starts clean. **shutil.rmtree** deletes a folder and everything in it; **ignore_errors=True** means "don't complain if it isn't there". **os.makedirs** creates the input folder again.
- **se.write_batch(..., se.readings(0, 10))** drops a file with minutes 0 to 9, 00:00:00 to 00:09:50: 25 × 6 × 10 = 1,500 readings.
- **spark.readStream** is **spark.read** for streams. **.schema(SCHEMA)** declares the columns; **.json("events_in")** reads JSON-lines files from the folder. In production this would be **.format("kafka")** with the broker's address and a topic; the rest of the code is unchanged.
- **isStreaming** confirms Spark treats this as an unbounded table. **printSchema()** (Chapter 48) shows the declared columns: **event_time** is a timestamp.

Nothing has been read yet. A streaming DataFrame is a plan, like Chapter 48's lazy DataFrames, until a query starts it. Two more ideas decide how the query runs:

Triggers decide when Spark runs a micro-batch: **processingTime="30 seconds"** for a steady job, **availableNow=True** to process everything waiting and stop (used here, so each step of the chapter is one run you can look at), or **continuous mode** for the lowest latency at some cost in features.

Checkpoints are how a streaming job survives restarts: Spark writes the files and offsets it has processed, and its running state, to the folder named in `checkpointLocation`. The next run reads the checkpoint and carries on where the last one stopped. Delete the checkpoint, or point the job at a new location, and it re-processes everything from the start; change the query's aggregation and the old checkpoint may no longer be usable. Treat it as part of the job's data.

50.5 Event time, windows, and watermarks

Two clocks matter.

- **Event time:** when the thing happened (the reading was taken at 00:02:10).
- **Processing time:** when your pipeline saw it (the file arrived at 00:14).

They differ, and the gap is where the hard problems live: a machine loses its network for ten minutes, a phone app syncs when it reconnects, a batch of events is retried. **Always aggregate on event time**, or your numbers change depending on how fast your pipeline happened to be.

Three kinds of window, by hand

A **window** groups events by event time. Take eight readings, taken at 00:01, 00:03, 00:04, 00:07, 00:13, 00:14, 00:21, and 00:22, and count them three ways.

Tumbling windows of 5 minutes sit one after another with no overlap, so each event is counted once. Window 00:00–00:05 includes 00:00 but not 00:05:

Window	Readings	Count
00:00–00:05	00:01, 00:03, 00:04	3
00:05–00:10	00:07	1
00:10–00:15	00:13, 00:14	2
00:15–00:20	none	no row
00:20–00:25	00:21, 00:22	2

Sliding windows of 10 minutes, starting every 5 minutes, overlap, so each event is counted twice: 23:55–00:05 has 3, 00:00–00:10 has 4, 00:05–00:15 has 3, 00:10–00:20 has 2, 00:15–00:25 has 2, and 00:20–00:30 has 2. That's 16 counts for 8 readings. Use them for "the last 10 minutes, updated every 5".

Session windows with a gap of 5 minutes group activity separated by quiet spells. A session starts at an event and stays open until 5 minutes pass with no new event: {00:01, 00:03, 00:04, 00:07} is one session, from 00:01 to 00:12 (the last event plus the gap), with 4 readings; {00:13, 00:14} runs from 00:13 to 00:19, with 2; {00:21, 00:22} from 00:21 to 00:27, with 2. Use them for "how long was this machine busy?"

Now check the hand counts in Spark. This cell makes a tiny DataFrame of the eight times, and a helper that counts rows per window and shows each window's start and end:

```
times = ["00:01", "00:03", "00:04", "00:07", "00:13", "00:14", "00:21", "00:22"]
toy = spark.createDataFrame([("2025-12-01 " + t + ":00",) for t in times], ["event_time"])
toy = toy.withColumn("event_time", F.to_timestamp("event_time"))

def count_windows(window):
    counts = toy.groupBy(window.alias("w")).count().orderBy("w")
    counts.select(F.date_format("w.start", "HH:mm").alias("start"),
                  F.date_format("w.end", "HH:mm").alias("end"),
                  "count").show()

count_windows(F.window("event_time", "5 minutes"))
```

```
+-----+-----+-----+
|start|  end|count|
+-----+-----+-----+
|00:00|00:05|    3|
|00:05|00:10|    1|
|00:10|00:15|    2|
|00:20|00:25|    2|
+-----+-----+-----+
```

- **createDataFrame** (Chapter 48) builds the DataFrame from a list of rows. Each row is a **tuple** with one item, written `(value,)`: the comma is what makes it a tuple. **F.to_timestamp** turns the text into a timestamp.
- **F.window("event_time", "5 minutes")** is a tumbling window: it gives each row the 5-minute window its event time falls in. Grouping by it and counting gives one row per window that has events.
- A window is one column holding **two values**, a `start` and an `end`. **.alias("w")** names it `w`, so `"w.start"` and `"w.end"` reach inside it.
- **F.date_format(..., "HH:mm")** prints a timestamp as hours and minutes. **orderBy("w")** sorts by window, earliest first.

Reading it. 3, 1, 2, 2, as by hand, and there's no row for 00:15–00:20: Spark never writes a row for a window with no events. The other two kinds are one argument away:

```
count_windows(F.window("event_time", "10 minutes", "5 minutes"))
```

```
+-----+-----+-----+
|start|  end|count|
+-----+-----+-----+
|23:55|00:05|    3|
|00:00|00:10|    4|
|00:05|00:15|    3|
|00:10|00:20|    2|
|00:15|00:25|    2|
|00:20|00:30|    2|
+-----+-----+-----+
```

- The third argument, **"5 minutes"**, is the slide: a new 10-minute window starts every 5 minutes. Without it, the window is tumbling.

```
count_windows(F.session_window("event_time", "5 minutes"))
```

```
+-----+-----+-----+
|start|  end|count|
+-----+-----+-----+
|00:01|00:12|    4|
|00:13|00:19|    2|
|00:21|00:27|    2|
+-----+-----+-----+
```

- `F.session_window("event_time", "5 minutes")` closes a session after 5 minutes without an event.

Both match the hand counts. The rest of this chapter uses **tumbling** windows of 5 minutes.

Watermarks: how long to wait for late events

A reading taken at 00:02 can arrive at 00:15. Should it still count in the 00:00–00:05 window? A streaming job can't wait forever, or it would hold every window open and never publish anything. A **watermark** is the promise: *"I will wait this long for late events, and then I'll close the window."* It's the knob that trades completeness for latency, and it also bounds how much the job must keep in memory.

The watermark rule. After each run, the watermark is the newest event time seen so far, minus the delay you chose (10 minutes here). During the next run, a reading older than that watermark is dropped. A window is final, and append mode writes it, once the watermark has passed its end.

Watch out: same word, different job. In Chapter 45 a watermark was the highest ID or timestamp a batch load had already read, so the next load could start after it. In streaming it's a moving cut-off in event time: "I've seen events up to 00:19:50, and I'll wait 10 more minutes for stragglers." Both mean "how far we've got", but a streaming watermark decides what gets dropped.

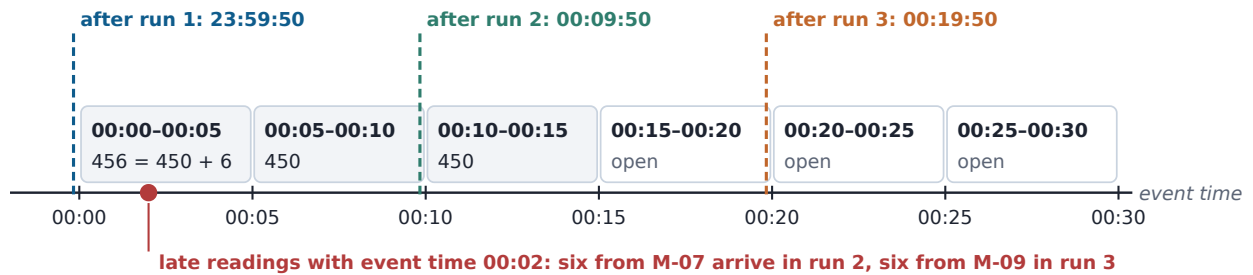
Here is what the rule does over three runs of the job. Each run brings a new file: ten minutes of readings on time, and in runs 2 and 3, six readings taken at 00:02 that arrive late. Work it out before looking at the code:

Run	Events arriving	Watermark in force	Six late readings (00:02)	Watermark after the run	Windows written
1	00:00 to 00:09:50	none yet	none	00:09:50 – 10 min = 23:59:50	none
2	00:10 to 00:19:50, plus six M-07 at 00:02	23:59:50	kept: 00:02 is later than 23:59:50	00:19:50 – 10 min = 00:09:50	00:00–00:05
3	00:20 to 00:29:50, plus six M-09 at 00:02	00:09:50	dropped: 00:02 is earlier than 00:09:50	00:29:50 – 10 min = 00:19:50	00:05–00:10, 00:10–00:15

After run 2, window 00:05–00:10 isn't written yet: it ends at 00:10:00, which the watermark, 00:09:50, hasn't passed. It waits for run 3. Figure 50.2 draws the same three runs.

Event time, 5-minute windows, and a watermark 10 minutes behind the newest event

Boxes: Bhiwandi Main readings per window after all three runs. Dashed lines: the watermark after each run.



Run 1: events 00:00 to 00:09:50

watermark in force: none yet; after the run: 23:59:50

No window ends by 23:59:50, so append mode writes nothing.

Run 2: events 00:10 to 00:19:50, plus six M-07 readings from 00:02

watermark in force: 23:59:50; after the run: 00:09:50

00:02 is later than 23:59:50, so the six are kept: $450 + 6 = 456$.

00:00-00:05 ends before 00:09:50: written. 00:05-00:10 ends at 00:10:00: it waits.

Run 3: events 00:20 to 00:29:50, plus six M-09 readings from 00:02

watermark in force: 00:09:50; after the run: 00:19:50

00:02 is earlier than 00:09:50, so the six are dropped, silently; 456 stays 456.

00:05-00:10 and 00:10-00:15 now end before 00:19:50: both are written.

A window is final once the watermark passes its end; a reading older than the watermark in force is dropped.

Figure 50.2 – The watermark decides which late events still count, and when a window is final. Same late readings, different arrival times, different answers.

The windowed query, one step at a time

First the calculation: readings, average temperature, and scrap readings per 5-minute window and plant, with a 10-minute watermark:

```
windowed = (stream
  .withWatermark("event_time", "10 minutes")
  .groupBy(F.window("event_time", "5 minutes"), "plant")
  .agg(F.count("*").alias("readings"),
    F.round(F.avg("temperature_c"), 2).alias("avg_temp_c"),
    F.sum(F.col("scrap").cast("int")).alias("scrap_readings"))
windowed.printSchema())
```

```
root
|-- window: struct (nullable = false)
|   |-- start: timestamp (nullable = true)
|   |-- end: timestamp (nullable = true)
|-- plant: string (nullable = true)
|-- readings: long (nullable = false)
|-- avg_temp_c: double (nullable = true)
|-- scrap_readings: long (nullable = true)
```

- `.withWatermark("event_time", "10 minutes")` sets the watermark: the newest `event_time` seen, minus 10 minutes. It must come before the `groupBy` it applies to.

- `.groupBy(F.window("event_time", "5 minutes"), "plant")` makes one group per 5-minute window and plant, like the toy example with a second grouping column.
- `F.count("*")`, `F.avg`, and `F.round(..., 2)` are Chapter 48's. `F.col("scrap").cast("int")` turns True and False into 1 and 0 (Chapter 48), so `F.sum` counts the scrap readings.
- The schema shows the `window` column holding a `start` and an `end`, as in the toy example.

Next, a helper that runs the query once. Each run of the job will call it:

```
def run_once():
    query = (windowed.writeStream
              .format("parquet")
              .option("path", "stream_out/windows")
              .option("checkpointLocation", "checkpoints/windows")
              .outputMode("append")
              .trigger(availableNow=True)
              .start())
    query.awaitTermination()
    return query
```

- `writeStream` is write for streams. `.format("parquet")` makes the sink a folder of Parquet files, and `.option("path", ...)` names it.
- `.option("checkpointLocation", "checkpoints/windows")` is where Spark keeps its place. Because every run uses the same checkpoint, run 2 carries on from where run 1 stopped: it reads only the new file, and it remembers the open windows and the watermark.
- `.outputMode("append")` writes a window only once it's final (the table after this list).
- `.trigger(availableNow=True)` processes whatever has arrived, then stops.
- `.start()` starts the query and returns at once, while Spark works in the background. `query.awaitTermination()` waits until the run is finished. The function returns the query, so you can ask it afterwards what it did.

Output modes decide what a sink receives:

Mode	Emits	Use for
append	Rows that are final (past the watermark)	Files, tables, downstream pipelines
update	Rows that changed in this micro-batch	Dashboards and alerting where "so far" is fine
complete	The whole result each time	Small aggregates only; the state grows forever

Last, a helper to look at what the sink holds so far:

```
def show_windows():
    result = spark.read.parquet("stream_out/windows")
    print("windows written so far:", result.count())
    (result.orderBy("window", "plant")
     .select(F.date_format("window.start", "HH:mm").alias("window_start"),
             F.date_format("window.end", "HH:mm").alias("window_end"),
             "plant", "readings", "avg_temp_c", "scrap_readings")
     .show(truncate=False))
```

- `spark.read.parquet` reads the sink as an ordinary batch table (Chapter 48). The whole chain is in round brackets so it can run over several lines.

- `show(truncate=False)` prints full values (Chapter 48).

Before you run the first run, predict from the hand table: how many windows will it write?

```
query = run_once()
show_windows()
```

```
windows written so far: 0
+-----+-----+-----+-----+-----+-----+
|window_start|window_end|plant|readings|avg_temp_c|scrap_readings|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
```

Reading it. The first run wrote **nothing at all**, and that's correct. In **append** mode a window is written only once the watermark has passed its end, so that the result never has to change. The first file's events end at 00:09:50, the watermark after the run is 10 minutes earlier, 23:59:50, and no window ends that early. This surprises everyone once.

Late data that still counts

Run 2 drops the next ten minutes into the folder, plus six M-07 readings taken at 00:02 that arrive only now:

```
on_time = se.readings(10, 20)
late = se.readings(2, 3, machine="M-07")
se.write_batch("events_in", "batch-02.json", on_time + late)
print("run 2:", len(on_time), "on time,", len(late), "late (event time 00:02)")

query = run_once()
show_windows()
```

```
run 2: 1500 on time, 6 late (event time 00:02)
windows written so far: 2
+-----+-----+-----+-----+-----+-----+
|window_start|window_end|plant      |readings|avg_temp_c|scrap_readings|
+-----+-----+-----+-----+-----+-----+
|00:00       |00:05     |Bhiwandi Main|456     |196.04    |4          |
|00:00       |00:05     |Chakan Pune  |300     |195.76    |4          |
+-----+-----+-----+-----+-----+-----+
```

- `se.readings(10, 20)` is 00:10:00 to 00:19:50: 1,500 readings. `se.readings(2, 3, machine="M-07")` is M-07's six readings from minute 2, 00:02:00 to 00:02:50.
- `on_time + late` joins the two lists, so the late readings sit in the same file as the new ones, as they would if a machine's network came back.

Reading it. The six readings from 00:02 arrived well after their window's end. The watermark in force was 23:59:50, and 00:02 is later than that, so they were **still counted**: Bhiwandi Main's 00:00–00:05 window shows **456 readings**, not the 450 that arrived on time (15 machines × 6 readings × 5 minutes = 450; M-07 is at Bhiwandi Main). Chakan Pune's 10 machines give 10 × 30 = 300. After the run the watermark moved to 00:09:50, which passed the end of 00:00–00:05, so that window was written. 00:05–00:10 ends at 00:10:00 and is still open.

Late data that doesn't

Run 3 does the same with six readings from M-09, also taken at 00:02:

```
very_late = se.readings(2, 3, machine="M-09")
se.write_batch("events_in", "batch-03.json", se.readings(20, 30) + very_late)

query = run_once()
show_windows()
```

```
windows written so far: 6
+-----+-----+-----+-----+-----+-----+
|window_start|window_end|plant      |readings|avg_temp_c|scrap_readings|
+-----+-----+-----+-----+-----+-----+
|00:00       |00:05     |Bhiwandi Main|456     |196.04    |4             |
|00:00       |00:05     |Chakan Pune  |300     |195.76    |4             |
|00:05       |00:10     |Bhiwandi Main|450     |195.88    |4             |
|00:05       |00:10     |Chakan Pune  |300     |195.87    |3             |
|00:10       |00:15     |Bhiwandi Main|450     |196.01    |5             |
|00:10       |00:15     |Chakan Pune  |300     |195.94    |6             |
+-----+-----+-----+-----+-----+-----+
```

Reading it. This time the watermark in force was 00:09:50, and the six M-09 readings from 00:02 are older than that, so they were **dropped**: the 00:00–00:05 window is unchanged at 456. Nothing failed, nothing was logged as an error, and the published numbers stayed stable. After the run the watermark moved to 00:19:50, so the windows 00:05–00:10 and 00:10–00:15 were written. That's the deal a watermark makes on your behalf, which is why it should be a deliberate decision rather than a default.

What the query reports

A query keeps a progress report for each micro-batch of its run in `query.recentProgress`: a list of dictionaries. Look at the keys of the first one:

```
progress = query.recentProgress
print(len(progress), "reports in this run")
print(list(progress[0].keys()))
```

```
2 reports in this run
['id', 'runId', 'name', 'timestamp', 'batchId', 'batchDuration', 'durationMs', 'eventTime', 'stateOperators',
 'sources', 'sink', 'numInputRows', 'inputRowsPerSecond', 'processedRowsPerSecond', 'observedMetrics']
```

Four keys matter here. **batchId** numbers the micro-batches. **numInputRows** is how many new rows the micro-batch read. **eventTime** holds the watermark the micro-batch used. **stateOperators** is a list with one entry per step that keeps state; its first entry holds **numRowsTotal**, the rows kept in state, and **numRowsDroppedByWatermark**, the rows dropped as too late. Print those for each report:

```
for p in progress:
    state = p["stateOperators"][0]
    print(f"batch {p['batchId']}: rows {p['numInputRows']}>4}, "
          f"watermark {p['eventTime']['watermark'][11:19]}, "
          f"dropped {state['numRowsDroppedByWatermark']}, "
          f"windows in state {state['numRowsTotal']}")
```

```
batch 4: rows 1506, watermark 00:09:50, dropped 1, windows in state 10
batch 5: rows    0, watermark 00:19:50, dropped 0, windows in state 6
```

- The watermark is text like `2025-12-01T00:09:50.000Z`; `[11:19]` slices out the time (Chapter 17). `:>4` right-aligns the number in 4 characters.

Reading it. Batch ids count every micro-batch since the checkpoint was created, across all three runs: run 1 was batches 0 and 1, run 2 was 2 and 3, and run 3, this one, was **4 and 5**. Each run's second micro-batch reads no rows: Spark adds it to move the watermark forward and write the windows that have just become final.

- **Batch 4** read 1,506 rows, the 1,500 on-time readings plus the six M-09 stragglers, using the watermark `00:09:50` set at the end of run 2. That's why the six were too late. The metric says **1** row dropped, not 6, because it counts *state rows*, not events. All six were M-09 readings from `00:02`, so they fell in the same group: window `00:00–00:05`, plant Bhiwandi Main (M-09 is one of Bhiwandi's machines M-01 to M-15). Spark had already combined them into one partial row before the watermark check, and that one row was dropped. After batch 4, state held 10 rows: 5 open windows × 2 plants.
- **Batch 5** read nothing, used the new watermark `00:19:50`, and wrote the two windows it had passed, `00:05–00:10` and `00:10–00:15`, for both plants. State shrank to 6 rows: the windows starting `00:15`, `00:20`, and `00:25`, for both plants.

How to choose a watermark. Measure the gap between event time and arrival time for a week, and pick a value that covers the great majority of it (a high percentile), then decide what to do with the rest: drop them, route them to a "late" table for a daily correction (Chapter 46's backfill, section 46.5), or widen the watermark and accept the delay. Tell the people using the numbers which you chose.

50.6 Writing a stream into a table

Streaming sinks produce many small files, because each micro-batch writes its own. Chapter 49's problem, at speed. First count the files the three runs wrote, and the rows in each:

```
import pyarrow.parquet as pq

out = "stream_out/windows"
files = sorted(f for f in os.listdir(out) if f.endswith(".parquet"))
rows = [pq.read_table(os.path.join(out, f)).num_rows for f in files]
print("parquet files written by the stream:", len(files))
print("rows in each file, smallest first:", sorted(rows))
```

```
parquet files written by the stream: 9
rows in each file, smallest first: [0, 0, 0, 0, 0, 1, 1, 2, 2]
```

- `sorted(f for f in os.listdir(out) if f.endswith(".parquet"))` lists the Parquet files in the sink's folder, leaving out Spark's bookkeeping files.
- `pq.read_table(...).num_rows` reads one file with PyArrow (Chapter 49) and counts its rows. The files' names are random, so `sorted(rows)` puts the counts in order, smallest first, to make them easy to

read.

Reading it. Nine files for six rows, and five of the nine are **empty**: Spark writes a file for every micro-batch, including those with no final windows to write, such as run 1's.

Now load them into a Delta table the way a stream would, one append per file, and count the table's data files. The table's log (Chapter 49) knows every data file and its size, so ask it rather than the folder, which also works the same on Windows, on a Mac, and on object storage:

```
import pyarrow as pa
from deltalake import DeltaTable, write_deltalake

shutil.rmtree("delta_windows", ignore_errors=True)
for f in files:
    write_deltalake("delta_windows", pq.read_table(os.path.join(out, f)), mode="append")

def data_files(path):
    added = pa.table(DeltaTable(path).get_add_actions(flatten=True))
    sizes = added.column("size_bytes").to_pylist()
    return f"data files {len(sizes)}, average {sum(sizes) / len(sizes) / 1e3:.1f} KB"

print("delta table now :", data_files("delta_windows"))
```

```
delta table now : data files 4, average 2.2 KB
```

- **write_deltalake(..., mode="append")** adds each file's rows to the table (Chapter 49); the first append creates it.
- **data_files** is a shorter version of Chapter 49's **file_report** (section 49.5):
get_add_actions(flatten=True) returns one row per data file in the table's current version, read from its log, and **pa.table(...)** turns that into a PyArrow table whose **size_bytes** column holds each file's size.

Reading it. Nine appends made four data files: appending an empty table adds no data file. Now compact:

```
DeltaTable("delta_windows").optimize.compact()
print("after compaction:", data_files("delta_windows"))
```

```
after compaction: data files 1, average 2.4 KB
```

Reading it. Compaction (Chapter 49) merged the four files into one. At real volumes this is a maintenance job that must run: a stream writing every 30 seconds makes at least 2,880 files a day per table partition, times the number of files each trigger writes (up to `spark.sql.shuffle.partitions`, 4 here). By the end of a month, reads are crawling.

That's the last Spark cell of the chapter, so stop Spark:

```
spark.stop()
```

- **spark.stop()** shuts the Spark session down and frees the memory Java was holding. A later **getOrCreate()** would start a new session.

The pattern that works in production:

1. Stream into a table format (Delta, Iceberg, Hudi) rather than loose files, so readers get consistent snapshots while writes continue.
2. Use a trigger interval that balances latency against file count: 30 seconds to a few minutes is common.
3. Run **compaction** and **vacuum** on a schedule (Chapter 49), and monitor the file count.
4. Make the sink **idempotent** (upsert by key, or exactly-once via checkpoints plus a transactional sink), so a replayed batch doesn't double anything.
5. Apply Chapter 47's checks to the stream's output table, not only to batch tables.

50.7 Operating a stream

A batch job either ran or didn't. A stream is always running, so you monitor its health instead.

Signal	What it tells you	Alert when
Consumer lag	How far behind the end of the log you are	Lag grows steadily rather than fluctuating
Watermark age	How far behind wall-clock time your event time is	It stalls, which means a source stopped sending
Rows dropped as late	Events arriving outside the watermark	Any sustained increase
State size	Memory held for open windows	It grows without bound: usually a missing watermark
Batch duration versus trigger	Whether processing keeps up	Duration approaches the trigger interval
Failed restarts	The job died and couldn't resume	Immediately: a stream that isn't running is silent

Three habits save most incidents.

- **Replay is your recovery tool.** Retention on the log (often 7 days) means you can reset a group's offsets and re-process, provided your sink is idempotent.
- **A dead-letter path beats a crashing job.** Events that can't be parsed go to a side table with the error, rather than stopping the pipeline at 3 a.m.
- **Schema changes are the usual cause of outages.** A producer adds a field: fine. A producer renames one: your job breaks. This is Chapter 47's data contract, applied to events. Teams enforce it with a **schema registry**: a shared service that stores each topic's agreed schema and refuses events from a producer that breaks it.

50.8 Riverstone's plant monitoring case

Put the pieces together, as the plant team actually needs them.

The need. A machine drifting hot produces scrap continuously. The supervisor wants to know within minutes, not the next morning, and wants the day's totals to be correct even if a machine's network drops for a while.

The design.

- **Producer:** each machine's controller sends a reading every 10 seconds to a `sensor-readings` topic, keyed by machine id, so one machine's readings stay in order.
- **Alerting path (seconds):** a small consumer group reads the topic, holds the last few minutes per machine, and raises an alert when the average over 2 minutes exceeds the machine's limit. Alerts are de-duplicated by machine and window, and suppressed for 30 minutes after firing, so one drifting machine doesn't send twelve messages.
- **Aggregation path (minutes):** a Spark Structured Streaming job with 5-minute windows and a 10-minute watermark writes per-plant summaries into the Delta table from Chapter 49, as in section 50.5. For per-machine rows, add `machine_id` to the `groupBy`: the state grows 25 times larger, one row per machine instead of per plant.
- **Correction path (daily):** a batch job re-reads the day from the archive and rewrites yesterday's windows, so late and dropped events are corrected once a day. Streaming for speed, batch for truth. Chapter 62 names this design, a fast streaming path plus a batch path that corrects it, the **Lambda architecture**.
- **Quality:** Chapter 47's checks run on the daily corrected table, and a freshness check on the streaming table alerts if no window has been written for 15 minutes.

What this costs. A broker or a managed equivalent, a streaming job that must be watched, an on-call rota, and the batch pipeline that still exists. That's the honest price, and it's why the last part of this section is about whether to pay it.

When Riverstone would not build this. If the plant already has an alarm on the machine controller, the streaming alert adds nothing. If nobody is on the floor at night to act on it, the alert is decoration. If the scrap cost is small, a 10-minute batch job is the better answer. Chapter 58 covers the automation-and-AI version of this same decision.

APPLY

Common mistakes

Mistake	Symptom	Fix
Building a stream because "real-time" was asked for	Complexity with no decision changed	Turn it into latency, consequence, and who acts (50.1)
Aggregating on processing time	Numbers change if the pipeline is slow	Aggregate on event time (50.5)
No watermark on a windowed job	State grows until the job dies	Set a watermark; monitor state size

Mistake	Symptom	Fix
A watermark chosen by default	Silent drops, or hours of delay	Measure the lateness distribution and choose (50.5)
Assuming exactly-once	Duplicate alerts and double counts	Assume at-least-once; de-duplicate (50.3)
Keeping the de-duplication record in memory	Duplicates come back after every real restart	Store sent keys on disk, in a table (50.3)
Auto-committing offsets	Events lost if the consumer dies after an auto-commit but before the work; duplicates if it dies after the work but before the next auto-commit (the story below)	Turn auto-commit off; commit after the work succeeds (50.3)
Ordering assumed across a topic	"Impossible" sequences in the data	Order holds within a partition; key by the entity
Schema inference on a stream	A new file changes the job's behaviour	Declare the schema; use a registry (50.4, 50.7)
Deleting or moving checkpoints	Everything re-processed, or state lost	Treat checkpoints as data (50.4)
Never compacting the sink	Reads slow down week by week	Scheduled compaction and vacuum (50.6)
No dead-letter path	One malformed event stops the pipeline	Route bad events aside with their error
Monitoring only "is it running?"	Silent lag and stalled watermarks	Lag, watermark age, state size, drops (50.7)
No batch correction path	Dropped late events are wrong forever	Reprocess the day in batch (50.8)

In the real world: the alert that cried wolf

Riverstone's first streaming alert went live on a Tuesday, watching machine temperatures against a fixed limit of 205 °C.

By Thursday, the plant supervisors had muted the channel. In two days it had sent 1,900 messages. Three things were wrong, and each is in this chapter.

It alerted on single readings. A reading every 10 seconds means a machine hovering near the limit trips the alert dozens of times an hour, and a single noisy reading fires it too. The fix was a 2-minute window: alert on the average, not the instant.

It re-sent everything after every restart. The consumer committed offsets automatically every few seconds, but the deployment restarted the job several times, and each restart re-read from a slightly older offset and re-sent alerts that had already gone. The fix was a de-duplication key of machine plus window, kept for an hour, and committing only after alerts were delivered.

It treated all machines alike. The extrusion lines run hotter by design, so their normal operation looked like an emergency. The fix was a per-machine limit read from a small table, with the plant team owning the values.

After the changes, the same two days would have produced 11 alerts, all of them real, and one of them caught a heater that was failing gradually.

The plant manager's verdict, at the review, was the sentence Meera kept: "The system was right every time and useful none of the time."

What made this work.

- **They counted the alerts** instead of arguing about whether it was noisy.
- **Every fix was about the design, not the threshold:** windows instead of readings, de-duplication instead of hope, per-machine limits instead of one number.
- **The owners of the limits were the people who understood the machines.**

Project: minute-level machine monitoring

Goal: build the plant monitoring design from section 50.8 end to end, on the practice data, and decide whether it's worth running.

Tools you'll need

- **Python 3.14** in the book's virtual environment, with `pyspark`, `deltalake`, `duckdb`, and `pyarrow`, and **Java 21**.
- **The Chapter 50 companion folder** (`companion/ch50/`): `mini_log.py` (the local message log) and `sensor_events.py` (replays the Chapter 48 archive as events). Generate the Chapter 48 dataset first.
- **Versions used for the outputs shown:** Python 3.11.15, PySpark 4.2.0, deltalake 1.6.6, DuckDB 1.5.6, PyArrow 25.0.1, OpenJDK 21.0.10, Apache Kafka 4.3.1, `kafka-python` 3.0.11.
- **For real Kafka:** the single broker from section 50.3, with `kafka-python` (or `confluent-kafka`) as the client; managed services include Amazon MSK, Confluent Cloud, Azure Event Hubs, and Google Pub/Sub. Spark connects with `format("kafka")`.
- **Other engines worth knowing:** Apache Flink (the strongest event-time engine), Kafka Streams, and cloud streaming services.

Option A: Riverstone. Use the companion scripts.

Option B: your own events. Any event-like data you have permission to use: web logs, IoT readings, transactions.

Steps

1. **Produce a stream:** replay one day of readings into the message log, keyed by machine, in batches of a few minutes.
2. **Write a consumer** that keeps a 2-minute average per machine and raises alerts above a per-machine limit held in a small table.
3. **Make it survive a crash:** stop the consumer mid-batch, restart it, and show that no alert is sent twice. Say which guarantee you implemented, and where the record of sent alerts lives.
4. **Build the aggregation job** in Structured Streaming: 5-minute windows, a watermark you justify, written to a Delta table.

5. **Demonstrate late data** twice: once accepted within the watermark, once dropped outside it. Record both counts.
6. **Add the correction path**: a batch job that rewrites yesterday's windows from the archive, and a check that compares streaming totals with corrected totals.
7. **Compact the sink** and record the file count before and after.
8. **Write the monitoring plan**: which signals, what thresholds, who is alerted, and what the runbook says.
9. **Write the decision memo**, one page: the latency the business needs, what it costs to run, and your recommendation, including the batch-only alternative.

Stretch goals

- Add a session window per machine (activity separated by gaps of more than 15 minutes) and describe what it tells the plant team.
- Replay the same day with a 1-minute watermark and compare dropped counts.
- Run the producer and the alerting consumer against the real broker from section 50.3, changing only the client code.

REVIEW

Recap

- **Streaming is for decisions whose value decays in minutes.** Most "real-time" requests are answered better by a shorter batch schedule.
- An event log is **append-only**, split into **partitions**; a **key** decides the partition, so order holds within a key, not across a topic.
- **Consumer groups** track their own **offsets**; a committed offset is the next position to read, and **lag** is the first metric to watch.
- Delivery is **at-least-once** in practice: expect duplicates and de-duplicate by a key kept on disk. Here a crash re-sent all 82 alerts until a stored key stopped them.
- **Event time** is what you aggregate on; **processing time** is when you happened to see it.
- **Tumbling** windows count each event once, **sliding** windows overlap, and **session** windows end after a quiet gap.
- **Windows** need a **watermark**: it sets how long you wait for stragglers and bounds state. Six late readings were counted inside a 10-minute watermark (456 readings instead of 450), and six more were dropped once the watermark had moved on.
- **Append mode emits only final windows**, which is why the first run produced nothing.
- Streaming sinks make **small files**: stream into a table format and compact on a schedule.
- Operate a stream by watching **lag**, **watermark age**, **dropped rows**, **state size**, **batch duration**, and **restarts**, with a **dead-letter path** and a **schema contract**.
- Pair streaming with a **daily batch correction**: speed from the stream, truth from the batch.

Key terms

batch · streaming · latency · micro-batch · event log · topic · partition · key · offset · committed offset · consumer group · lag · retention · replay · producer · consumer · broker · replica · at-most-once · at-least-once · exactly-once · sink · de-duplication key · business key · idempotency key · Kafka · schema registry · Structured Streaming · unbounded table · schema · trigger · checkpoint · event time · processing time · window (tumbling, sliding, session) · watermark · lateness · output mode (append, update, complete) · state · no-data batch · dead-letter path · small files · compaction · dashboard refresh

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ You can turn "real-time" into a latency, a decision, and an owner.
- ☐ You can explain topics, partitions, keys, offsets, consumer groups, lag, and retention.
- ☐ You can say exactly what order guarantees a log gives, and what it doesn't.
- ☐ You can explain at-most-once, at-least-once, and exactly-once, and say which you'd get by default.
- ☐ You can show a duplicate happening and fix it with a de-duplication key that survives a restart.
- ☐ You can read Kafka producer and consumer code and name the settings that matter.
- ☐ You can count events in tumbling, sliding, and session windows by hand.
- ☐ You can write a Structured Streaming job with a declared schema and a checkpoint.
- ☐ You can explain event time versus processing time and why aggregations use event time.
- ☐ You can trace a watermark run by run, and say what happens to data outside it.
- ☐ You can explain why an append-mode window produced no output in the first run.
- ☐ You can keep a streaming sink healthy with compaction and idempotent writes.
- ☐ You can list the signals to monitor on a running stream.
- ☐ You can argue both sides of "should this be streaming?" for a real case.

Exercises

Warm-up

1. For each, say whether you'd build streaming, a frequent batch job, or nothing: (a) a dashboard the sales head opens each morning; (b) blocking a fraudulent payment; (c) telling a supervisor a machine is running hot; (d) a weekly scrap report; (e) showing a customer where their delivery is.
2. A topic has 4 partitions and events are keyed by machine id. Which of these are guaranteed: (a) all events for M-07 are read in order; (b) all events across all machines are read in order; (c) two consumers in one group can each read a different partition; (d) two different groups can read the same partition independently?
3. Explain at-most-once and at-least-once in terms of *when the offset is committed*, and say which one produces duplicates.

Core

4. In section 50.2, the 300 events split 144 and 156 across two partitions. Explain why the split isn't exactly even, and what would happen to ordering if the producer used a random key instead of the machine id.
5. Section 50.3 sent the same 82 alerts twice. Write the de-duplication rule you'd use in production, say where you'd store it, and how long you'd keep it.
6. The first windowed run emitted nothing. Explain why, in terms of the watermark and append mode, and say what output mode you'd use for a live dashboard that's allowed to show "so far" numbers.
7. Six readings with event time 00:02 arrived in run 2 and were counted; six more from 00:02 arrived in run 3 and were dropped. Give the watermark in force during each run and show the arithmetic that decides each outcome.
8. Section 50.5's progress output says 1 row was dropped as too late, although 6 events were late. Explain the difference between events and state rows here.
9. A streaming job writes every 30 seconds into a table partitioned by day. How many files a day does that produce, and what two things would you schedule to keep reads fast? (Chapter 49.)

Stretch

10. Design the alerting consumer from section 50.8: window length, per-machine limits, de-duplication, suppression after firing, what the alert message says, and what happens when the consumer falls behind by an hour.
11. Riverstone's plant network drops for 25 minutes and then delivers all the missing readings at once. Trace what happens with a 10-minute watermark: which windows are correct, which are wrong, what the monitoring shows, and how the daily correction fixes it.
12. Compare Kafka with the file source used in this chapter for Riverstone's case, on: latency, replay, ordering, operational cost, and what would have to change in the Spark job. Then say what you'd recommend for a company Riverstone's size.

Think about it (no code needed)

13. The plant manager said the alerting system was "right every time and useful none of the time". Explain what that means for how you'd measure an alerting system's success, and what you'd report monthly.
14. Streaming makes numbers arrive sooner but also makes them provisional, since late data can change them. How would you explain that to a sales head who sees a revenue figure change between 9 a.m. and 10 a.m., and what design choices would you offer?

Answers

1. (a) **Nothing**, or a nightly batch: it's read once a day. (b) **Neither**: blocking a payment happens in the application's own request path, in milliseconds; a data pipeline is too slow, though it can score risk in advance. (c) **Streaming**, or a frequent batch if minutes are acceptable and the scrap cost is low. (d) **Batch**, weekly. (e) **Streaming** (or frequent updates from the courier's feed), because the customer is watching it change.
2. Guaranteed: (a), because one key always maps to one partition and a partition is read in order; (c), partitions are the unit of parallelism within a group; (d), groups track offsets independently. **Not guaranteed: (b)** — there's no global order across partitions, only within each one.
3. **At-most-once** commits the offset *before* doing the work, so a crash in between loses the event: no duplicates, possible loss. **At-least-once** commits *after* the work succeeds, so a crash in between re-reads the event: no loss, **duplicates possible**. At-least-once is the default in practice, which is why de-duplication belongs in the design.
4. The partition is chosen by a hash of the key, and a hash spreads 25 machine ids unevenly across 2 partitions: 12 machines hashed to partition 0 and 13 to partition 1. Each machine sent exactly 12 readings (2 minutes $\times$ 6), so the partitions hold $12 \times 12 = 144$ and $13 \times 12 = 156$. With a **random key**, events for one machine would scatter across partitions, and their relative order would be lost: a later reading could be processed before an earlier one, which breaks any per-machine logic such as "temperature rising for 2 minutes".
5. A business key: **machine id + window start** (or machine id + event time for single-event alerts, as in section 50.3). Store it where the consumer can check it after a restart: a small table, as in section 50.3, or a key-value store with expiry, keyed as `alert:M-07:2025-12-01T00:05`. Keep it at least as long as the maximum replay you'd ever do, plus your suppression window; an hour is fine for alerts fired per 5-minute window, a day is safer if you re-process the stream. The same key doubles as the suppression record.
6. In **append** mode a windowed result is emitted only when it can no longer change, which means when the **watermark** has passed the window's end. After run 1 the newest event time was 00:09:50, so the watermark (10 minutes behind) was 23:59:50 of the previous day: no window's end had been passed, so nothing was final and nothing was written. For a live dashboard that may show partial numbers, use **update** mode, which emits windows whose values changed in this micro-batch, and mark the latest window as provisional in the interface.

7. After run 1, the newest event time was 00:09:50, so the watermark in force during run 2 was **23:59:50** (00:09:50 minus 10 minutes). The late M-07 readings had event time 00:02, which is later than 23:59:50, so they were accepted and folded into the 00:00–00:05 window, which had not yet been emitted: 450 on time plus 6 late equals **456**. After run 2, the newest event time was 00:19:50, so the watermark in force during run 3 was **00:09:50**. The M-09 readings in run 3 again had event time 00:02, which is **earlier** than 00:09:50, so they were dropped, and the window stayed at 456.

8. The metric counts rows removed from the **aggregation's state**, not raw input events. The query groups by window and plant, and all six late readings were M-09's from 00:02: the same window (00:00–00:05) and the same plant (Bhiwandi Main). Spark had combined them into a single partial row before the watermark check applied, and that one row was dropped. Six events, one row. It matters when reading the metric: a spike of 1 can represent thousands of events, so pair it with input counts.

9. Every 30 seconds is 2 per minute, 120 per hour, **2,880 files a day** per partition, and more if each micro-batch writes several files (up to `spark.sql.shuffle.partitions`). Schedule (1) **compaction** (OPTIMIZE) to merge them into files of 128–512 MB, and (2) **vacuum** to remove the files compaction replaced, with a retention that still allows your rollback window. Increasing the trigger interval is a third, cheaper lever.

10. A defensible design: **2-minute tumbling windows per machine** (long enough to smooth noise, short enough to act); **per-machine limits** in a small table owned by the plant team, with a default for new machines; alert when the window's average exceeds the machine's limit **and** the previous window also did, so a single spike doesn't fire; **de-duplication** by machine plus window start, stored on disk for an hour; **suppression** of 30 minutes per machine after an alert, with a reminder if still hot at the end of it; the message says machine, line, plant, the window's average and limit, how long it has been high, and a link to the dashboard and runbook. If the consumer **falls behind by an hour**, the right behaviour is to skip to the latest offsets for *alerting* (an hour-old alert is worse than useless) while recording the gap, and to let the aggregation path process the backlog normally. That decision should be written down, because it's a trade-off, not an obvious truth.

11. With a 10-minute watermark: readings arriving 25 minutes late are **outside** it. Other machines keep sending, so the watermark keeps moving and the windows covering the outage are written with the readings that did arrive: for the affected machines they either have no row at all or a partial count. When the missing readings land, they're **dropped**, so those windows are permanently wrong in the streaming table: they under-report output and scrap. Monitoring shows it: during the outage, the **freshness** check on those machines' readings fires, and if the whole plant's network is down the **watermark stalls**; when the backlog lands, **lag** spikes briefly and **rows dropped as too late** jumps. The **daily correction** fixes it: the batch job re-reads the full archive for the day, recomputes every window, and rewrites them with an atomic partition overwrite (Chapters 46 and 49), so by the next morning the table is right. If the plant team needs the intraday numbers to survive outages, widen the watermark (more delay, more state) or route dropped events to a late table processed hourly.

12. **Latency**: Kafka delivers in milliseconds; a file source depends on how often files are written, so seconds to minutes. **Replay**: Kafka replays from retained offsets without the producer's involvement; files replay only if you kept them. **Ordering**: Kafka guarantees order within a partition, and keys give you per-machine order; a folder of files has no ordering guarantee beyond what's inside each file. **Operational cost**: Kafka means a cluster (or a managed service), monitoring, and upgrades; files mean

object storage. **Changes to the Spark job:** the source (`format("kafka")` with brokers, topic, and starting offsets), parsing the value from bytes, and offsets living in Kafka rather than in the file source's checkpoint tracking; the windowing, watermark, and sink code are unchanged.

Recommendation for Riverstone: start with the file or micro-batch route and a managed service only if second-level latency is genuinely needed; a company of this size gets most of the value from 1–5 minute latency, and avoids running a broker. Revisit when there are several consumers of the same events, which is where a log starts to pay for itself.

13. It means correctness isn't the measure; **action** is. Measure an alerting system by: how many alerts fired, how many led to someone doing something (the useful ones), how many were ignored or muted, how long from the underlying event to the alert, and how many real events were **missed**. Report monthly: alerts per machine per week, the share acted on, median time to acknowledge, false-positive causes grouped, and any incident the system missed. A rising alert count with a falling action rate is the signal to change the design, exactly as in the story.

14. Explain it in their terms: *"The 9 a.m. figure is what we know so far; some orders reach the system a few minutes late, so the number can still move. By 10 a.m. it's settled, and the end-of-day figure is final and reconciled with the ERP."* Then offer design choices: (1) label the live figure as provisional, with a "final at" time, which costs nothing; (2) delay the live figure by the watermark so it never changes, at the cost of being older; (3) show both a live and a confirmed number side by side; (4) keep the live number for direction only and put decisions on the daily corrected table. The choice depends on whether they act on the intraday number or only watch it.

NEXT

Where this leads

- **Chapter 51, Data Activation**, sends results into business systems, where at-least-once delivery becomes idempotency keys against a CRM or ERP.
- **Chapter 52, The Cloud, Containers & Infrastructure as Code**, covers running a stream somewhere it can stay up: containers, restarts, secrets, and cost.
- **Chapter 47** applies to streams: freshness on the sink, quality checks on the corrected table, and alerting that doesn't cry wolf.
- **Chapter 49** is where the sink lives: table formats, compaction, and time travel.
- **Chapter 58, Intelligent Automation**, revisits the alerting decision with AI in the loop, and this chapter's rule stands: the design matters more than the model.
- **Chapter 61, Distributed Systems & Trade-offs**, explains what ordering and delivery guarantees cost when the log runs across many machines.
- **Chapter 62, Data Architecture Patterns**, names section 50.8's design, a fast streaming path plus a batch path that corrects it, as the Lambda architecture, and compares it with Kappa, which treats everything as a stream.
- **Part 8:** streaming questions appear in the data engineering interview chapters, and "design a real-time alerting system" is a common case in Chapter 77.

Chapter 51. Data Activation: Reverse ETL, APIs & System Integration

Chapter at a glance

You will learn to: explain why a number sitting in a warehouse or a dashboard changes nothing on its own · describe reverse ETL and the kinds of fields companies sync back into operational systems · write to a system through an API with authentication, upserts, pagination, rate limits, and retries · make a write **idempotent** with an idempotency key, and prove a retried write doesn't double up · choose a system of record and a conflict rule before two systems can disagree · trigger a webhook, receive the event within seconds instead of polling, and check its signature · compare integration patterns: point-to-point, hub-and-spoke, message queues, and iPaaS · decide between custom code, orchestrator tasks, low-code tools, and enterprise platforms · explain why RPA is fragile, and when file-based integration (SFTP, EDI) is still the right answer · build an audit trail and a reconciliation report for writes, not only reads.

Before you start: Chapter 45 (ingestion, idempotency, APIs), Chapter 46 (orchestration and delivery), Chapter 47 (contracts and incidents), and Chapters 12 and 13 (SQL, including window functions).

Time needed: 14–18 hours of reading and practice, spread over two to three weeks.

Tools: Python with `psycopg2` and `requests` (both already installed, in Chapters 45 and 18). PostgreSQL 16. The Chapter 51 companion folder, which includes a sandbox CRM and a webhook receiver, both running on your own computer.

Practice data: Riverstone's leads and their stage history, and Riverstone's customers and orders, from `riverstone_source`, a fresh copy of `riverstone_2025`. Every output shown is from a real run against the sandbox CRM.

In this chapter

START	Why this matters · In plain English	246
LEARN	51.0 Setting up the practice environment	247
	51.1 The loop nobody finishes	249
	51.2 Computing what to sync	250
	51.3 Writing to a system through an API	255
	51.4 Making writes idempotent, and proving it	260
	51.5 System of record and conflict rules	263
	51.6 Webhooks: pushing instead of polling	264
	51.7 Reconciliation for writes	267
	51.8 Integration patterns	270
	51.9 Choosing how to build it	271
	51.10 When there's no API: RPA and file drops	272
APPLY	Common mistakes · In the real world · Project	273
REVIEW	Recap · Key terms · Check yourself	276
PRACTISE	Exercises	278
	Answers	279
NEXT	Where this leads	281

START

Why this matters

Every chapter in Part 5 so far has moved data **toward** the warehouse: ingest it, clean it, model it, check it, store it well. That data now knows things nobody in the business has noticed yet: which leads are hot, which customers have quietly stopped ordering, which machine is drifting hot. None of that changes anything by sitting in a table.

Data activation is the last mile: pushing what the warehouse knows back into the systems where people actually work. A sales rep doesn't open a dashboard before every call; they open the CRM. A dispatch clerk doesn't run a query; they look at the ERP. If the insight isn't there, it doesn't exist for them.

This is also where the stakes change. A wrong number in a dashboard is embarrassing. A wrong number **written into the CRM** can change what a customer is charged, what a rep is told to say, or which orders get held. Chapter 47's severities apply here at their sharpest: a bad reverse-ETL sync is not a late report, it's data corruption in a system other people depend on. This chapter builds that sync properly, on Riverstone's own leads and orders, against a sandbox CRM that behaves like a real one: it needs a token, it fails sometimes, and it remembers what you've already told it.

In plain English

Think about a factory floor with a whiteboard and a computer system.

- The computer system **knows** which machines are running hot, from sensors nobody on the floor watches directly.
- Someone has to **write it on the whiteboard** the supervisor actually reads. That's activation.

- If the same warning gets written twice because someone's marker slipped, the supervisor might stop a machine that's actually fine. **Writing it exactly once, however many times you try**, is idempotency.
- The whiteboard and the computer system can **disagree**: if a supervisor wipes off a warning early and the system writes it back, whose word counts? That's the **system of record**.
- A runner who **rushes over the moment something changes**, instead of waiting for someone to walk past the board, is a **webhook**.
- If the computer system has no way to talk to the whiteboard at all, someone might have to **transcribe it by hand every hour**. That's what RPA and file drops stand in for, and why they're fragile: a hand copying a whiteboard makes mistakes a direct wire never would.

LEARN

51.0 Setting up the practice environment

This chapter writes into another system, so it needs something safe to write into. The companion folder `companion/ch51/` has everything, and all of it runs on your own computer:

File	What it does
<code>reset_ch51.py</code>	Creates <code>riverstone_source</code> , a fresh copy of <code>riverstone_2025</code> (as in Chapter 45), and empties the sandbox's storage. Run it any time to start again.
<code>crm_sandbox.py</code>	The sandbox CRM : a small local web server that behaves like a real CRM's API, for reading and for writing. It holds Riverstone's leads and customer accounts.
<code>webhook_receiver.py</code>	A small local server that receives webhooks (section 51.6).
<code>lead_scores.sql</code> , <code>follow_up_due.sql</code>	The two queries you'll read in section 51.2.
<code>sync_leads.py</code>	The chapter's computations and keys collected in one file, for the project.

Step 1. Your practice folder

As in Chapter 45, copy the folder `companion/ch51` and paste it inside `work`, so you have `work/ch51` to practise in.

Step 2. Nothing new to install

`psycpg2-binary` was installed in Chapter 45 (section 45.2), and `requests` and `python-dotenv` in Chapter 18. The other modules this chapter uses (`json`, `time`, `hashlib`, and `hmac`) come with Python.

Step 3. Settings in `.env`

Copy the `.env` file from `work/ch45` into `work/ch51`, so `RIVERSTONE_SOURCE` points at the same database, and add two lines:

```
CRM_API_TOKEN=practice-token-51
WEBHOOK_SECRET=practice-secret-51
```


- **CRM_API_TOKEN** is the token the CRM gave your integration, to prove who is calling (Chapter 45, section 45.9).
- **WEBHOOK_SECRET** is a secret shared between the CRM and your webhook receiver, used to check that a webhook really came from the CRM (section 51.6).

Both values here are the sandbox's public practice values. Real ones are secrets: they live in `.env` or a secrets store, never in code or in Git. Chapter 52 (section 52.6) shows every layer of that in production.

`reset()` connects to PostgreSQL's maintenance database, `postgres`, to copy the database. If your PostgreSQL needs a password (usual on Windows), add a third line with your own password, in the same form as Chapter 45's connection string: `RIVERSTONE_PG=dbname=postgres user=postgres password=your-password host=localhost`.

Step 4. Start the notebook

In VS Code, create a notebook `ch51.ipynb` in `work/ch51`, with the `.venv` kernel (Chapter 17, section 17.0). Run each code block in this chapter as its own cell, top to bottom, **without restarting the kernel**: later cells use names that earlier cells create, such as `scores` and `sync_record`. If you restart, run the cells again from the top.

The first cell:

```
import os, json, time, hashlib, hmac
import psycpg2, requests
from dotenv import load_dotenv

load_dotenv()
from reset_ch51 import reset
from crm_sandbox import seed_crm, simulate_failure, start_server as start_crm
from webhook_receiver import start_server as start_receiver, read_inbox

reset()
SRC = os.environ.get("RIVERSTONE_SOURCE", "dbname=riverstone_source")
print("riverstone_source ready")
```

```
riverstone_source ready
```

How it works, line by line.

- `json` turns Python objects into JSON text (Chapter 17), `time` waits and measures time, `hashlib` computes fingerprints (Chapter 45, section 45.5), and `hmac` signs messages with a secret (section 51.6).
- `load_dotenv()` copies `.env` into the environment. It comes before the companion imports, because they read their settings when they load.
- `from crm_sandbox import ... start_server as start_crm` imports a function under a new name. Both companion servers have a function called `start_server`; `as` gives each its own name, so the second import doesn't replace the first.
- `reset()` rebuilds `riverstone_source` from `riverstone_2025` and empties the sandbox's storage folders, so every run of this chapter starts from the same state.
- `SRC` is the connection string, exactly as in Chapter 45.

Now start the two servers and fill the sandbox CRM:

```
crm = start_crm()
receiver = start_receiver()
leads_n, accounts_n = seed_crm()
BASE = "http://127.0.0.1:8051/api"
TOKEN = os.environ.get("CRM_API_TOKEN", "practice-token-51") # practice default only
HEADERS = {"Authorization": f"Bearer {TOKEN}"}
print("CRM:", crm.server_address, " receiver:", receiver.server_address)
print("the CRM holds", leads_n, "leads and", accounts_n, "customer accounts")
```

```
CRM: ('127.0.0.1', 8051) receiver: ('127.0.0.1', 8052)
the CRM holds 43 leads and 23 customer accounts
```

- `start_crm()` and `start_receiver()` start the two servers in the background of the notebook, at ports 8051 and 8052 on this computer, as Chapter 45's practice API did at 8045.
- `seed_crm()` fills the sandbox from `riverstone_source`: all of Riverstone's leads, and an account for each customer except the newest, customer 24, who signed up in November and has no CRM account yet (section 51.4 creates it). It returns the two counts.
- `BASE` is the start of every address the CRM answers on. `TOKEN` comes from the environment; the default after it is the sandbox's public token, so the notebook runs even without `.env`. `HEADERS` sends it as a bearer token.

Tool note. The outputs in this chapter were produced with Python 3.11.15, requests 2.33.1, psycpg2 2.9.13, python-dotenv 1.2.3, and PostgreSQL 16.13.

51.1 The loop nobody finishes

Chapter 7 drew the full flow from a question to an answer. Here's the piece most companies build, and then stop before finishing.

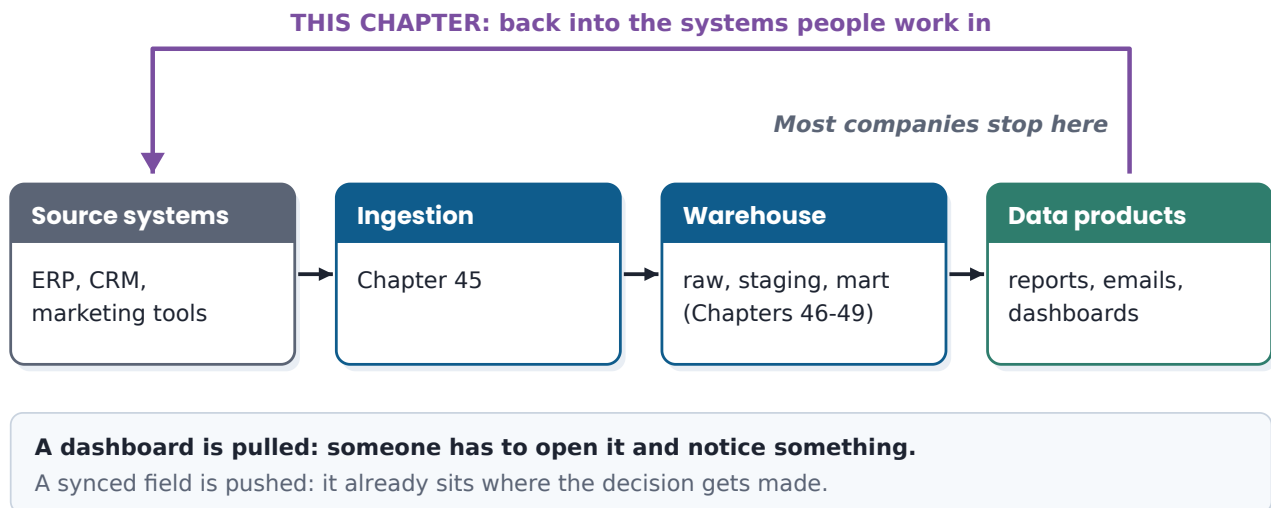


Figure 51.1 – Most data platforms stop at the dashboard. Activation closes the loop back into the systems people work in every day.

A **dashboard** is pull: someone has to open it, notice something, and act. A **synced field in the CRM** is push: it's sitting where the action already happens, next to the customer's name, when the rep is already looking. The difference in how often it gets used is not small.

Fields worth syncing back, and what they change:

Field	Computed from	Changes what
Lead score	Stage, source, recency (this chapter)	Which leads a rep calls first
Follow-up-due flag	Order dates and delivery status (this chapter)	Which quiet customers a rep calls this week
Customer health score	Order frequency, support tickets, payment history	Who account management checks in on
Credit limit or hold	Payment history, exposure	Whether an order can be placed at all
Reorder flag	Stock levels, usage rate	Whether a purchase order gets raised
Overdue-payment flag	Invoices and payments	Whether finance chases a customer this week
Churn risk	Usage decline, contract dates	Whether a renewal call gets scheduled

Every one of these is a number the warehouse can compute, once it has the data. The work in this chapter is getting it to land somewhere it changes a decision, safely.

51.2 Computing what to sync

Riverstone will sync two things into its CRM: a **lead score** on each lead, so reps call the right leads first, and a **follow-up-due flag** on each customer account, so reps call the customers who have gone quiet.

"Today" in this chapter. Throughout this chapter, today is **6 January 2026**, the same simulated calendar as Chapter 46: the nightly run starts at 06:30 IST that morning. Every date rule below counts back from that day, which the SQL writes as `DATE '2026-01-06'`, so every run of the chapter gives the same answer.

Lead scoring

The rule is deliberately simple and fully documented, because a score nobody can explain doesn't earn trust from the sales team:

score = points for the lead's current stage + points for how it arrived + a recency bonus if it entered that stage in the last 14 days, capped at 100. A Lost lead scores 0, whatever else is true.

Current stage	Points
New	10
Contacted	30
Quoted	55
Won	100
Lost	0, and the whole score is 0

Source	Points
Trade fair	20
Referral	15
Website	10
IndiaMART listing	10
Cold call	5

- **Recency:** +10 if the lead entered its current stage on or after 23 December 2025, 14 days before today.
- **Lost:** a lost lead should never rank above an active one, so it scores 0 whatever its source or recency.
- **The cap:** only a Won lead can pass 100 (100 plus its source points), so every Won lead scores exactly 100. The highest score for a lead still in play is Quoted 55 + Trade fair 20 + recency 10 = 85.

By hand first. To score a lead you need its source and its current stage, with the date it entered that stage. The table `lead_stage_history` has one row for each stage a lead has entered. This query, run in `psql` or `DBeaver` against `riverstone_source` (Chapter 12), picks each lead's newest row:

```
WITH latest AS (
  SELECT lead_id, stage, entered_at,
         ROW_NUMBER() OVER (PARTITION BY lead_id ORDER BY entered_at DESC) AS rn
  FROM lead_stage_history
)
SELECT l.lead_id, l.company_name, l.source, s.stage, s.entered_at::date AS stage_entered
FROM leads AS l
JOIN latest AS s ON s.lead_id = l.lead_id AND s.rn = 1
WHERE l.lead_id <= 7
ORDER BY l.lead_id;
```

lead_id	company_name	source	stage	stage_entered
1	Anand Stores	Referral	Contacted	2025-04-13
2	Bright Kitchens	Website	Quoted	2025-06-29
3	Bright Kitchens	Website	New	2025-06-12
4	BRIGHT KITCHENS	Website	New	2025-06-12
5	Cafe Mocha Lane	Website	New	2025-01-02
6	Delta Hospitality	Cold call	Lost	2025-05-27
7	Elite Mart	Referral	Won	2025-07-23

(7 rows)

- `ROW_NUMBER() OVER (PARTITION BY lead_id ORDER BY entered_at DESC)` numbers each lead's stages from the newest (Chapter 13), so `rn = 1` is the stage the lead is in now.
- `::date` keeps only the date part of `entered_at` (Chapter 12's cast shorthand, section 12.8).
- `WHERE l.lead_id <= 7` keeps the printout short; the real query scores every lead.

Three of these, by hand:

Lead	Stage	Source	Recency	Score
2, Bright Kitchens	Quoted 55	Website 10	0 (entered 29 June 2025)	55 + 10 + 0 = 65
6, Delta Hospitality	Lost	(Cold call 5)	0	Lost, so 0
7, Elite Mart	Won 100	Referral 15	0	115, capped to 100

No lead in this data entered its stage on or after 23 December (the latest date is 4 December), so nobody gets the recency bonus today. Exercise 4 has a lead that does.

The same rule in SQL. The file `lead_scores.sql` in your practice folder turns the two tables and the three rules into SQL. The next Python cell runs it.

```

WITH latest AS (
    SELECT lead_id, stage, entered_at,
           ROW_NUMBER() OVER (PARTITION BY lead_id ORDER BY entered_at DESC) AS rn
    FROM lead_stage_history
),
points AS (
    SELECT l.lead_id, l.company_name, l.source, s.stage,
           CASE s.stage WHEN 'New' THEN 10 WHEN 'Contacted' THEN 30
                    WHEN 'Quoted' THEN 55 WHEN 'Won' THEN 100 ELSE 0 END AS stage_points,
           CASE l.source WHEN 'Trade fair' THEN 20 WHEN 'Referral' THEN 15
                    WHEN 'Website' THEN 10 WHEN 'IndiaMART listing' THEN 10
                    WHEN 'Cold call' THEN 5 ELSE 0 END AS source_points,
           CASE WHEN s.entered_at >= DATE '2026-01-06' - 14 THEN 10 ELSE 0 END AS recency_points
    FROM leads AS l
    JOIN latest AS s ON s.lead_id = l.lead_id AND s.rn = 1
)
SELECT lead_id, company_name, source, stage,
       CASE WHEN stage = 'Lost' THEN 0
            ELSE LEAST(stage_points + source_points + recency_points, 100) END AS score
FROM points
ORDER BY lead_id;

```

- `latest` is the query you just ran, without the `WHERE`.
- `points` works out the three parts, one column each, with `CASE` (Chapter 12). `CASE s.stage WHEN 'New' THEN 10 ...` is the short form of `CASE WHEN s.stage = 'New' THEN 10 ...`: it compares one column with each value in turn. A stage or source not in the table gets `ELSE 0`.
- `DATE '2026-01-06' - 14` subtracts 14 days from a date: 23 December 2025, the start of the recency window.
- The last `SELECT` applies the two overrides. `LEAST(a, b)` returns the smaller of its values, so `LEAST(total, 100)` is the cap. The outer `CASE` makes a Lost lead 0 before anything else.

Now run it from Python and keep the scores:

```

def fetch_rows(sql, params=None):
    conn = psycopg2.connect(SRC)
    with conn, conn.cursor() as cur:
        cur.execute(sql, params)
        rows = cur.fetchall()
    conn.close()
    return rows

with open("lead_scores.sql") as f:
    score_rows = fetch_rows(f.read())
scores = {str(lead_id): {"company_name": name, "stage": stage, "score": score}
          for lead_id, name, source, stage, score in score_rows}
print(len(scores), "leads scored")
for lead_id in ["2", "6", "7"]:
    print(lead_id, scores[lead_id])

```

```

43 leads scored
2 {'company_name': 'Bright Kitchens', 'stage': 'Quoted', 'score': 65}
6 {'company_name': 'Delta Hospitality', 'stage': 'Lost', 'score': 0}
7 {'company_name': 'Elite Mart', 'stage': 'Won', 'score': 100}

```

- `fetch_rows()` is a smaller version of Chapter 45's helper: it opens a connection, runs one query, and returns the rows as a list of tuples. Its second argument, `params`, would fill `%s` placeholders in

the query with values, as in Chapter 45; this chapter's queries have none, so it stays `None`.

- with `open("lead_scores.sql") as f`: opens the file, and `f.read()` reads it as one string: the query.
- The dictionary comprehension (Chapter 17) makes one entry per row. `for lead_id, name, source, stage, score in score_rows` unpacks each row's five columns into five names. The key is `str(lead_id)`, text, because the CRM's addresses and JSON use text ids.

Reading it. All 43 leads are scored, and the three agree with the table worked by hand: 65, 0, and 100. Check a computed field by hand like this every time: a score nobody can rebuild from its own rule is a score nobody should trust, however tidy the code looks.

Simplification note. A learned lead-scoring model would use more signals, such as the number of contacts, the deal size, and what happened to similar past leads. That's a classification model, the kind Part 4 built on Riverstone's CRM leads (Chapters 35 to 39); its predicted probability could be synced with exactly this chapter's mechanics. This score is deliberately transparent: every point traces to a rule a sales manager can read in one sentence, which matters more early on than a cleverer model nobody can explain.

The follow-up-due flag

Riverstone's finance system tracks invoices and payments separately, and this book doesn't have that data, so the chapter doesn't pretend to compute an overdue-payment flag. It syncs a flag it can compute honestly from orders:

A customer is **follow-up due** if they ordered in the last six months (since 6 July 2025) but have had **no delivered order in the last 45 days** (since 22 November 2025).

This is a recency signal, a customer who has gone quiet, not a payment status. A customer who owes money but took a delivery last week isn't flagged. Any real deployment should replace or sharpen it with its owner's actual definition, agreed as a Chapter 47 data contract. The query is in `follow_up_due.sql`:

```
SELECT c.customer_id, c.customer_name,
       MAX(o.order_date) AS last_order,
       MAX(CASE WHEN o.status = 'Delivered' THEN o.order_date END) AS last_delivered
FROM customers AS c
JOIN orders AS o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING MAX(o.order_date) >= DATE '2026-01-06' - INTERVAL '6 months'
      AND COALESCE(MAX(CASE WHEN o.status = 'Delivered' THEN o.order_date END), DATE '1900-01-01')
      < DATE '2026-01-06' - 45
ORDER BY c.customer_id;
```

customer_id	customer_name	last_order	last_delivered
2	Patel Kitchenware	2025-11-09	2025-11-09
8	Blue Bay Cafe	2025-11-09	2025-11-09
9	Lakeview Resorts	2025-12-21	2025-10-15
13	Om Sai Provisions	2025-07-22	2025-07-22
20	Tasty Tiffins	2025-10-26	2025-10-26
22	Festive Gifts Co	2025-11-19	2025-11-19

(6 rows)

- `MAX(CASE WHEN o.status = 'Delivered' THEN o.order_date END)` is the date of the latest delivered order. A `CASE` with no `ELSE` gives `NULL` for other orders, and `MAX` ignores `NULL`s.
- `HAVING` keeps the groups, one per customer, that pass both tests (Chapter 12). `INTERVAL '6 months'` is Chapter 13's date arithmetic.
- `COALESCE(..., DATE '1900-01-01')` turns "never had a delivery" into a very old date, so such a customer is flagged too, instead of silently dropping out as `NULL`.

Reading it. Six customers are follow-up due. Look at Lakeview Resorts: it placed an order on 21 December, but that order is still Pending, and its last delivery was 15 October, so it's flagged. Should an order on its way count? The rule says no. Whether that's right is a question for the flag's owner, which is exactly why the definition belongs in a contract, not only in code.

The sync needs a value for **every** customer, `True` or `False`, so that a customer who orders again gets the flag cleared:

```
with open("follow_up_due.sql") as f:
    due = {str(row[0]) for row in fetch_rows(f.read())}
customer_ids = {str(row[0]) for row in fetch_rows("SELECT customer_id FROM customers ORDER BY
    customer_id")}
follow_up = {cid: cid in due for cid in customer_ids}
print(len(follow_up), "customers,", sum(follow_up.values()), "follow-up due:", sorted(due, key=int))
```

```
24 customers, 6 follow-up due: ['2', '8', '9', '13', '20', '22']
```

- `{str(row[0]) for row in ...}` is a **set** comprehension: the shape of Chapter 17's dictionary comprehension, but with no `key: value`, so it builds a set (Chapter 17), where each id appears once. `row[0]` is the first column, the customer id.
- `cid in due` is `True` or `False`, so `follow_up` maps every customer to its flag.
- `sum()` of booleans counts the `Trues`, because `True` counts as 1. `sorted(due, key=int)` sorts the ids as numbers, so "13" comes after "9".

51.3 Writing to a system through an API

Chapter 45 read from a CRM's API. Writing is the same shape with sharper edges: a bad read wastes a query; a bad write changes someone else's system.

Authentication for writes

Most write APIs use the same authentication as reads (Chapter 45's bearer tokens), but with a tighter **scope** (the list of things a token is allowed to do): a read scope might cover the whole CRM, while a write scope might be limited to a few fields or objects. **OAuth 2.0** is the standard for delegated write access: your application gets a short-lived **access token** after a user, or a **service account** (a login that belongs to an application rather than a person), grants permission, and it refreshes the token before it expires, rather than holding one permanent password-like key forever. The sandbox CRM uses a fixed bearer token to keep the examples short; production integrations almost always use OAuth for anything that writes.

Four HTTP methods for writing

Chapter 2 introduced requests, Chapter 18 sent GET requests, and Chapter 20 sent a POST to a chat channel's webhook. Writing uses four methods, and the difference between two of them causes real damage:

Method	What it asks the system to do
GET	Read a record. Changes nothing.
POST	Create a new record.
PUT	Replace the whole record with what you send. Any field you leave out is gone.
PATCH	Change only the fields you send. Everything else stays as it was.

See the difference on lead 1. First, read it:

```
lead = requests.get(f"{BASE}/leads/1", headers=HEADERS, timeout=10).json()
print(lead)
```

```
{'lead_id': '1', 'company_name': 'Anand Stores', 'email': 'anandstores@example.com', 'source': 'Referral',
  'owner_id': 3}
```

Now send only a score, with PUT, and read the lead again:

```
resp = requests.put(f"{BASE}/leads/1", json={"score": 45}, headers=HEADERS, timeout=10)
print(resp.status_code, resp.json())
print(requests.get(f"{BASE}/leads/1", headers=HEADERS, timeout=10).json())
```

```
200 {'record': 'leads/1', 'action': 'replaced'}
{'lead_id': '1', 'score': 45}
```

- `requests.put(url, json=...)` sends a PUT with a JSON body. `json=` turns the dictionary into JSON text and tells the server so.

Reading it. The CRM did exactly what PUT means: the record is now the id plus a score. The company name, the email, the source, and the owner are gone. Put the sandbox back, and try the same change with PATCH:

```
seed_crm()                                # put the sandbox back as it was
resp = requests.patch(f"{BASE}/leads/1", json={"score": 45}, headers=HEADERS, timeout=10)
print(resp.status_code, resp.json())
print(requests.get(f"{BASE}/leads/1", headers=HEADERS, timeout=10).json())
```

```
200 {'record': 'leads/1', 'action': 'updated', 'fields': ['score'], 'replayed': False}
{'lead_id': '1', 'company_name': 'Anand Stores', 'email': 'anandstores@example.com', 'source': 'Referral',
 'owner_id': 3, 'score': 45}
```

Reading it. PATCH changed one field, score, and left everything else alone. A reverse-ETL sync should almost always use PATCH, and name only the fields it owns. This chapter's closing story is what happens when it doesn't.

The idempotency key

A sync will sometimes send the same write twice: a retry after a timeout, or a whole run started again. The protection is an **idempotency key**: a short text sent with each write, in a header called **Idempotency-Key**. The CRM remembers every key it has applied. If a write arrives with a key it has already seen, it changes nothing and sends back its earlier answer, marked **"replayed": True**.

So the key must be **the same whenever the write is the same**. Build it from the record and the values being written:

```
def idempotency_key(record, payload):
    content = record + ":" + json.dumps(payload, sort_keys=True)
    return hashlib.sha256(content.encode()).hexdigest()[:16]

print(idempotency_key("leads/2", {"score": 65, "stage": "Quoted"}))
print(idempotency_key("leads/2", {"stage": "Quoted", "score": 65}))
print(idempotency_key("leads/2", {"score": 66, "stage": "Quoted"}))
```

```
d6f8907536e7b40d
d6f8907536e7b40d
50bbc6a7d7d61904
```

- **record** is the record's address in the CRM, such as **"leads/2"**, so two records with the same values get different keys.
- **json.dumps(payload, sort_keys=True)** turns the values into text with the keys in alphabetical order. The same dictionary always gives the same text, whatever order it was built in: that's why the first two keys match.
- **hashlib.sha256(...).hexdigest()** is the SHA-256 fingerprint from Chapter 45 (section 45.5); **.encode()** turns the text into bytes first, as there. **[:16]** keeps the first 16 of its 64 characters: plenty to tell one write from another, and shorter in headers and logs.

Reading it. The same values give the same key, in any order; a score of 66 instead of 65 gives a completely different one.

One write, step by step

Build the first write, for lead 1:

```
payload = {"score": scores["1"]["score"], "stage": scores["1"]["stage"]}
RUN_TIME = "2026-01-06T06:30:00+05:30"
body = {**payload, "score_updated_at": RUN_TIME}
key = idempotency_key("leads/1", payload)
print(body)
print(**HEADERS, "Idempotency-Key": key)
```

```
{'score': 45, 'stage': 'Contacted', 'score_updated_at': '2026-01-06T06:30:00+05:30'}
{'Authorization': 'Bearer practice-token-51', 'Idempotency-Key': 'b5564a8c80e53ded'}
```

- `payload` holds the values the sync owns: the score and the stage it was computed from.
- `{**payload, "score_updated_at": RUN_TIME}` builds a new dictionary: `**payload` copies every key and value of `payload` into it, and then one more key is added. `payload` itself is unchanged.
- `score_updated_at` tells a rep in the CRM when the score was last calculated: the run at 06:30 IST on 6 January. The key is built from `payload`, **not** from `body`: the time says when the run happened, not what was written, and a key that includes a time changes every time (section 51.4 comes back to this).
- The last line shows the headers the write will carry: the token, plus the key.

The function that sends one write, and retries only what might succeed next time:

```
TIMEOUT = 2 # seconds; the sandbox answers in a few milliseconds
RETRY_STATUSES = {429, 503}

def sync_record(record, body, key, max_attempts=3):
    headers = {**HEADERS, "Idempotency-Key": key}
    for attempt in range(1, max_attempts + 1):
        try:
            resp = requests.patch(f"{BASE}/{record}", json=body, headers=headers, timeout=TIMEOUT)
        except (requests.Timeout, requests.ConnectionError) as e:
            problem, wait = type(e).__name__, 2 ** (attempt - 1)
        else:
            if resp.status_code not in RETRY_STATUSES:
                resp.raise_for_status()
                return resp.json()
            problem = f"{resp.status_code} {resp.reason}"
            wait = int(resp.headers.get("Retry-After", 2 ** (attempt - 1)))
        if attempt < max_attempts:
            print(f" {record}: {problem}, waiting {wait}s (attempt {attempt})")
            time.sleep(wait)
        raise RuntimeError(f"{record}: gave up after {max_attempts} attempts (last: {problem})")

print(sync_record("leads/1", body, key))
```

```
{'record': 'leads/1', 'action': 'updated', 'fields': ['score', 'score_updated_at', 'stage'], 'replayed': False}
```

How it works, line by line. The loop is Chapter 45's `get_with_retry()` (section 45.9), turned into a write:

- `requests.patch(f"{BASE}/{record}", json=body, ...)` sends a PATCH to the record's address, such as `.../api/leads/1`. A part of the address that names one record is called a **path parameter**.
- The **key arrives from outside** the function, and the headers are built once, before the loop, so every attempt sends the same key. That's the whole point: a retry must look like the same write.
- **Timeouts and dropped connections** (`requests.Timeout`, `requests.ConnectionError`) are retried, with waits of 1, then 2 seconds. `TIMEOUT` is short because the sandbox is on your own computer; Chapter 45 used 10 seconds for a real API.
- **429 Too Many Requests** and **503 Service Unavailable** are retried, waiting for `Retry-After` seconds when the CRM sends that header. Any other error, such as 401 or 404, goes to `raise_for_status()` and stops at once: retrying won't fix it.
- The message is printed only when another attempt follows (`attempt < max_attempts`); after the last attempt the function raises a clear error instead.
- The answer, `resp.json()`, says which record changed, whether it was updated or created, which fields, and `"replayed": False`: this write was new.

Now the first nightly sync of every lead. The sandbox has one more piece of **test scaffolding**, `simulate_failure()`, which makes the CRM fail one write on purpose, so you can watch the retry work. A real CRM doesn't have it.

```
simulate_failure("leads/4", "429")      # the CRM will refuse lead 4's next write once
outcomes = {"new": 0, "replayed": 0}
for lead_id, s in scores.items():
    payload = {"score": s["score"], "stage": s["stage"]}
    key = idempotency_key(f"leads/{lead_id}", payload)
    result = sync_record(f"leads/{lead_id}", {**payload, "score_updated_at": RUN_TIME}, key)
    if result["replayed"]:
        outcomes["replayed"] += 1
    else:
        outcomes["new"] += 1
print(outcomes)
```

```
leads/4: 429 Too Many Requests, waiting 1s (attempt 1)
{'new': 42, 'replayed': 1}
```

- The loop goes through every scored lead, builds its payload and key, and sends it.
- `outcomes` counts how many writes were new and how many the CRM recognized as repeats.

Reading it. Lead 4's first attempt was refused with 429; the loop waited the one second the CRM asked for and tried again with the same key, and it went through. 42 writes were new. One was a replay: lead 1, whose identical write the previous cell had already sent. The CRM recognized the key and changed nothing.

51.4 Making writes idempotent, and proving it

Say it as plainly as Chapter 45 did for loads: **assume a write can happen more than once, and design for that on purpose**. A network timeout doesn't tell you whether the request arrived; retrying is correct, and correctness depends on the retry being harmless.

Upserts: creating a record that isn't there yet

The follow-up flags go on customer **accounts**. Customer 24, Prime Wholesale, has no CRM account yet, so an ordinary update would fail with 404. The sandbox's accounts address works as an **upsert**: a PATCH to `accounts/<ERP customer id>` updates the account with that id, or creates it if there isn't one. It's Chapter 45's `INSERT ... ON CONFLICT ... DO UPDATE` (section 45.5), done through an API. Many CRM APIs offer the same thing, often called an upsert by **external id**: an id that belongs to another system, here the ERP's customer id.

```
payload = {"follow_up_due": follow_up["24"]}
print(sync_record("accounts/24", payload, idempotency_key("accounts/24", payload)))
print(requests.get(f"{BASE}/accounts/24", headers=HEADERS, timeout=TIMEOUT).json())
```

```
{'record': 'accounts/24', 'action': 'created', 'fields': ['follow_up_due'], 'replayed': False}
{'customer_id': '24', 'follow_up_due': False}
```

Reading it. `"action": "created"`: the account didn't exist, so the upsert made it, with the flag. Sent again, the same write would be a replay, and the account would stay as it is. An upsert is safe to repeat in the same way Chapter 45's SQL upsert was.

A write that times out

The hardest failure is the one where you don't know what happened. Make the CRM apply the next write to account 9, Lakeview Resorts, and then answer too late:

```
simulate_failure("accounts/9", "slow") # the CRM will apply the next write, then answer 3 s late
payload = {"follow_up_due": follow_up["9"]}
print(sync_record("accounts/9", payload, idempotency_key("accounts/9", payload)))
```

```
accounts/9: ReadTimeout, waiting 1s (attempt 1)
{'record': 'accounts/9', 'action': 'updated', 'fields': ['follow_up_due'], 'replayed': True}
```

Reading it. After 2 seconds with no answer, `requests` raised `ReadTimeout`, and the function waited a second and tried again. From the notebook's side, the first attempt might have failed on the way there or on the way back; there's no way to tell. In fact it had arrived and been applied. The retry carried the **same key**, so the CRM answered `"replayed": True` and applied nothing twice. For a flag, a double write would be harmless; for "add a ₹500 credit to this account", it would cost money. The key makes both cases safe, and the caller's code doesn't need to know which failure happened. (What if you change `TIMEOUT` to 5 seconds? The CRM's answer, 3 seconds late, would arrive in time, and there'd be no retry at all. A timeout shorter than the other system's slowest normal answer turns slow replies into retries.)

Now sync every account's flag:

```
outcomes = {"new": 0, "replayed": 0}
for cid, due_flag in follow_up.items():
    payload = {"follow_up_due": due_flag}
    result = sync_record(f"accounts/{cid}", payload, idempotency_key(f"accounts/{cid}", payload))
    if result["replayed"]:
        outcomes["replayed"] += 1
    else:
        outcomes["new"] += 1
print(outcomes)
```

```
{'new': 22, 'replayed': 2}
```

Reading it. 24 accounts: 22 new writes, and two replays, accounts 24 and 9, which the two cells before had already written.

A write that's genuinely repeated

Now lead 4. Send its current values again, then a changed value, then the changed value again:

```
payload = {"score": scores["4"]["score"], "stage": scores["4"]["stage"]}
print("same values again:", sync_record("leads/4", payload, idempotency_key("leads/4", payload)))

changed = {**payload, "score": payload["score"] + 1}
print("a real change      :", sync_record("leads/4", changed, idempotency_key("leads/4", changed)))
print("that change again:", sync_record("leads/4", changed, idempotency_key("leads/4", changed)))
```

```
same values again: {'record': 'leads/4', 'action': 'updated', 'fields': ['score', 'score_updated_at', 'stage'],
                    'replayed': True}
a real change      : {'record': 'leads/4', 'action': 'updated', 'fields': ['score', 'stage'], 'replayed': False}
that change again: {'record': 'leads/4', 'action': 'updated', 'fields': ['score', 'stage'], 'replayed': True}
```

- changed copies payload and replaces one value: the score plus 1. It stands for a genuine change, such as tomorrow's score after the lead moves stage.

Reading it. The first write matches the nightly sync's key exactly, so the CRM replayed it and did nothing. The changed score has a new key, so it was applied (`"replayed": False`). Sending that again was a replay. So far, the key does exactly what it should.

When a value goes back

Now suppose the score returns to what it was, 20. The warehouse sends 20 again:

```
print("back to 20:", sync_record("leads/4", payload, idempotency_key("leads/4", payload)))
print("the CRM now says:", requests.get(f"{BASE}/leads/4", headers=HEADERS, timeout=TIMEOUT).json()
      ["score"])
```

```
back to 20: {'record': 'leads/4', 'action': 'updated', 'fields': ['score', 'score_updated_at', 'stage'],
              'replayed': True}
the CRM now says: 21
```

Reading it. This is a real bug. The key for "lead 4, score 20, stage New" was used in the first sync, so the CRM treats the new write as a repeat and ignores it. The warehouse says 20, the CRM says 21, and **every future sync with this key will be ignored too**: a value that goes back to an earlier value can never be sent again. A key built from content alone can't tell "the same write, retried" from "the same values, written again on purpose later".

The fix: put the run in the key

What should count as "the same write" is: the same record, the same values, **in the same run**. Add the run's id to the key. Here the run id is the day the scheduled run is for, like a Chapter 46 partition key:

```
def run_key(record, payload, run_id):
    content = record + ":" + run_id + ":" + json.dumps(payload, sort_keys=True)
    return hashlib.sha256(content.encode()).hexdigest()[:16]

print(run_key("leads/4", payload, "2026-01-06"))
print(run_key("leads/4", payload, "2026-01-06"))
print(run_key("leads/4", payload, "2026-01-07"))
```

```
a5d6feaec99c52c8
a5d6feaec99c52c8
67329d74f025d291
```

Reading it. Within one run, every retry of a write has the same key, so retries are still harmless. Tomorrow's run has a new key, so tomorrow's 20 is applied, even though 20 was sent before. Section 51.7 runs "tomorrow" with this key and checks the result.

Real APIs also **forget** keys after a while. Stripe's API reference, for example, says its keys may be removed once they are at least 24 hours old (checked 29 September 2026). That's one more reason not to depend on a content-only key: whether a revert is ignored would depend on how long the other system remembers.

Watch out: an idempotency key must stay the same across every retry of one operation. Create it once, before the first attempt, and reuse it, as `sync_record()` does. A key built inside the retry loop from "now" or from a fresh random value changes on every attempt and protects nothing. Building it from the record, the values, and the run id guarantees that, and makes it reproducible.

Choosing what goes in the key

Built from	Behaviour	Use when
Record + values + run id (recommended, this chapter)	A retry within a run is a no-op; each new run can apply any value, including one sent before	Most reverse-ETL syncs
Record + values only	A retry is a no-op, but a value that goes back to an earlier one is never sent again (section 51.4)	Only if the receiver forgets keys quickly, and you know how quickly

Built from	Behaviour	Use when
Record + run id	One write per record per run, even if the values changed during the run	You want exactly "one write per scheduled run"
A random id (a UUID) created once per operation, and stored with it	Two identical operations are both applied; a retry of either is not	Payments and order creation, where the same content can legitimately be sent twice (two identical orders). Stripe's API reference suggests this pattern (V4 UUIDs)

51.5 System of record and conflict rules

Two systems holding the same fact will eventually disagree. Decide the rule **before** that happens, not while a customer is on the phone.

A **system of record** is the system whose value wins when there's a conflict (Chapter 45, section 45.1). For Riverstone:

Field	System of record	Why
Order status, quantities	The ERP	It's where the transaction happens
Lead stage, contact history	The CRM	Reps update it directly; the warehouse only reads it
Lead score, follow-up flag	The warehouse , written into the CRM	They're computed, not entered by a person; the CRM should never let someone hand-edit them and expect it to stick
Negotiated discount	The CRM , read into the warehouse	A person agreed it; a pipeline must never silently overwrite a human decision

The last two rows are the trap. A field can be **computed downstream** (safe to overwrite from the warehouse every run) or **entered upstream by a person** (must never be overwritten without an explicit, agreed process). Confusing the two is the single most common cause of the "the sync erased something" incident in this chapter's closing story.

A minimal conflict policy, written down before the first sync runs:

1. Name the system of record for every synced field, individually: not "the CRM is the system of record" as a blanket rule.
2. For computed fields, the pipeline always wins; document it as such in the field's description in the CRM itself, so a rep sees "auto-calculated, updates nightly" (and `score_updated_at`) rather than guessing.
3. For fields a person can edit, the sync either **never writes them** or writes only when the field is currently empty, and logs every write for review.
4. Any exception needs a name attached to the decision, and a date.

51.6 Webhooks: pushing instead of polling

Everything so far runs on a schedule. Some events are worth acting on **the moment they happen**: this chapter's project is a high-value lead that shouldn't sit unnoticed until tomorrow's sync.

A **webhook** (Chapter 2, section 2.8: "an API in reverse") is the receiving side of an event: instead of you asking "anything new?" every minute, the other system calls **you**, with the event, the moment it occurs. Chapter 20 posted to one, a chat channel's incoming webhook; here you receive them. It's the same idea as Chapter 50's log, without the log: a single HTTP push instead of a stream to poll.

First, tell the CRM where to send the event:

```
resp = requests.post(f"{BASE}/webhooks", headers=HEADERS, timeout=TIMEOUT,
                    json={"url": "http://127.0.0.1:8052/", "event": "lead.created_high_value"})
print(resp.status_code, resp.json())
```

```
201 {'registered': {'url': 'http://127.0.0.1:8052/', 'event': 'lead.created_high_value'}}
```

POST /api/webhooks **registers** an address and the event that should trigger it. Real webhook providers work the same way: you tell them where to send things, usually in a settings screen or through their API. 201 means "created": the registration exists.

Now a rep creates a high-value lead in the CRM. The sandbox fires `lead.created_high_value` when a new lead's `deal_value` is ₹2,00,000 or more:

```
t0 = time.time()
new_lead = requests.post(f"{BASE}/leads", headers=HEADERS, timeout=TIMEOUT, json={
    "company_name": "Grand Horizon Hotels", "email": "procurement@grandhorizon.example",
    "source": "Trade fair", "deal_value": 350000}).json()
print("lead created:", new_lead["lead_id"], new_lead["company_name"])

for _ in range(20):
    # wait up to 2 seconds, checking every 0.1 s
    if read_inbox():
        break
    time.sleep(0.1)
print("webhook arrived in under a second:", time.time() - t0 < 1)
print("event:", read_inbox()[0]["body"])
```

```
lead created: 9001 Grand Horizon Hotels
webhook arrived in under a second: True
event: {'event_id': 'evt_1', 'event': 'lead.created_high_value', 'lead': {'lead_id': '9001', 'company_name':
    'Grand Horizon Hotels', 'email': 'procurement@grandhorizon.example', 'source': 'Trade fair', 'deal_value':
    350000}}
```

How it works, line by line.

- `time.time()` is the current time in seconds, so `time.time() - t0` is how long everything after `t0` took.
- `POST /api/leads` creates the lead. Its `deal_value` of ₹3,50,000 is over the threshold, so the CRM calls the registered address.
- The `for` loop checks the receiver's inbox up to 20 times, 0.1 seconds apart, and stops as soon as something is there: a small, local version of polling, used only to measure the webhook.
- `read_inbox()[0]["body"]` is the first event received. It carries its own id, `evt_1`, the event's name, and the new lead.

Reading it. The event arrived within a second; in fact, it was there before the CRM had even answered the `POST`, because the sandbox calls the webhook **inside** its own request to keep the demo simple. A real provider puts the event in a queue and sends it separately, typically within seconds. Compare that with polling the CRM's leads every five minutes: the same lead could sit unseen for most of that window.

Making webhook receivers safe

A webhook receiver is a small public API, and it inherits every lesson from Chapters 45 and 46:

- **Verify the sender.** Anyone who finds the address can send it a request. Real providers sign each request (usually an HMAC signature in a header); check it before trusting the body.
- **Respond fast, then process.** Acknowledge receipt immediately and do the real work afterwards (from a queue, or a folder, as here), because a slow response makes the sender retry, and you'll receive the same event twice. (The sandbox itself breaks this rule when it calls the webhook inside its own request; a real provider doesn't.)
- **Expect duplicates.** Providers document that webhooks are **at-least-once** (Chapter 50's term, exactly): de-duplicate by the event's own id.
- **Handle being unreachable.** If your receiver is down, does the sender retry, and for how long? Know the answer before you rely on it, and keep a scheduled reconciliation sync as a safety net under any webhook.

Signatures: HMAC

An **HMAC** (hash-based message authentication code) is a hash of the message mixed with a secret that only the sender and the receiver know. The CRM computes it from the body it sends and puts it in a header, `X-Signature`. The receiver computes it again from the body it received. If they match, the body came from someone who knows the secret and wasn't changed on the way; a forger without the secret can't produce the right value.


```
SECRET = os.environ.get("WEBHOOK_SECRET", "practice-secret-51").encode()
body = b'{"event_id": "evt_100", "event": "test"}'
signature = hmac.new(SECRET, body, hashlib.sha256).hexdigest()
print("genuine :", signature[:16])
forged = b'{"event_id": "evt_100", "event": "test!"}'
print("forged  :", hmac.new(SECRET, forged, hashlib.sha256).hexdigest()[:16])
print("same?   :", hmac.compare_digest(signature, hmac.new(SECRET, body,
    hashlib.sha256).hexdigest()))
```

```
genuine : 4f679cd25f9d90b7
forged  : 1d3d91ecca2d9904
same?   : True
```

- `b'...'` is a **bytes** value, the raw form a request body travels in. `.encode()` turns the secret's text into bytes too.
- `hmac.new(secret, message, hashlib.sha256)` mixes the two with SHA-256; `.hexdigest()` gives the result as text. The printout shows the first 16 characters.
- One extra character in the body gives a completely different signature.
- `hmac.compare_digest(a, b)` compares two signatures. Use it instead of `==`: it takes the same time whether the first character differs or the last, so an attacker can't guess a signature one character at a time by timing the answers.

The receiver in `webhook_receiver.py` does all four things from the list above. Here is its request handler. You don't run this cell; it's inside the receiver, which the notebook started in section 51.0:

```
def do_POST(self):
    body = self.rfile.read(int(self.headers.get("Content-Length", 0)))
    expected = hmac.new(SECRET, body, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(expected, self.headers.get("X-Signature", "")):
        return self.reply(401, {"ok": False, "error": "bad signature"})
    event = json.loads(body)
    if event["event_id"] in Handler.seen:
        return self.reply(200, {"ok": True, "duplicate": True})
    Handler.seen.add(event["event_id"])
    save_to_inbox(event)
    return self.reply(200, {"ok": True})
```

- `self.rfile.read(...)` reads the raw body, exactly `Content-Length` bytes. The signature is checked on these bytes, **before** anything else, and a mismatch gets **401** with nothing kept.
- `json.loads(body)` turns the checked body into a dictionary. `Handler.seen` is a set of event ids already received; a repeat is answered 200, so the sender stops retrying, and skipped.
- `save_to_inbox()` writes the event to the inbox folder, and the handler answers 200 at once. Any slow work happens later, from the inbox.

Now send it the genuine request, the same request again, and the forged body with the genuine signature:

```
signed = {"Content-Type": "application/json", "X-Signature": signature}
for label, data in [("genuine", body), ("same again", body), ("forged", forged)]:
    r = requests.post("http://127.0.0.1:8052/", data=data, headers=signed, timeout=TIMEOUT)
    print(label, r.status_code, r.json())
print("events in the inbox:", len(read_inbox()))
```

```
genuine 200 {'ok': True}
same again 200 {'ok': True, 'duplicate': True}
forged 401 {'ok': False, 'error': 'bad signature'}
events in the inbox: 2
```

- **data=** sends the bytes exactly as they are. (**json=** would turn a dictionary into new bytes, and the signature must match the bytes that were signed.)

Reading it. The genuine event was accepted, its repeat was acknowledged as a duplicate and skipped, and the forged body was refused with 401. The inbox holds two events: Grand Horizon Hotels' evt_1 and the test's evt_100.

51.7 Reconciliation for writes

Chapter 47 reconciled what a pipeline **read**. The same discipline applies to what it **writes**, and it's skipped just as often. Reconciling a write means reading the destination back and comparing it with what the warehouse says it should hold.

First, read everything back, page by page:

```
def read_all(kind, id_field, page_size=20):
    records, page, pages = [], 1, 0
    while page is not None:
        resp = requests.get(f"{BASE}/{kind}", params={"page": page, "page_size": page_size},
                           headers=HEADERS, timeout=TIMEOUT)
        resp.raise_for_status()
        body = resp.json()
        records.extend(body["data"])
        page, pages = body["next_page"], pages + 1
    return {r[id_field]: r for r in records}, pages

crm_leads, pages = read_all("leads", "lead_id")
print("leads:", len(crm_leads), "in", pages, "pages")
crm_accounts, pages = read_all("accounts", "customer_id")
print("accounts:", len(crm_accounts), "in", pages, "pages")
```

```
leads: 44 in 3 pages
accounts: 24 in 2 pages
```

- **params={"page": page, "page_size": page_size}** adds the two settings to the address, as `?page=1&page_size=20`. `page_size=20` asks for 20 records a page, so the 44 leads take three pages; with `page_size=50` (the sandbox's largest) they would take one, and with 10, five.

- The `while` loop is Chapter 45's pagination: ask for page 1, then whatever `next_page` the CRM returns, until it's `None` on the last page. `pages` counts the requests.
- The dictionary comprehension indexes the records by their id, so a lead can be looked up as `crm_leads["4"].id_field` names the id column, which differs between leads and accounts.
- The function returns two values, the records and the page count, and `crm_leads, pages = ...` unpacks them.

Reading it. 44 leads, the 43 Riverstone leads plus Grand Horizon Hotels, created in the CRM in section 51.6. 24 accounts, including the one the upsert created.

Now compare, record by record:

```
def find_mismatches():
    crm_leads, _ = read_all("leads", "lead_id")
    crm_accounts, _ = read_all("accounts", "customer_id")
    found = []
    for lead_id, s in scores.items():
        crm_value = crm_leads[lead_id].get("score")
        if crm_value != s["score"]:
            found.append((f"leads/{lead_id}", s["score"], crm_value))
    for cid, due_flag in follow_up.items():
        crm_value = crm_accounts[cid].get("follow_up_due")
        if crm_value != due_flag:
            found.append((f"accounts/{cid}", due_flag, crm_value))
    return found

print("checked:", len(scores), "leads and", len(follow_up), "accounts")
for record, warehouse_value, crm_value in find_mismatches():
    print(f"MISMATCH {record}: warehouse {warehouse_value}, CRM {crm_value}")
```

```
checked: 43 leads and 24 accounts
MISMATCH leads/4: warehouse 20, CRM 21
```

- `find_mismatches()` reads the CRM again each time it's called, so it always compares with the CRM as it is now. `_` is the usual name for a value you don't need, here the page count.
- It loops over the **warehouse's** records, not the CRM's, because the question is "did everything we meant to write arrive?". Grand Horizon Hotels isn't in the warehouse yet (tomorrow's ingestion will bring it), so it isn't checked.
- `.get("score")` returns `None` instead of stopping with an error if the CRM record has no score, for example a record the sync never reached.

Reading it. One mismatch: lead 4, 20 in the warehouse against 21 in the CRM. That's section 51.4's +1, which the content-only key then couldn't undo. The report caught it the same day, which is its job. Re-running today's sync wouldn't fix it: every write would be a replay, as "back to 20" showed.

Tomorrow's run, with the run-id key

Here is the nightly sync as the project would run it: every lead and every account, keyed with `run_key()`, with each write's result recorded:

```
def nightly_sync(run_id, run_time):
    jobs = [(f"leads/{i}", {"score": s["score"], "stage": s["stage"]}) for i, s in scores.items()]
    jobs += [(f"accounts/{c}", {"follow_up_due": d}) for c, d in follow_up.items()]
    log = []
    for record, payload in jobs:
        key = run_key(record, payload, run_id)
        body = {**payload, "score_updated_at": run_time} if record.startswith("leads/") else payload
        try:
            result = sync_record(record, body, key)
            outcome = "replayed" if result["replayed"] else "new"
        except (requests.HTTPError, RuntimeError) as e:
            outcome = "rejected: " + str(e)
        log.append({"run": run_id, "record": record, "sent": payload, "key": key, "outcome": outcome})
    with open("sync_state/audit_log.jsonl", "a") as f:
        for row in log:
            f.write(json.dumps(row) + "\n")
    return log
```

- `jobs` is the list of writes: one (record, payload) pair per lead, then `+=` adds one per account.
- Each write gets its `run_key()`. Leads also carry `score_updated_at`; accounts don't.
x if condition else y picks one of two values (Chapter 17).
- **try / except** (Chapter 17) catches a write the CRM refused, such as a 404, or one that failed every attempt, and records it as **rejected** instead of stopping the whole run: one bad record shouldn't block the other 66.
- `log` gets one row per write. The `with open(..., "a")` block **appends** every row to `sync_state/audit_log.jsonl`, one JSON object per line, a common format for logs called JSON Lines. That file is the audit trail below.

Now pretend it's 7 January, the warehouse's values haven't changed overnight, and run it, then reconcile:

```
log = nightly_sync("2026-01-07", "2026-01-07T06:30:00+05:30")
outcomes = [row["outcome"] for row in log]
print("attempted :", len(log))
print("confirmed :", sum(1 for o in outcomes if not o.startswith("rejected")))
print("new       :", outcomes.count("new"))
print("replayed  :", outcomes.count("replayed"))
print("rejected  :", sum(1 for o in outcomes if o.startswith("rejected")))
print("mismatched:", len(find_mismatches()))
print("audit row :", log[3])
```

```
attempted : 67
confirmed : 67
new       : 67
replayed  : 0
rejected  : 0
mismatched: 0
audit row : {'run': '2026-01-07', 'record': 'leads/4', 'sent': {'score': 20, 'stage': 'New'}, 'key':
             '67329d74f025d291', 'outcome': 'new'}
```

- `sum(1 for o in outcomes if ...)` counts the items that pass the test: it adds 1 for each. `outcomes.count("new")` counts exact matches.
- `log[3]` is the fourth write, lead 4's, as it's stored in the audit trail.

Reading it. All 67 writes, 43 leads and 24 accounts, were attempted and confirmed, none rejected, and the reconciliation now finds **no mismatches**: with a new run id, lead 4's 20 was applied again. Every write was new, though, even the 66 whose values hadn't changed. That's the price of the run-id key: each run re-sends everything. At thousands of records you'd send only the rows whose values changed since the last run, Chapter 45's incremental idea (section 45.4) applied to writes, and keep the run id in the key for those.

A reverse-ETL reconciliation report should show, every run:

- how many records the warehouse tried to sync;
- how many the destination confirmed;
- how many writes were replays (already applied) versus new changes;
- anything the destination rejected, with its error;
- any record where the destination's value doesn't match what was sent, with both values.

This is Chapter 46's gating check, pointed the other way: **hold the "sync succeeded" status, and alert, until the reconciliation confirms it**, exactly as the Daily Sales Flash isn't delivered until its numbers match the ERP.

An audit trail

Keep a durable log of every write attempt, separate from the CRM's own record: time, record, old value if known, new value, idempotency key, and outcome. `nightly_sync()` keeps a simple one in `audit_log.jsonl`; a production version also stores the time of each write and the CRM's old value, read just before writing. When someone asks "why does this lead have this score", or "who changed this field last Tuesday", the audit trail answers it without guessing. It's cheap to build (one row per write), and it's the first thing anyone reaches for during an incident.

51.8 Integration patterns

As the number of systems grows, *how* they connect matters as much as any single sync.

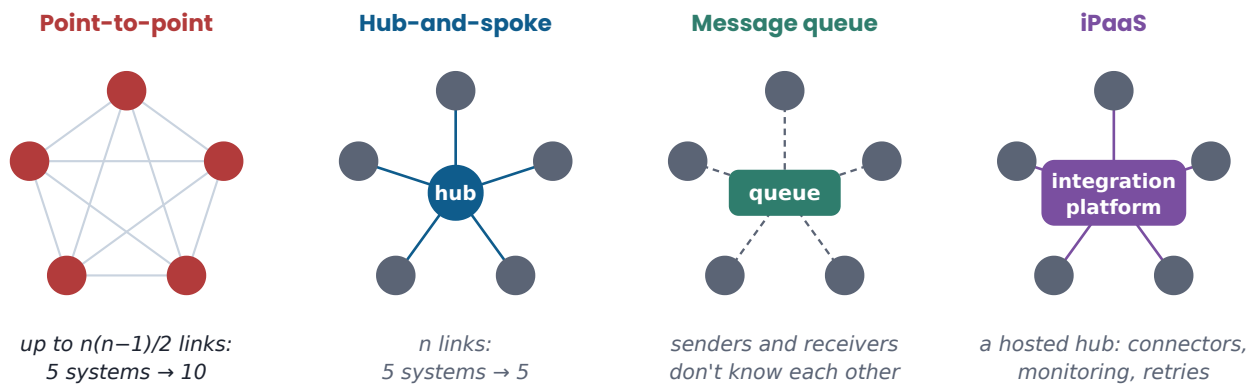


Figure 51.2 – Four ways to wire systems together. The number of connections is the thing to watch as the company grows.

Pattern	Shape	Strength	Weakness
Point-to-point	Every system talks directly to every other	Simple to start, no new infrastructure	Grows roughly as the square of the system count: n systems need up to $n(n-1)/2$ links (5 systems, 10 links; 10 systems, 45); nobody can see the whole picture
Hub-and-spoke	Every system talks only to a central hub	New systems add one connection, not many	The hub becomes critical; needs its own reliability
Message queue / event bus	Systems publish and subscribe to events, decoupled	Producers and consumers don't need to know about each other; naturally handles bursts	Needs the infrastructure from Chapter 50; harder to trace a single request end to end
iPaaS (integration platform as a service)	A hosted hub with pre-built connectors, retries, and monitoring	Fast to build with, less code to own	Ongoing cost; you depend on the vendor's connector for each system

Riverstone at its current size (a handful of systems: ERP, CRM, the warehouse, one or two marketing tools) is well served by **hub-and-spoke with the warehouse as the hub**, which is exactly what Chapters 45–51 have built without naming it: everything reads from and writes through the warehouse and its pipelines, rather than the CRM talking directly to the ERP. Message queues and iPaaS earn their cost once there are many systems, many events, or a team too small to maintain custom connectors for each one.

51.9 Choosing how to build it

Option	Best for	Cost
Custom code (this chapter)	A handful of well-understood syncs; full control over idempotency and reconciliation	Engineering time to build and maintain
Orchestrator tasks (Chapter 46)	Syncs that fit naturally next to your existing pipelines	Some setup; scales with what you already run
Low-code / iPaaS (Zapier, Workato, Tray.io, n8n)	Many simple syncs, built quickly, by people who aren't primarily engineers	Per-task or per-connector pricing; less control over retries and idempotency

Option	Best for	Cost
Enterprise integration platforms (MuleSoft, Boomi, Informatica)	Large organizations with many systems and compliance needs	Significant cost and specialist skill to run
Vendor-native reverse ETL (Census, Hightouch)	Syncing warehouse tables into standard SaaS tools with minimal code	Subscription cost; excellent fit when the destination is a well-supported SaaS app

The decision tree is short: **if a reverse-ETL tool already supports your exact destination and the sync is "a warehouse column into a CRM field," use it**, since idempotency and pagination are solved for you. **Build custom code when the logic is bespoke** (Riverstone's lead score, a conflict rule specific to your business) or when no connector exists for your system, as is common with older ERPs.

51.10 When there's no API: RPA and file drops

Not every system Riverstone deals with has an API. Some suppliers' portals only offer a login page. Some of Riverstone's own older tools were never built to be integrated with.

RPA, and why it's fragile

Chapter 7 introduced **RPA** (robotic process automation): software that clicks through a screen the way a person would, reading and typing into the same interface a human uses. It's sometimes the only option, and it's genuinely useful when there's truly no other way in.

It's fragile because it depends on things that were never a contract: a button's position, a label's exact wording, how long a page takes to load. A vendor redesigns their portal, and every recorded click breaks at once, silently, often producing wrong data rather than an obvious error, because the bot clicked whatever is now in that position on the screen.

If you must use RPA: treat every run as production-critical (monitoring, alerts, a human review step for anything unusual), prefer tools that read the page's underlying structure over raw pixel coordinates, and revisit the decision every time the target system changes. An API sometimes appears later, or a data export replaces the need entirely.

File-based integration: still normal, not a failure

SFTP drops (a file placed on a secure server on a schedule) and **EDI** (Electronic Data Interchange, a decades-old standard format for business documents like purchase orders and invoices) remain the backbone of a huge amount of manufacturing and B2B integration. A supplier who has exchanged EDI purchase orders with hundreds of partners for twenty years has no reason to build you an API.

Everything Chapter 45 taught about files applies directly: declared columns, a load log, reconciliation against a control total, and a data contract with the sending party about format and timing. The only difference here is direction: you may be the one **producing** the file, in which case the same discipline applies to what you send, with a stable format, a clear naming convention, and validation before it leaves, so a malformed export doesn't become someone else's 3 a.m. incident.

Common mistakes

Mistake	Symptom	Fix
Building a beautiful dashboard nobody visits	The insight exists; nothing changes	Push it where the decision is already made (51.1)
Syncing a field without checking who else edits it	A human's careful edit vanishes on the next sync	Name the system of record per field (51.5)
No idempotency key on writes	A retried sync doubles a discount or creates a duplicate lead	Send a key with every write, and prove replays are no-ops (51.3, 51.4)
Key built inside the retry loop, from "now" or a fresh random value	The key changes every retry, so it protects nothing	Create the key once, before the first attempt, and reuse it on every retry (51.4)
Key built from the values alone	A value that goes back to an earlier one is never sent again; reconciliation flags the same record every night	Add the run id to the key (51.4)
Retrying only one kind of failure	A timeout or a 429 crashes the sync halfway	Retry 429, 503, timeouts, and dropped connections, respecting Retry-After; fail fast on other errors (51.3)
Using PUT to change one field	Every field you didn't send is wiped	PATCH, naming only the fields the sync owns (51.3)
Trusting a write "succeeded" because no exception was thrown	A field silently didn't take, and nobody notices for weeks	Reconcile writes the same run, not later (51.7)
Polling every few minutes when the moment matters	A high-value lead sits unnoticed for most of the interval	A webhook, with a reconciliation sync as a backstop (51.6)
Treating a webhook receiver as always-on and always-fast	Duplicate events; missed events during downtime	Verify signatures, respond fast, de-duplicate, expect retries (51.6)
Point-to-point connections between every pair of systems	Adding one system means touching five others	Hub-and-spoke, or a queue, as the system count grows (51.8)
Choosing RPA when an API exists	Fragile, breaks on every UI change	Check for an API or export first; RPA is a last resort (51.10)
No audit trail for writes	"Why does this record say this?" has no answer	Log every write attempt with old value, new value, and outcome (51.7)
Building custom code for a well-supported SaaS destination	Reinventing pagination, retries, and auth a tool already solved	Check for a reverse-ETL or iPaaS connector first (51.9)

In the real world: the sync that erased a discount

Three months after Riverstone's lead-scoring sync went live, a sales rep phoned Meera, upset. A customer she'd spent weeks negotiating a loyalty discount for had opened their CRM record that morning to find the discount field blank.

Meera traced it in under an hour, because the audit trail this chapter builds made it possible. The nightly reverse-ETL job wasn't touching discounts: it only wrote lead scores. But a second sync, built six weeks earlier by a different contractor to push "customer segment" into the CRM, had been written against an older export of the customer table that didn't include the newly added discount column. Its update statement replaced the **entire customer record** rather than patching one field, because the contractor had used `PUT` (replace everything) instead of `PATCH` (change only what's specified), and nobody had reconciled the write afterward.

The discount wasn't malicious, and it wasn't even the sync this chapter is about. It was a `PUT` where a `PATCH` was needed, on a field nobody had named a system of record for, with no reconciliation to catch it, and no audit trail on that particular job because it had been built in a hurry outside the team's normal process.

The fixes were exactly this chapter's checklist. Every write became a `PATCH` naming its fields explicitly, never a full-record replace. Every synced field got an entry in a small internal document naming its system of record: discount is **the CRM**, never written by any pipeline. Every sync gained the same reconciliation report as the lead-score job. And the "customer segment" sync, rebuilt properly, took half a day.

At the retrospective, the plant manager's line from Chapter 50 came up again, adapted: *a sync that silently overwrites what a person entered is not a small bug. It's a system nobody can trust with anything a person has typed by hand.*

What made this work.

- **The cause wasn't the sync this chapter builds:** it was a different one, built without the same discipline, which is exactly why the discipline has to be a rule for every sync, not a one-off best effort.
- **`PATCH`, not `PUT`,** for anything that shouldn't touch fields it doesn't know about.
- **A named system of record per field** would have made "never write to discount" an explicit, checked rule instead of an assumption.
- **Reconciliation would have caught it the same night**, not three months later from an upset customer.

Project: sync Riverstone's lead scores and follow-up flags

Goal: a nightly, idempotent sync of lead scores and follow-up-due flags into the sandbox CRM, with a reconciliation report, plus a webhook that alerts sales within a minute of a high-value lead.

Tools you'll need

- **Python** with `psycopg2` and `requests`, in the book's virtual environment (installed in Chapters 18 and 45).
- **The Chapter 51 companion folder** (`companion/ch51/`, copied to `work/ch51` in section 51.0): `reset_ch51.py`, `crm_sandbox.py` (the sandbox CRM's API), `webhook_receiver.py` (the local webhook receiver), `lead_scores.sql` and `follow_up_due.sql` (the two queries), and `sync_leads.py` (the scoring, the flags, and both keys, as functions). Run Python from that folder.
- **Versions used for the outputs shown:** Python 3.11.15, `requests` 2.33.1, `psycopg2` 2.9.13, PostgreSQL 16.13.
- **Worth knowing about:** reverse-ETL tools (Census, Hightouch), iPaaS platforms (Zapier, Workato, Tray.io, n8n, MuleSoft, Boomi), and RPA tools (UiPath, Automation Anywhere, Power Automate Desktop).

Option A: Riverstone. Use the companion environment.

Option B: your own systems. Any write API you have permission to use, ideally with a sandbox or test mode. Never practice writes against a production system without explicit permission.

Steps

1. **Name the system of record** for every field you'll sync, and write down the conflict rule for each, before writing any code.
2. **Compute the values** to sync (adapt `sync_leads.py`'s approach, or use your own logic), documented in one sentence per rule.
3. **Build the sync function** with an idempotency key built from the record, the values, and the run id, created before the first attempt; retries on 429, 503, timeouts, and dropped connections only; and PATCH requests that touch only the fields you intend.
4. **Prove idempotency:** send the same write twice in one run and show the second call is a no-op; change the content and show it's applied; send an earlier value again in a new run and show it's applied too.
5. **Add the reconciliation report:** records attempted, confirmed, mismatched (with both values), replayed versus new, and rejected.
6. **Add an audit trail** table or log: timestamp, entity, fields, idempotency key, outcome.
7. **Register a webhook** for one high-value event, and measure the time from the event to your receiver getting it.
8. **Make the webhook receiver safe:** acknowledge fast, de-duplicate by event id, and add a scheduled reconciliation sync as a backstop in case an event is ever missed.
9. **Write the incident scenario:** what would happen if this sync ran with a stale or wrong value for a day, who would notice, and how they'd find out. Add one check that would have caught it.

Stretch goals

- Turn the sync into a Chapter 46 Dagster asset, gated by a Chapter 47 check before it's allowed to write.
- Simulate the CRM being unreachable for a whole run, and design the retry-and-alert behavior (Chapter 46's patterns, applied to writes).
- Compare your custom sync against what a reverse-ETL tool's free tier would let you build for the same fields, and note what you'd trade.

REVIEW

Recap

- **Data activation** closes the loop from source systems through the warehouse and back into the systems people actually work in. A dashboard is pulled; a synced field is pushed to where the decision already happens.
- **Reverse ETL** writes computed fields (lead scores, follow-up flags, health scores, credit holds) into the CRM, ERP, and marketing tools.
- Writes use the same **API mechanics** as reads (authentication, pagination, retries on 429, 503, and timeouts) with sharper consequences, and OAuth is standard for write access. An **upsert** creates a record or updates it, matched on an id both systems share.
- An **idempotency key** makes retries safe: a repeated write returns `replayed: True` and changes nothing. Create it once, before the first attempt, from the record, the values, and the **run id**: a key from the values alone can never re-send a value that went back to an earlier one.
- **PATCH changes only named fields; PUT replaces the whole record.** Using PUT where PATCH was needed is a real, common cause of silently erased data.
- Name the **system of record** for every synced field individually, and write the **conflict rule** before two systems disagree in front of a customer.
- **Webhooks** push events the moment they happen; treat receivers as needing signature verification (an **HMAC** of the body with a shared secret), fast acknowledgement, and de-duplication by event id, and always keep a reconciliation sync as a backstop.
- **Reconcile writes**, not only reads, every run, and keep an **audit trail** of every attempt.
- **Integration patterns** (point-to-point, hub-and-spoke, message queues, iPaaS) trade simplicity against how well they scale as systems multiply.
- Choose **custom code, orchestrator tasks, low-code tools, or enterprise platforms** based on how bespoke the logic is and how well-supported the destination already is.
- **RPA** is fragile because it depends on a screen's layout rather than a contract; **file-based integration** (SFTP, EDI) is not a failure mode, it's still normal, and deserves the same discipline as any other file load.

Key terms

data activation · reverse ETL · lead score · customer health score · credit hold · reorder flag · churn risk · follow-up-due flag · scope · service account · OAuth 2.0 · access token · GET, POST, PUT, PATCH · path parameter · upsert (by external id) · idempotency key · replay · run id · system of record · conflict rule · webhook · HMAC (event signature) · at-least-once webhook delivery · point-to-point integration · hub-and-spoke integration · message queue / event bus · iPaaS (integration platform as a service) · enterprise integration platform · RPA (robotic process automation) · SFTP · EDI (Electronic Data Interchange) · audit trail · JSON Lines · write reconciliation

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ You can explain why a correct number in a warehouse can still change nothing.
- ☐ You can name three fields worth syncing back into operational systems and what each changes.
- ☐ You can write a `PATCH` request with authentication, and explain why `PATCH` and `PUT` aren't interchangeable.
- ☐ You can build an idempotency key that makes a retry a no-op but lets a new run re-send an earlier value, and prove both.
- ☐ You can explain system of record and write a conflict rule for a specific field.
- ☐ You can register and receive a webhook, check its HMAC signature, and say what else makes a receiver safe.
- ☐ You can reconcile a set of writes and explain what a mismatch means.
- ☐ You can compare point-to-point, hub-and-spoke, message queues, and iPaaS, and say which fits a given company size.
- ☐ You can explain why RPA is fragile and when it's still the right tool.
- ☐ You can describe SFTP and EDI integration and apply Chapter 45's file discipline to a file you produce.
- ☐ You can design an audit trail that answers "why does this record say this?"

Exercises

Warm-up

1. For each field, say whether the warehouse or a human system should be its system of record, and why: (a) a computed customer health score; (b) a manually negotiated discount; (c) an order's shipping status; (d) a lead score computed nightly.
2. Explain the difference between `PATCH` and `PUT` in one sentence each, and say which one the sync that erased a discount should have used.
3. A webhook fires twice for the same event, five seconds apart. Name two ways your receiver could be built to make that harmless.

Core

4. Using the rules in section 51.2, compute the lead score for a lead that is **Contacted**, sourced from a **Trade fair**, that entered its stage 3 days before the chapter's "today". Show your working.
5. In section 51.4, lead 4's "same values again" and its "back to 20" both returned `replayed: True`. Explain precisely why, referring to what the key was built from, and say why the first is correct and the second is a bug.
6. Write the idempotency key logic for a sync that should apply "once per scheduled run" rather than "once per distinct value" (the third row of the table in section 51.4). What business situation would make you prefer that behavior?
7. The reconciliation in section 51.7 found one mismatch, for lead 4. Explain what caused it, why re-running the section 51.3 sync couldn't repair it, and what did. Is the mismatch a bug, or the reconciliation working as intended? Justify your answer.
8. Riverstone wants to add a sync for "customer segment" into the CRM. Using section 51.5's framework, write the two-line conflict policy for that field, including what happens if a rep manually changes the segment in the CRM.
9. A supplier sends purchase order confirmations by SFTP once a day, as a fixed-width text file. List the four things from Chapter 45's file discipline you'd apply before trusting a single row of it.

Stretch

10. Design the reconciliation report you'd run every night for the lead-score sync at real volume (thousands of leads, not 43), including what you'd summarize versus what you'd list row by row, and what would trigger an alert versus a note in a log.
11. A vendor discontinues the CRM's API in favor of a new one with different field names and a different authentication scheme. Walk through what in this chapter's design changes and what stays the same (idempotency keys, reconciliation, system of record, the webhook contract).
12. Compare building the lead-score sync as (a) custom Python in an orchestrator, (b) a low-code iPaaS flow, and (c) a vendor reverse-ETL tool, for a company with one part-time data person. Recommend one, with reasons, and say what would change your recommendation if the company had a five-person data team instead.

Think about it (no code needed)

13. "A sync that silently overwrites what a person entered is not a small bug." Explain what makes this failure mode worse than a wrong number in a report, using the ideas from Chapter 47 on severity.
14. RPA and file-based integration are both presented in this chapter as reasonable choices in the right circumstances, not as mistakes. Explain to a colleague who wants to "modernize everything to APIs" why that isn't always the right goal.

Answers

1. (a) **Warehouse**: it's computed from data the CRM doesn't hold, and no person should hand-edit a calculated score. (b) **The CRM (a human system)**: a person negotiated it, and a pipeline must never silently overwrite that. (c) **The ERP**: it's where the transaction and the physical dispatch happen. (d) **Warehouse**: same reasoning as (a), nightly computed, not entered by a person.
2. **PATCH** changes only the fields named in the request, leaving everything else untouched. **PUT** replaces the entire record with what's sent, so any field left out is wiped. The discount-erasing sync should have used **PATCH**, naming only the customer-segment field, so the discount field would never have been touched.
3. (1) **De-duplicate by the event's own id**: keep a short-lived record of ids already processed, and skip a repeat. (2) **Make the processing itself idempotent**, in the style of this chapter's writes, so that even without explicit de-duplication, handling the same event twice produces the same end state rather than a doubled effect.
4. From section 51.2's tables: Contacted = 30 points (stage), Trade fair = 20 points (source). The stage was entered 3 days before today, inside the 14-day window, and the lead isn't Lost, so +10 recency. $30 + 20 + 10 = 60$, below the cap of 100, so no capping applies.
5. Both keys were built from the record and the values only (section 51.3's `idempotency_key`). "Same values again" sent lead 4's score and stage exactly as the first sync had, so the key matched the one the CRM had stored, and it returned its earlier answer, `replayed: True`, without touching the record. That is correct: nothing needed to change. "Back to 20" also sent those same values, after the score had been changed to 21, so it produced the same old key and was ignored too. That is the bug: the CRM still says 21, and no later sync with a content-only key can ever set it back to 20. Adding the run id to the key (section 51.4's `run_key`) fixes it.
6. A sample answer, the third row of the table in section 51.4, built like `run_key()` without the values:

```
def run_scoped_key(record, run_id):
    content = record + ":" + run_id
    return hashlib.sha256(content.encode()).hexdigest()[:16]
```

- `record` is the record's address, such as "leads/4", and `run_id` is the run's day, such as "2026-01-07". The payload is left out on purpose.
- `content.encode()` turns the text into bytes, `hashlib.sha256(...).hexdigest()` fingerprints it, and `[:16]` keeps 16 characters, exactly as in `idempotency_key()`.

- `return` hands the key back to the caller, which creates it once, before the first attempt.

This makes retries within the *same scheduled run* a no-op regardless of whether the score changed mid-run, but a *new run* always gets a new key, so its write is applied even if the value happens to be identical to last time. You'd prefer this when you want a clean, auditable "one write per run" record, for example a compliance requirement that every nightly sync is logged exactly once per lead per night, rather than being silently skipped because the value hadn't moved.

7. Section 51.4 pushed a "+1" test value into lead 4 (21), standing in for a genuine change, and then tried to send the warehouse's 20 again. With a key built from the values alone, that write had the same key as the first sync's, so the CRM replayed it and kept 21. Re-running the section 51.3 sync would produce exactly the same keys, so every write would be replayed and lead 4 would stay wrong forever. What repaired it was the next run with `run_key()`: a new run id gives new keys, so 20 was applied, and the next reconciliation was clean. The mismatch itself was the reconciliation **working as intended**: it caught the drift the same day. The bug was the key design, which is what to fix. In a real deployment the same report line needs investigating: was it a manual edit in the CRM (perhaps this field shouldn't be blindly overwritten), a partly failed sync, or a second job writing the same field (as in the closing story)? The report's job is to surface the question, not answer it.

8. A sample two-line policy: *"Customer segment: system of record is the warehouse; the nightly sync always overwrites this field with the computed value. If a rep manually changes it in the CRM, that change will be overwritten by the next sync within 24 hours; reps should request a change to the segmentation rule instead of hand-editing individual records."* The key elements: name the system of record explicitly, and say plainly what happens to a manual edit, so nobody is surprised later.

9. (1) **Declare the columns and their types explicitly** rather than letting a reader guess from a fixed-width file with no header. (2) **Land the raw file unchanged** before parsing, so the original is always available if the parsing rule turns out wrong. (3) **Reconcile row counts (and a control total if the supplier provides one) against the file** before trusting the loaded data. (4) **Check for schema drift**: confirm the field widths and layout match what was agreed, and stop with a clear error if a column has shifted, rather than silently misreading every field after it.

10. At real volume: **summarize** by default, with total attempted, total confirmed, total replayed, total newly changed, total mismatched, total rejected, broken down by error type for rejections, and only **list row by row** the records that are mismatched or rejected, since those are what a person needs to act on; a report listing every one of ten thousand successful matches is noise. **Alert** (page someone) when: the mismatch or rejection rate crosses a threshold (say, more than 1% of attempted writes), when the whole sync fails to run, or when a previously-matching record now mismatches for no logged reason. **Log without alerting**: normal-range mismatches under investigation, expected rejections (like a lead deleted in the CRM since the last run), and routine replay counts.

11. **Changes**: the authentication code (new OAuth flow instead of a bearer token), the field names in the request and response bodies, and the base URL and any pagination parameter names. **Stays the same**: the idempotency key design (still built from your own entity id and content, not from anything the new API dictates), the reconciliation report's logic (still comparing your computed value against what the destination confirms), the system-of-record decisions (unchanged by which vendor holds the CRM), and the webhook contract's shape, if the new API also supports webhooks (register a URL,

verify signatures, de-duplicate by event id) even though the exact payload will differ. The lesson: a well-designed sync isolates vendor-specific details (auth, field names, URLs) behind a thin layer, so a vendor migration touches a small, obvious part of the code.

12. For one part-time data person: recommend **(c) a vendor reverse-ETL tool** if the CRM is a well-supported mainstream product, because idempotency, pagination, retries, and authentication are solved by the vendor, leaving the person to define the one thing that's actually specific to Riverstone: the lead-scoring SQL. Custom code (a) demands ongoing maintenance a part-time person can't reliably provide; a low-code iPaaS flow (b) is a reasonable middle ground if no reverse-ETL connector exists for the destination. With a **five-person data team**, the calculus shifts toward **(a) custom code in the orchestrator**, because the team can build it once with full control over reconciliation, idempotency keys tailored to the business's conflict rules, and audit trails that match their own standards. At that scale, avoiding per-row vendor pricing on a reverse-ETL tool also starts to matter, and the team has the capacity to own what they build.

13. A wrong number in a report is a **single, visible, correctable** failure: someone reads it, it's wrong, it gets fixed, and the damage is bounded to whoever acted on that one report before the correction went out (both failures are S1 under Chapter 47: wrong data has reached people; the difference is the blast radius and how visible it is). A sync that silently overwrites a human's entry is worse on every axis: it's **invisible** until someone happens to look at the specific record; it **destroys information** that may not exist anywhere else (the negotiated discount lived only in that field); it can **recur indefinitely**, silently re-erasing a fix every time the sync runs, until someone identifies the actual cause; and it **damages trust in the system itself**, since once a rep learns the CRM can silently lose their work, they stop trusting the sync for everything, not just the one field that broke.

14. The goal was never "have an API". It was **reliable, correct data movement between systems, at a cost proportional to what's at stake**. An API is usually the best way to get that, but a twenty-year-old EDI relationship with a supplier that has worked reliably for two decades, validated by the same file-checking discipline as any other data source, isn't broken just because it's old; rebuilding it around a new API the supplier doesn't have would cost real effort to deliver the same reliability that already exists. Similarly, RPA against a portal with no API isn't a failure of engineering: it's occasionally the only bridge available, and the right response is to operate it carefully (monitoring, human review, revisiting the decision as the target changes), not to pretend it doesn't exist or refuse to use it. "Modernize everything" is a good instinct pointed at the wrong target: point it at reliability and cost, and let the technology choice follow from that, case by case.

NEXT

Where this leads

- **Chapter 52, The Cloud, Containers & Infrastructure as Code**, deploys syncs like this one somewhere they stay running: secrets management for tokens such as `CRM_API_TOKEN` (section 52.6), containers, and scheduled infrastructure.
- **Chapter 47** applies directly: the reconciliation and audit-trail habits here are Chapter 47's checks, pointed at writes instead of reads.

- **Chapter 46** is where a real deployment of this sync would live, as an orchestrated, idempotent, checked pipeline step.
- **Chapter 50's** at-least-once delivery is the same guarantee webhooks make, from the other direction.
- **Chapter 58, Intelligent Automation**, builds AI-driven actions on top of exactly this activation layer, with the same idempotency and system-of-record discipline.
- **Chapter 63, Automation Architecture & Governance**, returns to integration patterns at the whole-company level.
- **Part 8:** Chapter 78 (Automation & Integration) drills webhooks, idempotency, and CRM syncs; Chapter 77 includes system design cases built on exactly this loop.

Chapter 52. The Cloud, Containers & Infrastructure as Code

Chapter at a glance

You will learn to: say what IaaS, PaaS, and SaaS rent you, and where the shared responsibility line falls for each · explain regions and availability zones · write a least-privilege access policy and read every field of it · plan a network with CIDR arithmetic you can do by hand · install Docker, build an image of Chapter 46's pipeline, and run it · run a small stack of containers with Docker Compose · read a Kubernetes CronJob, Deployment, and Service and say what each line does · write a small Terraform configuration, check it, and read its plan · build a CI/CD workflow and trace exactly which jobs run for a given event · keep secrets out of code and images · estimate what running a pipeline in the cloud actually costs · choose a deployment target that fits a company Riverstone's size.

Before you start: Chapter 26 (the terminal, Git, pull requests, `.gitignore` and `.env`, GitHub Actions). Chapter 29 (tests with `pytest`). Chapter 34 (environment variables, IP addresses, ports, private address ranges, and WSL on Windows). Chapter 45 (the practice source and CRM API), Chapter 46 (the pipeline this chapter deploys), Chapter 47 (checks and incidents), Chapter 49 (storage and its cost), Chapter 51 (webhooks).

Time needed: 20–26 hours of reading and practice, spread over three to four weeks. Docker (section 52.2) and Terraform (section 52.4) are hands-on and take the most time; read Kubernetes (section 52.3) for its vocabulary, and try it on your laptop only if you want to.

Tools: Python 3.14 in the book's virtual environment, with `pyyaml`, `python-hcl2`, and `dockerfile-parse` (section 52.0); Docker (section 52.2); Terraform (section 52.4). No cloud account is needed.

In this chapter

START	Why this matters · In plain English	284
LEARN	52.0 Setting up	285
	52.1 Cloud fundamentals	286
	52.2 Containers	292
	52.3 Orchestrating containers	303
	52.4 Infrastructure as Code	309
	52.5 CI/CD	316
	52.6 Secrets, done properly	320
	52.7 Cost	321
	52.8 Choosing a deployment target	323
APPLY	Common mistakes · In the real world · Project	323
REVIEW	Recap · Key terms · Check yourself	326
PRACTISE	Exercises	328
	Answers	329
NEXT	Where this leads	332

A note on what actually runs in this chapter. Everything that can run on a laptop runs for real, and its output is shown: the network arithmetic, a real Docker image of Chapter 46's pipeline, built and run until it delivers the 2 January Flash, a two-container Docker Compose stack, and Terraform's own checks and its plan. Some things need services a laptop doesn't have: a Kubernetes cluster, a cloud account, and a repository on GitHub with a cloud account behind it. Those files are still real and complete, and they're checked as far as a laptop allows: the Kubernetes files against Kubernetes' own rules, the workflow's trigger logic simulated in Python against the actual file. Faking the output of what wasn't run would be worse than saying so plainly, so anything that needs a cluster or a cloud account is marked **not executed**, in the same style Chapter 46 used for Airflow.

START

Why this matters

Chapter 46 built a pipeline. Chapter 51 gave it something worth writing to. Right now, both of them exist only as files on your computer, running when you happen to type a command. Getting them to run reliably, on a schedule, reachable by the right people and nobody else, without your laptop being involved at all, is a different problem, and it's the one this chapter solves.

It's also where a great deal of a data engineer's actual working life happens. Job descriptions ask for "cloud experience" and "containers" more often than they ask for any single analytics tool, not because the cloud is more important than the data, but because nothing you've built in Part 5 runs anywhere real until it's deployed properly. This chapter takes Riverstone's pipeline from "works on my machine" to "works, reliably, on infrastructure someone else can also understand," using the same rigor the rest of this book has applied to data: nothing asserted that hasn't been checked.

In plain English

Think about Riverstone opening a second warehouse in another city.

- **Cloud regions** are like choosing which city to build in: closer to your customers is faster, and rules differ by location.
- **The shared responsibility model** is like renting a unit in a business park: the landlord secures the building, the fence, and the road; you lock your own door and decide who gets a key.
- **Identity and access** is the list of who holds which keys: the delivery driver's key opens the loading bay, not the accounts office.
- **A container** is a warehouse's stock, packed identically every time, so it behaves the same wherever it's unloaded, rather than depending on what happens to already be in that particular building.
- **Kubernetes** is the logistics coordinator who decides which building gets which containers, replaces one that's damaged, and adds more when a big order comes in.
- **Infrastructure as Code** is a written building plan, not a verbal instruction to a contractor: anyone can read it, review it before it's built, and rebuild the exact same warehouse in a different city.

- **CI/CD** is the checklist that runs automatically before anything ships: inspected, tested, and only then loaded onto the truck.

LEARN

52.0 Setting up

Step 1. Your practice folder. As in earlier chapters, copy the folder `companion/ch52` and paste it inside `work`, so you have `work/ch52`. It holds:

File or folder	What it is	Section
<code>pipeline_image/</code>	Everything needed to build a container image of Chapter 46's pipeline: <code>Dockerfile</code> , <code>requirements.txt</code> , <code>.dockerignore</code> , <code>run_flash.py</code> , copies of <code>ingest.py</code> and <code>riverstone_pipeline.py</code> from Chapter 46, <code>docker-compose.yml</code> , and <code>.env.example</code>	52.2
<code>k8s/</code>	Two Kubernetes files: <code>pipeline-cronjob.yml</code> and <code>flash-page.yml</code>	52.3
<code>terraform/main.tf</code>	Storage and an identity for the pipeline, as code	52.4
<code>terraform_practice/main.tf</code>	A tiny configuration for practising reading a plan, with no cloud account	52.4
<code>.github/workflows/deploy.yml</code>	The CI/CD workflow	52.5
<code>simulate_workflow.py</code>	Works out which jobs of a workflow run for a given event	52.5

`.github` and `.env.example` start with a dot, so your file browser may hide them. They're there; `ls -a` in a terminal shows them (Chapter 26, section 26.0).

Step 2. Three small libraries. Open a terminal, go into your practice folder, activate the book's virtual environment, and install the three parsers this chapter's checks use:

```
# terminal
$ cd work/ch52
$ source ../../.venv/bin/activate
$ python -m pip install pyyaml==6.0.3 python-hcl2==8.1.4 dockerfile-parse==2.0.1
```

- **pyyaml** reads YAML files, the format Docker Compose, Kubernetes, and GitHub Actions all use.
- **python-hcl2** reads HCL, the language Terraform files are written in (section 52.4).
- **dockerfile-parse** reads a Dockerfile (section 52.2) into Python lists and dictionaries.

On Windows PowerShell, activate with `..\..\venv\Scripts\Activate.ps1`, as in Chapter 45. Add the three lines to your `requirements.txt`. Then create a notebook, `ch52.ipynb`, in `work/ch52`, choose the `.venv` kernel, and check the install in its first cell:

```
import yaml, hcl2, dockerfile_parse
print("ok")
```

```
ok
```

`import yaml, hcl2, dockerfile_parse` loads the three libraries (each is imported under a slightly different name from the one you installed), and `print("ok")` runs only if all three loaded. If you see `ModuleNotFoundError` instead, the environment wasn't active when you installed: activate it and install again. Run every code block in this chapter as its own cell, top to bottom. Docker and Terraform are programs, not Python libraries; each is installed in the section that first uses it.

52.1 Cloud fundamentals

What you rent: IaaS, PaaS, SaaS

Chapter 2 described the cloud in two sentences: computers, storage, and networks in someone else's data centre that you rent over the internet. The largest providers are Amazon Web Services (AWS), Microsoft Azure, and Google Cloud. Businesses rent at three levels, depending on how much they want to manage themselves:

Level	You rent...	You still manage...	Examples	At Riverstone
IaaS (infrastructure as a service)	virtual computers, storage, and networks	the operating system, software, and data	a virtual server to run your own database	a virtual server (AWS calls it EC2) running the pipeline, which you patch and look after
PaaS (platform as a service)	a ready-to-use platform, such as a managed database	your data and how you use it	a cloud PostgreSQL service that handles backups and updates for you	a managed container service running the pipeline's container (section 52.8), or managed PostgreSQL (AWS calls it RDS)
SaaS (software as a service)	a finished application, used through a browser	your data and your settings	Gmail, Google Drive, Microsoft 365, CRM and accounting software	the CRM that Chapter 51 syncs to, and Gmail

The further down the table, the less you manage. That moves the shared-responsibility line, later in this section.

This chapter's examples use AWS, the provider that appears most often in job listings. Every idea has an equivalent on the others, usually under a different name:

What it is	AWS	Microsoft Azure	Google Cloud
Object storage (files, by name)	S3	Blob Storage	Cloud Storage
Identity and access	IAM	Entra ID and role-based access control	Cloud IAM
Running containers without managing servers	ECS on Fargate	Container Apps	Cloud Run
Managed Kubernetes	EKS	AKS	GKE
A safe place for secrets	Secrets Manager	Key Vault	Secret Manager

S3 (Simple Storage Service) stores files, which AWS calls **objects**, in named containers called **buckets**. Chapter 49's sensor archive would live in one.

Regions and availability zones

A **region** is a geographic area a cloud provider operates in (Mumbai, Singapore, Frankfurt). Each region contains multiple **availability zones**: physically separate data centers with independent power and networking, close enough for fast connections between them but far enough apart that one flooding or losing power doesn't take out the others.

Two decisions follow directly:

- **Choose a region near your users and your data's legal home.** Riverstone's customers, staff, and (per Chapter 45's note on data protection law) most of its personal data belong in an Indian region. AWS's Mumbai region is called `ap-south-1`; you'll see that name in section 52.4.
- **Spread anything that must stay up across at least two availability zones.** A pipeline running in one zone goes down when that zone has a bad afternoon; the same pipeline split across two zones usually doesn't.

The shared responsibility model

Every major cloud provider splits security between itself and you. The provider secures the **cloud**: the physical data centers, the hardware, and the **virtualization layer** (the software that splits one physical server into many virtual machines, the kind of program Chapter 6 mentioned for running Windows on a Mac). You secure what you put **in** the cloud.

The line moves with what you rent. On IaaS you also patch the operating system; on PaaS the provider does; on SaaS you manage only your data, your users, and your settings. **What never moves:** your data, who can access it, and how you configure the service are always yours. A misconfigured storage bucket left open to the internet is never the provider's fault under this model, however it happened. Chapter 64 covers the governance side of this line in depth; this chapter is about not crossing it by accident.

Who secures what, at each service level

Each cell says who is responsible; the heavy line is where the split falls.

	IaaS	PaaS	SaaS
Your data, and who can access it	You	You	You
How you configure the service	You	You	You: your settings
Application code	You	You	Provider
Operating system and runtime	You	Provider	Provider
Virtualization layer	Provider	Provider	Provider
Hardware and data centres	Provider	Provider	Provider
	e.g. a virtual server	e.g. managed PostgreSQL	e.g. a CRM, Gmail

The line moves with what you rent. What never moves:

your data, who can access it, and how you configure the service are always yours.

Figure 52.1 – The provider draws the line differently for IaaS, PaaS, and SaaS, but data, access, and configuration are always the customer's.

Identity and access management (IAM)

IAM controls who, or what, can do what. Four words carry the whole idea:

- An **identity** is who is asking. It's either a **user** (a person with a login) or a **role**: an identity with no password that a program **assumes**, "puts on", to get **temporary credentials**, which expire on their own after a while, usually within an hour.
- A **policy** is a document that lists what an identity may do. AWS writes policies in JSON (Chapter 2).
- Each policy holds one or more **statements**, and each statement has three parts: an **Effect** (Allow or Deny), the **Actions** it covers, and the **Resources** they apply to.
- Every AWS resource has an **ARN** (Amazon Resource Name), its unique ID, such as `arn:aws:s3:::riverstone-sensor-archive-production` for a bucket.

The habit that matters most is **least privilege**: grant exactly the permissions a task needs, and no more, and prefer a role a pipeline assumes over a permanent password a person shares. For Riverstone's pipeline, that means a dedicated identity that can read and write the sensor archive's bucket and nothing else, not an administrator's full access borrowed for convenience. Here is that policy:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::riverstone-sensor-archive-production",
                 "arn:aws:s3:::riverstone-sensor-archive-production/*"]
  }]
}
```

Reading it, field by field.

- **"Version": "2012-10-17"** is the version of AWS's policy language, not a date to keep up to date. Every current policy uses exactly this value.
- **"Statement": [...]** is a list, in square brackets, even when it holds one statement.
- **"Effect": "Allow"** grants what follows. Anything not allowed is denied: AWS starts from "no".
- **"Action"** lists three things: read a file (`s3:GetObject`), write one (`s3:PutObject`), and list what's in the bucket (`s3:ListBucket`). Deleting (`s3:DeleteObject`) isn't there, so the pipeline can't delete anything.
- **"Resource"** names the bucket itself (listing applies to the bucket) and `/*`, every object inside it (reading and writing apply to objects).

Across the people and programs that touch the archive, least privilege looks like this:

Who	Identity	May do
The pipeline	a role it assumes	<code>GetObject</code> , <code>PutObject</code> , <code>ListBucket</code> on this one bucket
An analyst	a user	<code>GetObject</code> and <code>ListBucket</code> on this bucket: read only
An administrator	a separate "break-glass" role, used only in an emergency and logged	everything, but nobody uses it day to day

Section 52.4 creates the pipeline's role and this policy as code.

Watch out: "it's just for now" access outlives its reason. Temporary broad permissions granted to unblock a deadline are one of the most common routes into cloud security incidents, because removing them is nobody's job once the deadline passes. Grant scoped access from the start; it costs a few extra minutes and no meetings six months later.

Networking: VPCs, subnets, and security groups

A **VPC** (virtual private cloud) is your own isolated slice of the provider's network. Inside it, **subnets** divide the address space into smaller ranges, usually one **public** (reachable from the internet, for things like a **load balancer**: a front door that spreads incoming requests across several copies of a service) and one **private** (not directly reachable, for a pipeline and a database) per availability zone. A **security group** is a firewall attached to a resource, allowing only the specific traffic it needs.

Subnets are sized in **CIDR notation**: an address, a slash, and a number, such as `10.0.0.0/16`. This isn't a diagram to memorize; it's arithmetic you can check, first by hand.

CIDR by hand. An IPv4 address (Chapter 34, section 34.9) is 32 bits (Chapter 2), written as four numbers from 0 to 255, such as `10.0.0.0`. The number after the slash says how many of those 32 bits are **fixed**: every address in the range starts with them. The rest are free.

- `/16`: 16 bits fixed, $32 - 16 = 16$ free, so $2^{16} = 65,536$ addresses.
- `/24`: $32 - 24 = 8$ bits free, so $2^8 = 256$ addresses.
- Splitting a `/16` into `/24`s fixes $24 - 16 = 8$ more bits, so there are $2^8 = 256$ subnets.

`10.0.0.0/16` sits inside `10.x.x.x`, one of the **private** ranges from Chapter 34: addresses used inside a network and never routed on the public internet.

Now the same arithmetic in Python, with its standard `ipaddress` module:

```
import ipaddress

vpc = ipaddress.ip_network("10.0.0.0/16")
print(vpc.num_addresses)
print(2 ** (32 - 16))
```

```
65536
65536
```

- `import ipaddress` loads Python's built-in module for IP addresses; there's nothing to install.
- `ipaddress.ip_network("10.0.0.0/16")` turns the text into a network object that knows its own size.
- `.num_addresses` is how many addresses the range holds. The second line does the by-hand sum, and they agree.

Next, split it into `/24` subnets:

```
subnets = list(vpc.subnets(new_prefix=24))
print(len(subnets), "subnets of", subnets[0].num_addresses, "addresses each")
print("first:", subnets[0], " second:", subnets[1], " last:", subnets[-1])
```

```
256 subnets of 256 addresses each
first: 10.0.0.0/24  second: 10.0.1.0/24  last: 10.0.255.0/24
```

- `vpc.subnets(new_prefix=24)` splits the network into pieces with a /24 prefix. It hands them out one at a time, as a **generator** (Chapter 33), so `list(...)` collects them all into a list you can count and index.
- `subnets[0]`, `subnets[1]`, `subnets[-1]` are the first, second, and last. Each /24 moves the third number along by one: 10.0.0.0, 10.0.1.0, up to 10.0.255.0.

Finally, Riverstone's plan: two public and two private subnets, one of each per availability zone. **Before you run it, predict:** how many addresses of a 256-address subnet can the pipeline actually use?

```
plan = {
    "public-a (load balancer, zone A)": subnets[0],
    "public-b (load balancer, zone B)": subnets[1],
    "private-a (pipeline, database, A)": subnets[2],
    "private-b (pipeline, database, B)": subnets[3],
}
for name, subnet in plan.items():
    usable = subnet.num_addresses - 5          # AWS reserves 5 addresses in every subnet
    print(f"{name} {str(subnet):<12} usable: {usable}")
print("remaining /24 subnets:", len(subnets) - len(plan))
```

```
public-a (load balancer, zone A) 10.0.0.0/24 usable: 251
public-b (load balancer, zone B) 10.0.1.0/24 usable: 251
private-a (pipeline, database, A) 10.0.2.0/24 usable: 251
private-b (pipeline, database, B) 10.0.3.0/24 usable: 251
remaining /24 subnets: 252
```

- `plan` is a dictionary from a subnet's job to the subnet itself; `.items()` gives each pair in turn.
- `subnet.num_addresses - 5` is the number of addresses you can actually give to machines. **AWS (and Azure) reserve five addresses in every subnet** for their own use: the network address, a router, DNS, one kept for the future, and the last address. **Google Cloud reserves four.** This chapter uses AWS's rule.
- `str(subnet):<12` turns the subnet into text and pads it to 12 characters, so the columns line up.

Reading it. Four /24 subnets of 251 usable addresses each, far more than Riverstone's pipeline will ever need, and 252 left over for the future. What happens if you change `new_prefix=24` to `new_prefix=26`? Each subnet shrinks to $2^6 = 64$ addresses (59 usable), and there are $2^{10} = 1,024$ of them. Exercise 4 asks when that's the better choice.

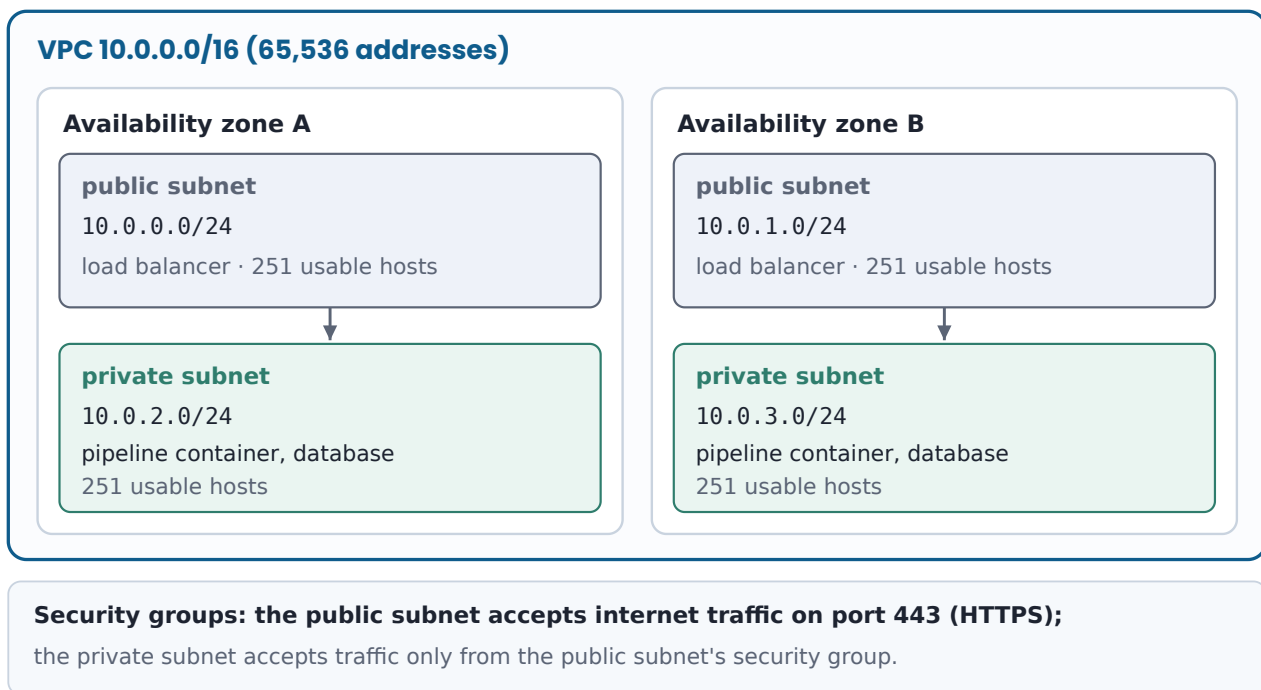


Figure 52.2 – Riverstone's VPC, matching the four subnets computed in this section: one public and one private subnet per availability zone.

The rule for security groups, expressed the way Chapter 47 expressed data tests: **security groups deny all incoming traffic until you add a rule** (outgoing traffic is allowed by default), **so add only the specific rule a real need justifies**. The load balancer accepts HTTPS on port 443 (Chapter 34) from the internet. The pipeline's security group allows inbound traffic on its one port from the load balancer's security group specifically, not from `0.0.0.0/0` (every address on the internet). Riverstone's database's security group allows inbound traffic on its port, 5432, from the pipeline's security group, and nothing else.

52.2 Containers

Why containers exist

"It works on my machine" is a symptom of one thing: the code depends on details of that machine, whether library versions, environment variables, or configuration files, that aren't written down anywhere. A **container** packages an application with everything it needs to run, as a single, versioned artifact that behaves identically wherever it's run.

An **image** is the packaged artifact; a **container** is a running instance of that image. You build an image once and run as many containers from it as you need. Images are shared through a **registry**, a library of images: **Docker Hub** is the public one, and every cloud has a private one. Each image has a name and a **tag**, written name:tag, such as `python:3.14-slim`; the tag says which version.

Installing Docker, and your first container

Docker is the most common tool for building and running containers.

- **Windows and macOS:** install **Docker Desktop** from docker.com. It's free for personal use, education, and small businesses; larger companies need a paid plan, which your employer would arrange. On Windows it runs on WSL 2, the Linux layer Chapter 34 (section 34.1) installed, and its installer asks to use it.
- **Linux (and WSL, if you prefer not to use Docker Desktop):** install **Docker Engine** from your distribution's instructions on docs.docker.com. Add yourself to the `docker` group, so you don't need `sudo` for every command, then log out and in again.

Start Docker Desktop (Docker Engine starts on its own), open a terminal, and check it:

```
# terminal
$ docker --version
Docker version 29.3.1, build c2be9cc

$ docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

- `docker --version` proves the command is installed. Your version number will be newer or older; anything from the last year or two behaves the same for this chapter.
- `docker run hello-world` downloads a tiny image called `hello-world` from Docker Hub and starts a container from it. The container prints its message and stops. The first time, Docker also prints a few lines saying it can't find the image locally and is downloading it.
- `--rm` removes the stopped container afterwards, so finished containers don't pile up.
- The message names the two halves of Docker: the **client** (the `docker` command you type) and the **daemon** (a background program that does the work). If you see "Cannot connect to the Docker daemon", Docker Desktop isn't running: start it and try again.

A Dockerfile for the Riverstone pipeline

A **Dockerfile** is a script that builds an image, one instruction at a time. The folder you build from is called the **build context**: Docker can copy only files inside it. So `pipeline_image/` holds everything the image needs, and nothing it doesn't:

File	What it is
<code>ingest.py</code> , <code>riverstone_pipeline.py</code>	Chapter 46's pipeline, copied unchanged
<code>run_flash.py</code>	Runs the Flash once, for one day, and exits (below)
<code>requirements.txt</code>	The Python packages, pinned to the versions Chapters 45 and 46 used
<code>.dockerignore</code>	Files Docker must never copy into the image
<code>Dockerfile</code>	The build instructions

Chapter 46's practice helpers, `mock_crm_api.py`, `make_files.py`, and `apply_day.py`, stand in for the ERP and the CRM. They're for testing, so they stay out of the image.

What runs in the container. Chapter 46 ran the pipeline under `dagster dev`, which keeps a web page and a scheduler running all day. That's the right tool on your laptop. Dagster's own documentation says it isn't for production, and it isn't what a once-a-day job needs anyway: in the cloud, the platform's scheduler starts the container at 6:30, the container builds the Flash, and it stops, so you pay for minutes, not days (section 52.7). `run_flash.py` does exactly that one job:

```
import argparse
from datetime import datetime, timedelta
from zoneinfo import ZoneInfo
from riverstone_pipeline import defs

parser = argparse.ArgumentParser(description="Run the Daily Sales Flash for one day.")
parser.add_argument("--day", help="the day to build, as YYYY-MM-DD (default: yesterday, India time)")
args = parser.parse_args()
yesterday = datetime.now(ZoneInfo("Asia/Kolkata")) - timedelta(days=1)
day = args.day or yesterday.strftime("%Y-%m-%d")

job = defs.resolve_job_def("daily_flash_job")
result = job.execute_in_process(partition_key=day, raise_on_error=False)
print(f"Flash for {day}: ", "succeeded" if result.success else "FAILED")
raise SystemExit(0 if result.success else 1)
```

How it works, line by line. You don't run this in the notebook; it runs inside the container.

- `argparse` reads an optional `--day` from the command line, as in Chapter 18 (section 18.15). `description=` and `help=` are the text `python run_flash.py --help` prints. `args.day` is `None` when `--day` isn't given.
- `yesterday` is now, in India time, minus one day (`timedelta`, `ZoneInfo`: Chapter 20). `strftime("%Y-%m-%d")` writes it as `2026-01-02`. `args.day` or `...` uses the given day if there is one, otherwise `yesterday`: the same day Chapter 46's 6:30 schedule builds.
- `defs` is the `Definitions` object from `riverstone_pipeline.py`. `resolve_job_def("daily_flash_job")` fetches the job by name, as Chapter 46 fetched the schedule with `resolve_schedule_def`.

- `execute_in_process(partition_key=day, raise_on_error=False)` runs the whole job for that one day, in this process, like Chapter 46's `materialize()`.
- `raise SystemExit(0 if result.success else 1)` ends the program with **exit code 0** if the run worked and 1 if it didn't (Chapter 26, section 26.0). The platform that starts the container reads that code to decide whether the morning's run failed.

`requirements.txt` pins every package:

```
dagster==1.13.23
dagster-webserver==1.13.23
duckdb==1.5.6
psycopg2-binary==2.9.13
requests==2.33.1
python-dotenv==1.2.3
```

`psycopg2-binary` is the PostgreSQL driver Chapter 45 installed. The `-binary` version comes with everything it needs already compiled, so the image needs no C compiler and no system packages to install it. `dagster-webserver` is only for the local web page, later in this section.

`.dockerignore` lists what Docker must never copy into the image, the image-side twin of Chapter 26's `.gitignore`:

```
.env
__pycache__/
warehouse/
outbox/
alerts/
exports/
```

`.env` holds your passwords (Chapter 45, section 45.2). An image can be copied to anyone who can pull it, so a password copied into one is as good as published. The four folders are data the pipeline writes, which belongs outside the image.

Here's the Dockerfile:

```
FROM python:3.14-slim

RUN useradd --create-home pipeline

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY ingest.py riverstone_pipeline.py run_flash.py ./
RUN mkdir warehouse exports outbox alerts /home/pipeline/dagster_home \
    && chown pipeline warehouse exports outbox alerts /home/pipeline/dagster_home

ENV PYTHONUNBUFFERED=1
ENV DAGSTER_HOME=/home/pipeline/dagster_home
ENV RIVERSTONE_SOURCE="dbname=riverstone_source"

USER pipeline

CMD ["python", "run_flash.py"]
```


Every line, and why it's in that order.

Line	What it does
FROM python:3.14-slim	Starts from Docker Hub's official Python image, the version Chapter 17 installed. <code>slim</code> is a small Debian Linux with Python and little else: less to download, less to keep secure.
RUN useradd --create-home pipeline	RUN runs a command while the image is built. This one creates an ordinary user called <code>pipeline</code> with a home folder (<code>--create-home</code>), to run as later. It comes early because it never changes, so Docker can reuse it (caching, below).
WORKDIR /app	Makes <code>/app</code> the working folder for everything after it, creating it if needed, like <code>cd</code> that also creates the folder.
COPY requirements.txt .	Copies one file from the build context into the image. <code>.</code> means "the working folder", <code>/app</code> .
RUN pip install --no-cache-dir -r requirements.txt	Installs the packages. <code>--no-cache-dir</code> tells pip not to keep its download cache, which would only make the image bigger.
COPY ingest.py riverstone_pipeline.py run_flash.py ./	Copies the three program files. With several files, the destination must end in <code>/</code> , so <code>./</code> rather than <code>..</code> .
RUN mkdir ... \	Creates the four folders the pipeline writes to, and Dagster's home folder. The <code>\</code> at the end of a line continues the command on the next line.
&& chown pipeline ...	<code>&&</code> runs the second command only if the first worked (Chapter 26). <code>chown</code> gives the folders to the <code>pipeline</code> user, because everything created so far belongs to root. Chaining keeps both in one RUN, and one RUN makes one layer.
ENV PYTHONUNBUFFERED=1	ENV sets an environment variable in every container. This one makes Python print each line at once instead of saving lines up, so logs appear as they happen.
ENV DAGSTER_HOME=...	Where Dagster keeps its files when you run <code>dagster dev</code> in the container, later in this section.
ENV RIVERSTONE_SOURCE="dbname=riverstone_source"	A default that isn't secret. The real connection string, with its password, arrives when the container starts (section 52.6).
USER pipeline	Everything from here, including the running container, runs as <code>pipeline</code> , not root, so a flaw in the application can't easily take over the whole container.
CMD ["python", "run_flash.py"]	What runs when a container starts. The list form, with each word in quotes, runs the program directly; it's called the exec form . You can replace it when you start a container, as you'll do below.

Order matters for caching. Docker builds an image in **layers**, one for each `RUN`, `COPY`, or `ADD` instruction, and it reuses a cached layer when nothing that affects it has changed. The packages are installed **before** the application code is copied. If the code were copied first, editing a single line of `ingest.py` would invalidate every layer after it, including the slow package install. Copying dependencies first means a code change rebuilds in seconds, not minutes.

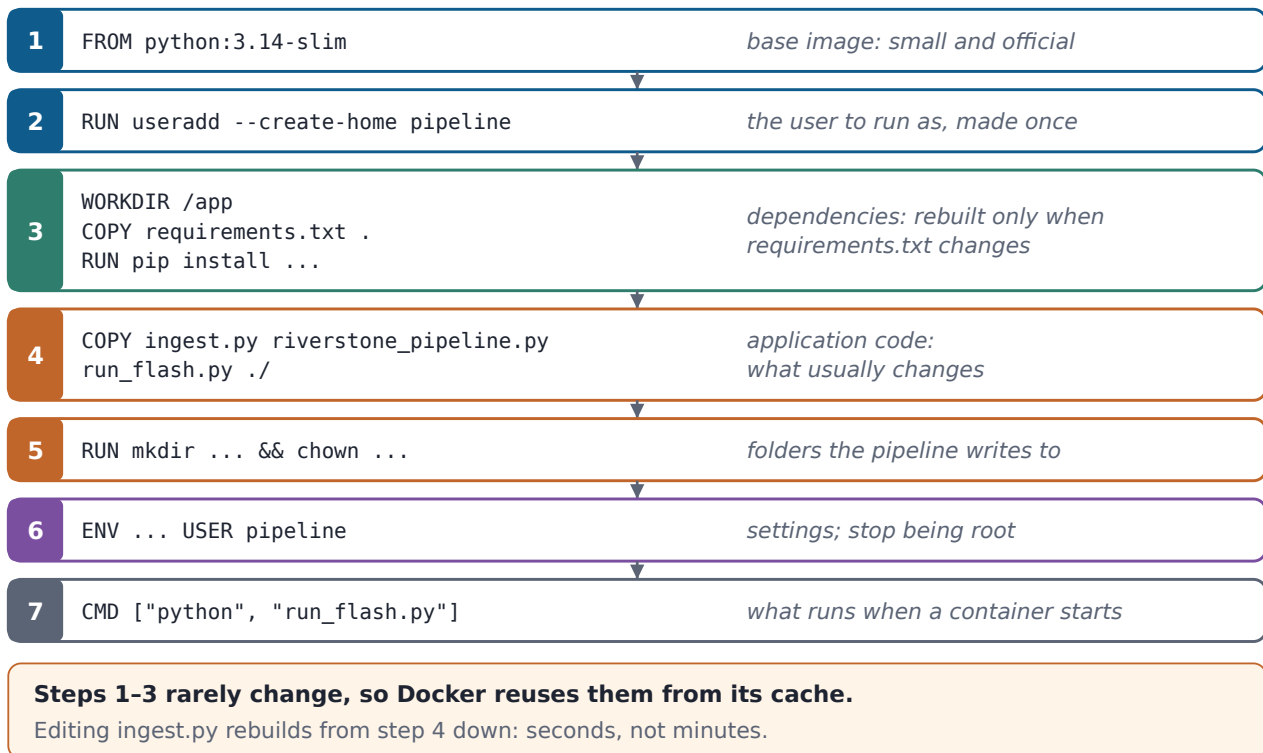


Figure 52.3 – The Dockerfile's steps, in build order. Ordering dependencies before code is what makes rebuilds fast.

Building and running the image

In a terminal, go into `pipeline_image` and build:

```
# terminal
$ cd pipeline_image
$ docker build -t riverstone/pipeline:1.4.0 .
```

- **docker build** reads the `Dockerfile` in the build context and builds an image.
- **-t riverstone/pipeline:1.4.0** names and tags it. `riverstone/` is the owner's name, the way Docker Hub groups images; `1.4.0` is this version of the pipeline.
- `.` at the end is the build context: this folder.

The first build downloads the Python image and the packages, which takes a minute or two, and prints a line for each step, ending with `naming to docker.io/riverstone/pipeline:1.4.0 done`. Now list your images, and start a container just to ask who it runs as:

```
# terminal
$ docker image ls riverstone/pipeline
IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
riverstone/pipeline:1.4.0  3834b8617d20      573MB       134MB

$ docker run --rm riverstone/pipeline:1.4.0 whoami
pipeline
```

- **docker image ls riverstone/pipeline** lists images with that name. The image takes 573 MB on disk; 134 MB is what gets downloaded, because the layers are compressed. Your ID will differ.

- **docker run --rm riverstone/pipeline:1.4.0 whoami**: anything after the image name replaces its CMD. `whoami` prints the current user: `pipeline`, not `root`, as the Dockerfile asked.

Now run the pipeline in the container, for real. It needs what it needed in Chapter 46: the practice source, `riverstone_source`, in your PostgreSQL; the practice CRM API; your `.env` file; and the folders. Set them up as Chapter 46's first run did (section 46.3): in its notebook, run `reset()` and then `apply_day(1)`, shut the notebook's kernel down so it lets go of the warehouse file, and start `python mock_crm_api.py` in a second terminal. Your `.env` from Chapter 45 names `host=localhost`, which is what the container needs. Then, in `work/ch46`:

```
# terminal
$ docker run --rm --network host --env-file .env -v ./exports:/app/exports -v ./warehouse:/app/warehouse -v \
  ./outbox:/app/outbox riverstone/pipeline:1.4.0 python run_flash.py --day 2026-01-02 2> run.log
Flash for 2026-01-02: succeeded

$ echo $?
0

$ cat outbox/flash_2026-01-02.txt
Riverstone Daily Sales Flash - 2026-01-02
Orders booked: 2
Net bookings (excl. cancelled, net of discounts): Rs 38,710.00
```

How it works, piece by piece.

- **--network host** lets the container use your computer's own network, so `localhost` inside it means your computer, where PostgreSQL and the practice API are listening. On Linux this just works. On Docker Desktop, host networking must first be switched on (Settings, Resources, Network, *Enable host networking*; it needs Docker Desktop 4.34 or later and a free Docker sign-in).
- **--env-file .env** passes every line of your `.env` file to the container as environment variables. That's how the password reaches the pipeline without ever being in the image.
- **-v ./exports:/app/exports** mounts a **volume**: the folder `exports` on your computer appears inside the container as `/app/exports`. Anything the container writes there stays on your computer after the container is gone. The warehouse and outbox are mounted the same way, so the container reads and writes Chapter 46's own files.
- On Linux, if the run fails because it can't write to `warehouse` or `outbox`, your user ID differs from the container user's; `chmod a+w warehouse outbox` (Chapter 34, section 34.5) lets it write, for practice.
- **python run_flash.py --day 2026-01-02** replaces CMD, to build the practice day instead of yesterday.
- **2> run.log** sends Dagster's own messages (about 50 lines of its event log, which it writes to standard error) into a file (Chapter 34, section 34.2), leaving just the program's last line on screen.
- **echo \$?** prints the exit code: 0, success, the value a scheduler would check.

Reading it. The container built 2 January's Flash, 2 orders for ₹38,710.00, the same numbers Chapter 46 produced on your laptop. Nothing in the container came from your computer except the settings you passed in and the folders you mounted: the Python, the packages, and the code are the image's own, which is the whole point.

Watch the cache work. Make a one-line change to `ingest.py` (add a comment at the end) and build again, this time with the tag `1.4.1`. The build prints a numbered line for each step; these are the lines for the five steps before the code:

```
# terminal
$ docker build -t riverstone/pipeline:1.4.1 .
#6 [3/7] WORKDIR /app
#6 CACHED
#7 [4/7] COPY requirements.txt .
#7 CACHED
#8 [2/7] RUN useradd --create-home pipeline
#8 CACHED
#9 [5/7] RUN pip install --no-cache-dir -r requirements.txt
#9 CACHED
#10 [6/7] COPY ingest.py riverstone_pipeline.py run_flash.py ./
#10 DONE 0.0s
#11 [7/7] RUN mkdir warehouse exports outbox alerts /home/pipeline/dagster_home    && chown pipeline warehouse
    exports outbox alerts /home/pipeline/dagster_home
#11 DONE 0.1s
```

`CACHED` means Docker reused the layer instead of rebuilding it. Everything up to the package install was reused, and only the code and the step after it were rebuilt, in a fraction of a second. (Docker sometimes prints the steps slightly out of order, as here; the `[n/7]` numbers give the real order.) What happens if you change `requirements.txt` instead? Step 5 and every step after it rebuild, and the packages download again. Delete the extra image when you're done: `docker image rm riverstone/pipeline:1.4.1`.

Try the web page too. `dagster dev` works in the container, for looking around on your laptop: the Compose file below starts it. Two things make it laptop-only. Its web page and scheduler run all day, which costs money in the cloud for a job that needs minutes (section 52.7). And it keeps its run history in files inside the container, in `DAGSTER_HOME`, so the history disappears whenever the container is replaced. A production Dagster runs its web page and its scheduler as separate programs, with the history kept in a PostgreSQL database; Dagster's deployment guide covers it.

Checking the Dockerfile with a parser

A **parser** reads a file into a structure a program can check: here, lists and dictionaries. Back in your notebook, in `work/ch52`:

```
from dockerfile_parse import DockerfileParser

d = DockerfileParser("pipeline_image")
print(d.structure[0])
```

```
{'instruction': 'FROM', 'startline': 0, 'endline': 0, 'content': 'FROM python:3.14-slim\n', 'value': 'python:3.14-slim'}
```

- `DockerfileParser("pipeline_image")` points the parser at the folder; it reads the file called `Dockerfile` in it.
- `d.structure` is a list with one dictionary per instruction. The first one shows the shape: the instruction's name, its first and last line (counting from 0), the full text, and the value after the instruction's name. Comments, if the file had any, would appear as entries named `COMMENT`.

Now check the facts that matter:

```
print("base image :", d.baseimage)
print("instructions:", [s["instruction"] for s in d.structure])
print("layers (RUN/COPY/ADD):", sum(1 for s in d.structure if s["instruction"] in ("RUN", "COPY", "ADD")))
print("runs as user:", [s["value"] for s in d.structure if s["instruction"] == "USER"])
```

```
base image : python:3.14-slim
instructions: ['FROM', 'RUN', 'WORKDIR', 'COPY', 'RUN', 'COPY', 'RUN', 'ENV', 'ENV', 'ENV', 'USER', 'CMD']
layers (RUN/COPY/ADD): 5
runs as user: ['pipeline']
```

- `d.baseimage` is the image named in `FROM`.
- The list comprehension collects every instruction's name, in order.
- `sum(1 for s in ... if ...)` counts the instructions that make a layer: five (two `COPY`, three `RUN`).
- The last line collects the value of every `USER` instruction: the container drops root.

Reading it. The parser confirms the file is well formed and lets you check facts about it, such as "it never runs as root", in a test. A **linter** goes further: a tool such as `hadolint` reads a Dockerfile and flags risky patterns, such as installing system packages without pinning their versions. You've also built the image for real, which is the final check.

Running more than one container together

A single pipeline usually needs a database alongside it during development. **Docker Compose**, which comes with Docker Desktop (on Linux, install the `docker-compose-plugin` package), describes a small stack of containers as one file, `docker-compose.yml`. Compose reads its passwords from a `.env` file in the same folder. Copy `.env.example` to `.env` and put your own values in it:

```
POSTGRES_PASSWORD=change-me-locally
CRM_API_TOKEN=practice-token-45
```

`.env` is in the `.gitignore` of your Chapter 26 repository and in `.dockerignore`, so it never reaches Git or an image. `.env.example`, with no real values, is the one you commit, so others know which settings to provide. Here's the Compose file:

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: riverstone_source
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 10

  pipeline:
    build: .
```

```

command: ["dagster", "dev", "-f", "riverstone_pipeline.py", "-h", "0.0.0.0", "-p", "3000"]
depends_on:
  postgres:
    condition: service_healthy
environment:
  RIVERSTONE_SOURCE: "dbname=riverstone_source host=postgres user=postgres
    password=${POSTGRES_PASSWORD}"
  CRM_API_TOKEN: ${CRM_API_TOKEN}
ports:
  - "3000:3000"
restart: unless-stopped

volumes:
  pgdata:

```

Line by line.

Line	What it means
services:	The containers in the stack, each under its own name. The names double as network names: inside the stack, the database is reachable at the host name <code>postgres</code> .
image: postgres:16	Use the official PostgreSQL 16 image from Docker Hub, the version the book uses.
POSTGRES_DB: riverstone_source	The image creates a database with this name on first start. Without it, only a database called <code>postgres</code> would exist, and the pipeline's connection would fail. It starts empty; the point here is the wiring.
POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}	<code>\$(NAME)</code> is filled in by Compose, when you run it, from your shell's environment or the <code>.env</code> file. The password is never written in this file.
volumes: - pgdata:...	Keeps the database's files, in the container's folder <code>/var/lib/postgresql/data</code> , in a named volume called <code>pgdata</code> , which Docker stores for you. The data survives when the containers are removed.
ports: - "5433:5432"	<code>laptop:container</code> . Port 5433 on your laptop reaches PostgreSQL's port 5432 in the container. Not 5432 on the laptop, because your own PostgreSQL already uses it. Handy locally; in the cloud a database port is never opened this way (section 52.1's security groups).
healthcheck:	How Docker tells whether PostgreSQL is ready: every 5 seconds (<code>interval</code>), up to 10 times (<code>retries</code>), it runs <code>pg_isready</code> , PostgreSQL's own "are you accepting connections?" command. <code>CMD-SHELL</code> means "run this as a shell command".
build: .	Build this service's image from the Dockerfile in this folder.
command: [...]	Replaces the image's <code>CMD</code> for this service: here, <code>dagster dev</code> , the web page and scheduler, on all network addresses (<code>-h 0.0.0.0</code>) and port 3000 (<code>-p 3000</code>), so the page is reachable from outside the container.
depends_on: postgres: condition: service_healthy	Start the pipeline only once the database's health check passes. Without the condition, Compose waits only for the database's container to <i>start</i> , and PostgreSQL may not accept connections for a few more seconds.
RIVERSTONE_SOURCE: "... host=postgres ..."	The connection string, pointing at the <code>postgres</code> service by name.
restart: unless-stopped	If the pipeline's container stops on its own, start it again, unless you stopped it.
volumes: pgdata: (at the bottom)	Declares the named volume used above.

Check it with the parser before running it:

```
import yaml

with open("pipeline_image/docker-compose.yml") as f:
    compose = yaml.safe_load(f)
print("services:", list(compose["services"]))
print("pipeline waits for:", compose["services"]["pipeline"]["depends_on"])
print("postgres health check:", compose["services"]["postgres"]["healthcheck"]["test"])
print("pipeline environment:", list(compose["services"]["pipeline"]["environment"]))
```

```
services: ['postgres', 'pipeline']
pipeline waits for: {'postgres': {'condition': 'service_healthy'}}
postgres health check: ['CMD-SHELL', 'pg_isready -U postgres']
pipeline environment: ['RIVERSTONE_SOURCE', 'CRM_API_TOKEN']
```

- `yaml.safe_load(f)` reads YAML into dictionaries and lists. `safe_load` refuses anything but plain data, which is the right default for files from anywhere.
- The prints walk the structure with keys, as with any dictionary.

Reading it. Two services, and the pipeline waits until PostgreSQL is actually ready, not just started. Both secrets arrive through environment variables from `.env`. Now run it, in `pipeline_image`:

```
# terminal
$ docker compose up -d
Network pipeline_image_default Creating
Network pipeline_image_default Created
Volume pipeline_image_pgdata Creating
Volume pipeline_image_pgdata Created
Container pipeline_image-postgres-1 Creating
Container pipeline_image-postgres-1 Created
Container pipeline_image-pipeline-1 Creating
Container pipeline_image-pipeline-1 Created
Container pipeline_image-postgres-1 Starting
Container pipeline_image-postgres-1 Started
Container pipeline_image-postgres-1 Waiting
Container pipeline_image-postgres-1 Healthy
Container pipeline_image-pipeline-1 Starting
Container pipeline_image-pipeline-1 Started

$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	
STATUS	PORTS				
pipeline_image-pipeline-1	pipeline_image-pipeline	"dagster dev -f rive..."	pipeline	14 seconds ago	Up 7 ↗
seconds	0.0.0.0:3000->3000/tcp				
pipeline_image-postgres-1	postgres:16	"docker-entrypoint.s..."	postgres	14 seconds ago	Up 13 ↗
seconds (healthy)	0.0.0.0:5433->5432/tcp				

- **`docker compose up -d`** builds what needs building, creates a private network and the volume, and starts both containers. `-d` (detached) runs them in the background and gives you the prompt back. Notice `Waiting` then `Healthy`: the pipeline started only after the health check passed.
- **`docker compose ps`** lists the stack's containers and their state. For any container, `docker ps` does the same.

`docker compose logs pipeline` prints what the pipeline container has written; after a few seconds its last line reads `Serving dagster-webserver on http://0.0.0.0:3000`, and `http://127.0.0.1:3000` in your browser shows Chapter 46's graph. To stop:

```
# terminal
$ docker compose down
Container pipeline_image-pipeline-1 Stopping
Container pipeline_image-pipeline-1 Stopped
Container pipeline_image-pipeline-1 Removing
Container pipeline_image-pipeline-1 Removed
Container pipeline_image-postgres-1 Stopping
Container pipeline_image-postgres-1 Stopped
Container pipeline_image-postgres-1 Removing
Container pipeline_image-postgres-1 Removed
Network pipeline_image_default Removing
Network pipeline_image_default Removed
```

`docker compose down` stops and removes the containers and the network, but not the named volume, so the database's data is still there next time. (`docker stop <name>` stops a single container.)

52.3 Orchestrating containers

What Kubernetes actually solves

Running one container by hand is easy. Running many, across many machines, so that a crashed one restarts, traffic reaches a healthy one, and a busy period gets more capacity automatically, is what **Kubernetes** does. It's worth understanding even for a company Riverstone's size, because it's the vocabulary most cloud job descriptions and most managed platforms are built on. **Read this section for its vocabulary.** Trying it on your laptop is optional, and described at the end of the section.

Object	What it is
Cluster	A group of machines that Kubernetes manages as one
Node	One machine in the cluster
Pod	The smallest deployable unit: one or more containers that always run together, on the same node
Job	Runs a Pod until it finishes successfully, once
CronJob	Creates a Job on a schedule, like the scheduler in Chapter 46
Deployment	Keeps a stated number of copies (replicas) of a Pod running all the time, and rolls out new versions
Service	A stable network address in front of a changing set of Pods, so other things can find them reliably
Secret	A piece of sensitive configuration (a token, a password), stored separately from the container image
Namespace	A way of dividing one cluster into separate areas, for different teams or environments

You describe the objects you want in YAML files called **manifests**, and Kubernetes makes the cluster match them.

A CronJob for the Daily Sales Flash

The Flash is a batch job: it should start at 6:30 every morning, run once, and stop. That's a **CronJob**. Here is the smallest one that would work:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-flash
  namespace: riverstone
spec:
  schedule: "30 6 * * *"
  timeZone: "Asia/Kolkata"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: pipeline
              image: riverstone/pipeline:1.4.0
```

Line by line.

- **apiVersion: batch/v1** says which part of Kubernetes' vocabulary the object comes from, and which version of it. Batch objects (Job, CronJob) are in `batch/v1`; Deployments are in `apps/v1`; the oldest, core objects, such as Service, just say `v1`.
- **kind: CronJob** is the type of object.
- **metadata:** names it: `name: daily-flash`, and `namespace: riverstone`, the area of the cluster it lives in, so Riverstone's objects stay apart from everyone else's.
- **spec:** (specification) is what you want. **schedule: "30 6 * * *"** is the same cron expression Chapter 46's schedule printed: minute 30, hour 6, every day. **timeZone: "Asia/Kolkata"** makes that 6:30 India time; without it, the cluster's own clock decides, often UTC.
- **concurrencyPolicy: Forbid** says never to start a run while the previous one is still going. A Flash must never be built twice at once (Chapter 46's idempotency lessons).
- **jobTemplate:** is the Job to create each morning, and inside it, **template:** is the Pod that Job runs. The nesting reads: a CronJob makes Jobs, a Job makes a Pod.
- **restartPolicy: OnFailure** restarts the container if it exits with a non-zero code: `run_flash.py`'s exit code, doing its job.
- **containers:** is a YAML list: each item starts with `-`. This Pod has one container, named `pipeline`, started from the image built in section 52.2. The container runs the image's `CMD`, so each morning it builds yesterday.

A real run needs two more things: its secrets, and limits on what it may use. Added to the container, they make the companion file `k8s/pipeline-cronjob.yaml`:

```
- name: pipeline
  image: riverstone/pipeline:1.4.0
  env:
    - name: RIVERSTONE_SOURCE
      valueFrom:
        secretKeyRef:
          name: riverstone-db-secret
          key: connection-string
    - name: CRM_API_TOKEN
      valueFrom:
        secretKeyRef:
          name: riverstone-crm-secret
          key: token
  resources:
    requests:
      cpu: "250m"
      memory: "512Mi"
    limits:
      cpu: "1"
      memory: "1Gi"
```

- **env**: sets environment variables in the container, a list of `name` plus where the value comes from. **valueFrom: secretKeyRef**: takes it from a Kubernetes Secret: the one named `riverstone-db-secret`, its entry with the key `connection-string`. The values never appear in this file, which you can safely commit.
- **resources.requests** is what the Pod is guaranteed, and what the cluster uses to choose a node with room for it. **limits** is the ceiling. `m` means thousandths of a CPU core, so `250m` is a quarter of a core and `"1"` a whole one. `Mi` and `Gi` are mebibytes and gibibytes, the binary units of Chapter 2 (1 Mi = 1,024 × 1,024 bytes).

Check the whole file:

```
with open("k8s/pipeline-cronjob.yaml") as f:
    cron = yaml.safe_load(f)
    pod = cron["spec"]["jobTemplate"]["spec"]["template"]["spec"]
    container = pod["containers"][0]
    print(cron["kind"], cron["metadata"]["name"], "in namespace", cron["metadata"]["namespace"])
    print("schedule:", cron["spec"]["schedule"], cron["spec"]["timeZone"])
    print("image :", container["image"])
    print("secrets :", [e["valueFrom"]["secretKeyRef"]["name"] for e in container["env"]])
    print("cpu request/limit:", container["resources"]["requests"]["cpu"], "/", container["resources"]["limits"]
          ["cpu"])
```

```
CronJob daily-flash in namespace riverstone
schedule: 30 6 * * * Asia/Kolkata
image : riverstone/pipeline:1.4.0
secrets : ['riverstone-db-secret', 'riverstone-crm-secret']
cpu request/limit: 250m / 1
```

- `pod = ...` walks down the nesting, CronJob to Job to Pod, once, so the next lines stay short.
- `container["env"]` is a list of dictionaries; the list comprehension picks each one's Secret name.

Reading it. One CronJob, 6:30 India time, the image from section 52.2, two Secrets referenced and never inlined. A parser checks that the YAML is well formed, not that Kubernetes would accept it. For that, a tool called kubeconform checks manifests against Kubernetes' own rules without a cluster; it was run on both of this section's files and reported `Valid: 3, Invalid: 0` (the second file holds two objects).

Services that run all the time: Deployment and Service

A CronJob suits work that starts and stops. A web page must answer at any moment, so it runs all the time, usually in more than one copy. Suppose Riverstone later adds a small internal web page showing the latest Flash (Exercise 11 imagines it), called `flash-page`. It keeps no data of its own: it reads the warehouse. That's `k8s/flash-page.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flash-page
  namespace: riverstone
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flash-page
  template:
    metadata:
      labels:
        app: flash-page
    spec:
      containers:
        - name: web
          image: riverstone/flash-page:1.0.0
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: flash-page
  namespace: riverstone
spec:
  selector:
    app: flash-page
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```

Line by line, the new parts.

- **replicas: 2** keeps two copies running. If one crashes, or its node fails, Kubernetes starts a replacement, and the other keeps answering meanwhile.
- **Labels** are name tags: `labels: app: flash-page` under `template.metadata` puts the tag `app=flash-page` on every Pod the Deployment makes. **selector: matchLabels: app: flash-page**

tells the Deployment which Pods are its own, so it can count them. The two must match, or Kubernetes rejects the Deployment.

- **containerPort: 8080** records the port the web page listens on inside the container.
- **readinessProbe** is how Kubernetes decides a Pod is ready for traffic: it asks for `/healthz` on port 8080 every few seconds, starting 5 seconds after the container starts (`initialDelaySeconds`), and sends a Pod traffic only while the answer is a success. **The path must be one the application actually serves**; this page would serve `/healthz` for exactly this purpose. Check before you deploy: Dagster's web server, for example, answers `/server_info`. A **liveness probe**, written the same way, goes one step further: when it fails, Kubernetes restarts the container.
- `---` separates two YAML documents in one file: the Deployment, then the Service.
- **The Service** finds its Pods by the same label (`selector: app: flash-page`) and gives them one stable name, `flash-page`, on port 80, forwarding to port 8080 (`targetPort`) on whichever Pod is ready.
- **type: ClusterIP** means the Service is reachable only from inside the cluster, which suits an internal page. To reach something from the internet you'd use `type: LoadBalancer`, which asks the cloud for a load balancer like the one in Figure 52.2, or an **Ingress**, one shared entrance for several services.

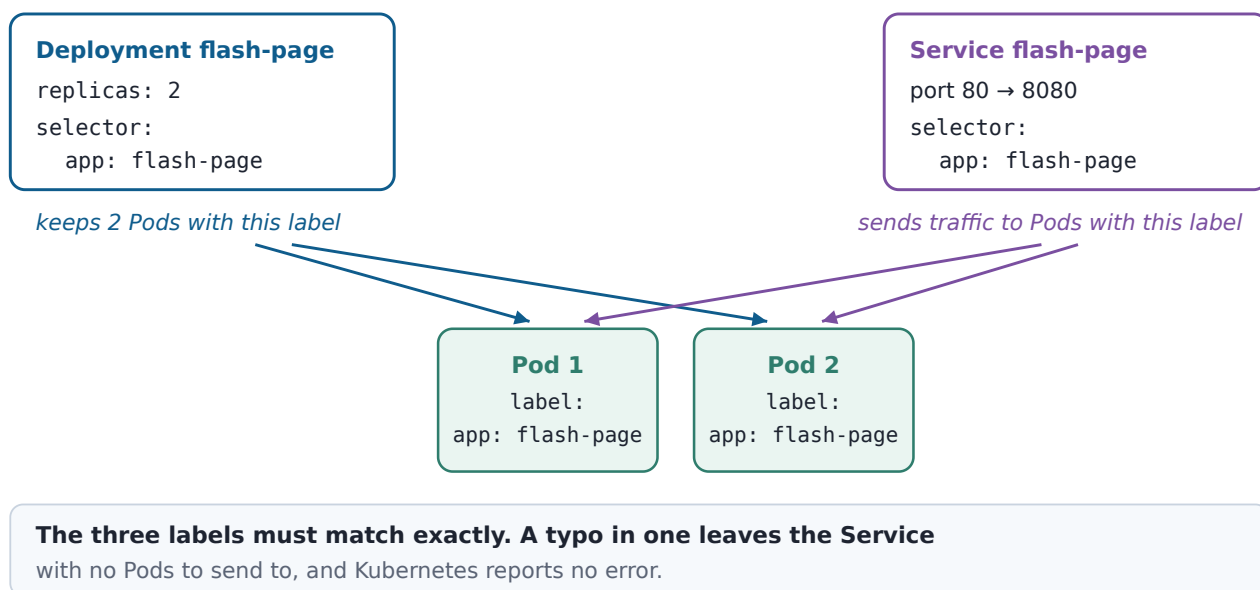


Figure 52.4 – Labels are how a Deployment counts its Pods and how a Service finds them.

Why the Flash isn't a Deployment with two replicas. Two copies suit a web page because either copy can answer any request. A scheduled job is the opposite: if two copies each ran Chapter 46's 6:30 schedule, the Flash would be built and sent **twice** every morning. Replicas are for services that keep no state of their own; a job that must run exactly once belongs in a CronJob, or on a scheduler outside the cluster.

Check the file:

```
with open("k8s/flash-page.yaml") as f:
    deployment, service = yaml.safe_load_all(f)
print(deployment["kind"], "replicas:", deployment["spec"]["replicas"])
print("labels match:", deployment["spec"]["selector"]["matchLabels"] == deployment["spec"]["template"]
      ["metadata"]["labels"]
      == service["spec"]["selector"])
print(service["kind"], "port", service["spec"]["ports"][0]["port"], "->", service["spec"]["ports"][0]
      ["targetPort"],
      f"({service['spec']['type']})")
```

```
Deployment replicas: 2
labels match: True
Service port 80 -> 8080 (ClusterIP)
```

- `yaml.safe_load_all(f)` reads every document in the file, one per `---`; `deployment, service = ...` unpacks the two.
- The second print compares the three label dictionaries in one chained `==`: `True` only if all three are equal.

Secrets for the cluster

Both CronJob Secrets have to exist in the cluster before the first run. A Secret can be written as a manifest:

```
apiVersion: v1
kind: Secret
metadata:
  name: riverstone-crm-secret
  namespace: riverstone
stringData:
  token: replace-me
```

Never commit a file like this with a real value in it. A Secret is only **base64-encoded** when Kubernetes stores it (a reversible way of writing bytes as text, not encryption), and anyone who reads the file or the Git history has the token. Create Secrets from your `.env` file instead, so the value never lands in a file you might commit:

```
# terminal
$ kubectl create secret generic riverstone-crm-secret --namespace riverstone --from-env-file=.env
```

`kubectl create secret generic` makes a Secret named `riverstone-crm-secret`, with one entry per line of `.env`. Typing a value directly on the command line with `--from-literal=token=...` also works, but, as Chapter 34 warned about `export`, it lands in your shell's history. Section 52.6 covers the better options.

Trying it on your laptop (optional, not executed here)

The book's practice machine can't run Kubernetes' own containers, so nothing in this subsection is executed here, and no output is shown. These steps are for your laptop, with Docker running.

1. Install **kind** ("Kubernetes in Docker", a whole cluster inside one Docker container) and **kubectl**, Kubernetes' command-line tool, from their documentation at kind.sigs.k8s.io and kubernetes.io.
2. `kind create cluster` creates a one-node cluster; `kubectl get nodes` lists it, with `Ready` in the status column.
3. `kind load docker-image riverstone/pipeline:1.4.0` copies your image into the cluster, which can't see your laptop's images otherwise.
4. `kubectl create namespace riverstone`, then create the two Secrets, as above.
5. `kubectl apply -f k8s/pipeline-cronjob.yaml` creates the CronJob. **apply** means "make the cluster match this file"; run it again after editing the file and it changes only what differs.
6. `kubectl get cronjobs -n riverstone` lists it with its schedule. `kubectl create job flash-now --from=cronjob/daily-flash -n riverstone` starts one run now, without waiting for 6:30; `kubectl get pods -n riverstone` shows its Pod, and `kubectl logs <pod name> -n riverstone` shows what the container printed. In the cluster, `localhost` means the Pod itself, so the connection string in `riverstone-db-secret` must name a database the Pod can reach.
7. For `flash-page.yaml` you'd need an image of that page. With one, `kubectl get pods` would list two Pods; `kubectl delete pod <one of them>` and a second `kubectl get pods` shows a new Pod with a new name taking its place: the Deployment keeping its promise of two.
8. `kind delete cluster` removes everything.

When you don't need it

Kubernetes solves problems that come with scale and many services. Riverstone, with one pipeline, doesn't have that problem yet. A single managed container service (Chapter 46's job, deployed as one container on a managed platform such as AWS ECS on Fargate, which can start a container on a schedule) is simpler, cheaper, and has almost nothing to operate. **Reach for Kubernetes when you have several services that need to be scheduled, scaled, and networked together**, not because the term appears in job postings. Section 52.8 returns to this choice directly.

52.4 Infrastructure as Code

Clicking through a cloud provider's console to create a bucket works once. It doesn't tell the next person what was created, why, or how to make an identical one in another region. **Infrastructure as Code (IaC)** describes infrastructure in files that can be reviewed, versioned, and applied repeatedly, the same discipline Chapter 29 taught for application code.

Terraform is the most widely used tool for this, and it works with every major cloud provider through the same workflow: write configuration, run `terraform plan` to see exactly what would change, and `terraform apply` to make it happen. Terraform keeps a **state file** recording what it created, which is how it knows the difference between "create this" and "this already exists, leave it alone." Terraform talks to each cloud through a **provider**, a plug-in it downloads. (OpenTofu, a free fork of Terraform, reads the same files with the command `tofu` instead of `terraform`.)

Installing Terraform

- **Windows:** `winget install Hashicorp.Terraform` in PowerShell.
- **macOS:** `brew install hashicorp/tap/terraform` with Homebrew.
- **Linux and WSL:** follow the instructions for your distribution at developer.hashicorp.com/terraform/install.

Then check:

```
# terminal
$ terraform -version
Terraform v1.16.4
on linux_amd64
```

Your version may be newer; this chapter's files were checked with 1.16.4, and any recent version reads them. `linux_amd64` is the operating system and processor type; yours may say `windows_amd64` or `darwin_arm64` (a Mac).

A configuration for the sensor archive, in four parts

Terraform files are written in **HCL** (HashiCorp Configuration Language) and end in `.tf`. The companion's `terraform/main.tf` creates the storage behind Chapter 49's sensor archive and the identity that's allowed to touch it. It has four parts.

Part 1: settings, state, provider, and variables.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  backend "s3" {
    bucket  = "riverstone-tfstate"
    key     = "platform/terraform.tfstate"
    region  = "ap-south-1"
    use_lockfile = true
  }
}

provider "aws" {
  region = var.aws_region
}

variable "aws_region" {
  description = "AWS region for Riverstone's data platform"
  type        = string
  default     = "ap-south-1" # Mumbai
}

variable "environment" {
  description = "Deployment environment"
  type        = string
  default     = "production"
}
```

- HCL is made of **blocks**: a type, sometimes one or two labels in quotes, then settings inside `{ }`, each written `name = value`. `#` starts a comment.
- **terraform { required_providers { ... } }** says which providers this configuration needs. `source = "hashicorp/aws"` is the AWS provider, published by HashiCorp, Terraform's maker. `version = "~> 6.0"` accepts any 6.x version (6.0 or later, but below 7.0), so a future 7.0 with breaking changes can't arrive by surprise.
- **backend "s3" { ... }** says where the **state** lives: not on your laptop, but in an S3 bucket (`bucket`), under a file name (`key`), in Mumbai (`region`). `use_lockfile = true` makes Terraform write a small lock file next to the state while it works, so two people running Terraform at once can't corrupt each other's changes. (Older setups lock with a DynamoDB table instead.) The state bucket is created once, by hand or by a separate small configuration, before this one is used.
- **provider "aws" { region = var.aws_region }** configures the provider. **Credentials never go in this file.** The provider finds them in your environment: variables such as `AWS_PROFILE`, or a login made with AWS's own command-line tool. In CI they come from a role (section 52.5).
- **variable "aws_region" { ... }** declares an input: a description for people, a type, and a default used when nobody sets it. `var.aws_region` reads it. Variables let one file build Mumbai production or Singapore staging without editing.

What the state holds. The state file records every resource Terraform created and every attribute of it, including, for some resources, **passwords and keys in plain text**. So keep it in an encrypted, access-controlled bucket, never in Git: add `*.tfstate`, `*.tfstate.backup`, and `.terraform/` to your `.gitignore`.

Part 2: the bucket, with versioning.

```
resource "aws_s3_bucket" "sensor_archive" {
  bucket = "riverstone-sensor-archive-${var.environment}"

  tags = {
    Project      = "analyst-to-architect"
    Environment  = var.environment
    ManagedBy    = "terraform"
  }
}

resource "aws_s3_bucket_versioning" "sensor_archive" {
  bucket = aws_s3_bucket.sensor_archive.id

  versioning_configuration {
    status = "Enabled"
  }
}
```

- **resource "aws_s3_bucket" "sensor_archive"** has two labels: the **type** of thing to create (an S3 bucket, from the AWS provider) and a **name** that Terraform uses for it inside your files. The name never appears in AWS.
- **"riverstone-sensor-archive-\${var.environment}"**: `${ ... }` inside a string inserts a value, so this becomes `riverstone-sensor-archive-production`. Bucket names are shared across all of AWS, which is why they're long and specific.

- `tags = { ... }` attaches labels to the bucket, a **map** of names to values. Tags are how a company finds, and pays for, what each project owns (Chapter 65, section 65.4).
- `aws_s3_bucket.sensor_archive.id` is a **reference**: `type.name.attribute`, the ID of the bucket above. A reference also tells Terraform the order: it must create the bucket before it can turn versioning on for it.
- `versioning_configuration { status = "Enabled" }` is a **nested block**, a block inside a resource. Versioning keeps every version of every object, so an overwritten or deleted file can be brought back. Versioning is what let Chapter 49's team restore the deleted days in its closing story; there it happened to be on, and here it's on by design.

Part 3: what happens to old versions.

```
resource "aws_s3_bucket_lifecycle_configuration" "sensor_archive" {
  bucket = aws_s3_bucket.sensor_archive.id

  rule {
    id      = "old-versions-to-cold-storage"
    status = "Enabled"

    filter {}

    noncurrent_version_transition {
      noncurrent_days = 90
      storage_class   = "GLACIER"
    }
  }
}
```

- A **lifecycle configuration** is a set of rules S3 applies to objects automatically. This one has one rule, with an `id` for people and `status = "Enabled"`.
- `filter {}`, empty, means "apply to every object in the bucket". A filter could name a folder instead.
- `noncurrent_version_transition` acts only on **previous versions**: when a file is overwritten, the old version becomes "noncurrent", and 90 days after that (`noncurrent_days`) it moves to `GLACIER`, a cold storage class that is much cheaper to keep and slower and dearer to read back. The current data never moves. That keeps versioning's safety net without paying hot-storage prices for it.

Part 4: the pipeline's identity.

```
resource "aws_iam_role" "pipeline_role" {
  name = "riverstone-pipeline-${var.environment}"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "ecs-tasks.amazonaws.com" }
      Action    = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy" "pipeline_s3_access" {
  name = "sensor-archive-read-write"
  role = aws_iam_role.pipeline_role.id
}
```

```

policy = jsonencode({
  Version = "2012-10-17"
  Statement = [{
    Effect = "Allow"
    Action = ["s3:GetObject", "s3:PutObject", "s3:ListBucket"]
    Resource = [
      aws_s3_bucket.sensor_archive.arn,
      "${aws_s3_bucket.sensor_archive.arn}/*"
    ]
  }]
})

output "bucket_name" {
  value = aws_s3_bucket.sensor_archive.bucket
}

```

A role has **two** policies, and they answer different questions:

Policy	Question it answers	Here
Trust policy (<code>assume_role_policy</code>)	<i>Who</i> may put this role on?	Containers run by AWS ECS, the managed container service Riverstone deploys to (section 52.8)
Permission policy (<code>aws_iam_role_policy</code>)	<i>What</i> may the role do, once on?	Section 52.1's three actions, on this one bucket

- `jsonencode({ ... })` turns an HCL map into JSON text. Read it as "write this out as the JSON policy of section 52.1": `Version`, `Statement`, `Effect`, `Action`, `Resource` are the same fields.
- **Principal** = `{ Service = "ecs-tasks.amazonaws.com" }` is the **principal**, who is trusted: the ECS service, starting a container. On Kubernetes on AWS (EKS), the trust policy would name the cluster's identity provider instead.
- **Action** = `"sts:AssumeRole"` is the permission to put the role on. **STS** (Security Token Service) is the part of AWS that hands out temporary credentials.
- **role** = `aws_iam_role.pipeline_role.id` attaches the permission policy to the role above. `aws_s3_bucket.sensor_archive.arn` is the bucket's ARN, which AWS assigns when it creates the bucket; the reference fills it in.
- **output "bucket_name"** prints the created bucket's name after `apply`, so other configurations (or a human) can use it without hunting through a console.

Checking it, for real

Terraform checks its own files without a cloud account. In a terminal, in `work/ch52/terraform`:

```

# terminal
$ terraform init -backend=false
$ terraform fmt -check
$ terraform validate
Success! The configuration is valid.

```

- **terraform init** prepares the folder. It downloads the AWS provider into a hidden `.terraform/` folder, printing `Installing hashicorp/aws v6.66.0...` (or a newer 6.x), and records the exact version and its checksums in `.terraform.lock.hcl`, which you do commit, so everyone gets the

same provider. It ends with Terraform has been successfully initialized!. **-backend=false** skips connecting to the state bucket, which you don't have yet. Without it, `init` would try to reach S3 and stop.

- **terraform fmt -check** checks the layout (indentation, lined-up = signs) and prints nothing when it's already right. Without `-check`, `fmt` fixes the layout for you.
- **terraform validate** checks that every block, setting, and reference is valid for this provider version: a misspelled setting or a reference to a resource that doesn't exist fails here.

The same file, read into Python:

```
import hcl2

with open("terraform/main.tf") as f:
    cfg = hcl2.load(f)
    print("top-level blocks:", list(cfg.keys()))
    print("resources:", [list(r.keys())[0].strip(' ') for r in cfg["resource"]])
    print("variables:", [list(v.keys())[0].strip(' ') for v in cfg["variable"]])
    print("outputs :", [list(o.keys())[0].strip(' ') for o in cfg["output"]])
```

```
top-level blocks: ['terraform', 'provider', 'variable', 'resource', 'output', '__comments__']
resources: ['aws_s3_bucket', 'aws_s3_bucket_versioning', 'aws_s3_bucket_lifecycle_configuration',
            'aws_iam_role', 'aws_iam_role_policy']
variables: ['aws_region', 'environment']
outputs   : ['bucket_name']
```

- `hcl2.load(f)` reads the file into dictionaries: one key per block type, each holding a list of blocks.
- `cfg["resource"]` is a list of one-key dictionaries, `{"aws_s3_bucket": {...}}`. `list(r.keys())[0]` takes the key. The parser keeps the quotation marks from resource `"aws_s3_bucket"`, so `.strip(' ')` removes them.
- `__comments__` is the parser keeping the file's comments. Ignore it.

Reading it. Five resources, two variables, one output: the configuration you just read, part by part.

Plan, apply, and why review matters

- **terraform plan** shows what would change, without changing anything. Reading a plan carefully, especially any line that destroys something, is the single habit that prevents most infrastructure incidents. `terraform plan -out=plan.tfplan` saves the plan, and `terraform apply plan.tfplan` then does exactly what was reviewed, nothing more.
- **terraform apply** makes the change, after showing the plan and asking for confirmation (or automatically in CI, once a plan has been reviewed).
- **State** must be stored somewhere shared and locked (the `backend` block above), not on a laptop, or two people running Terraform at once can corrupt each other's changes.
- **Every change goes through a pull request** (Chapter 26), reviewed like any other code change, precisely because "it's just infrastructure" is how the closing story in this chapter goes wrong.

Reading a plan

`terraform plan` compares the configuration with the state and with what really exists, and prints what it would do. This plan is real: for resources that don't exist yet, Terraform needs to ask AWS nothing, so it was run with no account at all. Here are the first lines of its 130, one resource, and the last line:

```
Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
```

```
+ create
```

```
Terraform will perform the following actions:
```

```
# aws_s3_bucket_versioning.sensor_archive will be created
+ resource "aws_s3_bucket_versioning" "sensor_archive" {
  + bucket = (known after apply)
  + id      = (known after apply)
  + region  = "ap-south-1"

  + versioning_configuration {
    + mfa_delete = (known after apply)
    + status      = "Enabled"
  }
}
```

```
Plan: 5 to add, 0 to change, 0 to destroy.
```

- **+** means create. The other symbols are **~** update in place, **-** destroy, and **-/+** destroy and create again (replace), for a change that can't be made in place.
- **(known after apply)** is a value AWS decides when it creates the resource, such as the bucket's ID.
- **Plan: 5 to add, 0 to change, 0 to destroy.** is the summary. **Read the whole plan anyway:** the summary tells you *that* something will be destroyed, not *what*.

Practice: what a destroy looks like

The companion's `terraform_practice/main.tf` uses `terraform_data`, a resource built into Terraform that creates nothing outside it, so it needs no provider download and no cloud account. Its two resources stand in for two firewall rules. In `work/ch52/terraform_practice`, run `terraform init`, then `terraform apply` and type `yes`. Then delete the second resource block, `rule_pipeline_to_database`, from `main.tf`, and plan again. This is the whole plan, apart from its closing note.

```
# terminal
$ terraform plan
terraform_data.rule_https_from_internet: Refreshing state... [id=1bf20d3e-1853-90ca-7e0d-5d31525208f5]
terraform_data.rule_pipeline_to_database: Refreshing state... [id=bc1847c9-674b-e6ff-747b-39b57bf3b9b9]

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

# terraform_data.rule_pipeline_to_database will be destroyed
# (because terraform_data.rule_pipeline_to_database is not in configuration)
- resource "terraform_data" "rule_pipeline_to_database" {
  - id      = "bc1847c9-674b-e6ff-747b-39b57bf3b9b9" -> null
  - input   = "allow 5432 from the pipeline to the database" -> null
  - output  = "allow 5432 from the pipeline to the database" -> null
}

Plan: 0 to add, 0 to change, 1 to destroy.
```

(The plan ends with a note about saving it with `-out`; see below. Your IDs will differ.) **Reading it.**

Terraform wants to destroy the rule **because it is not in configuration**: to Terraform, the file is the truth, and anything it manages that the file no longer mentions should go. Keep that sentence in mind for this chapter's closing story. `terraform destroy` removes everything a configuration created; run it here to clean up.

Simplification note. `main.tf` passes `terraform validate`, and its plan is real, but it has never been applied to a real AWS account in producing this book. It creates the storage and the identity; the rest of a deployment, such as the container registry, the ECS cluster, the task definition that tells ECS how to run the image, and the 6:30 schedule, would be more resources in the same style. Treat it as a correct starting point to adapt, not as a file to copy and apply unread.

52.5 CI/CD

Continuous integration runs checks (tests, linting) automatically on every change. **Continuous deployment** (or delivery) takes a change that passes those checks and ships it, automatically or with one approval. Together, **CI/CD** is what makes "every change is reviewed and tested before it reaches production" true by construction, rather than by everyone remembering to do it.

Chapter 26 (section 26.8) built a one-job check with **GitHub Actions**. Here's a three-job workflow for Riverstone's pipeline, `.github/workflows/deploy.yml`. GitHub runs workflows only from the `.github/workflows/` folder of a repository.

```
name: Riverstone pipeline CI/CD

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
```

```

permissions:
  id-token: write
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.14"
      - run: python -m pip install -r pipeline_image/requirements.txt pytest
      - run: python -m pytest tests/

  build-and-push:
    needs: test
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: ${ secrets.AWS_DEPLOY_ROLE_ARN }
          aws-region: ap-south-1
      - id: ecr
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build and push the image
        env:
          IMAGE: ${ steps.ecr.outputs.registry }/riverstone/pipeline:${ github.sha }
        run: |
          docker build -t "$IMAGE" pipeline_image/
          docker push "$IMAGE"

  deploy:
    needs: build-and-push
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: ${ secrets.AWS_DEPLOY_ROLE_ARN }
          aws-region: ap-south-1
      - id: ecr
        uses: aws-actions/amazon-ecr-login@v2
      - id: render
        uses: aws-actions/amazon-ecs-render-task-definition@v1
        with:
          task-definition-family: riverstone-daily-flash
          container-name: pipeline
          image: ${ steps.ecr.outputs.registry }/riverstone/pipeline:${ github.sha }
      - uses: aws-actions/amazon-ecs-deploy-task-definition@v2
        with:
          task-definition: ${ steps.render.outputs.task-definition }}

```

Line by line, the parts Chapter 26 didn't cover.

Line	What it means
on: pull_request: branches: [main]	Run for a pull request whose target is main, whatever branch it comes from.
on: push: branches: [main]	Run when commits land on main, which is what merging a pull request does.

Line	What it means
<code>permissions: id-token: write</code>	Lets the workflow ask GitHub for a signed statement of which repository and branch it is: the key to the passwordless AWS login below. <code>contents: read</code> lets it read the code and nothing more.
<code>test:</code>	Installs the pipeline's packages plus <code>pytest</code> and runs the tests in <code>tests/</code> , the tests Chapter 29 (section 29.7) taught you to write.
<code>needs: test</code>	This job waits for <code>test</code> and runs only if it succeeded.
<code>if: github.event_name == 'push' && ...</code>	A condition. <code>github.event_name</code> is what started the workflow (push or pull_request); <code>github.ref</code> is the branch, written <code>refs/heads/main</code> . <code>&&</code> means both must be true. Values inside <code>\${{ ... }}</code> , and conditions in <code>if:</code> , are GitHub's expressions , worked out before the job starts.
<code>uses: aws-actions/configure-aws-credentials@v6</code>	A prepared action from AWS, pinned to major version 6, as Chapter 26 pinned its actions. With <code>role-to-assume</code> , it logs in to AWS by OpenID Connect (OIDC) : GitHub vouches for the workflow, and AWS hands back temporary credentials for the named role. No AWS password is stored anywhere. <code>with:</code> passes the action its settings.
<code>\${{ secrets.AWS_DEPLOY_ROLE_ARN }}</code>	Reads a repository secret , which you add on GitHub under the repository's Settings, <i>Secrets and variables, Actions</i> . GitHub stores it encrypted and hides it in logs (section 52.6). This one holds the ARN of a deploy role, with its own trust and permission policies, set up once in Terraform.
<code>id: ecr</code> and <code>uses: aws-actions/amazon-ecr-login@v2</code>	Logs Docker in to ECR , AWS's private image registry. Without a login, <code>docker push</code> is refused. <code>id: ecr</code> names the step so later steps can read its outputs : <code>steps.ecr.outputs.registry</code> is the registry's address.
<code>env: IMAGE: ...</code>	An environment variable for this step: the full image name, registry plus <code>riverstone/pipeline</code> , tagged with <code>github.sha</code> , the commit's ID, so every image says exactly which code it holds.
<code>run: </code>	The <code> </code> lets <code>run:</code> hold several lines, one command each: build the image, then push it to the registry.
<code>environment: production</code>	Ties the job to a GitHub environment called <code>production</code> , which can require a named person's approval before the job runs.
<code>amazon-ecs-render-task-definition@v1</code>	Fetches the current task definition , ECS's recipe for running the container (image, memory, the role from section 52.4), for the family <code>riverstone-daily-flash</code> , and writes a copy with the new image in it.
<code>amazon-ecs-deploy-task-definition@v2</code>	Registers that copy as a new revision. Given no ECS service to update, it only registers, which is what a scheduled task needs.

Not executed here. `build-and-push` and `deploy` need an AWS account, a registry, and the ECS resources, so they're shown as GitHub Actions would run them. The daily schedule that starts the task at 6:30 (on AWS, an EventBridge Scheduler schedule) is set up once, like the bucket; whether it picks up a newly registered revision by itself depends on how the schedule names the task definition, so check your scheduler's documentation and write the answer into your runbook (Project, step 8). The file was checked with `actionlint`, a checker for workflow files, which reported no problems.

Working out exactly what runs, for real

Reading a workflow file and knowing what it will actually do for a given event is a skill worth having independently of any specific tool. `simulate_workflow.py` applies the workflow's own rules in Python. Its core is two functions:

```
def matches_trigger(workflow, event_name, branch):
    on = workflow.get(True) or workflow.get("on")    # PyYAML can read a bare 'on:' as True
    if event_name not in on:
        return False
    branches = (on[event_name] or {}).get("branches")
    return not branches or branch in branches

def jobs_that_run(workflow, event_name, branch):
    context = {"event_name": event_name, "ref": f"refs/heads/{branch}"}
    if not matches_trigger(workflow, event_name, branch):
        return []
    would_run = {}
    for name, job in workflow["jobs"].items():
        needs = job.get("needs")
        needs = [needs] if isinstance(needs, str) else (needs or [])
        would_run[name] = all(would_run.get(n) for n in needs) and eval_if(job.get("if"), context)
    return [name for name, runs in would_run.items() if runs]
```

- **matches_trigger** looks the event up in the `on:` block, then checks the branch against its `branches:` list, if there is one. YAML treats the bare word `on` as "true", so PyYAML may store the key as `True`, which is why the first line tries both.
- **jobs_that_run** goes through the jobs in order. A job runs if every job it `needs` runs and its `if:` condition is true. `eval_if` (in the file) handles the two tests this workflow uses, on `github.event_name` and `github.ref`, joined by `&&`.
- It deliberately **doesn't** model the rest of GitHub's rules: path filters, tags, manual runs (`workflow_dispatch`), schedules, or a job that fails. It answers one question, "which jobs would start?", for this kind of workflow.

```
from simulate_workflow import load_workflow, jobs_that_run

wf = load_workflow(".github/workflows/deploy.yml")
cases = [
    ("pull_request", "main", "a pull request targeting main"),
    ("push", "main", "a direct push to main"),
    ("push", "develop", "a push to a branch other than main"),
]
for event, branch, description in cases:
    print(f"{description:<38} -> {jobs_that_run(wf, event, branch)}")
```

```
a pull request targeting main      -> ['test']
a direct push to main              -> ['test', 'build-and-push', 'deploy']
a push to a branch other than main -> []
```


Reading it, and why each result is correct.

- A **pull request targeting main** triggers the workflow (the `pull_request` rule matches), so `test` runs. `build-and-push` has `if: github.event_name == 'push' && ...`, which is false for a pull request, so it, and `deploy` after it, don't run. This is exactly right: **you want tests to run on every proposed change, and deployment to happen only once it's merged.**
- A **direct push to main** matches the `push` trigger, `test` runs, its condition is satisfied so `build-and-push` runs, and `deploy`'s condition is satisfied too: all three run in order, because `needs:` makes each wait for the one before it to succeed.
- A **push to any other branch** matches nothing in the `on:` block at all: the workflow doesn't start.

Try the test job for real. Copy `.github/workflows/deploy.yml`, `pipeline_image/`, and a `tests/` folder with at least one `pytest` test into your Chapter 26 repository, push a branch, and open a pull request to `main`. The **Actions** tab shows the workflow: `test` runs and should pass, and the other two jobs show as skipped, exactly as the simulation said. (Merging would then start `build-and-push`, which fails at the AWS login without an account. That's expected.)

What good CI/CD adds beyond "it runs"

- **Fail fast, fail clearly.** A failed test should stop the pipeline from deploying, with a message that says what broke, not a general failure.
- **Environments with protection rules.** The `environment: production` line can require a named approver before that job runs, which is where a human checkpoint belongs for the highest-stakes step.
- **Rollback is a redeploy, not a rescue mission.** Every image is tagged with its commit (`{{ github.sha }}`), so rolling back is deploying the previous commit's image, a normal, tested action rather than an emergency improvisation.

52.6 Secrets, done properly

Chapter 45 said tokens don't belong in code. Here's the full version of where they do belong, tying together everything this chapter has built:

Layer	Where the secret lives	Not here
Local development	A <code>.env</code> file, listed in <code>.gitignore</code> and <code>.dockerignore</code> , never committed	Hardcoded in a script
Docker and Docker Compose	Passed in when the container starts (<code>--env-file .env</code> , or <code>\${VARIABLE}</code> in the Compose file)	Baked into the image with <code>ENV</code> or <code>COPY</code>
Kubernetes	A Secret object (better: sealed or provider-managed, since raw Secrets are only base64-encoded)	A ConfigMap, or committed YAML
CI/CD	The platform's own encrypted secrets store (GitHub Actions secrets, and similar in every CI tool), or no secret at all, with OIDC	A workflow file, even a private one
Cloud infrastructure	A managed secrets service (AWS Secrets Manager, Azure Key Vault, Google Secret Manager)	A Terraform variable's default value, or the state file in Git

The common failure across every layer is the same: a secret that was "just for testing" ends up committed to Git, and Git remembers everything, including in a repository's history, forever, even after the file is deleted. **Rotate a leaked credential immediately, and treat "leaked to the wrong channel" the same as "leaked publicly."** Chapter 47's incident process (section 47.8) applies directly: detect, classify severity, contain by rotating, fix, review.

52.7 Cost

Chapter 49 estimated the sensor archive's storage cost. Compute, and how you run it, changes the total more than storage usually does. Price the pipeline's container on **AWS Fargate** in Mumbai, which bills for the CPU and memory a container reserves, for as long as it runs:

```
FARGATE_VCPU_HOUR = 0.04256    # $ per vCPU per hour: AWS price list, Asia Pacific (Mumbai), checked 29 Sep 2026
FARGATE_GB_HOUR = 0.004655    # $ per GB of memory per hour, same source
price_per_hour = 0.5 * FARGATE_VCPU_HOUR + 1 * FARGATE_GB_HOUR    # half a vCPU and 1 GB
print(f"${price_per_hour:.4f} an hour")
```

```
$0.0259 an hour
```

- A **vCPU** is one virtual CPU core. The container is sized like section 52.3's requests: half a core (0.5) and 1 GB of memory (1).
- By hand: $0.5 \times 0.04256 = 0.02128$, plus 0.004655, is 0.025935, about 2.6 cents an hour.

Now run it two ways: always on, or only for the hour or so a day the pipeline needs (generously; the Flash itself takes a minute or two). **Before you run it, predict** the ratio between the two.

```
HOURS_PER_MONTH = 730                # 24 × 365 ÷ 12
always_on = price_per_hour * HOURS_PER_MONTH
scheduled = price_per_hour * 30        # about an hour a day
print(f"always on : ${always_on:.2f} a month")
print(f"scheduled : ${scheduled:.2f} a month")
print(f"ratio      : {always_on / scheduled:.1f} times")
```

```
always on : $18.93 a month
scheduled : $0.78 a month
ratio      : 24.3 times
```

- By hand: $0.025935 \times 730 = 18.93$; $\times 30 = 0.78$; and the ratio is simply $730 \div 30 = 24.3$, whatever the price.

Add Chapter 49's storage. Chapter 49 priced S3 in AWS's US East region, at \$0.023 per GB-month; in Mumbai, where section 52.4 puts the bucket, S3 Standard lists at \$0.025 for the first 50 TB (AWS price list, checked 29 September 2026):

```
S3_GB_MONTH = 0.025
storage = 277 * S3_GB_MONTH                # Chapter 49's rollout archive: 277 GB
total = storage + scheduled
print(f"storage : ${storage:.2f} a month")
print(f"compute : ${scheduled:.2f} a month")
print(f"total   : ${total:.2f} a month")
```

```
storage : $6.93 a month
compute : $0.78 a month
total   : $7.70 a month
```

Reading it. A container running around the clock costs about **24 times more** than the same container running only when the pipeline needs it. This is the same lesson as Chapter 49's storage-scanning arithmetic, applied to compute: **the cheapest infrastructure decision is usually "don't run it when nothing needs it,"** not a smaller container. Scheduled or serverless compute (a container that starts on a trigger and stops when it's done, Chapter 46's job model exactly) is very often the right default for a batch pipeline. An always-on service is for things that must answer requests at any moment, though even a webhook receiver (Chapter 51) can be serverless (section 52.8), paying per request rather than per hour.

And the platform has a price too. A managed Kubernetes cluster charges for itself, before it runs anything:

```
EKS_CLUSTER_HOUR = 0.10          # $ per cluster per hour: AWS price list, Mumbai, checked 29 Sep 2026
cluster = EKS_CLUSTER_HOUR * HOURS_PER_MONTH
print(f"cluster fee alone       : ${cluster:.2f} a month")
print(f"the same job on Kubernetes : ${cluster + scheduled + storage:.2f} a month")
print(f"always on, 2 copies, on K8s : ${cluster + 2 * always_on + storage:.2f} a month")
```

```
cluster fee alone       : $73.00 a month
the same job on Kubernetes : $80.70 a month
always on, 2 copies, on K8s : $117.79 a month
```

Reading it. Total estimate for Riverstone's pipeline and its sensor archive, run as a scheduled container: **about \$8 a month**. The same job on managed Kubernetes costs about ten times as much, and an always-on pipeline in two copies there about fifteen times as much: the price of the platform, not the pipeline. It's a small number for a real reason: most of what this book has built runs briefly, once a day, on a modest amount of data. That won't stay true forever, and knowing how to estimate it, rather than guessing, is what lets you notice when it stops being true.

Simplification note. These are list prices for Linux containers on x86 processors, before taxes, free tiers, or discounts, and they change; check your provider's price page or calculator before relying on them. The estimate also leaves out small items such as the container registry's storage and log storage, and the one-minute minimum Fargate bills for each run.

52.8 Choosing a deployment target

Option	Service level	Fits	Operational cost
A single managed container service (AWS ECS on Fargate, AWS App Runner, Google Cloud Run, Azure Container Apps)	PaaS	One or a few independent services, including scheduled jobs, like Riverstone's pipeline today	Very low: little to configure, scales automatically within limits
A managed orchestrator (AWS ECS with several services, Google Cloud Run for services with more control)	PaaS	A handful of services that need to talk to each other, without full Kubernetes complexity	Low to moderate
Managed Kubernetes (EKS, GKE, AKS)	PaaS for the cluster's control, IaaS-like for its nodes	Many services, teams, and workloads sharing a platform	Moderate to high: someone has to own cluster upgrades, networking, and access
Self-managed Kubernetes on virtual servers	IaaS	Rarely justified below significant scale, or strict on-premises requirements	High: you own everything a managed service would have handled
Serverless functions (AWS Lambda, Google Cloud Functions)	FaaS (functions as a service), a form of PaaS	Short, event-triggered tasks (a webhook receiver, a small transform)	Very low, pay-per-invocation

For Riverstone today, one scheduled container on a managed container service is the right size: on AWS, an ECS task on Fargate, started at 6:30 India time, using the role from section 52.4 and the image the CI/CD workflow in section 52.5 builds and registers. It costs a few dollars a month (section 52.7) and needs nobody dedicated to operating it. The Kubernetes files in section 52.3 are worth having written and understood, because the vocabulary is universal and the day this company runs ten services instead of one, the migration is a known, well-trodden path rather than a redesign from nothing.

APPLY

Common mistakes

Mistake	Symptom	Fix
Granting broad access "for now"	Permissions nobody remembers to remove	Scope access narrowly from the start (52.1)
A security group open to the whole internet	Anyone can reach a database or pipeline	Allow only the specific source that needs it (52.1)
Copying application code before installing dependencies in a Dockerfile	Every build reinstalls everything, even for a one-line code change	Copy dependency files first, install, then copy code (52.2)
Running a container process as root	A vulnerability inside the container has full control of it	Create and switch to a non-root user (52.2)
No <code>.dockerignore</code>	<code>.env</code> , with its passwords, is copied into the image	List <code>.env</code> and data folders in <code>.dockerignore</code> (52.2)

Mistake	Symptom	Fix
<code>depends_on</code> without a health check	The pipeline starts before the database accepts connections, and fails on its first connection	<code>condition: service_healthy</code> and a <code>healthcheck</code> (52.2)
Running a scheduler in several replicas	The job runs once per copy: two Flashes every morning	A <code>CronJob</code> or a platform schedule for batch jobs; replicas only for services with no state of their own (52.3)
A readiness probe on a path the application doesn't serve	The Pod never receives traffic, or the probe proves nothing	Probe a path the application actually serves for that purpose (52.3)
Reaching for Kubernetes for one service	Complexity with nothing to show for it	Use it when several services need scheduling together (52.3, 52.8)
Clicking through a console instead of writing IaC	Nobody can reproduce what exists, or review a change before it happens	Terraform (or equivalent), reviewed like code (52.4)
Applying Terraform without reading the plan	An unexpected <code>destroy</code> runs in production	Read every plan; treat <code>-</code> lines as a stop sign (52.4)
Storing Terraform state on a laptop, or in Git	Two people's changes corrupt each other; secrets in the state leak	Shared, locked, encrypted remote state (52.4)
Secrets in a committed file, "just this once"	The credential is in Git history forever, even after deletion	A secrets store at every layer (52.6); rotate immediately if it happens
Running compute around the clock for a job that runs once a day	A bill many times larger than necessary	Scheduled or serverless compute for batch work (52.7)
No environment protection on the deploy step	A merge to main ships straight to production with no checkpoint	Required approval on the production environment (52.5)

In the real world: the afternoon the warehouse went dark

Riverstone's contract data engineer was cleaning up the Terraform for the pipeline's network, and removed one security group rule he believed unused. He ran `terraform apply`, saw the familiar list of small changes, and confirmed it without reading past the first few lines.

The plan had listed that removal, and, further down, a second one: a rule a previous engineer had added by hand in the cloud console months earlier. Terraform managed the security group's whole list of rules, so a rule that wasn't in the file looked exactly like a rule that shouldn't exist, and the plan said it would delete it, on a line nobody had scrolled down to see. That second rule was the one letting the pipeline reach the database.

Twenty minutes later, Meera couldn't load the sales dashboard, and the morning's Flash hadn't been sent. Restoring the rule took ten minutes once the cause was found, and finding the cause took two hours, because nothing had logged **why** connectivity had disappeared, only that it had.

The fixes were the ones this chapter has built toward. Every resource that existed was **imported** into Terraform, so the file and reality agreed completely, with no manual console changes left unrecorded anywhere. Every `apply` after that went through a pull request, with the **plan's full output posted for**

review, not just a summary. And a Chapter 47-style check was added: a scheduled test that tries a real database connection from a throwaway container and alerts within minutes if it fails, rather than waiting for a missed report to be the first sign.

At the retrospective, the engineer made the point himself before anyone else needed to. "The plan told me exactly what was about to happen. I didn't read it, because it looked routine."

What made this work.

- **The cause was a gap between reality and the code describing it** (people call it **drift**), not a bug in Terraform, which did precisely what it was told.
- **Reading the plan is the review step**, and skipping it is the same mistake as merging code no one looked at.
- **Importing everything into code** closed the gap for good, rather than patching this one incident.
- **A synthetic connectivity check** meant the next version of this failure would be caught in minutes, not discovered by a missing report.

Project: deploy Riverstone's pipeline

Goal: a complete, reviewed deployment package for the Chapter 46 pipeline: a container image you have built and run, a Compose file for local development, orchestration files, infrastructure as code, and a CI/CD workflow, checked as far as a laptop allows, with an honest runbook for what remains to actually deploy it.

Tools you'll need

- **Python 3.14** in the book's virtual environment, with `pyyaml`, `python-hcl2`, and `dockerfile-parse` (section 52.0).
- **Docker** Desktop or Docker Engine, with Compose (section 52.2).
- **Terraform** (section 52.4), or OpenTofu.
- **Your Chapter 26 GitHub repository**, for the workflow's test job.
- **Optional:** kind and kubectl, to try section 52.3 on your laptop.
- **The Chapter 52 companion folder** (`companion/ch52/`), copied to `work/ch52` (section 52.0), and your Chapter 46 practice folder, `work/ch46`, with PostgreSQL, the practice source, and the practice CRM API as in Chapter 46.
- **Versions used for the outputs shown:** Python 3.11.15 for the notebook cells (the outputs are the same on 3.14), PyYAML 6.0.3, python-hcl2 8.1.4, dockerfile-parse 2.0.1; Docker 29.3.1 with Compose 5.1.1; the image's Python 3.14 with Dagster 1.13.23; Terraform 1.16.4 with the AWS provider 6.66.0; PostgreSQL 16.

Option A: Riverstone. Extend the companion files.

Option B: your own project. Any application you've built in this book, containerized the same way.

Steps

1. **Write a Dockerfile** for a pipeline of your choice, ordered for caching, running as a non-root user, with a `.dockerignore` and no secrets baked in. Check it with `dockerfile-parse`, then build it and run it until it produces its output.
2. **Write a docker-compose.yml** for local development, with secrets from `.env` and a health check on the database. Check it with PyYAML, run it with `docker compose up -d`, and stop it with `docker compose down`.
3. **Write a Kubernetes CronJob** for the same job, and write down, in your own words, what would happen if its Pod crashed halfway through, and why a Deployment with two replicas would be wrong for it.
4. **Write a small Terraform configuration** for one piece of infrastructure your project needs (storage, a role, a network rule). Run `terraform init -backend=false, fmt, and validate`, and write the IAM policy with the narrowest permissions you can justify.
5. **Write a CI/CD workflow** with at least a test job and a deploy job gated on the test passing and the branch being `main`. Simulate its trigger logic in Python for three events, as this chapter did, and run the test job for real in your Chapter 26 repository.
6. **Write your secrets plan**: where each secret lives at each layer, referencing section 52.6's table.
7. **Estimate the monthly cost** of running it, using section 52.7's approach with your provider's current prices, and compare an always-on option with a scheduled one.
8. **Write the deployment runbook**: the exact steps a person would follow to deploy this for real, including what they'd check in the Terraform plan before approving it, how the schedule finds the newest image, and what they'd do if the deploy step failed.

Stretch goals

- Run section 52.3's CronJob in a kind cluster, and start one run with `kubectl create job --from=cronjob/...`
- If you have a free-tier cloud account, apply the Terraform configuration to a throwaway project and destroy it afterward, reading every line of both plans.
- Add a synthetic health check to your CI/CD workflow, in the style of this chapter's closing story.

REVIEW

Recap

- **IaaS, PaaS, and SaaS** rent you progressively more, and leave you progressively less to manage.
- **Regions and availability zones** are about geography and independence; spread anything critical across at least two zones.
- The **shared responsibility model** moves its line with what you rent, but your data, who can access it, and how you configure the service are always yours.
- **Least privilege IAM**: a role with a trust policy (who may use it) and a permission policy (what it may do), scoped to one resource.

- **CIDR arithmetic** by hand: a /16 VPC split into /24 subnets gives $2^8 = 256$ subnets of 256 addresses, 251 usable under AWS's rule.
- **Containers** package an application with everything it needs; **Dockerfiles** should install dependencies before copying code, keep secrets out with `.dockerignore`, and run as a non-root user. You built one and ran Chapter 46's Flash in it.
- **Docker Compose** runs a small stack; a health check makes the pipeline wait until its database is ready.
- **Kubernetes** objects (CronJob, Deployment, Service, Secret) solve scheduling, healing, and networking for many services. Batch jobs are CronJobs; replicas are for services with no state of their own.
- **Infrastructure as Code**, checked with `validate` and read with `terraform plan` before every `apply`, makes infrastructure reviewable, versioned, and reproducible; anything the file doesn't mention, Terraform plans to destroy.
- **CI/CD** runs tests on every change and deploys only what passes; a workflow's trigger and `if` conditions can be traced precisely, as this chapter did.
- **Secrets** belong in a dedicated store at every layer, never in code, images, or committed files, and a leak anywhere means an immediate rotation.
- **Compute cost is dominated by whether something runs when nothing needs it**: the same container cost about 24 times more running constantly than running on Riverstone's actual schedule, and a Kubernetes cluster costs more than the whole pipeline.
- **Choose the simplest deployment target that fits today's number of services**, and treat Kubernetes as a destination to grow into, not a default to start with.

Key terms

IaaS · PaaS · SaaS · region · availability zone · shared responsibility model · IAM (identity and access management) · identity · role · temporary credentials · policy · statement · ARN · least privilege · trust policy · permission policy · VPC (virtual private cloud) · subnet · CIDR notation · load balancer · security group · container · image · registry · tag · Docker · Dockerfile · build context · layer · build cache · `.dockerignore` · volume · exit code · Docker Compose · health check · Kubernetes · cluster · node · Pod · Job · CronJob · Deployment · replica · label and selector · Service · readiness probe · liveness probe · Secret (Kubernetes) · namespace · manifest · resource requests and limits · Infrastructure as Code (IaC) · Terraform · HCL · provider · state file · backend · plan · apply · drift · CI/CD · continuous integration · continuous deployment · GitHub Actions · workflow trigger · job · OIDC · secrets manager · serverless · managed container service

(All terms are defined in the Glossary, Appendix A.)

Check yourself

- ☐ You can say what IaaS, PaaS, and SaaS rent you, and where the shared responsibility line falls for each.

- ☐ You can explain regions and availability zones in your own words.
- ☐ You can read an IAM policy field by field, and write one that grants the narrowest access a task needs.
- ☐ You can compute how many usable addresses a given CIDR range provides, and split a VPC into subnets by hand.
- ☐ You can explain why containers solve "it works on my machine", and the difference between an image and a container.
- ☐ You can write a Dockerfile ordered for caching, running as a non-root user, then build and run it.
- ☐ You can run a two-container stack with Docker Compose that waits for its database to be ready.
- ☐ You can read a Kubernetes CronJob, Deployment, and Service and say what each line does, and say which one a batch job needs.
- ☐ You can check a Terraform configuration with `validate`, and read a plan's `+`, `~`, `-`, and `-/+` lines.
- ☐ You can trace exactly which jobs in a CI/CD workflow run for a given event, using its actual trigger and `if` conditions.
- ☐ You can name where a secret should live at every layer, from a laptop to production.
- ☐ You can estimate the cost difference between always-on and scheduled compute.
- ☐ You can recommend a deployment target for a company of a given size, and say when Kubernetes earns its complexity.

PRACTISE

Exercises

Warm-up

1. A cloud provider's storage bucket is left publicly readable by mistake. Whose responsibility is that under the shared responsibility model, and why?
2. A VPC is `10.20.0.0/16`. How many total addresses does it contain, and how many `/24` subnets can it be split into?
3. In a Dockerfile, which should come first: `COPY requirements.txt .` and `RUN pip install`, or `COPY . .` (copying all the application code)? Explain why in terms of build caching.

Core

4. A `/24` subnet is created for a database that will only ever need 40 addresses. Using the reservation rule from section 52.1, how many usable addresses does it actually get, and what does that suggest about right-sizing a subnet?
5. Using this chapter's CI/CD workflow, trace what happens for a pull request from a feature branch targeting a branch called `develop` (not `main`). Which jobs run, and why?
6. Explain the difference between a Kubernetes `resources.requests` value and a `resources.limits` value, using the CronJob's CPU values (`250m` requested, `1` as the limit) as your example.

7. The Terraform example in section 52.4 grants the pipeline's role `s3:GetObject`, `s3:PutObject`, and `s3:ListBucket` on one bucket. Using section 52.1's least-privilege rule, explain why `s3:DeleteObject` and access to other buckets are deliberately left out.
8. A container costs \$0.05 an hour. Its job runs for about 2 hours a day. Using the cost model in section 52.7, calculate its monthly cost always on and scheduled, the ratio between them, and explain in one sentence why the ratio is smaller than section 52.7's 24.3.
9. In the closing story, `terraform plan` correctly showed the security group rule as something to be destroyed, and the engineer still caused an outage. What does this tell you about the limits of tooling alone, and what process change actually fixed it?

Stretch

10. Design the IAM setup for a second pipeline that needs to read from the sensor archive bucket but must never be able to write to it. Write the policy statement's `Action` and `Resource` fields the way section 52.4 did, and explain what would go wrong if you granted it the existing role instead of a new one.
11. Riverstone's pipeline grows from one service to four: ingestion, the Dagster web interface, a webhook receiver from Chapter 51, and a small internal dashboard. Using section 52.8, make the case for moving to a managed orchestrator or Kubernetes at this point, and say what you'd want to see before agreeing.
12. Write a GitHub Actions workflow (or extend this chapter's one) that adds a `staging` deployment on every push to a `staging` branch, separate from the existing `production` deployment on `main`. Trace, as this chapter did, which jobs would run for a push to `staging`.

Think about it (no code needed)

13. "Faking output would be worse than saying so plainly." Explain why a book that claims to have run code it hasn't would be more damaging to a reader than one that clearly labels what wasn't executed, using an example from earlier in this book (Chapter 46) if it helps.
14. A colleague argues that since the cloud is so cheap for a company Riverstone's size (a few dollars a month), cost discipline doesn't matter yet and can wait until the company is bigger. Argue against that position using this chapter's ideas.

Answers

1. It's the **customer's** responsibility. Under the shared responsibility model, the provider secures the underlying infrastructure and, for a managed storage service, the software of the service itself, but **configuration of that service**, including who can read a bucket, is squarely the customer's job at every service level. "Your data, who can access it, and how you configure the service are always yours" draws the line exactly here.
2. A /16 has $32 - 16 = 16$ bits free, so $2^{16} = \mathbf{65,536}$ total addresses. Splitting into /24 subnets uses $24 - 16 = 8$ more bits per subnet, so $2^8 = \mathbf{256}$ subnets.

3. COPY requirements.txt . and RUN pip install should come first. Docker caches each instruction's result and reuses it if nothing that affects it has changed. If `COPY . .` (all the code) came first, then editing any single application file would invalidate that layer and every layer after it, forcing a full dependency reinstall on every code change. Installing dependencies from a separate, rarely-changing file first means that layer stays cached across nearly every rebuild, as the `CACHED` lines in section 52.2 showed.

4. The subnet still gets a full /24 of 256 addresses, minus AWS's 5 reserved addresses, leaving **251 usable addresses**, of which only 40 will ever be used. This suggests the subnet is oversized for its purpose: a smaller /26 (64 addresses, 59 usable after reservations; 60 on Google Cloud) would comfortably fit 40 hosts with room to grow, leaving far more of the address space free for other subnets rather than reserving a needlessly large range for a small, fixed need.

5. Nothing runs. The workflow's `on: block` only triggers on `pull_request` targeting `main` and `push` to `main`. A pull request targeting `develop` doesn't match the `pull_request` trigger's branches: `[main]` restriction at all, so the workflow never starts, and none of `test`, `build-and-push`, or `deploy` run.

6. requests is what Kubernetes guarantees will be available to the container and uses when deciding which node to schedule it on: `250m` means a quarter of one CPU core is reserved for it. **limits** is the ceiling the container is never allowed to exceed: `1` means it can burst up to a full core under load, but no further, even if more is available on the node. A container using more than its request but less than its limit is normal; one hitting its limit gets throttled (for CPU) or killed and restarted (for memory).

7. Granting only `GetObject`, `PutObject`, and `ListBucket` on this one bucket means the role can read and write sensor data and see what's in the bucket, which is everything the pipeline actually does. Leaving out `s3:DeleteObject` means a bug in the pipeline, or a compromised credential, **cannot delete data**; it can still overwrite a file, but with the bucket's versioning (also in this configuration) the previous version is kept, which makes accidental or malicious data loss far harder. Leaving out access to other buckets means even a fully compromised pipeline credential can't touch Riverstone's other cloud storage, containing the blast radius of any single failure to exactly the one resource that needed access, which is least privilege in practice, not just in principle.

8. Always on: $0.05 \times 730 = \text{\$36.50}$ a month. Scheduled: $2 \text{ hours} \times 30 \text{ days} = 60 \text{ hours}$, $\times 0.05 = \text{\$3.00}$ a month. The ratio is $730 \div 60 \approx \textbf{12.2 times}$. It's smaller than 24.3 because the job itself runs twice as long each day, so there are half as many idle hours to stop paying for: the ratio depends only on how many of the month's 730 hours the job actually needs, not on the price.

9. It shows that **tooling correctly informing you is not the same as the information being acted on**: `terraform plan` did exactly its job, and the outage still happened, because the output wasn't read carefully. The process change that actually fixed it wasn't a better tool, it was **making the plan's full output part of a reviewed pull request**, so a second person, not under the time pressure of "just tidying up," had a chance to notice the destructive line before it was applied. Importing the hand-made rule into Terraform closed the drift that caused the surprise in the first place. Tools reduce the chance of an unreviewed mistake; only a review step catches the mistake a tool correctly reported and a person missed.

10. A sample policy statement: `Action = ["s3:GetObject", "s3:ListBucket"], Resource = [aws_s3_bucket.sensor_archive.arn, "${aws_s3_bucket.sensor_archive.arn}/*"]`, attached to a separate `aws_iam_role` from the existing pipeline role, with its own trust policy. Granting the **existing** role instead would give this new, read-only pipeline the same `PutObject` write access as the original pipeline, which violates least privilege for no reason: if this second pipeline is ever compromised or has a bug, it would be able to write or corrupt data it was only ever supposed to read, an entirely avoidable risk once you've noticed it needs a distinct, narrower role.

11. The case for moving up: with **four services** that need independent scaling, restarting, and internal networking (the dashboard talking to the pipeline's data, say), a single managed container service starts to mean four separate deployments to coordinate by hand, with no shared service discovery, health checking, or rollout strategy between them, exactly the coordination problem Kubernetes (or a lighter managed orchestrator like ECS with several services) exists to solve. Before agreeing, you'd want to see: that the four services genuinely need to talk to each other (if they're fully independent, four separate simple deployments may still be simpler than one shared platform); that someone is willing to own cluster upgrades and access control ongoing, not just the initial setup; and a cost comparison, since a managed cluster has a baseline cost of its own (\$73 a month for the EKS control plane alone in section 52.7, before any work runs), which only pays off once the coordination problem is real. You'd also check which services keep state: the webhook receiver and the Dagster web interface would need their state moved to a shared database before running in more than one copy.

12. A sample extension adds `staging` to both branches: lists in `on:`, and a new job, `deploy-staging`, with `if: github.event_name == 'push' && github.ref == 'refs/heads/staging'`, its own build step (or a `build-and-push` whose `if:` accepts both branches), and its own `environment:` `staging` with protection rules suited to a lower-stakes environment (perhaps no required approval, unlike production). Tracing a push to `staging`: the push trigger now matches, so `test` runs (it has no `if:`), and only the staging jobs satisfy their `if` conditions, while the production `build-and-push` and `deploy` jobs, still conditioned on `refs/heads/main`, correctly don't run. Without adding `staging` to `on:`, nothing would run at all, as in Exercise 5.

13. A book that quietly presents fabricated output as if it were real teaches the reader a false picture of what actually happens when code runs, which is far worse than a gap honestly marked, because the reader has no way to know which parts to distrust. Chapter 46 marked its `run_failure_sensor` and Airflow examples as not executed, precisely so a reader building on this book's examples knows exactly which parts have been proven to work as shown and which are correct-looking illustrations still needing their own testing in a real environment. This chapter did the same for its Kubernetes steps and the AWS jobs of its workflow. Once a single fabricated output is discovered, a reader reasonably starts doubting every other output in the book, even the ones that were genuinely verified, which destroys far more trust than the honest gap ever would.

14. Three things from this chapter argue against waiting. First, **habits formed on a small bill are the habits that scale**: a team that never learns to distinguish always-on from scheduled compute, or to read a cost estimate before deploying, doesn't suddenly develop that discipline when the bill crosses some threshold, they carry the same unexamined choices into a much larger number. Second, the **24-times cost difference** in section 52.7 between always-on and scheduled compute is a *ratio*, not an absolute; it applies exactly as much at Riverstone's current scale as it will at ten times the scale, and the sooner the right default is in place, the more total waste is avoided across the company's whole

future, not just today's few dollars. The platform choice matters the same way: the same job on a Kubernetes cluster cost about ten times as much. Third, cost estimation is itself a skill, exactly like the reconciliation habits Chapter 47 taught for data: the point of estimating early isn't the money saved on a small bill, it's building the muscle of noticing *when something has stopped being small*, which only works if you've been checking all along, not starting the habit the day the number becomes alarming.

NEXT

Where this leads

- **Chapter 65, FinOps: The Economics of Data Platforms**, returns to cost management at the level of an entire organization's cloud spend.
- **Chapter 64, Security, Privacy, Governance & Responsible AI**, builds on the shared responsibility model and least-privilege access from section 52.1.
- **Chapter 60, Designing Whole Systems**, returns to choosing platforms at the whole-company scale that section 52.8 only began; **Chapter 63, Automation Architecture & Governance**, to which tool runs which automation.
- **Chapter 46's** pipeline, **Chapter 47's** checks, **Chapter 49's** storage, **Chapter 50's** streaming jobs, and **Chapter 51's** syncs are all things this chapter's infrastructure would actually run.
- **Part 8:** cloud, containers, and deployment questions appear in the data engineering interview chapters, and system design cases in Chapter 77 routinely ask how you'd deploy and operate exactly what this chapter builds.